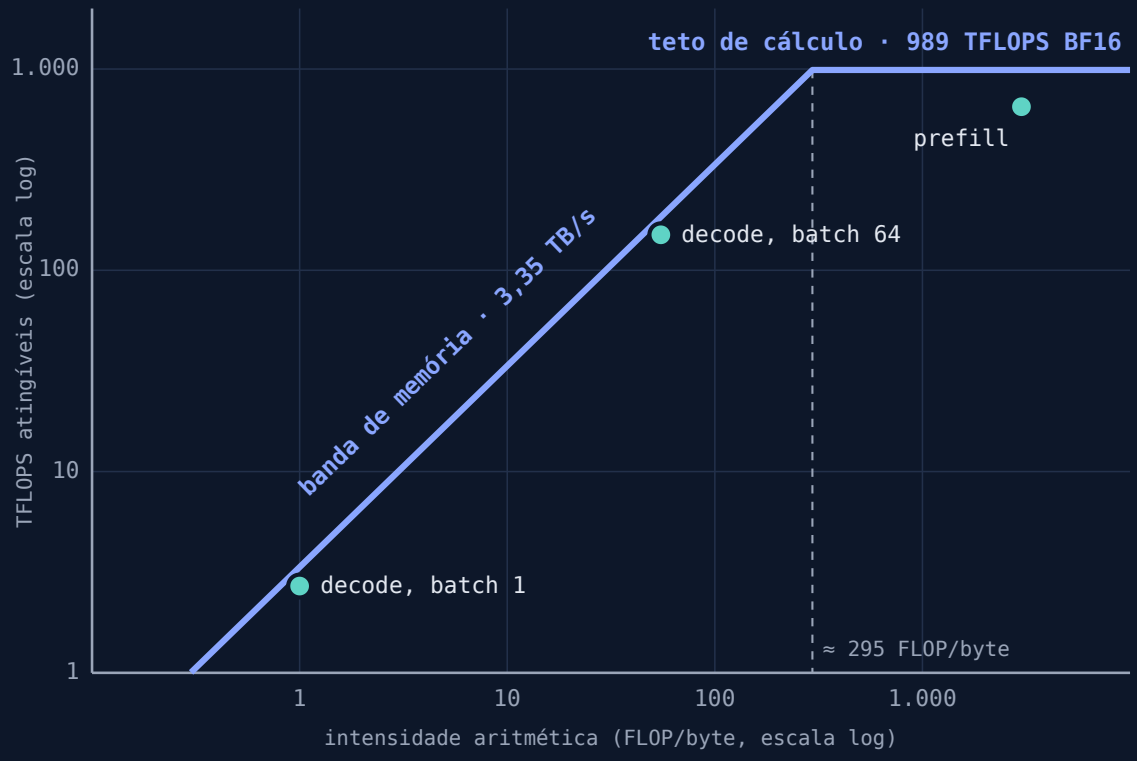
GUIA DE ENTREVISTAS · 1ª EDIÇÃO · 2026

Engenharia de Inferência

Os 14 temas e as 50 perguntas que caem em OpenAI, Anthropic, NVIDIA, Google e outras empresas, explicados do zero até o nível sênior.

14 capítulos · 50 perguntas comentadas · código testado



Júlia Cardoso

@juliacardoso.dev

GUIA DE ENTREVISTAS · 1ª EDIÇÃO · NOVEMBRO DE 2026

Engenharia de Inferência

Os 14 temas e as 50 perguntas que caem em entrevistas de engenharia de inferência em OpenAI, Anthropic, NVIDIA, Google e outras, explicados do zero até o nível sênior.

Júlia Cardoso @juliacardoso.dev

As 50 perguntas comentadas vêm do levantamento publicado pelo AI Engineering Insider no post “You should never walk into an Inference Engineering interview without knowing these 50 questions”. As respostas, explicações, diagramas e códigos desta edição são originais.

Valores de hardware, preços e números de desempenho são aproximados e servem para raciocínio; confira as especificações da geração com que você vai trabalhar.

Sumário

Introdução 9

O que é engenharia de inferência 9

O dia a dia da função 10

Carreira e salários 11

O que as vagas pedem 11

De onde vêm as perguntas 12

Como usar este livro 13

CAPÍTULO 1 **Fundamentos de Engenharia de Inferência** 16

1.1 A vida de uma requisição 16

1.2 Ler é mais caro que calcular 17

1.3 O modelo roofline 19

1.4 As métricas que importam 20

1.5 Custo por token 23

1.6 Inferência síncrona, assíncrona e em lote 23

1.7 O mapa de otimizações 24

Perguntas de entrevista comentadas 25

Q001 Qual é a sua experiência com otimização de inferência para LLMs? 25

Q002 Por que a inferência de LLMs é limitada por memória (memory-bound)? 27

Q003 O que são TTFT, TPOT e throughput, e como um compete com o outro? 29

Q004 Explique passo a passo como um forward pass de um transformer é executado de fato numa GPU. 31

Q007 Como você aborda a otimização de custos de um sistema de IA que está estourando o orçamento? 34

Q008 Inferência síncrona vs. assíncrona: quando usar cada uma? 37

CAPÍTULO 2 **Transformer e LLMs por dentro** 41

2.1 Do texto aos números: tokenização e embeddings 41

2.2 A estrutura de um decoder moderno 42

2.3 Atenção, do zero 43

2.4 Posições: RoPE 45

2.5 MLP, normalização e o resto 46

2.6 Sampling: escolhendo o próximo token 46

2.7 Prefill e decode por dentro 47

2.8 O KV cache, conceito 48

2.9 FlashAttention 49

Perguntas de entrevista comentadas 51

Q009 O que é um KV cache? O que é FlashAttention? (perguntadas em sequência) 51

Q010 Explique o que fica em cache em cada camada num KV cache e as armadilhas de corretude ao adicionar um (máscaras, posições, RoPE, shapes). 53

Q011 Explique prefill vs. decode e por que os sistemas de serving tratam cada um de forma diferente. 56

CAPÍTULO 3 **Arquitetura de GPU e CUDA** 60
3.1 CPU vs GPU: duas filosofias 60
3.2 Anatomia de uma GPU NVIDIA (Hopper / H100) 61
3.3 O modelo de programação: grid, blocos, threads, warps 62
3.4 A hierarquia de memória 64
3.5 Occupancy 66
3.6 Tensor cores 67
3.7 Lei de Amdahl 67
3.8 Multiplicação de matrizes: a progressão de otimizações 68
Perguntas de entrevista comentadas 70
Q017 Explique o modelo de execução do CUDA, a hierarquia de memória e a ocupação (occupancy). Depois, otimize uma GEMM com tiling e coalescing. 70
Q018 Explique a lei de Amdahl e como ela se aplica à multiplicação de matrizes em GPU e aos ganhos de velocidade de ponta a ponta. 72
Q019 Como você calcularia uma multiplicação de matrizes de forma eficiente numa GPU sem usar memória compartilhada? 75
Q020 Fale de um kernel CUDA que você escreveu: as otimizações, os desafios e exatamente o que você fez. 78

CAPÍTULO 4 **Otimização de Desempenho em GPU** 82
4.1 O método 82
4.2 Roofline a fundo: MFU e MBU 83
4.3 Ferramentas 84
4.4 Escondendo latência: a lei de Little 85
4.5 Fusão de kernels 86
4.6 Overhead de CPU e CUDA graphs 86
4.7 torch.compile por dentro 87
4.8 Medindo direito 88
Perguntas de entrevista comentadas 89
Q025 Um kernel atinge 30% da largura de banda de memória máxima numa H100. Como você descobre o porquê? 89
Q026 Sua redução é mais lenta com 64 threads por bloco do que com 256. O que está acontecendo? 92
Q027 Dê um exemplo de kernel que atingiu alta utilização dos tensor cores. Como você conseguiu isso? 95
Q028 Projete e faça o benchmark de um pipeline de inferência otimizado: TorchDynamo, TorchInductor, backends externos, fusão, CUDA graphs, quantização. 98

CAPÍTULO 5 **Kernels CUDA e Triton** 103
5.1 Esqueleto de um programa CUDA 103
5.2 GEMM FP32 completa, em três versões 104
5.3 Reduções 109
5.4 Triton 110
Perguntas de entrevista comentadas 113
Q033 Implemente um kernel CUDA FP32 de alto desempenho para C = A·B (A é m×k, B é k×n) e explique suas escolhas de arquitetura. 113
Q034 Projete uma redução CUDA determinística que garanta uma ordem de redução bem definida. (Dica dada: parecido com prefix sum.) 115
Q035 Como funcionam o TensorRT-LLM e o vLLM? (perguntada junto com "implemente um loop de inferência em PyTorch") 119

CAPÍTULO 6 **KV Cache e Gerenciamento de Memória** 124
6.1 Para onde vai a memória da GPU 124
6.2 A tabela que você deve saber montar 125
6.3 O problema da alocação ingênua 126
6.4 PagedAttention 126
6.5 Prefix caching 128
6.6 Quando a memória acaba: preempção 129
6.7 Hierarquia de armazenamento do KV cache 130
6.8 Diminuindo o próprio KV cache 130
Perguntas de entrevista comentadas 131
Q041 Projete e implemente um cache de chave-valor para inferência autorregressiva num decoder Transformer: shapes dos tensores no prefill e no decode, API de cache por camada, batching e beam search, precisão mista. 131
Q042 Sua inferência de LLM está ficando sem memória de GPU em conversas longas. O que você faz? 135
Q043 Um assistente de chat reenvia a conversa inteira a cada turno. Como evitar recalculá-la, e quais são as armadilhas? 138
Q044 Como funciona o PagedAttention do vLLM, e por que ele importa para a eficiência de memória da GPU? 141

CAPÍTULO 7 **Serving de LLMs e Continuous Batching** 145
7.1 Batching estático 145
7.2 Batching dinâmico (clássico) 146
7.3 Continuous batching 146
7.4 Anatomia de um scheduler 147
7.5 Chunked prefill 149
7.6 Head-of-line blocking 149
7.7 Os motores 150
Perguntas de entrevista comentadas 151
Q049 Projete um sistema de batching de inferência para uma única GPU que processe até 100 entradas por batch enquanto os usuários esperam de forma síncrona, maximizando a utilização sob restrições de computação. 151
Q050 Aqui está o design de um desenvolvedor júnior para um sistema de batching de inferência. Critique-o e proponha melhorias. 156
Q051 Quando o TensorRT-LLM é melhor que o vLLM? 158
Q052 Projete o escalonador de um motor de inferência com continuous batching. 160
Q053 Agrupe em batch requisições com tamanhos de prompt e de saída muito diferentes sem bloqueio head-of-line. 164

CAPÍTULO 8 **Quantização e Compressão de Modelos** 168
8.1 Por que quantizar 168
8.2 Formatos numéricos 169
8.3 Quantização linear: a matemática 170
8.4 Granularidade das escalas 171
8.5 O que quantizar 171
8.6 O problema dos outliers 172
8.7 Métodos de quantização pós-treino (PTQ) 173
8.8 QAT e QAD 174
8.9 Pruning 175
8.10 Distilação 175
Perguntas de entrevista comentadas 176
Q057 O que é quantização? O que são QAT e QAD? 176
Q058 Descreva técnicas comuns de otimização de IA para treinamento e inferência: quantização, pruning, distilação, fusão de kernels, otimizações de memória e de throughput. 178
Q059 Explique quantização e distilação de modelos para otimizar inferência: como escolher entre as duas? 181

CAPÍTULO 9 **Inferência Distribuída** 185
9.1 Por que distribuir 185
9.2 Data parallelism (réplicas) 186
9.3 Tensor parallelism 186
9.4 Pipeline parallelism 188
9.5 Expert parallelism (MoE) 189
9.6 Context parallelism 190
9.7 Combinando tudo 190
9.8 Desagregação de prefill e decode 190
Perguntas de entrevista comentadas 191
Q065 Explique paralelismo de tensor versus paralelismo de pipeline, e quando você usaria cada um. 191
Q066 Projete uma multiplicação de matrizes em múltiplas GPUs. 193
Q067 Por que você separaria (desagregaria) prefill e decode em máquinas diferentes, como faz o Mooncake? O que isso traz de ganho e qual é o custo? 196

CAPÍTULO 10 **Redes e Comunicação entre GPUs** 201
10.1 As interconexões 201
10.2 Anatomia de um nó HGX H100 202
10.3 As operações coletivas 203
10.4 O modelo α-β 203
10.5 Ring vs tree all-reduce 204
10.6 NCCL na prática 205
10.7 Kubernetes e GPUs 206
Perguntas de entrevista comentadas 208
Q073 Fale da sua experiência; depois, perguntas sobre GPUs, CI/CD, comunicação entre GPUs, Kubernetes e plugins da NVIDIA para Kubernetes. 208
Q074 All-reduce em anel vs. em árvore: custos e quando cada um é usado. 211
Q075 NVLink, PCIe e InfiniBand: quando cada um vira o gargalo na inferência? 213

CAPÍTULO 11 **Design de Sistemas de Inferência** 217
11.1 O roteiro de resposta 217
11.2 Contas de guardanapo 219
11.3 Arquitetura de referência 220
Perguntas de entrevista comentadas 221
Q081 Projete um serviço de inferência de LLM de alta concorrência: utilização da GPU, memória do KV cache, batching e escalonamento, divisão e junção de requisições. 221
Q082 Projete uma API de inferência baseada em GPU para servir vários modelos de ML: SLOs, batching dinâmico, versionamento e roteamento A/B, autoscaling em nós de GPU heterogêneos, backpressure, isolamento multi-tenant, carregamento e aquecimento (warm-up) de modelos. 224
Q083 Projete um sistema para distribuir arquivos grandes de pesos de modelo (dezenas a centenas de GB, divididos em shards) para uma frota de workers de inferência. 228

CAPÍTULO 12 **Confiabilidade em Produção e Observabilidade** 233
12.1 SLIs, SLOs e orçamento de erro 233
12.2 O que medir 234
12.3 Como a stack de inferência falha 235
12.4 Cauda de latência 236
12.5 Resposta a incidentes 237
Perguntas de entrevista comentadas 237
Q089 Como você investigaria um pod do Kubernetes travado no status Pending? 237
Q090 Explique a stack de inferência de LLMs e seus cenários de falha. 240
Q091 Construa um endpoint dedicado com garantias rígidas de latência de cauda e explique o que muda em relação ao pool compartilhado. 243

CAPÍTULO 13 **Programação e Depuração de Desempenho** 247
13.1 O método para a rodada de código 247
13.2 Profiling e depuração de desempenho 248
Perguntas de entrevista comentadas 250
Q097 Implemente um loop de inferência em PyTorch. 250
Q098 Implemente ordenação topológica em Python. 256
Q099 Serialize e desserialize uma árvore binária, com código limpo e tempo e espaço ótimos. 260

CAPÍTULO 14 **Arquitetura Avançada de Inferência** 267
14.1 Speculative decoding 267
14.2 Serving de muitos adaptadores LoRA 269
14.3 Saídas estruturadas (constrained decoding) 270
14.4 Modelos multimodais 271
Perguntas de entrevista comentadas 271
Q105 Projete uma API HTTP de alta concorrência para inferência de modelos de linguagem, em que workers de GPU processam requisições compatíveis em batches dinâmicos num pool limitado de GPUs. 271
Q106 Projete um serviço que aceite requisições de inferência online de baixa latência, agrupe requisições compatíveis em batches, roteie o trabalho para workers de GPU e suporte vários modelos e versões ao mesmo tempo. 274
Q107 Projete um sistema de upload de arquivos para um app de chat com IA que lide com uploads multimodais grandes e os coloque no contexto do modelo. 277

APÊNDICE A **Números e fórmulas de referência** 283
A.1 Hardware 283
A.2 Modelos de exemplo 284
A.3 Fórmulas 284
A.4 Regras práticas 285

APÊNDICE B **Glossário** 286

APÊNDICE C **Plano de estudo de 30 dias** 290

APÊNDICE D **As 50 perguntas** 291

APÊNDICE E **Referências** 295
Fonte das perguntas 295
Arquitetura de modelos 295
Atenção e kernels 295
Serving 296
KV cache 296
Quantização e compressão 296
Paralelismo 296
Speculative decoding 297
Documentação 297

Introdução

Um modelo de linguagem treinado é um arquivo. Ele pode ter 16 GB, 140 GB ou quase um terabyte de números, e sozinho não responde a ninguém. Para virar o produto que milhões de pessoas usam, alguém precisa colocar esse arquivo em GPUs, receber requisições pela internet, organizar quem é atendido primeiro, devolver tokens rápido, não estourar a memória, não gastar uma fortuna e continuar funcionando às 2 da manhã de um domingo.

Esse alguém é a pessoa de **engenharia de inferência**.

O que é engenharia de inferência

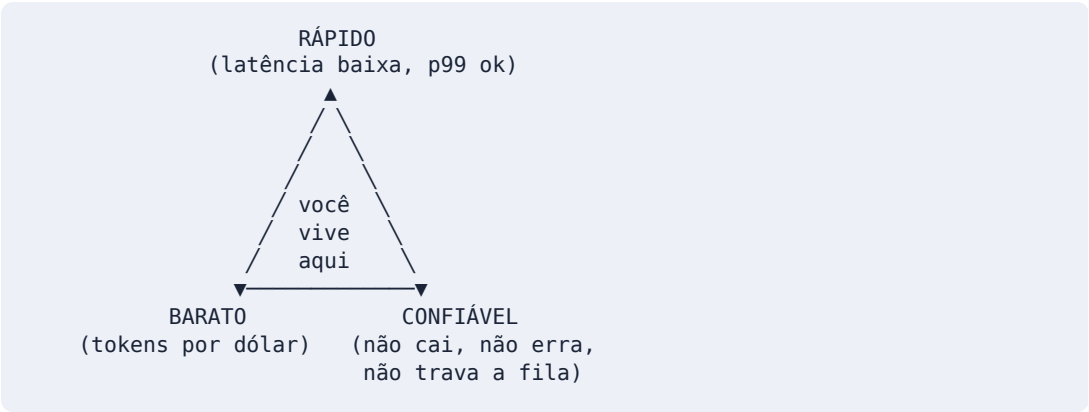
Em machine learning existem duas fases bem diferentes:

- **Treinamento**: o modelo aprende. Você mostra trilhões de tokens, calcula o erro, ajusta os pesos. Acontece uma vez (ou poucas vezes), dura semanas e usa milhares de GPUs ao mesmo tempo.
- **Inferência**: o modelo é usado. Alguém manda um prompt, o modelo gera uma resposta. Acontece bilhões de vezes, para sempre, enquanto o produto existir.

O custo do treinamento é grande, mas acontece poucas vezes. O custo da inferência se repete a cada requisição. Para um produto popular, o gasto acumulado com inferência pode passar o de treinamento. Por isso cada ponto percentual de eficiência em inferência vale muito dinheiro.

> **Analogia:** treinar um modelo é escrever uma receita depois de anos de testes. Fazer inferência é tocar o restaurante que serve essa receita para milhares de clientes por hora. A receita pode ser genial, mas se a cozinha for desorganizada, o prato chega frio, atrasado e caro. A engenharia de inferência cuida da cozinha.

O trabalho tem três objetivos que brigam entre si:



Deixar o serviço mais rápido quase sempre custa mais caro: você coloca menos usuários por GPU para cada um ser atendido mais depressa. Deixar mais barato quase sempre deixa mais lento: você enche a GPU de usuários e cada um espera mais. Deixar mais confiável custa capacidade reservada. A profissão inteira é escolher bem onde ficar nesse triângulo e mover o triângulo inteiro para melhor com técnicas novas.

O dia a dia da função

O post que originou a lista de perguntas deste livro resume bem: quem faz inferência "lê kernels de atenção, dimensiona frotas de GPUs, ajusta schedulers e fica de plantão quando a latência p99 dobra às 2 da manhã". Em termos concretos, uma semana típica pode incluir:

- **Kernel**: descobrir por que a atenção está usando só 40% da banda de memória da GPU e reescrever um pedaço em CUDA ou Triton.
- **Runtime**: ajustar o scheduler do motor de inferência (vLLM, TensorRT-LLM, SGLang ou um motor próprio) para misturar melhor prompts longos e curtos.
- **Modelo**: quantizar um modelo novo para FP8, medir a perda de qualidade e decidir se vale colocar em produção.
- **Sistema**: projetar como 2.000 GPUs recebem os pesos de um modelo novo de 400 GB sem derrubar a rede.
- **Capacidade**: calcular quantas GPUs são necessárias para o lançamento da semana que vem.
- **Operação**: investigar um incidente em que uma GPU com defeito deixava todas as requisições daquele nó lentas.

Não existe uma pessoa que faça tudo isso igualmente bem. As vagas costumam pender para um de três perfis:

Perfil	Foco	Linguagens e ferramentas
Kernel / performance	fazer cada operação rodar perto do limite do hardware	CUDA, C++, Triton, Nsight, PTX
Runtime / serving	motor de inferência, batching, KV cache, scheduler	Python, C++, PyTorch, vLLM, TensorRT-LLM
Infra / plataforma	frota de GPUs, Kubernetes, rede, deploy, confiabilidade	Go, Python, Kubernetes, NCCL, observabilidade

Este livro cobre os três, porque as entrevistas cobrem os três. Uma vaga de kernel ainda pergunta sobre batching, e uma vaga de infra ainda pergunta por que a inferência é limitada por memória.

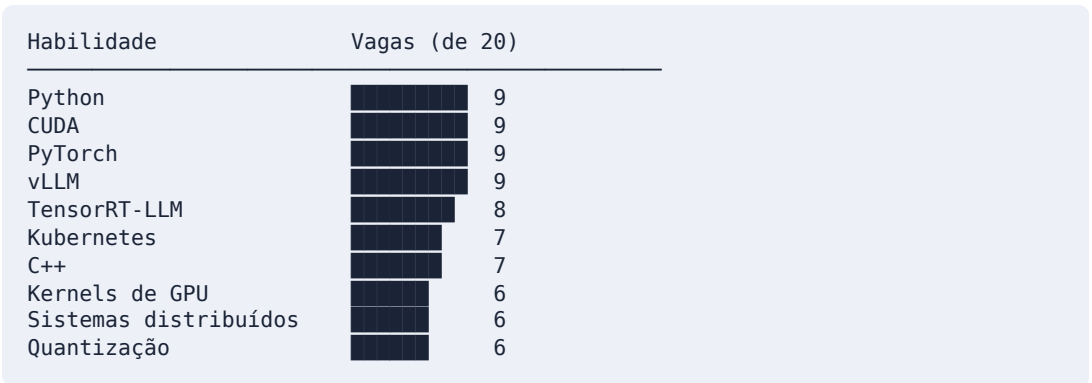
Carreira e salários

Engenharia de inferência é uma das especialidades mais bem pagas em IA hoje. Nas 20 vagas analisadas no levantamento que deu origem às perguntas, as faixas salariais base publicadas iam de cerca de **US$ 129 mil a US$ 685 mil por ano**. A xAI publicou uma faixa única, de US$ 180 mil a US$ 440 mil, para todos os níveis. Esses números são salário base nos Estados Unidos; ações e bônus vêm por cima e muitas vezes são a maior parte da remuneração.

O motivo é simples: uma melhoria de 20% no throughput de uma frota de dezenas de milhares de GPUs economiza dezenas de milhões de dólares por ano. Poucas pessoas sabem fazer isso, e todas as empresas de IA precisam.

O que as vagas pedem

O mesmo levantamento leu 20 vagas de 16 empresas. As habilidades que mais apareceram:



Três padrões apareceram com força:

1. **O KV cache é o recurso escasso.** Os entrevistadores voltam a ele o tempo todo: qual o tamanho, como paginar, como compartilhar, o que fazer quando acaba. Se você só puder estudar um assunto a fundo, estude este. O Capítulo 6 é inteiro sobre ele.
2. **Designs de batching dominam os processos da Anthropic.** Foram pelo menos oito enunciados relatados sobre batching, APIs de inferência em GPU ou distribuição de pesos. Os Capítulos 7, 11 e 14 cobrem esses designs.
3. **Vagas de nível Staff listam resultados, não ferramentas.** "Aumentar o throughput por GPU." "Ser dono do runtime sobre o qual todo acelerador é construído." Nesses níveis, prepare histórias de impacto com números, não listas de palavras-chave.

De onde vêm as perguntas

As 50 perguntas deste livro vêm do post *"You should never walk into an Inference Engineering interview without knowing these 50 questions"*, publicado pelo AI Engineering Insider. Os autores leram 20 vagas e reuniram relatos de entrevista do Glassdoor, PracHub, Taro, Aced (antigo Exponent), 1Point3Acres e LeetCode. O post publicou 50 das 112 perguntas do guia deles, cobrindo 14 categorias.

As respostas, explicações, diagramas e códigos deste livro foram escritos de forma independente e não reproduzem o guia pago do AI Engineering Insider. A lista de perguntas, as marcações e os links para as fontes foram mantidos como no post original, traduzidos para o português.

Como ler as marcações

Cada pergunta tem uma marcação que diz quanta evidência existe de que ela realmente caiu numa entrevista:

- **[Relatado]**: um relato de candidato cita a empresa (Glassdoor, PracHub, Taro, Aced, 1Point3Acres, LeetCode). A redação costuma ser parafraseada. Trate o **tema** como o sinal, não a frase exata.
- **[Fornecedor]**: um fornecedor de preparação ou banco público de perguntas atribui a pergunta a uma empresa, mas o relato original não pôde ser verificado.
- **[Prática]**: pergunta padrão do setor, escrita a partir das vagas. Nenhuma empresa é atribuída.

Depois da marcação vêm empresa, cargo, etapa do processo, ano, dificuldade e o link da fonte.

Como usar este livro

Cada capítulo segue a mesma estrutura:

1. **Por que importa**: o que o entrevistador está testando com o tema.
2. **Teoria do zero ao avançado**: analogia primeiro, depois números reais, depois detalhe técnico, diagramas e código.
3. **Perguntas de entrevista comentadas**: cada pergunta da lista com quatro partes:
   - *Resposta em 30 segundos*: o que você diz se tiver meio minuto.
   - *Resposta completa*: a versão que mostra profundidade.
   - *Perguntas de acompanhamento prováveis*: o que o entrevistador pergunta em seguida, com resposta curta.
   - *Armadilhas comuns*: os erros que derrubam candidatos.
4. **Resumo e checklist**: o que você precisa conseguir explicar sem consultar nada.

Você vai encontrar caixas de destaque ao longo do texto:

> **Analogia:** comparações do dia a dia para fixar a intuição.

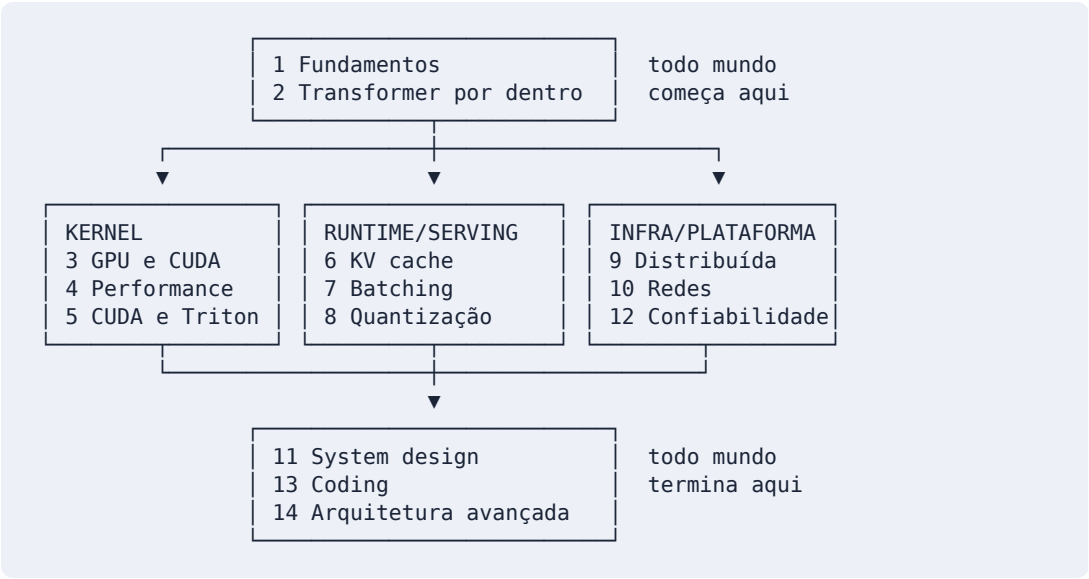
> **Fazendo a conta:** contas passo a passo com números de hardware e modelos reais. Refaça essas contas com lápis. Em entrevista de inferência, saber fazer conta de guardanapo é metade do jogo.

> **Dica de entrevista:** como apresentar o assunto para causar a melhor impressão.

> **Cuidado:** erros conceituais frequentes.

Ordem de leitura

Os capítulos foram escritos para serem lidos em ordem, mas você pode pular conforme a vaga:



Os apêndices trazem os números de referência que você deve saber de cabeça, um glossário, um plano de estudo de 30 dias, a lista das 50 perguntas com links e as referências (papers e documentação) para ir mais fundo.

Os números que vamos usar o tempo todo

Para as contas não ficarem abstratas, o livro usa sempre o mesmo hardware e os mesmos modelos de exemplo:

Item	Valor de referência
GPU H100 SXM	80 GB de HBM3, ~3,35 TB/s, ~989 TFLOPS BF16 denso
GPU A100 80GB SXM	80 GB de HBM2e, ~2,0 TB/s, ~312 TFLOPS BF16 denso
Llama 3 8B	32 camadas, d_model 4096, 32 heads de query, 8 heads de KV
Llama 3 70B	80 camadas, d_model 8192, 64 heads de query, 8 heads de KV

Esses valores são aproximados e mudam de geração para geração (B200, MI300X, TPUs). O raciocínio não muda. Em entrevista, o que conta é você saber de onde vem cada número e fazer a conta na frente do entrevistador.

> **Dica de entrevista:** quando der um número, diga a ordem de grandeza e a origem: “uma H100 tem uns 3 TB/s de banda; um modelo de 8B em BF16 tem 16 GB; então cada passo de decode leva no mínimo uns 5 ms”. Uma conta aproximada que você sabe defender vale mais que um número exato decorado.

Vamos começar pela pergunta que está por trás de todas as outras: por que gerar texto com um LLM é tão diferente de qualquer outro programa que roda numa GPU?

CAPÍTULO 1

Fundamentos de Engenharia de Inferência

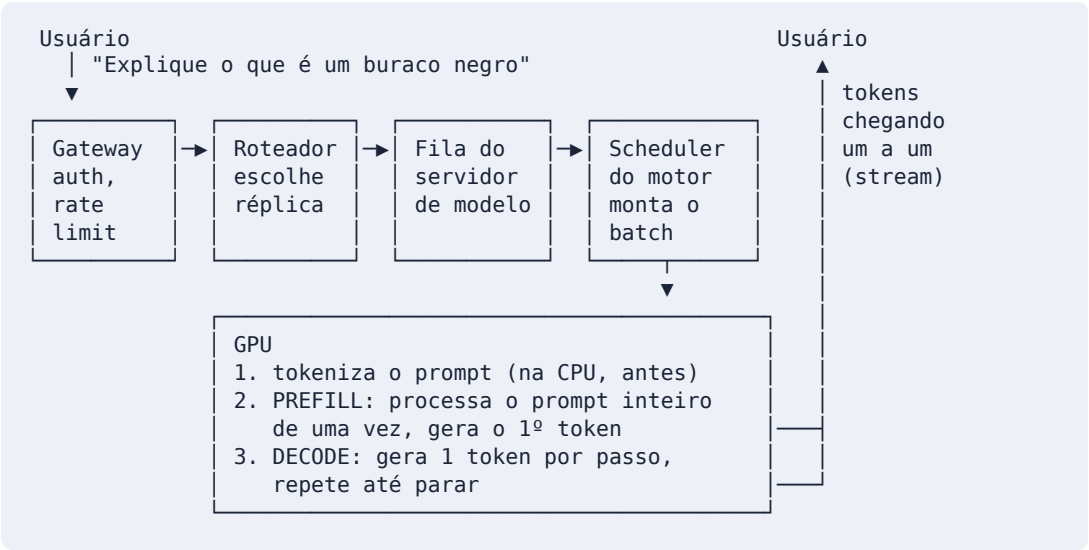
Este capítulo é a base de todo o resto do livro. Se você entender bem três ideias, metade das perguntas de entrevista fica fácil:

1. Gerar texto é um processo **sequencial**: um token de cada vez, e cada token depende do anterior.
2. Para gerar cada token, a GPU precisa **ler todos os pesos do modelo** da memória. Ler é muito mais lento que calcular.
3. A saída para esse gargalo é **atender muitos usuários ao mesmo tempo** (batching), e isso cria todos os trade-offs entre latência, throughput e custo.

O entrevistador que pergunta "por que a inferência é memory-bound?" ou "o que são TTFT e TPOT?" está testando se você tem esse modelo mental. Se tiver, você consegue raciocinar sobre qualquer sistema novo. Se não tiver, você só repete nomes de ferramentas.

1.1 A vida de uma requisição

Antes de entrar na GPU, vale ver o caminho completo de uma requisição num serviço de LLM:



Duas fases aparecem dentro da GPU, e elas têm comportamentos opostos:

- **Prefill** (pré-preenchimento): o modelo lê o prompt inteiro de uma vez. Se o prompt tem 2.000 tokens, a GPU processa os 2.000 em paralelo numa única passada. Isso é muito trabalho de cálculo e a GPU fica bem ocupada.
- **Decode** (decodificação): o modelo gera a resposta, um token por passada. Para gerar o token 501, ele precisa do token 500. Não dá para paralelizar dentro de uma mesma resposta.

O Capítulo 2 mostra o que acontece dentro do modelo em cada fase. Aqui o importante é a consequência: o prefill é limitado por **cálculo** e o decode é limitado por **memória**.

1.2 Ler é mais caro que calcular

Uma GPU tem dois recursos principais:

- **Capacidade de cálculo**, medida em FLOPS (operações de ponto flutuante por segundo). Uma H100 faz cerca de **989 trilhões** de operações BF16 por segundo nas unidades de matriz (tensor cores).
- **Banda de memória**, medida em bytes por segundo. A memória principal da H100 (HBM3, 80 GB) entrega cerca de **3,35 trilhões de bytes** por segundo.

Divida um pelo outro:

$$\frac{989 \times 10^{12}\ \text{FLOP/s}}{3,35 \times 10^{12}\ \text{B/s}} \approx 295\ \text{FLOP por byte}$$

Isso quer dizer: para a H100 ficar ocupada calculando, cada byte que ela lê da memória precisa ser usado em umas 295 operações. Se você faz menos que isso por byte, as unidades de cálculo ficam esperando os dados chegarem. Esse número tem nome: é o **ponto de equilíbrio** (ridge point) da GPU, e a métrica "operações por byte lido" se chama **intensidade aritmética**.

> **Analogia:** imagine um cozinheiro extremamente rápido (o cálculo) que depende de um ajudante buscando ingredientes na despensa (a memória). Se cada ingrediente que chega vira um prato complicado, com 300 passos, o cozinheiro fica sempre ocupado. Se cada ingrediente vira só um passo ("coloca o sal"), o cozinheiro passa o dia parado esperando o ajudante. O decode de um LLM é o segundo caso.

Quanto o decode faz por byte

Num passo de decode para **um** usuário, o modelo multiplica um vetor (o estado do token atual) por cada matriz de pesos. Cada peso é lido da memória uma vez e participa de uma multiplicação e uma soma: **2 operações**. Em BF16, cada peso ocupa **2 bytes**. Então:

$$\text{intensidade} = \frac{2 \text{ FLOP}}{2 \text{ bytes}} = 1 \text{ FLOP por byte}$$

A GPU aguenta 295 e você está dando 1. As unidades de cálculo ficam ociosas mais de 99% do tempo. Isso é o que significa dizer que o decode é **memory-bound** (limitado pela banda de memória).

> **Fazendo a conta:** Llama 3 8B em BF16 tem cerca de 8 bilhões de parâmetros × 2 bytes = **16 GB** de pesos.
>
> - Tempo para ler 16 GB a 3,35 TB/s: 16 / 3.350 = **~4,8 ms** por passo de decode.
> - Operações por token: ~2 × 8 bilhões = **16 GFLOP**. A 989 TFLOPS, isso leva **~0,016 ms**.
> - O passo é limitado pelos 4,8 ms de leitura, não pelos 0,016 ms de cálculo. A leitura é **300 vezes** mais lenta.
> - Velocidade máxima teórica para um usuário sozinho: 1 / 4,8 ms ≈ **210 tokens por segundo**. Na prática, com overheads, fica bem abaixo disso.

O batching resolve (em parte)

Agora imagine 64 usuários gerando ao mesmo tempo. Em vez de multiplicar a matriz por 1 vetor, você multiplica por 64 vetores empilhados (uma matriz de 64 linhas). Os pesos são lidos **uma vez** e usados 64 vezes. A intensidade aritmética sobe para cerca de 64 FLOP por byte.

O tempo do passo quase não muda, porque continua dominado pela leitura dos 16 GB. Mas agora cada passo gera 64 tokens em vez de 1. O throughput multiplica por quase 64, de graça.

```
Tempo por passo de decode (Llama 3 8B, H100, só pesos)

 batch    leitura    cálculo    passo     tokens/s total
 ─────    ───────    ───────    ─────     ──────────────
     1    4,8 ms     0,02 ms    4,8 ms           ~210
    16    4,8 ms     0,26 ms    4,8 ms         ~3.300
    64    4,8 ms     1,0 ms     4,8 ms        ~13.000
   256    4,8 ms     4,1 ms     ~5 ms         ~50.000
   512    4,8 ms     8,3 ms     ~8 ms         ~60.000
          (a partir daqui, o cálculo vira o gargalo)
```

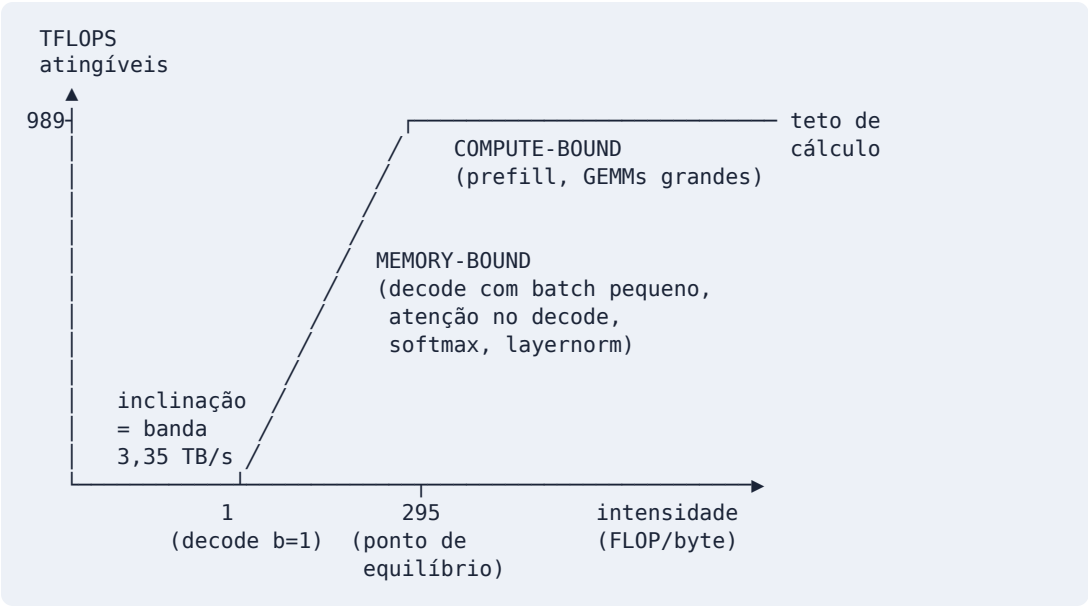
Esses números são o limite ideal, considerando só os pesos. Na vida real, dois fatores atrapalham:

1. **O KV cache**. Cada usuário tem um cache de atenção que também precisa ser lido em todo passo, e ele **não é compartilhado** entre usuários. Com 64 usuários de 4.000 tokens de contexto, o KV cache do 8B tem uns 33 GB (a conta está no Capítulo 6), mais que os próprios pesos. Batching não amortiza essa leitura.
2. **A memória acaba**. Os 80 GB da GPU precisam caber os pesos e os KV caches de todos os usuários. É isso que limita o tamanho do batch na prática.

Por isso o KV cache é o assunto mais cobrado em entrevistas: ele é o que impede o batching de resolver tudo.

1.3 O modelo roofline

O **roofline** (telhado) é um gráfico que resume tudo isso. No eixo X fica a intensidade aritmética de um kernel. No eixo Y, a performance que ele consegue atingir.



A regra:

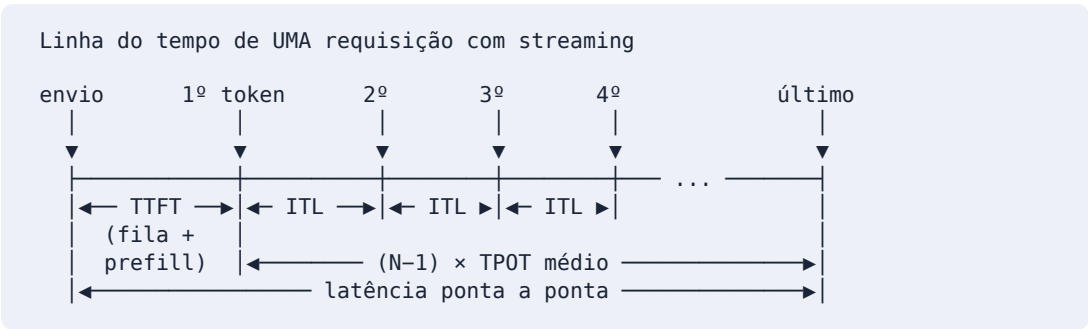
$$\text{performance} = \min(\text{pico de FLOPS},\ \text{banda} \times \text{intensidade})$$

À esquerda do ponto de equilíbrio, você está limitado pela memória: a única forma de ficar mais rápido é ler menos bytes (quantizar, fundir kernels, reaproveitar dados) ou aumentar a intensidade (batching). À direita, você está limitado pelo cálculo: precisa usar melhor os tensor cores ou fazer menos operações.

O Capítulo 4 usa o roofline a fundo para diagnosticar kernels. Por enquanto, guarde a ideia: **antes de otimizar qualquer coisa, descubra de que lado do telhado você está**.

1.4 As métricas que importam

Um serviço de LLM é avaliado por métricas de **latência** (quanto um usuário espera) e de **throughput** (quanto trabalho o sistema faz). Todo entrevistador espera que você saiba defini-las com precisão.



Métrica	O que é	Dominada por	Quem sente
TTFT (time to first token)	tempo do envio até o primeiro token	fila + prefill	usuário de chat: "travou?"
ITL (inter-token latency)	tempo entre dois tokens seguidos	um passo de decode	fluidez do texto aparecendo
TPOT (time per output token)	média de tempo por token depois do primeiro	decode	velocidade de leitura
Latência E2E	do envio ao último token	TTFT + (N−1) × TPOT	API sem streaming, agentes
Throughput	tokens (ou requisições) por segundo do sistema	tamanho do batch, eficiência	quem paga a conta
Goodput	throughput que **cumpre o SLO**	tudo junto	o negócio

A fórmula que liga as três primeiras:

$$\text{latência E2E} = \text{TTFT} + (N - 1) \times \text{TPOT}$$

onde N é o número de tokens gerados.

> **Fazendo a conta:** um chat com TTFT de 400 ms e TPOT de 25 ms gera uma resposta de 500 tokens em 0,4 + 499 × 0,025 ≈ **12,9 segundos**. O usuário vê o primeiro token em 0,4 s e lê a uma velocidade de 40 tokens por segundo, mais rápido do que uma pessoa lê (umas 4 a 5 palavras por segundo, ou seja, uns 6 a 8 tokens por segundo). Para chat, um TPOT abaixo de ~50 ms já parece fluido. Para um agente que encadeia 20 chamadas, os 12,9 s viram mais de 4 minutos, e aí cada milissegundo de TPOT importa.

Percentis: a média mente

Latência nunca deve ser reportada só pela média. Use **percentis**:

- **p50** (mediana): metade das requisições é mais rápida que isso.
- **p90, p99**: 90% ou 99% são mais rápidas. O resto é a **cauda** (tail).

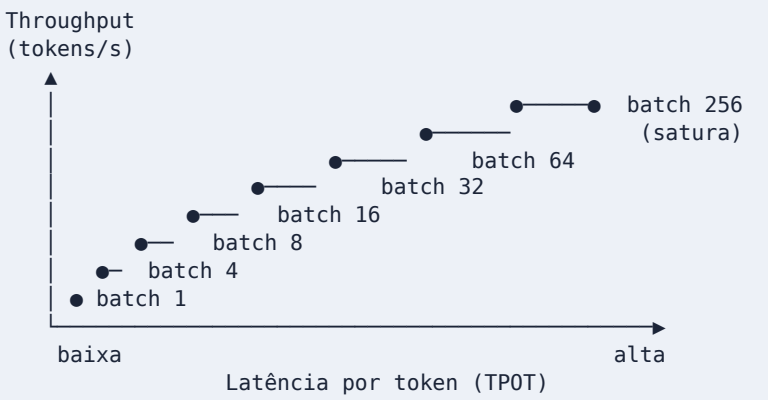
Num serviço com 10 milhões de requisições por dia, o p99 representa 100 mil requisições por dia. São pessoas reais tendo uma experiência ruim. Além disso, se uma tela depende de 10 chamadas em paralelo, a chance de pelo menos uma cair no p99 é $1 - 0,99^{10} \approx 9,6\%$. A cauda de um componente vira a experiência comum do sistema.

Goodput: throughput que serve

Um sistema que gera 50 mil tokens por segundo, mas com TTFT de 30 segundos, não serve para chat. **Goodput** é o número de requisições por segundo que o sistema atende **cumprindo o SLO** (por exemplo, TTFT p99 < 1 s e TPOT p99 < 50 ms). É a métrica certa para comparar configurações. O paper do DistServe (Zhong et al., 2024) popularizou o termo para inferência de LLM.

O trade-off fundamental

Aumentar o batch aumenta o throughput, mas cada passo de decode fica mais lento (mais KV cache para ler, mais cálculo), e novas requisições esperam mais para entrar. A curva típica:



Escolher o ponto de operação = escolher o maior throughput que ainda cumpre o SLO de latência.

Toda otimização boa de inferência faz uma de duas coisas: **move a curva inteira** para cima e para a esquerda (mais throughput com a mesma latência), ou permite **operar num ponto melhor** da curva com segurança.

1.5 Custo por token

No fim, o negócio olha uma métrica: **quanto custa gerar um milhão de tokens**.

$$\text{custo por 1M tokens} = \frac{\text{custo da GPU por hora}}{\text{tokens por hora}} \times 10^6$$

> **Fazendo a conta (valores hipotéticos):** suponha uma H100 a US$ 2,50 por hora e um serviço que sustenta 2.500 tokens de saída por segundo nessa GPU, cumprindo o SLO.
>
> - Tokens por hora: 2.500 × 3.600 = 9.000.000.
> - Custo por 1M tokens: 2,50 / 9 = **US$ 0,28**.
>
> Se uma otimização aumenta o throughput para 3.500 tokens/s com o mesmo SLO, o custo cai para US$ 0,20. Numa frota de 10.000 GPUs rodando 24 horas por dia, a mesma melhoria de 40% equivale a economizar umas 2.900 GPUs, ou cerca de US$ 63 milhões por ano ao preço hipotético acima.

Duas observações que impressionam entrevistadores:

1. **Tokens de entrada e de saída têm custos diferentes.** Prefill processa muitos tokens por passada e usa bem a GPU. Decode gera um token por passada por usuário. Por isso as APIs comerciais costumam cobrar mais caro o token de saída do que o de entrada.
2. **Utilização não é eficiência.** O `nvidia-smi` pode mostrar "100% GPU utilization" com a GPU fazendo pouco trabalho útil. Essa métrica só diz que algum kernel estava rodando naquele intervalo, não quanto do hardware ele usava. Meça tokens por segundo por GPU e MFU/MBU (veja o Capítulo 4).

1.6 Inferência síncrona, assíncrona e em lote

Nem toda inferência precisa responder na hora. Existem três modos principais:

```
SÍNCRONA (online)
cliente ──req──▶ servidor ──▶ GPU
cliente ◀──resp (ou stream)── servidor
o cliente fica esperando com a conexão aberta

ASSÍNCRONA (job)
cliente ──req──▶ API ──▶ fila ──▶ worker ──▶ GPU
cliente ◀── "job 123 aceito" (na hora)
... depois ...
cliente ──GET /jobs/123──▶ API     (polling)
  ou   API ──POST webhook──▶ cliente (callback)

BATCH (offline)
cliente envia arquivo com 1 milhão de prompts
sistema processa quando há GPU sobrando (horas)
cliente baixa arquivo com resultados
```

	Síncrona	Assíncrona	Batch offline
Latência esperada	ms a segundos	segundos a minutos	horas
Exemplo	chat, autocomplete	geração de relatório, vídeo	rotular dataset, avaliações
SLO principal	TTFT, TPOT	tempo de conclusão	custo, prazo (ex.: 24 h)
Uso da GPU	precisa de folga para picos	fila suaviza picos	enche os vales de uso
Falhas	cliente vê erro, tenta de novo	sistema faz retry sozinho	retry por item

A grande vantagem dos modos assíncronos e em lote é **econômica**: o tráfego síncrono tem picos (horário comercial) e vales (madrugada). Você precisa de GPUs para o pico, e elas ficam ociosas no vale. Trabalho assíncrono e em lote preenche esses vales com prioridade baixa, e por isso pode ser vendido mais barato.

> **Dica de entrevista:** quando falar de síncrono vs assíncrono, fale de **prioridades e preempção**. Um bom sistema mistura os dois na mesma frota: requisições síncronas têm prioridade, e as de batch usam a capacidade que sobra e podem ser pausadas ou interrompidas quando o tráfego síncrono sobe.

1.7 O mapa de otimizações

Para fechar a teoria, um mapa das principais alavancas, com o capítulo que aprofunda cada uma:

Alavanca	O que ataca	Ganho típico	Capítulo
Continuous batching	GPU ociosa entre requisições	grande em throughput	7
PagedAttention	desperdício de memória do KV cache	batch maior	6
Prefix caching	prefill repetido (system prompt, chat)	TTFT e custo	6
Quantização (FP8, INT4)	bytes lidos por passo	~1,5–3× no decode	8
FlashAttention	tráfego de memória da atenção	atenção bem mais rápida	2
Fusão de kernels, CUDA graphs	overhead de lançamento e leituras extras	10–30% em decode	4
Tensor/pipeline parallel	modelo não cabe, latência	permite servir	9
Speculative decoding	decode sequencial	~2–3× em latência	14
Prefill/decode separados	interferência entre fases	goodput	9

Os ganhos variam muito por modelo, hardware e carga. Na entrevista, fale de faixas e do mecanismo, nunca de um número mágico.

Perguntas de entrevista comentadas

Q001 Qual é a sua experiência com otimização de inferência para LLMs?

> **[Relatado]** Groq · Research Engineer · Londres · relatado em 2025 · Médio · fonte

Resposta em 30 segundos

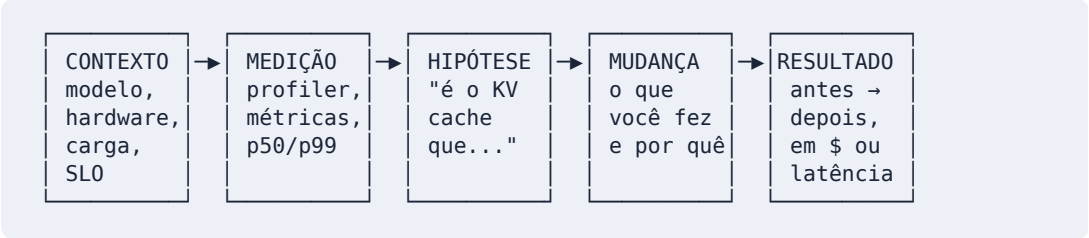
“Trabalhei principalmente em [área]. O caso mais relevante foi [projeto]: o serviço tinha [problema medido], eu descobri com [ferramenta/medição] que o gargalo era [causa], fiz [mudança] e o resultado foi [número antes → depois] em [métrica que o negócio usa]. Também tenho experiência com [2 ou 3 técnicas], e hoje estou estudando [tema atual].”

Resposta completa

Essa pergunta parece um bate-papo, mas é uma triagem técnica disfarçada. O entrevistador quer saber três coisas:

1. **Você mediu antes de mexer?** Quem otimiza sem medir chuta.
2. **Você entende o porquê?** "Usei vLLM e ficou mais rápido" não mostra entendimento. "O decode estava limitado por memória e o batch médio era 6 porque o KV cache fragmentava; o PagedAttention elevou o batch para 40" mostra.
3. **Você conhece o espaço de técnicas?** Quantização, batching, KV cache, kernels, paralelismo, speculative decoding.

Use a estrutura **contexto → medição → hipótese → mudança → resultado → aprendizado**:



Exemplo de resposta modelo (adapte à sua experiência real; nunca invente números numa entrevista):

> "No meu último trabalho eu cuidava do serviço de um modelo de 13B para um assistente interno, em A100s. O TTFT p99 passava de 6 segundos no horário de pico, e o SLO era 2 segundos. Medindo, vi que o problema não era o prefill em si: requisições ficavam até 4 segundos na fila, porque usávamos batching estático e uma requisição com prompt de 8 mil tokens travava o batch inteiro. Migrei para um motor com continuous batching e chunked prefill, com chunks de 512 tokens. O TTFT p99 caiu para 1,3 segundo e o throughput subiu 2,4 vezes, o que nos deixou desligar 5 das 12 GPUs. Depois testei quantização FP8 dos pesos; a perda na nossa avaliação interna ficou dentro da margem de ruído, e ganhamos mais uns 30% de throughput. O que aprendi é que o primeiro gargalo quase sempre está no scheduling, não no kernel."

Se você ainda não tem experiência profissional com isso, seja honesto e mostre projeto próprio com medições: "Ainda não fiz isso em produção. No meu projeto pessoal servi um Llama 3 8B numa RTX 4090, medi TTFT e TPOT com batch 1 a 64, comparei

FP16 com AWQ INT4 e entendi por que o ganho no decode foi maior do que no prefill." Isso vale muito mais do que fingir.

Perguntas de acompanhamento prováveis

- **Como você mediu?** Cite ferramentas e métricas concretas: benchmark de carga com distribuição real de tamanhos de prompt, TTFT/TPOT p50 e p99, `torch.profiler`, Nsight Systems, métricas do motor (tamanho do batch, uso do KV cache, preempções).
- **Qual foi o maior trade-off?** Por exemplo: chunked prefill melhorou o TTFT, mas piorou um pouco o TPOT dos usuários em decode. Mostre que você notou e decidiu de propósito.
- **O que você faria diferente?** Mostre maturidade: "teria montado o benchmark de carga reproduzível antes de mexer em qualquer coisa".
- **Como garantiu que a qualidade não caiu?** Avaliação antes e depois (benchmarks da tarefa, comparação lado a lado, taxa de erro em produção), principalmente para quantização.

Armadilhas comuns

- Listar ferramentas ("usei vLLM, TensorRT, Triton...") sem nenhum número ou causa.
- Dar números sem dizer **o que** foi medido (throughput de quê? com qual batch? qual percentil?).
- Atribuir a si o trabalho do time inteiro. Diga "eu" para o que você fez e "nós" para o resto.
- Não mencionar a qualidade do modelo depois de otimizações que podem alterá-la.

Q002 Por que a inferência de LLMs é limitada por memória (memory-bound)?

> **[Fornecedor]** Compilado de relatos de candidatos, sem empresa citada · Triagem · 2026 · Fácil · fonte

Resposta em 30 segundos

"Porque no decode cada token novo exige ler todos os pesos do modelo e todo o KV cache da memória, mas faz só umas 2 operações por peso lido. Isso dá uma intensidade aritmética perto de 1 FLOP por byte com batch 1, enquanto uma H100 precisa

de uns 300 para ficar limitada por cálculo. A GPU passa a maior parte do tempo esperando a memória. O batching sobe a intensidade para os pesos, mas o KV cache é por usuário e não se amortiza, e a memória limita o tamanho do batch."

Resposta completa

Comece pela distinção entre as fases, porque a pergunta só é verdadeira para o decode:

- **Prefill** processa T tokens de uma vez. Cada peso lido é usado T vezes. Com prompts de algumas centenas de tokens, a intensidade passa do ponto de equilíbrio e o prefill é **compute-bound**.
- **Decode** processa 1 token por usuário por passo. Cada peso lido é usado uma vez por usuário do batch.

Depois mostre a conta (seção 1.2): 16 GB de pesos do 8B lidos a 3,35 TB/s levam ~4,8 ms; o cálculo leva ~0,016 ms. A leitura domina.

Em seguida, explique por que o batching não resolve tudo:

```
Bytes lidos por passo de decode
─────────────────────────────────────────────────────────
pesos          W bytes        (lidos 1× para o batch todo)
KV cache       B × L × k      (cada usuário tem o seu)
               B = batch, L = contexto, k = bytes/token
─────────────────────────────────────────────────────────
FLOPs úteis    ≈ 2 × P × B    (P = parâmetros)

intensidade ≈ 2PB / (W + B·L·k)
  - com contexto curto: cresce com B (bom)
  - com contexto longo: o termo do KV domina e a
    intensidade empaca, por maior que seja o batch
```

A atenção do decode é intrinsecamente memory-bound: cada usuário lê o próprio KV cache e faz poucas operações com ele (um produto escalar e uma soma ponderada por token armazenado). Não há reaproveitamento entre usuários.

Por último, liste as consequências práticas, porque é aí que você mostra senioridade:

1. **Quantizar ajuda o decode mais do que o prefill.** Pesos em INT4 leem 4× menos bytes que BF16. Num regime memory-bound, isso vira quase direto velocidade.
2. **Tamanho do batch é limitado por memória, não por cálculo.** Por isso técnicas de memória (PagedAttention, GQA, quantização do KV cache) aumentam o throughput.

3. **A banda de memória é o número mais importante da GPU para decode.** Por isso a geração de GPUs com mais HBM e mais banda (H200, B200) dá ganhos grandes em inferência mesmo quando o FLOPS cresce menos.
4. **Speculative decoding funciona** porque usa o cálculo ocioso: verificar vários tokens de uma vez custa quase o mesmo que gerar um.

Perguntas de acompanhamento prováveis

- **O prefill também é memory-bound?** Em geral não. Com prompts de centenas de tokens ou mais, as multiplicações de matriz têm intensidade alta e ficam compute-bound. Prompts muito curtos (poucas dezenas de tokens) podem continuar memory-bound.
- **A partir de qual batch o decode vira compute-bound?** Olhando só os pesos, perto do ponto de equilíbrio: algumas centenas na H100 em BF16. Com FP8 nos pesos e cálculo em FP8, o ponto muda (o pico de FP8 é ~2× maior, e os bytes caem pela metade). Na prática, a memória do KV cache costuma limitar o batch antes disso.
- **Como você confirmaria isso com um profiler?** No Nsight Compute, kernels de GEMV/GEMM do decode mostram throughput de memória (DRAM) perto do pico e throughput de SM/tensor core baixo. A métrica MBU (model bandwidth utilization) perto de 70–90% indica que você está bem otimizado para memória.
- **E MoE?** Num modelo Mixture of Experts, cada token usa só alguns especialistas. Com batch pequeno, você lê poucos especialistas; com batch grande, quase todos são ativados e os bytes lidos crescem. A intensidade por especialista é menor do que num modelo denso de mesmo tamanho ativo, o que torna o MoE ainda mais memory-bound no decode.

Armadilhas comuns

- Dizer “porque o modelo é grande e não cabe na GPU”. Isso é capacidade de memória, não banda. São problemas diferentes.
- Esquecer o KV cache e dizer que batching resolve tudo.
- Não diferenciar prefill de decode.
- Confundir “memory-bound” com “falta de memória (OOM)”.

Q003 O que são TTFT, TPOT e throughput, e como um compete com o outro?

> **[Fornecedor]** Banco público de perguntas · Triagem · 2026 · Fácil · fonte

Resposta em 30 segundos

"TTFT é o tempo até o primeiro token: fila mais prefill. TPOT é o tempo médio por token gerado depois disso: um passo de decode. Throughput é quantos tokens ou requisições por segundo o sistema entrega. Eles competem porque batches maiores aumentam o throughput, mas deixam cada passo de decode mais lento (TPOT sobe) e, se você prioriza o decode de quem já está rodando, novos prompts esperam mais (TTFT sobe). Se você prioriza prefills novos, o TTFT melhora e quem está no decode sofre pausas. O objetivo é o maior throughput que ainda cumpre o SLO de TTFT e TPOT, ou seja, maximizar o goodput."

Resposta completa

Defina com precisão (seção 1.4) e depois mostre os três conflitos:

Conflito 1 — Throughput vs TPOT. Batch maior → mais KV cache lido por passo e mais cálculo → cada passo demora mais → cada usuário vê tokens mais devagar. Mas o sistema gera mais tokens no total.

Conflito 2 — TTFT vs TPOT. Quando chega uma requisição nova, o scheduler pode:

```
Opção A: prioriza prefill novo (TTFT bom)
passo:   | D | D |PREFILL 4000 tokens| D | D | D |
usuário em decode vê um "soluço" de várias dezenas de ms

Opção B: prioriza decode (TPOT estável)
passo:   | D | D | D | D | D | D | D |PREFILL|
a nova requisição espera → TTFT ruim

Opção C: chunked prefill (meio-termo)
passo:   |D+p|D+p|D+p|D+p| D | D |
cada passo junta os decodes + um pedaço (chunk) do prefill
```

O chunked prefill (SARATHI, Agrawal et al.) e a desagregação de prefill e decode em máquinas diferentes (DistServe, Splitwise, Mooncake) são as duas respostas modernas a esse conflito. Estão nos Capítulos 7 e 9.

Conflito 3 — Throughput vs custo de cauda. Rodar a GPU a 95% de ocupação maximiza throughput, mas qualquer pico vira fila, e a fila explode o TTFT p99. Teoria de filas: o tempo de espera cresce de forma não linear quando a utilização se aproxima de 100%. Em sistemas simples, a espera média é proporcional a $\rho/(1-\rho)$, onde ρ é a utilização. Com 50% de uso, o fator é 1; com 90%, é 9; com 95%, é 19.

Para ligar tudo, use a fórmula $\text{E2E} = \text{TTFT} + (N-1) \times \text{TPOT}$ e explique que **o peso relativo de cada métrica depende do produto**:

Produto	Métrica principal	Por quê
Chat	TTFT e TPOT	o usuário está olhando a tela
Autocomplete de código	TTFT (bem baixo)	saídas curtas, precisa parecer instantâneo
Agente com muitas chamadas	latência E2E	as chamadas são sequenciais
API de resumo em lote	throughput / custo	ninguém está esperando
Voz	TTFT	silêncio na conversa é muito perceptível

Perguntas de acompanhamento prováveis

- **ITL e TPOT são a mesma coisa?** Quase. ITL é cada intervalo individual entre tokens; TPOT costuma ser a média por requisição. Os percentis de ITL mostram os "soluços" que a média do TPOT esconde.
- **Como você mede TTFT corretamente?** Do ponto de vista do cliente, do envio até o primeiro byte do primeiro token no stream. Medir só dentro do servidor ignora fila no gateway, rede e tokenização.
- **Throughput em tokens ou em requisições?** Depende. Tokens por segundo é melhor para comparar eficiência de hardware; requisições por segundo com SLO (goodput) é melhor para planejar capacidade. Sempre separe tokens de entrada e de saída.
- **Como o tamanho do prompt afeta cada métrica?** Prompt longo aumenta o TTFT (mais prefill) e o TPOT (mais KV cache lido em cada passo de decode).

Armadilhas comuns

- Definir TPOT incluindo o primeiro token. TPOT é sobre os tokens seguintes.
- Falar de média em vez de percentis.
- Dizer que throughput e latência "sempre" se opõem. Otimizações como quantização ou kernels melhores melhoram os dois.
- Esquecer o tempo de fila no TTFT.

Q004 Explique passo a passo como um forward pass de um transformer é executado de fato numa GPU.

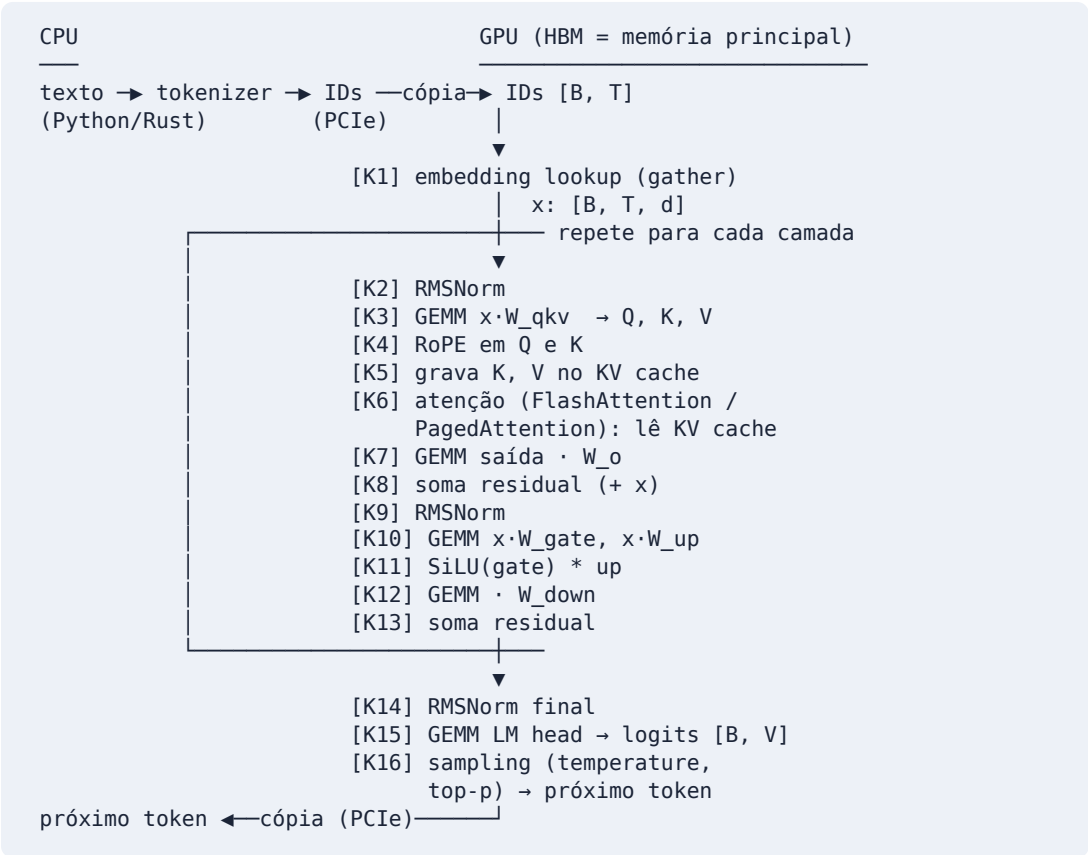
> **[Fornecedor]** Lista de perguntas comuns de inferência de um fornecedor de preparação · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"A CPU tokeniza o texto e copia os IDs para a GPU. A GPU faz um lookup na tabela de embeddings. Para cada camada, ela lança uma sequência de kernels: normalização, uma GEMM para Q, K e V, RoPE, o kernel de atenção que também lê e escreve o KV cache, a GEMM de projeção de saída, a soma residual, outra normalização, as GEMMs do MLP com a ativação no meio e outra soma residual. No fim, normalização final, a GEMM do LM head gera os logits e um kernel de sampling escolhe o próximo token. Pesos e ativações moram na HBM; cada kernel traz blocos para SRAM e registradores, calcula e escreve de volta. No decode, isso tudo é limitado pela leitura dos pesos, e o overhead de lançar centenas de kernels pesa; por isso usam-se fusão de kernels e CUDA graphs."

Resposta completa

Desenhe o fluxo para um modelo estilo Llama (o Capítulo 2 explica cada bloco em detalhe):



Agora detalhe o que acontece **dentro** de um kernel típico, porque é isso que separa uma resposta média de uma excelente:

1. **Lançamento.** A CPU chama a API do CUDA, que coloca o kernel numa fila (stream) da GPU. Cada lançamento custa alguns microssegundos de CPU. Um modelo de 80 camadas com ~13 kernels por camada lança mais de mil kernels por passo de decode. Se cada kernel leva poucos microssegundos de GPU, a CPU vira o gargalo.
2. **Grid de blocos.** O kernel é dividido em blocos de threads (por exemplo, cada bloco calcula um pedaço 128×128 da matriz de saída). O hardware distribui os blocos entre os **SMs** (streaming multiprocessors; a H100 tem 132).
3. **Movimento de dados.** Cada bloco carrega os pedaços de que precisa da **HBM** (grande e lenta) para a **shared memory** (pequena e rápida, dentro do SM) e para os **registradores**. Os **tensor cores** fazem as multiplicações de matriz nesses dados.
4. **Escrita.** O resultado vai de volta para a HBM, onde o próximo kernel vai ler.

```
HBM (80 GB, ~3,35 TB/s)
  │  pesos, ativações, KV cache
  ▼
L2 cache (50 MB, compartilhado por todos os SMs)
  ▼
SM: shared memory / L1 (até ~228 KB por SM)
  ▼
registradores (~256 KB por SM) ─► tensor cores / CUDA cores
```

Esse ciclo “lê da HBM, calcula, escreve na HBM” a cada kernel é a fonte de duas ineficiências:

- **Tráfego extra.** O resultado da RMSNorm é escrito na HBM só para a GEMM seguinte ler de novo. A **fusão de kernels** junta operações (por exemplo, RMSNorm + quantização, ou SiLU × up) para os dados intermediários não saírem do chip.
- **Overhead de lançamento.** Os **CUDA graphs** gravam a sequência inteira de kernels de um passo de decode uma vez e a reexecutam com um único lançamento. Em decode com batch pequeno, isso pode dar ganhos de dezenas de por cento.

Por fim, diga o que muda com mais de uma GPU: com tensor parallelism, cada GPU tem uma fatia das matrizes, e depois da projeção de saída da atenção e da GEMM `W_down` do MLP há um **all-reduce** (soma entre GPUs) pela NVLink. Isso dá dois all-reduces por camada (Capítulo 9).

> **Dica de entrevista:** depois de descrever o fluxo, diferencie prefill e decode no mesmo diagrama. “No prefill, x tem T linhas e as GEMMs são grandes e compute-bound. No decode, x tem uma linha por usuário, as GEMMs viram quase multiplicações matriz-vetor e tudo fica limitado pela leitura de pesos e do KV cache.” Isso mostra que você entende o desempenho, não só o fluxo.

Perguntas de acompanhamento prováveis

- **Quais desses kernels dominam o tempo?** Em prefill, as GEMMs (QKV, saída, MLP) e, com contextos longos, a atenção. Em decode com batch pequeno, as GEMMs (limitadas pela leitura dos pesos). Em decode com batch grande e contexto longo, a atenção (leitura do KV cache) cresce muito.
- **Onde fica o KV cache e quando ele é escrito?** Na HBM. K e V do token atual são calculados no kernel QKV e gravados no cache antes ou durante o kernel de atenção, que então lê o cache inteiro daquela sequência.
- **Por que o sampling pode ser um problema?** Ordenar um vocabulário de 128 mil entradas para top-p é caro se feito ingenuamente, e sincronizar com a CPU a cada token (para checar tokens de parada) adiciona latência. Motores modernos fazem o sampling na GPU e reduzem as sincronizações.
- **O que muda num modelo MoE?** O bloco MLP vira um roteador (gate) que escolhe os k especialistas por token, seguido de GEMMs agrupadas (grouped GEMM) por especialista e uma etapa para recombinar os resultados.

Armadilhas comuns

- Descrever só a matemática (“Q vezes K transposto...”) sem falar de kernels, memória e lançamento.
- Esquecer a tokenização na CPU e as cópias CPU↔GPU.
- Não mencionar onde o KV cache entra.
- Ignorar o overhead de lançamento de kernels no decode.

Q007 Como você aborda a otimização de custos de um sistema de IA que está estourando o orçamento?

> **[Fornecedor]** Banco público de perguntas · Comportamental-técnica · 2026 · Médio · fonte

Resposta em 30 segundos

“Primeiro eu meço para onde o dinheiro vai: custo por requisição e por funcionalidade, utilização real das GPUs, tokens de entrada e saída por chamada. Depois ataco em ordem de retorno sobre esforço: cortar desperdício (GPUs ociosas, chamadas duplicadas, prompts inchados, saídas longas demais), usar cache (prefix caching, cache de respostas), escolher o modelo certo para cada tarefa (roteamento para modelos menores), mover o que pode para batch assíncrono, e só então otimizações de serving: quantização, batching melhor, kernels. Toda mudança passa por avaliação de qualidade, e acompanho o custo por unidade de negócio, não só a conta total.”

Resposta completa

Mostre um processo. O entrevistador quer ver se você parte de dados e se pensa em produto, não só em GPU.

Passo 1 — Medir e decompor. Monte uma visão como esta:

```
Custo mensal: US$ 400k  (orçamento: US$ 250k)
├── Funcionalidade A (chat)              US$ 220k
│   ├── tokens de entrada                 70%  ◄ system prompt
│   │                                          de 3.000 tokens
│   │                                          em toda chamada!
│   └── tokens de saída                   30%
├── Funcionalidade B (resumo diário)     US$ 120k
│   └── roda síncrono às 9h, pico        ◄ podia ser batch
└── GPUs reservadas ociosas à noite      US$ 60k  ◄ desperdício
```

A métrica certa é **custo por unidade de valor**: custo por conversa, por documento resumido, por usuário ativo. Se a conta total sobe porque o uso triplicou, talvez o problema seja preço, não eficiência.

Passo 2 — Cortes baratos (dias).

- **Prompts**: system prompts enormes repetidos em toda chamada. Enxugar ou usar prefix caching (o prefixo comum é calculado uma vez e reaproveitado; Q043).
- **Saídas**: limitar `max_tokens`, pedir respostas mais concisas, formatos estruturados.
- **Chamadas duplicadas**: retries sem controle, a mesma pergunta processada várias vezes. Cache de respostas para entradas idênticas.
- **Ociosidade**: autoscaling, desligar réplicas fora do pico, juntar modelos pouco usados em menos GPUs.

Passo 3 — Mudanças de arquitetura (semanas).

- **Roteamento de modelos**: 70% das perguntas podem ser simples o bastante para um modelo menor. Um classificador barato (ou o próprio modelo pequeno com fallback) manda só as difíceis para o grande.
- **Assíncrono e batch**: tudo que não precisa de resposta imediata vai para filas de baixa prioridade que usam capacidade ociosa (Q008).
- **Distilação**: treinar um modelo menor especializado na tarefa (Capítulo 8).

Passo 4 — Eficiência de serving (semanas a meses).

- Quantização FP8/INT4, continuous batching, ajuste do tamanho do batch e do paralelismo, speculative decoding, GPUs mais adequadas (por exemplo, GPUs de inferência mais baratas para modelos pequenos).

Passo 5 — Governança. Painel de custo por equipe e funcionalidade, orçamento por time, alertas de anomalia. Sem isso, o custo volta a subir.

> **Dica de entrevista:** termine com o trade-off com qualidade: "Toda redução de custo que muda o modelo, o prompt ou a precisão passa por uma avaliação offline e um teste A/B. Economizar 40% e perder 5% de satisfação pode ser um mau negócio."

Perguntas de acompanhamento prováveis

- **Qual seria sua primeira ação na segunda-feira?** Instrumentar: custo por funcionalidade e por chamada, com contagem de tokens. Sem isso tudo é chute.
- **Como decidir entre um modelo maior quantizado e um menor em precisão total?** Avaliar os dois na tarefa real, com o mesmo orçamento de GPU. Em geral, modelos maiores quantizados para 8 bits perdem muito pouco; 4 bits depende do método e do modelo.
- **E se a empresa usa uma API externa (não hospeda o modelo)?** As alavancas mudam: prompt caching da API, batch API com desconto, escolha do modelo por tarefa, controle de tokens. Hospedar o próprio modelo só compensa acima de um volume em que as GPUs ficam bem utilizadas.
- **Como evitar que a otimização quebre algo?** Rollout gradual, métricas de qualidade e de erro por versão, possibilidade de rollback rápido.

Armadilhas comuns

- Pular direto para "vou escrever kernels" sem medir onde está o custo.
- Ignorar a qualidade.

- Otimizar o total em vez do custo por unidade de valor.
- Esquecer as GPUs ociosas: muitas vezes são o maior desperdício.

Q008 Inferência síncrona vs. assíncrona: quando usar cada uma?

> **[Fornecedor]** Banco público de perguntas · Triagem · 2026 · Fácil · fonte

Resposta em 30 segundos

"Síncrona quando há um humano ou sistema esperando a resposta naquele momento, com SLO de segundos: chat, autocomplete, voz. Assíncrona quando a resposta pode chegar depois: jobs longos, processamento de documentos, geração em lote, avaliações. A assíncrona usa fila, desacopla o pico de chegada da capacidade, permite retries e prioridade baixa, e usa GPU ociosa; o custo é complexidade (status, polling ou webhooks, idempotência) e latência maior. Muitos sistemas usam os dois na mesma frota, com o tráfego síncrono tendo prioridade."

Resposta completa

Use a tabela da seção 1.6 e depois aprofunde três pontos.

1. Critério de decisão. Pergunte: *quem está esperando, e quanto tempo aguenta?*

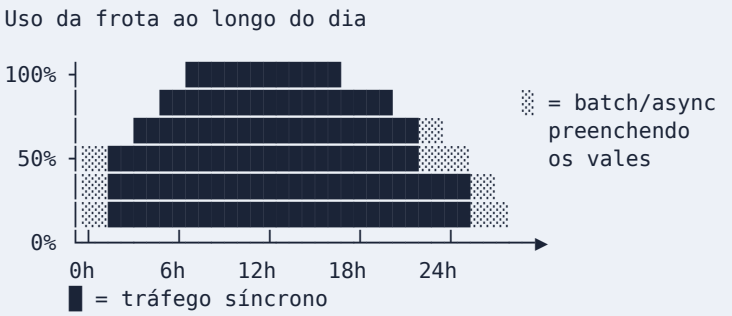
```
Alguém está esperando a resposta agora?
├── Sim
│   ├── A resposta sai em < ~30–60 s?
│   │   ├── Sim → SÍNCRONA (de preferência com streaming)
│   │   └── Não → ASSÍNCRONA com barra de progresso
│   │             (conexões longas caem: proxies, celular)
│   └──
└── Não
    ├── Precisa terminar até um prazo (ex.: 24 h)?
    │   └── BATCH offline, prioridade baixa
    └── Reage a um evento (upload, e-mail novo)?
        └── ASSÍNCRONA dirigida por eventos (fila)
```

2. O que a assíncrona exige.

- **Fila durável** (Kafka, SQS, Pub/Sub, Redis Streams) e workers que puxam trabalho.
- **Idempotência**: se o worker morrer no meio, o job é refeito. A chave de idempotência evita cobrar ou processar duas vezes.

- **Status do job**: `pendente → rodando → concluído/falhou`, com resultado guardado num armazenamento (objeto, banco).
- **Notificação**: polling (`GET /jobs/{id}`) ou webhook (o servidor chama o cliente). Webhook precisa de assinatura, retries e de um fallback de polling.
- **Backpressure**: se a fila crescer demais, recuse ou atrase novos jobs antes de a latência virar dias.

3. Por que isso importa para a GPU. O tráfego síncrono tem picos. Você dimensiona a frota para o pico com folga, e o vale fica ocioso. Jobs assíncronos e em lote com prioridade baixa preenchem o vale. O scheduler precisa conseguir **preemptar** (pausar ou interromper) trabalho de baixa prioridade quando chega tráfego síncrono. Também ajuda que jobs em lote toleram batches enormes e latência alta, então rodam no ponto de máximo throughput da curva da seção 1.4.



Perguntas de acompanhamento prováveis

- **Streaming é síncrono ou assíncrono?** Síncrono: a conexão fica aberta e os tokens chegam conforme são gerados (Server-Sent Events, WebSocket ou gRPC streaming). Melhora a latência percebida, mas a conexão continua ocupada até o fim.
- **Como garantir que um job assíncrono não roda duas vezes?** Não dá para garantir "exatamente uma vez" em sistemas distribuídos com falhas. Use entrega "pelo menos uma vez" com processamento idempotente: chave de deduplicação, escrita do resultado condicionada ao estado do job.
- **Como priorizar batch sem prejudicar o síncrono?** Filas separadas por prioridade, cotas, e preempção no scheduler do motor (pausar ou devolver para a fila requisições de baixa prioridade quando o KV cache aperta).
- **Quando uma requisição síncrona deveria virar assíncrona?** Quando o tempo esperado passa dos timeouts típicos (proxies e balanceadores costumam cortar conexões ociosas em dezenas de segundos ou alguns minutos) ou quando o cliente é móvel e a conexão é instável.

Armadilhas comuns

- Achar que assíncrono é sempre mais barato. Só é se a capacidade usada for a ociosa ou se permitir batches maiores.
- Esquecer idempotência e retries.
- Usar polling agressivo (milhares de clientes a cada 100 ms) e criar um problema de carga novo.
- Não falar de prioridade e preempção.

Resumo do capítulo

- Gerar texto tem duas fases: **prefill** (prompt inteiro de uma vez, compute-bound) e **decode** (um token por passo, memory-bound).
- No decode com batch 1, a intensidade aritmética é ~1 FLOP/byte, enquanto a H100 precisa de ~295 para ficar limitada por cálculo.
- Llama 3 8B em BF16: 16 GB de pesos → ~4,8 ms por passo só de leitura numa H100 → no máximo ~210 tokens/s para um usuário sozinho.
- **Batching** amortiza a leitura dos pesos, mas o **KV cache** é por usuário e limita o batch pela memória.
- O **roofline** diz se um kernel é limitado por memória ou por cálculo: performance = min(pico, banda × intensidade).
- **TTFT** = fila + prefill. **TPOT/ITL** = passo de decode. **E2E** = TTFT + (N−1) × TPOT. **Goodput** = throughput que cumpre o SLO.
- Reporte latência em **percentis** (p50, p99), nunca só a média.
- Custo por 1M tokens = custo da GPU por hora ÷ tokens por hora × 10^6.
- Síncrono para quem espera agora; assíncrono e batch para o resto, preenchendo a capacidade ociosa com prioridade baixa.

Checklist para a entrevista

- ☐ Explico prefill vs decode e por que um é compute-bound e o outro memory-bound.
- ☐ Faço a conta do tempo mínimo de um passo de decode para qualquer modelo e GPU.
- ☐ Sei o ponto de equilíbrio da H100 (~295 FLOP/byte) e de onde ele vem.
- ☐ Desenho o roofline e posiciono decode, prefill e operações elementares nele.
- ☐ Defino TTFT, ITL, TPOT, E2E, throughput e goodput sem hesitar.
- ☐ Explico os três conflitos: throughput vs TPOT, TTFT vs TPOT, utilização vs cauda.

- [ ] Calculo custo por milhão de tokens.
- [ ] Descrevo o forward pass como sequência de kernels, com o caminho dos dados na memória.
- [ ] Tenho uma história de otimização com contexto, medição, causa, mudança e resultado em números.
- [ ] Sei quando usar inferência síncrona, assíncrona e em lote, e o que a assíncrona exige.

CAPÍTULO 2

Transformer e LLMs por dentro

Você não precisa saber treinar um transformer para trabalhar com inferência. Mas precisa saber **exatamente o que ele calcula, em que ordem, com que formatos de tensor e quanto de memória cada parte usa**. Sem isso, não dá para entender por que o KV cache existe, por que a atenção é cara com contextos longos ou onde estão os bugs sutis que aparecem quando você mexe no motor de inferência.

Os entrevistadores testam este tema de dois jeitos: perguntas conceituais (“o que é o KV cache?”, “o que é FlashAttention?”) e perguntas de implementação (“implemente o cache e me diga o que pode dar errado”). Este capítulo prepara para as duas.

2.1 Do texto aos números: tokenização e embeddings

O modelo não lê letras. Um **tokenizer** quebra o texto em pedaços (tokens) e troca cada pedaço por um número inteiro (o ID do token). Os tokenizers modernos usam variações de **BPE** (byte-pair encoding): começam com bytes e juntam os pares mais frequentes até formar um vocabulário de, por exemplo, 128 mil tokens no Llama 3.

```
"Engenharia de inferência"
       | tokenizer (BPE)
       ▼
["Eng", "enh", "aria", " de", " infer", "ência"]   (ilustrativo)
       |
       ▼
[  6031,  12740,  10649,  409,  12309,  56743 ]    (IDs)
```

Fatos que importam em inferência:

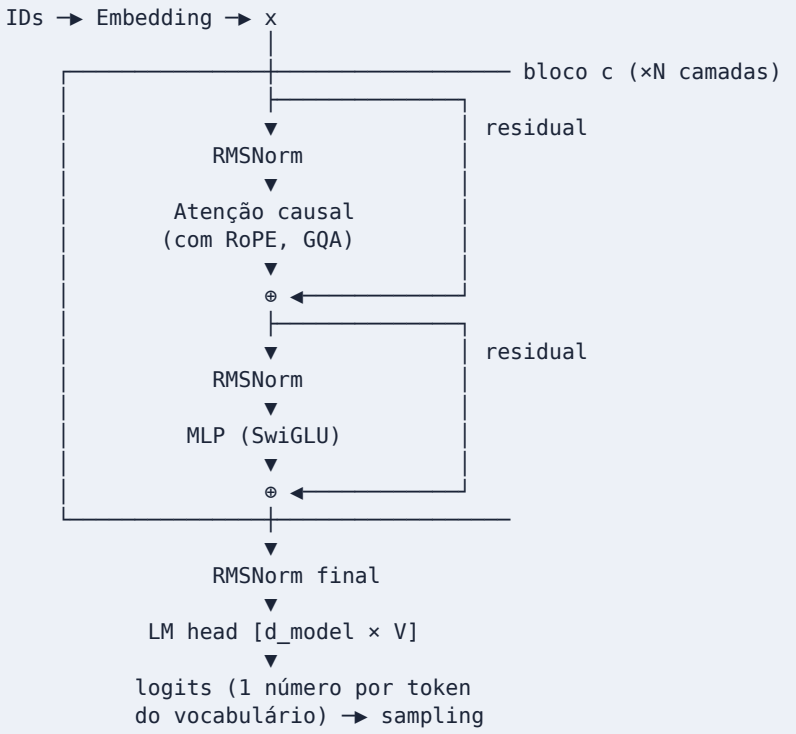
- **Português gasta mais tokens que inglês** na maioria dos tokenizers, porque eles foram treinados com mais texto em inglês. A mesma frase pode virar 20% a 50% mais tokens. Isso aumenta custo e latência.
- **A tokenização roda na CPU**, geralmente em Rust (biblioteca `tokenizers`). Para prompts gigantes, ela pode levar milissegundos e entra no TTFT.

- **Detokenizar no streaming é chato**: um caractere acentuado pode ocupar dois tokens. Se você mandar o texto do primeiro token sozinho, o usuário vê um caractere quebrado. O servidor precisa segurar bytes incompletos até formar UTF-8 válido.

Depois, cada ID vira um vetor por meio de uma **tabela de embeddings**: uma matriz de tamanho [vocabulário × d_model]. No Llama 3 8B, são 128.256 × 4.096 ≈ 525 milhões de parâmetros só nessa tabela. Na GPU, isso é um simples **gather**: copiar a linha correspondente a cada ID.

2.2 A estrutura de um decoder moderno

Os LLMs de hoje (Llama, Mistral, Qwen, DeepSeek e muitos outros) são **decoder-only**: uma pilha de blocos iguais, cada um com atenção e MLP, ligados por conexões residuais.



Os números do Llama 3 8B e 70B, que vamos usar nas contas:

	Llama 3 8B	Llama 3 70B
Camadas	32	80

	Llama 3 8B	Llama 3 70B
d_model	4.096	8.192
Heads de query	32	64
Heads de KV (GQA)	8	8
head_dim	128	128
Dimensão do MLP	14.336	28.672
Vocabulário	128.256	128.256

Fazendo a conta: de onde vêm os 8 bilhões? Para o Llama 3 8B:

- Atenção por camada: Q (4.096 × 4.096) + K (4.096 × 1.024) + V (4.096 × 1.024) + saída (4.096 × 4.096) ≈ 16,8 + 4,2 + 4,2 + 16,8 = **42 M**. K e V têm 1.024 colunas porque são só 8 heads × 128.
- MLP por camada: 3 matrizes de 4.096 × 14.336 ≈ **176 M**.
- Por camada: ~218 M. Vezes 32 camadas: **~6,98 B**.
- Embedding + LM head (não compartilhados): 2 × 525 M ≈ **1,05 B**.
- Total: **~8,0 B**. Note que o MLP tem cerca de 80% dos parâmetros das camadas.

2.3 Atenção, do zero

A atenção deixa cada token "olhar" para os tokens anteriores e puxar informação deles. Para cada token, o modelo calcula três vetores a partir do estado atual x:

- **Query** ($q = xW_Q$): "o que eu estou procurando?"
- **Key** ($k = xW_K$): "o que eu ofereço para quem procura?"
- **Value** ($v = xW_V$): "o conteúdo que eu entrego se alguém me escolher."

A atenção do token i compara seu query com os keys de todos os tokens $j \le i$, transforma as similaridades em pesos com softmax e faz uma média ponderada dos values:

$$\text{Atenção}(Q, K, V) = \text{softmax}(\frac{QK^\top}{\sqrt{d}} + M)V$$

- d é o head_dim (128). Dividir por $\sqrt{d}$ evita que os produtos fiquem grandes demais e o softmax sature.

- M é a **máscara causal**: 0 onde $j \le i$ e $-\infty$ onde $j > i$. Assim um token nunca vê o futuro.

> **Analogia:** pense numa biblioteca. A query é a pergunta que você leva ao balcão. Cada livro tem uma etiqueta na lombada (a key) e um conteúdo (o value). Você compara sua pergunta com todas as etiquetas, dá uma nota para cada livro e monta a resposta misturando os conteúdos, com mais peso para os livros com nota maior. A máscara causal diz que você só pode consultar livros que já chegaram na biblioteca.

```
Matriz de scores S = Q·Kᵀ para 5 tokens (com máscara causal)

            k0    k1    k2    k3    k4
        ┌─────┬─────┬─────┬─────┬─────┐
   q0   │  ●  │ -∞  │ -∞  │ -∞  │ -∞  │
   q1   │  ●  │  ●  │ -∞  │ -∞  │ -∞  │
   q2   │  ●  │  ●  │  ●  │ -∞  │ -∞  │
   q3   │  ●  │  ●  │  ●  │  ●  │ -∞  │
   q4   │  ●  │  ●  │  ●  │  ●  │  ●  │
        └─────┴─────┴─────┴─────┴─────┘

  Prefill: calcula o triângulo inteiro de uma vez.
  Decode do token 5: calcula só UMA linha nova (q5 contra
  k0..k5), e precisa de todos os k e v anteriores.
```

Multi-head, MQA, GQA e MLA

Uma única atenção captura um tipo de relação. Por isso os modelos usam várias **heads** em paralelo, cada uma com suas próprias projeções, e concatenam os resultados. O Llama 3 8B tem 32 heads de 128 dimensões ($32 \times 128 = 4.096$).

O problema para inferência: cada head precisa guardar seus próprios K e V para todos os tokens anteriores. Daí surgiram variações que **compartilham K e V entre heads de query**:

```
MHA (multi-head)      GQA (grouped-query)      MQA (multi-query)
Q: ■■■■■■■■■          Q: ■■■■■■■■■             Q: ■■■■■■■■■
K: ■■■■■■■■■          K: ■ ■ ■ ■  (grupos)      K: ■
V: ■■■■■■■■■          V: ■ ■ ■ ■                V: ■
8 heads de KV         2 heads de query          1 head de KV para
                      por head de KV            todas as queries
KV cache: 100%        KV cache: 25%             KV cache: 12,5%
```

- **MQA** (Shazeer, 2019): uma única head de K e V. Economiza muita memória, mas pode perder qualidade.

- **GQA** (Ainslie et al., 2023): grupos de heads de query compartilham uma head de KV. O Llama 3 70B usa 64 heads de query e 8 de KV: o KV cache fica **8 vezes menor** do que seria com MHA, com perda de qualidade pequena.
- **MLA** (multi-head latent attention, DeepSeek-V2): comprime K e V num vetor latente de baixa dimensão por token e reconstrói as heads na hora. Reduz ainda mais o cache, ao custo de mais cálculo.

Para inferência, isso é decisivo: **o KV cache é o que limita o batch**, e GQA/MLA atacam exatamente ele.

2.4 Posições: RoPE

A atenção, sozinha, não sabe a ordem dos tokens: trocar dois tokens de lugar não muda os scores. Os modelos modernos codificam a posição com **RoPE** (rotary position embedding, Su et al., 2021): cada par de dimensões de q e k é **girado** por um ângulo proporcional à posição do token.

Para o par de dimensões (x_1, x_2) na posição p, com frequência $\theta_i = b^{-2i/d}$ (a base b é 10.000 no Llama 2 e 500.000 no Llama 3):

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} \cos(p\theta_i) & -\sin(p\theta_i) \\ \sin(p\theta_i) & \cos(p\theta_i) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

O truque: o produto escalar entre um q girado pela posição m e um k girado pela posição n depende só da **diferença** $m - n$. A atenção passa a enxergar distâncias relativas.

```python
import torch

def rope(x, pos, base=500000.0):
    # x: [B, H, T, D]   pos: [T] com as posições ABSOLUTAS
    d = x.shape[-1]
    i = torch.arange(0, d, 2, dtype=torch.float32)
    inv_freq = base ** (-i / d)                    # [D/2]
    ang = pos.float()[:, None] * inv_freq[None]    # [T, D/2]
    cos = torch.cat([ang.cos(), ang.cos()], -1)    # [T, D]
    sin = torch.cat([ang.sin(), ang.sin()], -1)
    # convenção "rotate_half" (Llama no Hugging Face):
    # pares são (i, i + D/2), não (2i, 2i+1)
    x1, x2 = x[..., : d // 2], x[..., d // 2 :]
    rot = torch.cat([-x2, x1], dim=-1)
    return x * cos + rot * sin
```

> **Cuidado:** existem duas convenções de RoPE. Uma gira pares vizinhos (dimensões 0 e 1, 2 e 3...). A outra, usada pela implementação Llama do Hugging Face, gira a primeira metade com a segunda (dimensões 0 e 64, 1 e 65...). Os pesos de Q e K são permutados de acordo com a convenção. Misturar as duas não dá erro de execução, só gera texto ruim. É um bug clássico ao portar modelos entre motores.

2.5 MLP, normalização e o resto

MLP (SwiGLU). Depois da atenção, cada token passa sozinho por uma rede de duas camadas com uma porta:

$$\mathrm{MLP}(x) = W_{down}(\mathrm{SiLU}(xW_{gate}) \odot (xW_{up}))$$

São três GEMMs grandes. Como têm 80% dos parâmetros, dominam a leitura de pesos no decode. A multiplicação elemento a elemento ($\odot$) com SiLU é um ótimo candidato à **fusão de kernels**.

RMSNorm. Normaliza cada vetor pelo seu valor quadrático médio e multiplica por um peso aprendido:

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot g$$

É barata em FLOPs, mas lê e escreve o tensor inteiro: memory-bound. Por isso costuma ser fundida com a soma residual.

Residual. Cada bloco soma sua saída à entrada ($x + f(x)$). Isso estabiliza o treinamento e, em inferência, é só uma soma elemento a elemento.

LM head. A última GEMM projeta o vetor final (d_{model}) para o tamanho do vocabulário, gerando os **logits**: uma nota para cada token possível. Com vocabulários de 128 mil tokens, essa GEMM não é pequena: no 8B, são 525 M de parâmetros.

2.6 Sampling: escolhendo o próximo token

Os logits viram probabilidades com softmax. O **sampling** decide qual token sai:

Método	Como funciona	Uso
Greedy	pega o de maior probabilidade	determinístico, tarefas objetivas

Método	Como funciona	Uso
Temperatura T	divide os logits por T antes do softmax; $T < 1$ concentra, $T > 1$ espalha	controla criatividade
Top-k	sorteia só entre os k mais prováveis	corta a cauda de tokens absurdos
Top-p (nucleus)	sorteia entre os menores conjuntos cuja soma passa de p	adapta o corte à confiança do modelo
Penalidades	reduz logits de tokens já usados	evita repetição

```python
import torch

def sample(logits, temperature=0.7, top_p=0.9):
    # logits: [B, V]
    if temperature == 0:
        return logits.argmax(dim=-1)
    probs = torch.softmax(logits / temperature, dim=-1)
    sp, idx = probs.sort(dim=-1, descending=True)
    cum = sp.cumsum(dim=-1)
    # remove tokens depois de atingir top_p (mantém o 1º)
    sp[(cum - sp) > top_p] = 0.0
    sp = sp / sp.sum(dim=-1, keepdim=True)
    escolha = torch.multinomial(sp, num_samples=1)
    return idx.gather(-1, escolha).squeeze(-1)
```

Em inferência de produção, o sampling roda na GPU. Ordenar 128 mil valores por usuário a cada passo não é de graça; motores otimizados usam técnicas que evitam a ordenação completa.

2.7 Prefill e decode por dentro

Agora dá para ver com precisão o que muda entre as fases. Considere um batch de B sequências:

	PREFILL	DECODE (1 passo)
entrada x	[B, T, d]	[B, 1, d]
GEMMs	matriz × matriz (T linhas por seq.)	~matriz × vetor (1 linha por seq.)
Q	[B, H, T, 128]	[B, H, 1, 128]
K, V novos	T por sequência	1 por sequência
atenção	T × T (triângulo)	1 × (contexto todo)
KV cache	ESCRITO (T tokens)	LIDO (tudo) + 1 escrito
gargalo	cálculo (FLOPs)	memória (bytes)
duração típica	dezenas a centenas de ms	alguns a dezenas de ms
métrica afetada	TTFT	TPOT / ITL

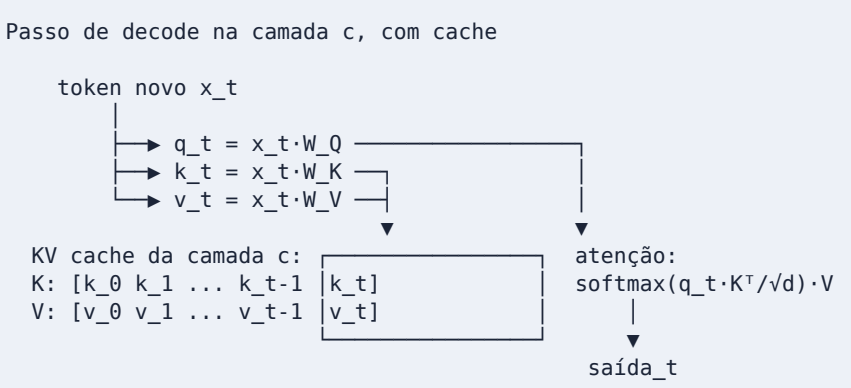
O custo da atenção cresce de forma diferente nas duas fases:

- No **prefill**, a matriz de scores tem $T \times T$ elementos: o custo é **quadrático** no tamanho do prompt. Dobrar o prompt quadruplica o trabalho da atenção (as GEMMs crescem linearmente).
- No **decode**, cada passo calcula uma linha nova: o custo é **linear** no contexto. Mas, somando todos os passos, gerar N tokens também custa algo proporcional a N^2.

2.8 O KV cache, conceito

Repare na linha “KV cache” da tabela. Sem cache, para gerar o token 1.001, o modelo teria que recalcular K e V dos 1.000 tokens anteriores, em todas as camadas. Para gerar o 1.002, recalcularia os 1.001. Isso é desperdício: **K e V de um token não mudam depois de calculados**, porque a máscara causal garante que um token passado nunca vê os tokens novos.

O **KV cache** guarda, para cada camada, os K e V de todos os tokens já processados. A cada passo de decode, o modelo calcula K e V só do token novo, adiciona ao cache e roda a atenção contra o cache inteiro.



O preço é memória. O tamanho por token é:

$$\text{bytes por token} = 2 \times n_{camadas} \times n_{kv_heads} \times d_{head} \times \text{bytes por elemento}$$

O “2” é porque guardamos K e V.

> **Fazendo a conta:**
>
> - Llama 3 8B em BF16: 2 × 32 × 8 × 128 × 2 bytes = **131.072 bytes = 128 KiB por token**.
> - Llama 3 70B em BF16: 2 × 80 × 8 × 128 × 2 = **327.680 bytes = 320 KiB por token**.
> - Uma conversa de 8.000 tokens no 70B: 8.000 × 320 KiB ≈ **2,6 GB**. Só uma conversa.
> - Se o 70B usasse MHA (64 heads de KV), seriam 8 vezes mais: ~21 GB por conversa de 8.000 tokens. GQA é o que torna esse modelo servível.

O Capítulo 6 aprofunda: gerenciamento, paginação, compartilhamento, offload e quantização do cache.

2.9 FlashAttention

A implementação ingênua da atenção faz três passos, cada um um kernel:

1. $S = QK^\top$ → escreve S (tamanho $T \times T$) na HBM.
2. $P = \text{softmax}(S)$ → lê S, escreve P na HBM.
3. $O = PV$ → lê P, escreve O.

Com T = 8.192 e 32 heads, a matriz S tem $8.192^2 \times 32 \approx 2{,}1$ bilhões de elementos por sequência, uns 4 GB em BF16. Escrever e ler isso várias vezes da HBM é lento, e a memória necessária cresce com T^2.

O **FlashAttention** (Dao et al., 2022) nunca materializa S inteira. Ele divide Q, K e V em blocos que cabem na SRAM (shared memory) do SM, calcula a atenção bloco a bloco e acumula o resultado. O desafio é o softmax, que precisa do máximo e da soma da linha inteira. A solução é o **softmax online**: manter, para cada linha, o máximo parcial m e a soma parcial l, e corrigir o acumulador quando um bloco novo traz um máximo maior.

```
FlashAttention (forward), para um bloco de queries Q_i:

  carrega Q_i na SRAM
  m ← -∞,  l ← 0,  O ← 0          (por linha)
  para cada bloco j:
      carrega K_j, V_j na SRAM
      S   ← Q_i · K_jᵀ / √d        (só na SRAM)
      m'  ← max(m, max_linha(S))
      P   ← exp(S − m')
      l   ← l · exp(m − m') + soma_linha(P)
      O   ← O · exp(m − m') + P · V_j
      m   ← m'
  O ← O / l
  escreve O na HBM                 (uma única vez)
```

O resultado é **exatamente** a mesma atenção (a menos de arredondamento), não uma aproximação. Você pode verificar com um script curto:

```python
import torch

def atencao_ingenua(q, k, v):
    s = q @ k.T / q.shape[-1] ** 0.5
    return torch.softmax(s, dim=-1) @ v

def atencao_online(q, k, v, bloco=64):
    d = q.shape[-1]
    m = torch.full((q.shape[0], 1), float("-inf"))
    l = torch.zeros(q.shape[0], 1)
    o = torch.zeros_like(q)
    for j in range(0, k.shape[0], bloco):
        s = q @ k[j:j + bloco].T / d ** 0.5
        m_novo = torch.maximum(m, s.max(-1, keepdim=True).values)
        p = torch.exp(s - m_novo)
        fator = torch.exp(m - m_novo)
        l = l * fator + p.sum(-1, keepdim=True)
        o = o * fator + p @ v[j:j + bloco]
        m = m_novo
    return o / l

q, k, v = (torch.randn(128, 64) for _ in range(3))
print(torch.allclose(atencao_ingenua(q, k, v),
                     atencao_online(q, k, v), atol=1e-5))  # True
```

Os ganhos:

- **Menos tráfego de HBM**: S e P nunca saem do chip. A atenção passa de memory-bound para muito mais perto do limite de cálculo no prefill.
- **Memória linear** em vez de quadrática: dá para usar contextos longos.
- **Fusão**: máscara, escala, softmax e dropout (no treino) num kernel só.

As versões seguintes melhoraram a eficiência no hardware:

- **FlashAttention-2** (2023): paraleliza também ao longo da sequência, reduz operações fora de GEMM e divide melhor o trabalho entre warps.
- **FlashAttention-3** (2024): específico para Hopper (H100). Usa TMA (cópias assíncronas de memória) e WGMMA (instruções de tensor core por grupo de warps), sobrepõe o softmax com as GEMMs e suporta FP8.
- Para **decode**, a forma do problema é outra (uma query por sequência, cache longo). Variantes como **Flash-Decoding** dividem o cache em pedaços processados em paralelo e juntam os resultados no fim, para ocupar todos os SMs mesmo com batch pequeno.

> **Dica de entrevista:** a frase que resume FlashAttention: "é uma atenção exata, ciente da hierarquia de memória (IO-aware), que usa tiling e softmax online para nunca escrever a matriz T×T na HBM". Se você disser "IO-aware" e explicar o softmax online, já está acima da média.

Perguntas de entrevista comentadas

Q009 O que é um KV cache? O que é FlashAttention? (perguntadas em sequência)

> **[Relatado]** NVIDIA · Senior Software Engineer · entrevista técnica · 2026 · Médio · fonte

Resposta em 30 segundos

"O KV cache guarda os vetores key e value de todos os tokens já processados, em cada camada, para que no decode o modelo calcule K e V só do token novo, em vez de recalcular o prefixo inteiro a cada passo. Isso troca cálculo por memória: no Llama 3 70B são uns 320 KB por token em BF16. FlashAttention é uma implementação exata da atenção que divide Q, K e V em blocos que cabem na SRAM e usa softmax online, para nunca escrever a matriz de scores T×T na memória da GPU. Isso reduz muito o tráfego de HBM e deixa a memória linear no tamanho da sequência. Os dois se complementam: o KV cache evita recalcular, o FlashAttention deixa a atenção que sobra mais rápida."

Resposta completa

Responda as duas partes com a mesma estrutura: **problema → ideia → custo → números**.

KV cache.

- *Problema*: geração autorregressiva. Sem cache, o passo t recalcula K e V de t tokens. Gerar N tokens custa $O(N^2)$ projeções a mais.
- *Ideia*: K e V de tokens passados nunca mudam (máscara causal). Guarde-os.
- *Custo*: memória. Fórmula $2 \times L \times H_{kv} \times d_h \times$ bytes por token (seção 2.8).
- *Consequências*: o cache limita o batch, cresce com o contexto e precisa ser gerenciado (paginação, compartilhamento de prefixos, preempção, offload). Também é lido inteiro a cada passo, o que faz a atenção do decode ser memory-bound.

FlashAttention.

- *Problema*: a atenção ingênua escreve e lê S e P ($T \times T$) na HBM. Memória quadrática e muito tráfego.
- *Ideia*: tiling + softmax online + fusão num único kernel; recomputação no backward (no treino) em vez de guardar P.
- *Custo*: kernel complexo, específico de hardware (por isso há FA2, FA3 e variantes para decode).
- *Consequências*: atenção bem mais rápida em sequências longas, contextos longos viáveis, mesma saída numérica (a menos de arredondamento).

Termine conectando: "No serving, os kernels de atenção do decode, como o PagedAttention do vLLM ou o FlashInfer, combinam as duas ideias: leem um KV cache paginado em blocos e aplicam a mesma lógica de softmax online."

Perguntas de acompanhamento prováveis

- **O FlashAttention reduz o número de FLOPs?** Não. Ele faz até um pouco mais (por causa das correções do softmax online e, no treino, da recomputação). O ganho vem de mover muito menos bytes entre HBM e SRAM.
- **O FlashAttention ajuda no decode?** Menos que no prefill. No decode, a matriz de scores tem só uma linha por sequência; o custo dominante é ler o KV cache. Variantes como Flash-Decoding ajudam a paralelizar essa leitura.
- **Por que não guardar também Q no cache?** Porque o query de tokens antigos nunca é usado de novo: cada token novo só precisa do próprio query contra os K e V antigos.
- **Como reduzir o tamanho do KV cache?** GQA/MQA/MLA (menos heads de KV), quantização do cache (FP8, INT8, INT4), sliding window, eviction de tokens pouco usados, offload para CPU. Detalhes no Capítulo 6.

- **O que é o softmax online, exatamente?** Uma forma de calcular o softmax em uma passada, mantendo o máximo parcial e a soma parcial e reescalando o que já foi acumulado sempre que aparece um máximo maior (Milakov e Gimelshein, 2018).

Armadilhas comuns

- Dizer que FlashAttention é uma aproximação. É exata.
- Dizer que o KV cache "acelera porque usa menos memória". É o contrário: usa mais memória para fazer menos cálculo.
- Esquecer que o cache é por camada e por head de KV.
- Não saber estimar o tamanho do cache.

Q010 Explique o que fica em cache em cada camada num KV cache e as armadilhas de corretude ao adicionar um (máscaras, posições, RoPE, shapes).

> **[Relatado]** Startups.com · Machine Learning Engineer · presencial · listada em 2026 · Médio · fonte

Resposta em 30 segundos

"Em cada camada guardo K e V depois da projeção e depois do RoPE, com shape [batch, kv_heads, tokens, head_dim]. As armadilhas: a máscara causal no prefill precisa considerar o deslocamento do cache e, com batch, o padding; as posições do token novo têm que continuar de onde o cache parou, por sequência; o RoPE deve ser aplicado uma única vez, antes de gravar K, e nunca de novo nos K do cache; com GQA, as heads de KV são expandidas para as de query só na hora da atenção; com beam search, o cache precisa ser reordenado quando os feixes mudam; e concatenar tensores a cada passo é quadrático, então prefiro pré-alocar."

Resposta completa

O que guardar, exatamente. Para cada camada *c*:

```
cache_k[c]: [B, H_kv, T_max, D]   (K já com RoPE aplicado)
cache_v[c]: [B, H_kv, T_max, D]   (V não leva RoPE)
comprimento[b]: quantos tokens válidos a sequência b tem
```

Não guarde: Q (nunca reusado), a saída da atenção, nem os estados do MLP. Não guarde K e V **antes** da projeção (os x da camada) a menos que queira recalcular as projeções para economizar memória, o que é um trade-off diferente.

Agora as armadilhas, uma por uma. Essa é a parte que o entrevistador quer ouvir.

1. Máscara causal com deslocamento. No decode com cache, o query tem 1 token e o key tem $T_{cache} + 1$. A máscara causal "triangular" padrão assume $T_q = T_k$. Se você reusar a máscara do prefill, ela fica desalinhada. A regra correta: o query na posição local i tem posição absoluta $T_k - T_q + i$ e pode ver keys $j \le T_k - T_q + i$.

```
import torch
import torch.nn.functional as F

def atencao_com_cache(q, k, v):
    # q: [B, H, Tq, D]   k, v: [B, H, Tk, D]  (Tk >= Tq)
    tq, tk = q.shape[-2], k.shape[-2]
    s = q @ k.transpose(-1, -2) / q.shape[-1] ** 0.5
    pos_q = torch.arange(tq).unsqueeze(1) + (tk - tq)
    pos_k = torch.arange(tk).unsqueeze(0)
    s = s.masked_fill(pos_k > pos_q, float("-inf"))
    return F.softmax(s, dim=-1) @ v
```

Isso também cobre o caso de **chunked prefill** e de **prefix caching**, em que um pedaço do prompt já está no cache e o resto chega como várias queries.

2. Padding no batch. Sequências de tamanhos diferentes no mesmo batch exigem padding. Com padding à esquerda (comum em geração com Hugging Face), os tokens de padding ocupam as primeiras posições do cache. Você precisa de uma **máscara de atenção por sequência** que impeça os tokens reais de olhar para o padding. Esquecer isso faz a saída mudar conforme o tamanho do batch: um bug que só aparece em produção.

3. Posições e RoPE. O token novo de cada sequência tem posição `comprimento[b]`, não `T_max` nem `0`. Com padding à esquerda, a posição não é o índice no tensor: precisa descontar o padding. Erros típicos:

- Reiniciar as posições em 0 a cada passo de decode: o modelo "acha" que todo token novo é o primeiro.
- Usar a mesma posição para todas as sequências do batch.
- Aplicar RoPE de novo nos K que vêm do cache (girando duas vezes).
- Guardar K sem RoPE e esquecer de girar na leitura.

```
Prefill de 5 tokens, depois decode:

posições:   0   1   2   3   4 | 5   6   7 ...
cache K:   k0  k1  k2  k3  k4 |k5  k6  k7
           (RoPE aplicado     | cada novo k recebe RoPE
            com pos 0..4)     | com SUA posição absoluta,
                              | uma única vez
```

4. Shapes e GQA. O cache tem H_{kv} heads (8), mas o query tem H_q (32 ou 64). Na atenção, cada head de KV serve H_q/H_{kv} heads de query. Não expanda o cache na memória (`repeat_interleave` no cache inteiro anula a economia do GQA); expanda só no cálculo, ou use um kernel que entenda GQA. E cuidado com a ordem: o agrupamento é `query head h` ↔ `kv head h // (H_q/H_kv)`, e ele precisa bater com o que o modelo aprendeu.

5. Alocação. `torch.cat([cache, k_novo])` a cada passo realoca e copia o cache inteiro: $O(T^2)$ em cópias e fragmentação de memória. Pré-aloque `T_max` e escreva com índice, ou use blocos (PagedAttention, Capítulo 6).

6. dtype. Calcular K e V em BF16 e guardar em FP16 (ou vice-versa) causa diferenças numéricas; quantizar o cache para FP8 exige fatores de escala por head ou por bloco. Seja consistente.

7. Beam search e cópias. Em beam search, os feixes trocam de ancestral a cada passo. O cache precisa ser **reordenado** (`cache.index_select(0, indices_dos_pais)`) ou, melhor, compartilhado por referência com copy-on-write.

8. Limite de contexto. Passar de `T_max` (ou da janela do modelo) silenciosamente corrompe o cache ou extrapola posições que o modelo não viu no treino. Trate explicitamente: erro, truncamento ou sliding window.

Como testar. O teste de ouro é simples: gere N tokens com greedy usando o cache e compare com a geração sem cache (recalculando tudo a cada passo). Os logits devem bater dentro da tolerância numérica, e os tokens devem ser idênticos na maioria dos casos. Repita com batch de tamanhos diferentes e padding, para pegar bugs de máscara.

Perguntas de acompanhamento prováveis

- **Por que gravar K depois do RoPE?** Para não girar todos os K do cache a cada passo. O custo: a posição fica "congelada" no cache, o que complica técnicas que mudam posições depois (por exemplo, remover tokens do meio do contexto).

- **Como lidar com sequências de tamanhos diferentes sem padding?** Com cache paginado e um kernel de atenção que recebe a lista de blocos e o comprimento de cada sequência (layout "ragged"). É o que vLLM e similares fazem.
- **Como o cache muda com sliding window attention?** Você só precisa guardar os últimos W tokens por camada (um buffer circular). Cuidado com as posições: elas continuam crescendo mesmo que o buffer gire.
- **Os resultados com e sem cache são bit a bit iguais?** Em geral não. A ordem das operações muda (kernels diferentes para prefill e decode), e o arredondamento em ponto flutuante não é associativo. Compare com tolerância.

Armadilhas comuns

- Falar só "guardo K e V" sem shapes, sem RoPE e sem máscara.
- Esquecer padding e GQA.
- Usar `torch.cat` no laço e não perceber o custo quadrático.
- Não propor um teste de equivalência com a versão sem cache.

Q011 Explique prefill vs. decode e por que os sistemas de serving tratam cada um de forma diferente.

> **[Fornecedor]** Banco público de perguntas de inferência, marcada como "pergunta padrão para sêniores" · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"Prefill processa o prompt inteiro numa passada, em paralelo, e preenche o KV cache: são GEMMs grandes, compute-bound, e definem o TTFT. Decode gera um token por passada, lendo todos os pesos e o KV cache inteiro para fazer pouco cálculo: é memory-bound e define o TPOT. Como os gargalos são opostos, misturar os dois num mesmo passo faz um atrapalhar o outro: um prefill longo trava o decode de todo mundo. Por isso os sistemas usam chunked prefill para misturar com controle, priorizam um ou outro conforme o SLO, usam kernels diferentes para cada fase e, em escala, chegam a separar prefill e decode em GPUs diferentes, transferindo o KV cache."

Resposta completa

Use a tabela da seção 2.7. Depois explique as quatro formas como os sistemas tratam as fases de maneira diferente:

1. Kernels diferentes. O prefill usa kernels de atenção otimizados para muitas queries (FlashAttention em tiles grandes). O decode usa kernels otimizados para uma query por sequência contra um cache longo e paginado (PagedAttention, FlashInfer, Flash-Decoding). Tamanhos de tile, paralelização e uso de tensor cores são diferentes.

2. Scheduling. O scheduler decide, a cada passo, quais sequências entram:

```
Sem cuidado: um prompt de 32k tokens chega
passo:  D  D  D [██████ PREFILL 32k ██████] D  D  D
                ▲ todos os usuários em decode
                  ficam centenas de ms sem token

Chunked prefill: o prompt entra em pedaços de, digamos, 2k
passo: [D+2k][D+2k][D+2k] ... [D+2k][D][D]
        cada passo tem um "orçamento" de tokens;
        decodes têm prioridade e o resto do orçamento
        vai para o pedaço de prefill
```

O orçamento de tokens por passo controla o trade-off: maior = TTFT melhor e passos de decode mais lentos; menor = o contrário. Detalhes no Capítulo 7.

3. Paralelismo e hardware diferentes. Prefill gosta de muito cálculo (tensor parallelism grande ajuda a reduzir o TTFT de prompts enormes). Decode gosta de banda de memória e capacidade para o KV cache (batch grande). Uma configuração única é um compromisso.

4. Desagregação. Em escala, sistemas como DistServe, Splitwise e Mooncake colocam prefill e decode em **pools de GPUs separados**. O prefill gera o KV cache e o transfere (via NVLink, InfiniBand/RDMA) para uma GPU de decode. Vantagens: cada pool é dimensionado e configurado para seu SLO (TTFT ou TPOT), e não há interferência. Custo: transferência do KV cache, que pode ter centenas de MB a alguns GB por requisição longa, e mais complexidade operacional. O Capítulo 9 aprofunda (Q067).

Fazendo a conta: quanto tempo leva o prefill de 8.000 tokens no Llama 3 70B em 8 H100?

- FLOPs das matrizes: $2 \times 70\text{ B} \times 8.000 \approx$ **1,12 PFLOP**.
- Atenção (QK^T e PV, com máscara causal ~metade): $2 \times 2 \times 80$ camadas $\times\ 8.000^2 \times 8.192 / 2 \approx$ **0,084 PFLOP**.
- Total ~1,2 PFLOP. Com 8×989 TFLOPS e uns 50% de eficiência real: $1,2 / (8 \times 0,989 \times 0,5) \approx$ **0,3 s**.
- Já o KV cache gerado: 8.000×320 KiB $\approx$ **2,6 GB**. É isso que a desagregação precisa transferir.

Perguntas de acompanhamento prováveis

- **Por que não fazer só batches de prefill e só batches de decode, alternando?** Funciona, mas os decodes ficam parados durante cada prefill (picos de ITL), e os prefills esperam os decodes. Chunked prefill junta os dois e aproveita a capacidade ociosa de cálculo do decode para fazer prefill "de carona".
- **O prefill pode ser memory-bound?** Com prompts muito curtos, sim. E a atenção do prefill com contexto muito longo cresce quadraticamente e pode dominar o tempo.
- **Como o prefix caching muda isso?** Se o começo do prompt já está no cache (system prompt, histórico do chat), o prefill só processa o sufixo novo. O TTFT cai muito em chats multi-turno (Q043).
- **Qual fase domina o custo?** Depende da carga. Em chat, as saídas costumam ser bem mais curtas que as entradas acumuladas, mas cada token de saída é mais caro. Em resumo de documentos, o prefill domina. Em geração longa (código, raciocínio), o decode domina.

Armadilhas comuns

- Dizer só "prefill é o prompt e decode é a resposta", sem falar de gargalos.
- Não conectar as fases às métricas (TTFT e TPOT).
- Esquecer a interferência entre as fases e as soluções (chunked prefill, desagregação).
- Achar que o prefill é sempre rápido: com prompts de 100 mil tokens, ele leva segundos.

Resumo do capítulo

- Um LLM decoder-only é: embedding → N blocos (RMSNorm, atenção, residual, RMSNorm, MLP, residual) → norma final → LM head → sampling.
- No Llama 3 8B, o MLP tem ~80% dos parâmetros das camadas; embedding e LM head somam ~1 B.
- Atenção = softmax($QK^T/\sqrt{d}$ + máscara)·V. A máscara causal impede ver o futuro.
- **GQA** compartilha K e V entre grupos de heads de query e reduz o KV cache (8× no Llama 3 70B).
- **RoPE** gira Q e K pela posição; o produto escalar passa a depender da distância relativa. Existem duas convenções incompatíveis.
- **Prefill**: T tokens de uma vez, compute-bound, define o TTFT. **Decode**: 1 token por passo, memory-bound, define o TPOT.
- **KV cache**: 2 × camadas × kv_heads × head_dim × bytes por token. 128 KiB (8B) e 320 KiB (70B) em BF16.
- **FlashAttention**: atenção exata e IO-aware, com tiling e softmax online; nunca escreve a matriz T×T na HBM.
- Armadilhas do KV cache: máscara com deslocamento, padding, posições por sequência, RoPE aplicado uma vez, GQA sem expandir na memória, pré-alocação, beam search.

Checklist para a entrevista

- [ ] Desenho um bloco de decoder moderno de memória e explico cada parte.
- [ ] Calculo os parâmetros de um modelo a partir da configuração.
- [ ] Escrevo a fórmula da atenção e explico a máscara causal.
- [ ] Explico MHA, MQA, GQA e MLA e o efeito de cada um no KV cache.
- [ ] Explico RoPE e por que o produto escalar depende da distância relativa.
- [ ] Calculo o tamanho do KV cache por token e por conversa.
- [ ] Explico o softmax online e escrevo o pseudocódigo do FlashAttention.
- [ ] Listo as armadilhas de corretude do KV cache e como testar a implementação.
- [ ] Comparo prefill e decode em entrada, gargalo, métrica e kernel.
- [ ] Explico chunked prefill e desagregação de prefill e decode.

CAPÍTULO 3

Arquitetura de GPU e CUDA

Quase todo o tempo de um LLM em produção é gasto dentro de kernels de GPU. Para otimizar inferência, você precisa saber como a GPU executa código: como as threads são organizadas, onde os dados moram, por que um acesso à memória pode ser 10 vezes mais lento que outro feito de jeito diferente, e por que "mais threads" nem sempre significa "mais rápido".

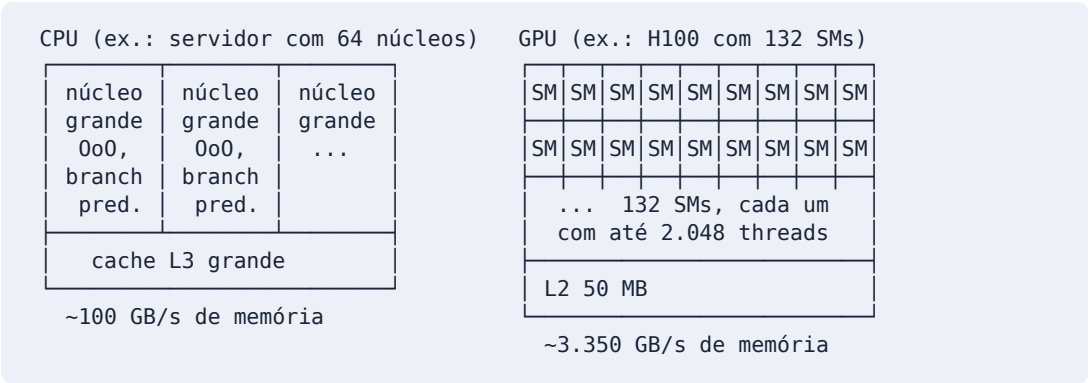
A NVIDIA, a AMD e qualquer empresa que escreve kernels testam isso diretamente: "explique o modelo de execução do CUDA", "o que é occupancy", "otimize esta multiplicação de matrizes". Este capítulo cobre a arquitetura e o modelo de programação. O Capítulo 4 cobre a metodologia de otimização, e o Capítulo 5 traz os kernels completos.

3.1 CPU vs GPU: duas filosofias

Uma CPU é feita para **latência**: poucos núcleos muito poderosos, com caches enormes, previsão de desvio e execução fora de ordem, tudo para terminar **uma** tarefa o mais rápido possível.

Uma GPU é feita para **throughput**: milhares de unidades simples que executam a mesma instrução em muitos dados ao mesmo tempo. Cada thread individual é lenta. A GPU compensa tendo tantas threads que sempre há alguma pronta para trabalhar enquanto outras esperam a memória.

> **Analogia:** a CPU é um carro esportivo que leva uma pessoa de São Paulo ao Rio em 4 horas. A GPU é uma frota de ônibus que leva 5.000 pessoas, cada uma demorando 6 horas. Para levar uma pessoa, o carro ganha. Para levar uma cidade inteira, os ônibus ganham de lavada. Multiplicar matrizes de bilhões de elementos é levar uma cidade inteira.



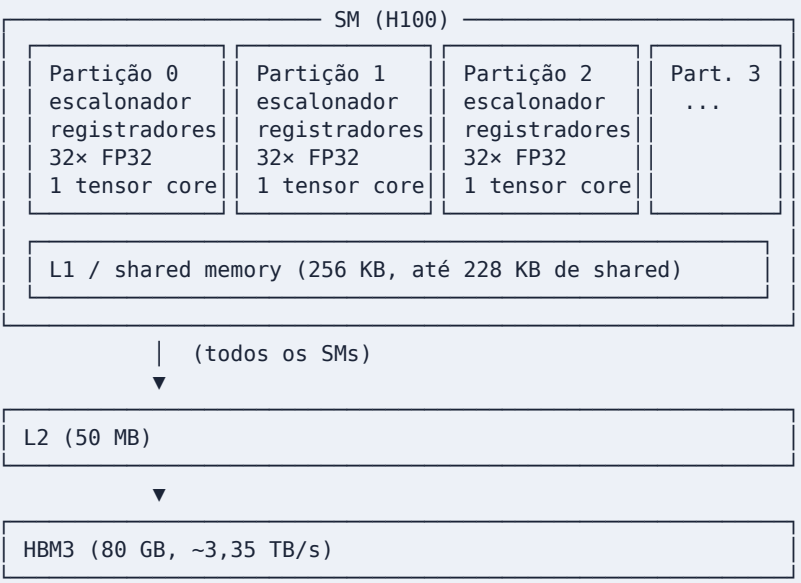
3.2 Anatomia de uma GPU NVIDIA (Hopper / H100)

A unidade básica é o **SM** (streaming multiprocessor). A H100 SXM tem **132 SMs**. Cada SM é dividido em **4 partições**, e cada partição tem:

- Um **escalonador de warps** que, a cada ciclo, escolhe um warp pronto e emite uma instrução dele.
- **32 unidades FP32** (os "CUDA cores"), unidades de inteiro e de FP64.
- **1 tensor core**, unidade especializada em multiplicação de matrizes pequenas.
- Um pedaço do **banco de registradores** (o SM inteiro tem 65.536 registradores de 32 bits, 256 KB).

As 4 partições compartilham:

- **Shared memory + L1**: 256 KB por SM, dos quais até **228 KB** podem ser shared memory.
- Unidades de load/store, de funções especiais (exp, sin...) e de textura.
- Na Hopper, a **TMA** (Tensor Memory Accelerator), que copia blocos de tensores da memória global para a shared memory de forma assíncrona.



Os números mudam de geração em geração, mas a estrutura (muitos SMs, warps, hierarquia de memória, tensor cores) se mantém desde a Volta (2017). As GPUs da AMD têm conceitos equivalentes com outros nomes: **CU** (compute unit) em vez de SM, **wavefront** de 64 (ou 32) threads em vez de warp, **LDS** em vez de shared memory e **matrix cores** em vez de tensor cores.

3.3 O modelo de programação: grid, blocos, threads, warps

Em CUDA, você escreve um **kernel**: uma função que roda em muitas threads ao mesmo tempo. Ao lançar, você define uma **grade** (grid) de **blocos**, cada bloco com várias **threads**:

```
// soma de vetores: cada thread cuida de um elemento
__global__ void soma(const float* a, const float* b,
                     float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {          // a última leva pode sobrar
        c[i] = a[i] + b[i];
    }
}

// no host (CPU):
int threads = 256;
int blocos = (n + threads - 1) / threads;  // arredonda p/ cima
soma<<<blocos, threads>>>(d_a, d_b, d_c, n);
```

A hierarquia lógica e como ela vira hardware:

```
LÓGICO (o que você programa)          FÍSICO (onde roda)
──────────────────────────────        ──────────────────────

Grid: todos os blocos                 GPU inteira
  │
  ├─ Bloco (até 1.024 threads)  ──►   roda inteiro num único SM;
  │   compartilha shared memory        vários blocos podem
  │   pode sincronizar                 dividir o mesmo SM
  │   (__syncthreads)
  │
  │   ├─ Warp (32 threads)      ──►   unidade de execução real:
  │   │   mesma instrução,            o escalonador emite uma
  │   │   ao mesmo tempo              instrução para os 32
  │   │
  │   │   └─ Thread             ──►   uma "faixa" (lane) do warp,
  │                                   com seus registradores
```

Regras que você precisa saber de cor:

- **Warp = 32 threads.** É a unidade que realmente executa. Blocos devem ter tamanho múltiplo de 32.
- **Blocos são independentes.** Não há ordem garantida entre blocos, e eles não sincronizam entre si dentro de um kernel (fora de recursos especiais como cooperative groups ou clusters na Hopper).
- **Máximo de 1.024 threads por bloco.**
- **Um SM roda vários blocos ao mesmo tempo**, até os limites de registradores, shared memory, warps (64 por SM na H100, ou seja, 2.048 threads) e blocos (32 por SM).

SIMT e divergência

O modelo se chama **SIMT** (single instruction, multiple threads). As 32 threads de um warp executam a mesma instrução. Se houver um `if` em que metade vai para um lado e metade para o outro, o warp executa **os dois caminhos**, desligando as threads que não participam de cada um. Isso é a **divergência de warp**.

```
if (threadIdx.x % 2 == 0) A(); else B();

tempo ──►
lanes pares:   [ A A A A ][ - - - - ]
lanes ímpares: [ - - - - ][ B B B B ]
               └ metade do warp ociosa em cada trecho ┘

if (threadIdx.x < 32) A(); else B();   (divide por warp)
warp 0: [ A A A A ]
warp 1: [ B B B B ]        sem divergência
```

Desde a Volta, cada thread tem seu próprio contador de programa (independent thread scheduling), o que permite padrões de sincronização mais flexíveis, mas não elimina o custo da divergência.

3.4 A hierarquia de memória

Este é o assunto mais importante do capítulo. Quase toda otimização de kernel é, no fundo, **mover dados para mais perto do cálculo e reutilizá-los o máximo possível**.

Nível	Tamanho (H100)	Escopo	Latência (ordem de grandeza)
Registradores	256 KB por SM	uma thread	~1 ciclo
Shared memory	até 228 KB por SM	um bloco	dezenas de ciclos
L1	(mesmo espaço físico da shared)	um SM	dezenas de ciclos
L2	50 MB	GPU inteira	centenas de ciclos
HBM (global)	80 GB	GPU inteira + host	centenas a ~mil ciclos

Os valores de latência são aproximados e variam com o acesso. O importante é a ordem de grandeza: **cada nível abaixo é muito mais lento e muito maior**.

> **Analogia:** registradores são o que você tem na mão. Shared memory é a bancada da cozinha, compartilhada pela equipe daquela estação. L2 é a despensa do restaurante. HBM é o depósito do outro lado da rua. Um bom cozinheiro busca uma caixa inteira do depósito de uma vez, deixa na bancada e usa muitas vezes, em vez de atravessar a rua para cada cebola.

Memória global e coalescing

Quando um warp lê da memória global, o hardware junta os 32 endereços e busca os **setores** de 32 bytes necessários. Se as 32 threads leem 32 floats **consecutivos** (128 bytes alinhados), isso vira 4 setores, a eficiência máxima. Isso é um acesso **coalescido**.

```
Coalescido: thread i lê a[i]
threads:  0   1   2   3  ...  31
endereço: |0 |4 |8 |12| ... |124|   → 128 bytes, 4 setores
          └──────────────────────┘     100% dos bytes usados

Com passo (stride) 32: thread i lê a[i*32]
threads:  0     1     2    ...   31
endereço: 0   128   256   ...  3968  → 32 setores diferentes
          cada setor de 32 B traz 4 B úteis: 12,5% de eficiência
```

Um caso clássico em GEMM: em C/C++ as matrizes são guardadas por linha (row-major). Se cada thread de um warp percorrer uma **coluna** diferente lendo elementos em linhas diferentes, os acessos ficam separados por uma linha inteira: nada coalescido. A solução é organizar quem lê o quê para que threads vizinhas leiam endereços vizinhos, e transpor ou reorganizar na shared memory.

Shared memory e bank conflicts

A shared memory é dividida em **32 bancos** de 4 bytes. Endereços consecutivos de 4 bytes ficam em bancos consecutivos. Se as 32 threads de um warp acessam 32 bancos diferentes, a leitura sai num ciclo. Se duas threads acessam **endereços diferentes no mesmo banco**, há um **conflito de banco** e os acessos são serializados. (Se várias threads leem o **mesmo** endereço, é broadcast e não há conflito.)

```
float tile[32][32];   // cada linha tem 32 floats = 32 bancos
Ler uma COLUNA: thread i lê tile[i][0]
  endereços: 0, 32, 64, ... (em floats) → todos no banco 0
  → conflito de 32 vias: 32× mais lento

float tile[32][33];   // +1 de padding por linha
  endereços: 0, 33, 66, ... → bancos 0, 1, 2, ... 31
  → sem conflito
```

Esse truque do **padding** (`[32][33]`) aparece em quase toda entrevista que envolve transposição ou GEMM com shared memory.

Registradores

São a memória mais rápida e são privados de cada thread. Os kernels mais rápidos de GEMM fazem cada thread calcular um **pedaço** da saída (por exemplo, 8×8 elementos) mantido inteiro em registradores. Mas cada SM tem um número fixo de registradores, e isso leva ao próximo conceito.

3.5 Occupancy

Occupancy é a razão entre os warps ativos num SM e o máximo que ele suporta (64 na H100). Ela importa porque a GPU esconde a latência da memória **trocando de warp**: enquanto um warp espera dados da HBM, o escalonador emite instruções de outro. Com poucos warps residentes, pode não haver ninguém pronto para trabalhar.

O que limita a occupancy:

1. **Registradores por thread.** 65.536 registradores por SM. Se cada thread usa 128, cabem 512 threads = 16 warps = **25%.**
2. **Shared memory por bloco.** Se cada bloco usa 100 KB, cabem 2 blocos por SM (228 KB).
3. **Threads por bloco** e o limite de blocos por SM.

> **Fazendo a conta:** kernel com 256 threads por bloco, 64 registradores por thread e 48 KB de shared memory por bloco, na H100.
>
> - Registradores por bloco: 256 × 64 = 16.384 → cabem 65.536 / 16.384 = **4 blocos**.
> - Shared memory: 228 / 48 = **4 blocos** (arredondando para baixo).
> - Warps: 4 blocos × 8 warps = 32 warps de 64 possíveis.
> - Occupancy: **50%**.
>
> Se você reduzir para 32 registradores por thread, os registradores deixam de ser o limite, mas a shared memory continua limitando em 4 blocos. Para subir, teria que reduzir a shared memory também.

> **Cuidado:** occupancy alta **não** é o objetivo final. Os kernels de GEMM mais rápidos rodam com occupancy baixa (às vezes 1 ou 2 warps por partição), porque usam muitos registradores para reaproveitar dados e escondem a latência com **paralelismo em nível de instrução** (ILP): cada thread tem várias leituras e multiplicações independentes em andamento ao mesmo tempo. Vasily Volkov mostrou isso em “Better Performance at Lower Occupancy” (2010). Occupancy é uma ferramenta para esconder latência, não uma métrica de sucesso.

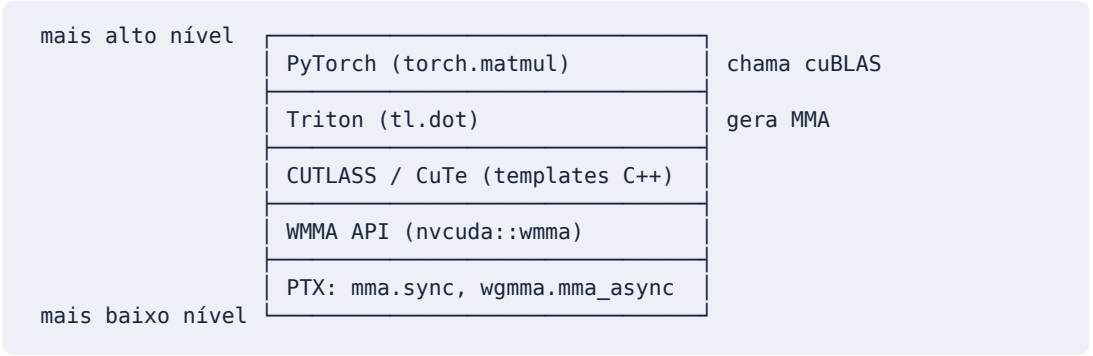
3.6 Tensor cores

Os **tensor cores** fazem uma operação só: multiplicação e acumulação de matrizes pequenas, $D = A \times B + C$, em um tile por instrução. Nas GPUs recentes:

- Volta e Turing: FP16.
- Ampere (A100): + BF16, TF32, INT8, INT4 e esparsidade estruturada 2:4.
- Hopper (H100): + FP8 (E4M3 e E5M2), instruções **WGMMA** que operam num grupo de 4 warps (128 threads) e leem operandos direto da shared memory.
- Blackwell (B200): + FP4 e FP6.

A diferença de throughput é enorme: na H100, o pico FP32 nos CUDA cores comuns é de ~67 TFLOPS, enquanto os tensor cores fazem ~989 TFLOPS em BF16 denso. Por isso, **quem não usa tensor cores para GEMMs está usando uma fração pequena da GPU**.

Você raramente programa tensor cores na mão. As camadas comuns:



O Capítulo 4 (Q027) mostra como chegar perto do pico dos tensor cores.

3.7 Lei de Amdahl

A lei de Amdahl diz quanto um sistema inteiro acelera quando você acelera só uma parte dele. Se uma fração p do tempo é acelerada s vezes:

$$\text{speedup total} = \frac{1}{(1-p) + \frac{p}{s}}$$

Mesmo com s infinito, o speedup máximo é $1/(1-p)$: a parte que você não otimizou vira o teto.

```
Antes:   [████████ GEMM 70% ████████][██ resto 30% ██]
GEMM 10× mais rápida:
Depois:  [█ 7% █][██ resto 30% ██]   total = 37% do original
         speedup = 1 / 0,37 ≈ 2,7×  (não 10×)
```

Isso vale em todos os níveis: um kernel dentro de uma camada, uma camada dentro de um passo, um passo dentro de uma requisição, uma requisição dentro de um fluxo de produto. A Q018 aprofunda.

3.8 Multiplicação de matrizes: a progressão de otimizações

GEMM (general matrix multiply, $C = A \cdot B$) é a operação mais importante em LLMs. Entender como otimizá-la passo a passo é a base para entender qualquer kernel.

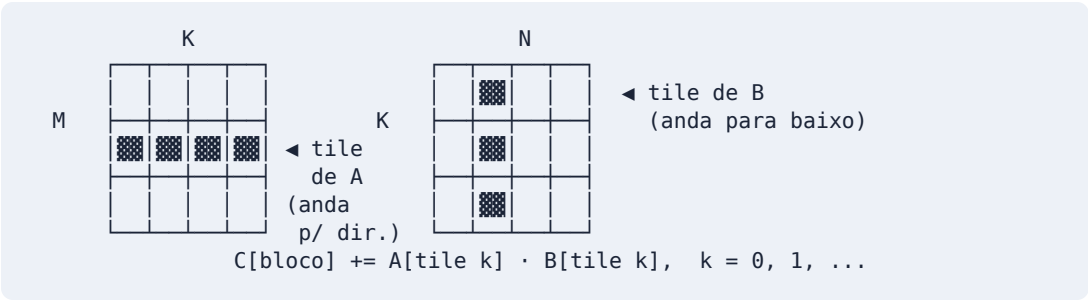
Seja A de tamanho $M \times K$, B de $K \times N$ e C de $M \times N$. São $2MNK$ FLOPs.

Nível 0 — ingênuo. Cada thread calcula um elemento de C, lendo uma linha de A e uma coluna de B da memória global:

```
__global__ void gemm_ingenua(const float* A, const float* B,
                             float* C, int M, int N, int K) {
    int lin = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (lin < M && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k)
            acc += A[lin * K + k] * B[k * N + col];
        C[lin * N + col] = acc;
    }
}
```

Cada multiplicação-soma (2 FLOPs) lê 2 floats (8 bytes): **0,25 FLOP/byte**, muito abaixo do ponto de equilíbrio. O L1 e o L2 ajudam um pouco, porque threads vizinhas leem os mesmos dados, mas esse kernel costuma ficar numa fração pequena do pico.

Nível 1 — tiling com shared memory. Cada bloco calcula um tile de C (por exemplo, 32×32). Ele carrega um tile de A e um de B para a shared memory, todas as threads do bloco usam esses dados, e o bloco avança no eixo K:



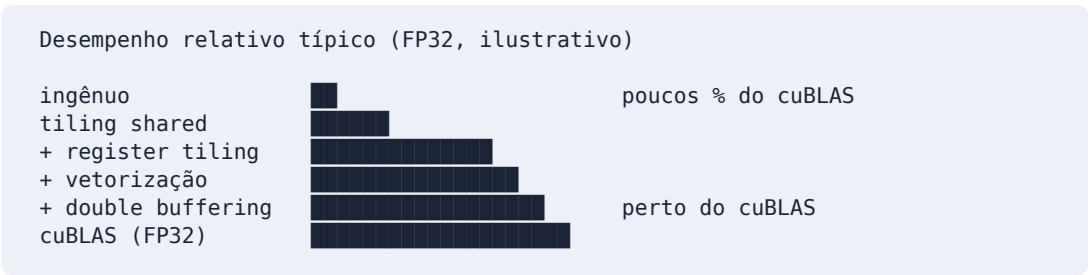
Com tile $T \times T$, cada elemento carregado da memória global é reutilizado T vezes. A intensidade sobe em um fator de aproximadamente T.

Nível 2 — tiling em registradores (register blocking). Cada thread calcula um micro-tile (ex.: 8×8) de C em registradores. Para cada passo em k, ela carrega 8 valores de A e 8 de B da shared memory e faz 64 multiplicações-somas. Reuso dentro da thread, menos leituras da shared memory.

Nível 3 — leituras vetorizadas e layout. Ler `float4` (16 bytes) por instrução, transpor A na shared memory para leituras sem conflito de banco, alinhar tudo.

Nível 4 — pipeline (double buffering). Enquanto o bloco calcula com o tile atual, já carrega o próximo (cópias assíncronas `cp.async` na Ampere, TMA na Hopper). O cálculo esconde a latência da memória.

Nível 5 — tensor cores. Trocar as multiplicações escalares por instruções MMA. Isso muda a ordem de grandeza do pico disponível.



As proporções exatas dependem da GPU e do tamanho da matriz. Mas a ordem das otimizações e o porquê de cada uma é exatamente o que o entrevistador quer ouvir. O código completo de cada nível está no Capítulo 5 (Q033).

Q017 Explique o modelo de execução do CUDA, a hierarquia de memória e a ocupação (occupancy). Depois, otimize uma GEMM com tiling e coalescing.

> **[Relatado]** NVIDIA · vaga de software de GPU · triagem · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

"Um kernel roda como uma grade de blocos; cada bloco vai inteiro para um SM e é dividido em warps de 32 threads que executam a mesma instrução (SIMT). A memória vai de registradores (por thread) a shared memory (por bloco), L2 (global) e HBM. Occupancy é a fração de warps ativos por SM, limitada por registradores, shared memory e tamanho do bloco, e serve para esconder latência. Para a GEMM: começo com uma thread por elemento, que tem intensidade de 0,25 FLOP/byte; faço tiling com shared memory para reaproveitar cada elemento T vezes; garanto que threads vizinhas leiam endereços vizinhos ao carregar os tiles (coalescing); depois uso micro-tiles em registradores, leituras vetorizadas, double buffering e, se a precisão permitir, tensor cores."

Resposta completa

Divida em duas partes e use diagramas.

Parte 1 — conceitos. Use as seções 3.3 a 3.5: grid → bloco → warp → thread; SIMT e divergência; a tabela da hierarquia de memória; occupancy e seus limites, com uma conta rápida. Diga explicitamente que occupancy alta não é o objetivo, e sim esconder latência.

Parte 2 — a GEMM com tiling. Escreva o kernel com shared memory. É o código mais pedido em entrevistas de GPU:

```
#define TILE 32

__global__ void gemm_tiled(const float* A, const float* B,
                           float* C, int M, int N, int K) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE + 1];  // padding: sem conflito

    int tx = threadIdx.x, ty = threadIdx.y;
    int lin = blockIdx.y * TILE + ty;     // linha de C
    int col = blockIdx.x * TILE + tx;     // coluna de C
    float acc = 0.0f;

    for (int k0 = 0; k0 < K; k0 += TILE) {
        // Carga coalescida: threads com tx vizinhos leem
        // endereços vizinhos (mesma linha, colunas seguidas).
        As[ty][tx] = (lin < M && k0 + tx < K)
                   ? A[lin * K + k0 + tx] : 0.0f;
        Bs[ty][tx] = (k0 + ty < K && col < N)
                   ? B[(k0 + ty) * N + col] : 0.0f;
        __syncthreads();          // tiles prontos para todos

        #pragma unroll
        for (int k = 0; k < TILE; ++k)
            acc += As[ty][k] * Bs[k][tx];
        __syncthreads();          // antes de sobrescrever
    }
    if (lin < M && col < N) C[lin * N + col] = acc;
}
// lançamento: dim3 bloco(TILE, TILE);
// dim3 grade((N + TILE - 1) / TILE, (M + TILE - 1) / TILE);
```

Explique cada decisão:

- **Coalescing**: `threadIdx.x` varia mais rápido dentro do warp. Por isso ele indexa a **coluna** na leitura de `A` (dentro de uma linha) e de `B`. As 32 threads de um warp leem 32 floats seguidos.
- **Dois `__syncthreads()`**: o primeiro garante que o tile está completo antes do cálculo; o segundo, que ninguém sobrescreve o tile enquanto outras threads ainda o usam.
- **Bordas**: matrizes que não são múltiplas de 32 recebem zero fora dos limites.
- **Padding**: no laço interno, `As[ty][k]` é o mesmo endereço para todas as threads do warp (broadcast), e `Bs[k][tx]` lê endereços consecutivos de uma linha (bancos diferentes). Com esse padrão específico o padding não é estritamente necessário, mas ele evita conflitos se você mudar o padrão de acesso, como ao transpor um tile.
- **Reuso**: cada elemento carregado da memória global é usado 32 vezes. A intensidade passa de 0,25 para cerca de 8 FLOP/byte.

Depois, diga qual seria o próximo passo e por quê: “Esse kernel ainda faz duas leituras de shared memory por multiplicação-soma. O próximo nível é cada thread calcular um micro-tile de 4×4 ou 8×8 em registradores, o que divide as leituras de shared memory por 4 a 8.”

> **Dica de entrevista:** antes de escrever código, faça a conta de intensidade aritmética de cada versão. Mostra que você otimiza com base em um modelo, não por tentativa e erro.

Perguntas de acompanhamento prováveis

- **Por que 32×32 e não 64×64?** Um bloco de 64×64 threads teria 4.096 threads, acima do limite de 1.024. Para tiles maiores, cada thread precisa calcular vários elementos (register tiling). Tiles maiores aumentam o reuso, mas consomem mais shared memory e registradores e reduzem a occupancy.
- **Como você verificaria se os acessos estão coalescidos?** No Nsight Compute, olhando os setores por requisição na seção de memória: em acessos de 4 bytes perfeitamente coalescidos, são 4 setores por requisição de warp. Valores maiores indicam desperdício.
- **O que é `__syncthreads()` dentro de um `if`?** Um bug: se só parte das threads do bloco chega ao barrier, o comportamento é indefinido (normalmente trava).
- **Quando a GEMM fica memory-bound mesmo com tiling?** Quando uma dimensão é pequena, como no decode (M = tamanho do batch, às vezes 1 a 16). Aí não há reuso suficiente de B, e o kernel vira, na prática, uma GEMV limitada pela leitura dos pesos.

Armadilhas comuns

- Esquecer o segundo `__syncthreads()`.
- Indexar com `threadIdx.y` na dimensão contígua e destruir o coalescing.
- Confundir occupancy com desempenho.
- Não tratar bordas quando M, N, K não são múltiplos do tile.

Q018 Explique a lei de Amdahl e como ela se aplica à multiplicação de matrizes em GPU e

aos ganhos de velocidade de ponta a ponta.

> **[Relatado]** NVIDIA · Software Engineer · rodada de fundamentos · listada em 2026 · Médio · fonte

Resposta em 30 segundos

“Amdahl diz que, se uma fração p do tempo fica s vezes mais rápida, o speedup total é 1/((1−p) + p/s), e o teto é 1/(1−p). Numa GEMM em GPU, a parte paralela é o cálculo dentro do kernel, mas há partes que não aceleram: cópias CPU↔GPU, lançamento do kernel, as bordas e a fase final em que poucos blocos sobram para poucos SMs. De ponta a ponta, se as GEMMs são 60% do passo de decode e eu as deixo 2× mais rápidas, o passo fica só 1,43× mais rápido. Por isso eu meço o perfil inteiro antes de escolher o que otimizar.”

Resposta completa

A fórmula e o teto. Mostre a fórmula da seção 3.7 e o caso limite: com $p = 0,9$, mesmo com $s = \infty$, o máximo é 10×.

Aplicação 1 — dentro de um kernel de GEMM. Mesmo um kernel “100% paralelo” tem partes seriais ou mal paralelizadas:

```
Tempo de uma GEMM pequena na GPU
├── lançamento do kernel (CPU → GPU)       alguns µs
├── "rampa" de início (blocos subindo)
├── corpo: todos os SMs ocupados           ◀ a parte que
│                                            escala
└── "cauda" (tail effect): última leva
    de blocos ocupa só parte dos SMs

Ex.: 140 blocos em 132 SMs → 1ª leva com 132 blocos,
2ª leva com só 8 → a 2ª leva demora o mesmo que a 1ª
com 94% dos SMs parados (efeito de quantização das ondas)
```

O efeito de **quantização de ondas** (wave quantization) é um exemplo direto de Amdahl: aumentar o número de SMs não acelera a segunda onda. Soluções: tiles de outro tamanho, **split-K** (dividir o eixo K entre blocos e somar no fim) ou kernels persistentes com escalonamento por **stream-K**.

Aplicação 2 — GEMM dentro do programa. Se a matriz está na CPU, você precisa copiar para a GPU pelo PCIe (~64 GB/s por sentido no Gen5, contra ~3.350 GB/s da HBM). Para uma GEMM de 4096×4096×4096 em FP32: 3 matrizes × 64 MB = 192 MB de cópias ≈ 3 ms no PCIe. O cálculo, 137 GFLOP, leva uns 2 ms em FP32 nos CUDA cores

de uma H100 e bem menos em TF32 nos tensor cores. A cópia domina, e otimizar o kernel quase não muda o total. Solução: manter dados na GPU, sobrepor cópias e cálculo com streams.

Aplicação 3 — ponta a ponta na inferência.

Fazendo a conta: perfil hipotético de um passo de decode:

Parte	% do tempo
GEMMs (QKV, saída, MLP, LM head)	60%
Atenção (leitura do KV cache)	25%
Normas, ativações, RoPE, sampling	10%
Overhead de CPU / lançamento	5%

- GEMMs 2× mais rápidas (quantização FP8): 1 / (0,4 + 0,3) = **1,43×**.
- Atenção 2× mais rápida: 1 / (0,75 + 0,125) = **1,14×**.
- As duas: 1 / (0,15 + 0,3 + 0,125) = **1,74×**.
- Depois disso, os 15% de normas e overhead viram 26% do novo tempo. Fusão de kernels e CUDA graphs passam a ser a melhor alavanca.

Aplicação 4 — no sistema. O TTFT inclui fila, tokenização, rede e prefill. Se a fila é 70% do TTFT p99, um prefill 2× mais rápido melhora pouco o p99; o que resolve é capacidade ou scheduling.

Gustafson como contraponto. A lei de Gustafson lembra que, com mais recursos, normalmente aumentamos o tamanho do problema (batch maior, contexto maior), e a fração paralela cresce. É por isso que GPUs maiores continuam valendo a pena para cargas que escalam.

Perguntas de acompanhamento prováveis

- **Como você descobre o p de cada parte?** Com um profiler de linha do tempo (Nsight Systems, `torch.profiler`), somando o tempo por categoria de kernel e os intervalos em que a GPU fica ociosa.
- **Qual a relação entre Amdahl e o overhead de lançamento?** Em decode com batch pequeno, os kernels são curtos e o tempo de CPU para lançá-los vira uma fração serial grande. CUDA graphs reduzem essa fração.

- **Amdahl vale para paralelismo entre GPUs?** Sim. Com tensor parallelism, as comunicações (all-reduce) não diminuem com mais GPUs e podem até crescer; elas viram o teto do speedup.

Armadilhas comuns

- Prometer speedup total igual ao speedup do kernel.
- Esquecer cópias e lançamento.
- Não saber aplicar a fórmula com números na hora.
- Ignorar o efeito de cauda/ondas em GPUs com muitos SMs.

Q019 Como você calcularia uma multiplicação de matrizes de forma eficiente numa GPU sem usar memória compartilhada?

> **[Relatado]** NVIDIA · GPU Architect · entrevista · relato antigo, ainda comum · Difícil · fonte

Resposta em 30 segundos

"Sem shared memory, o reuso tem que vir dos registradores e dos caches. Cada thread calcula um micro-tile da saída, por exemplo 8×8, mantido em registradores; a cada passo em K ela carrega 8 valores de A e 8 de B e faz 64 multiplicações-somas, o que dá 8 vezes mais reuso que uma thread por elemento. Organizo as threads para que as leituras sejam coalescidas e para que threads vizinhas leiam os mesmos dados ao mesmo tempo, aproveitando L1 e L2. Também dá para trocar dados entre threads do warp com warp shuffle, que funciona como uma shared memory de warp sem usar a shared memory. E os tensor cores, via mma.sync, operam direto em registradores."

Resposta completa

O entrevistador quer ver se você entende **por que** a shared memory é usada (reuso de dados entre threads) e se conhece as alternativas. São quatro.

1. Register tiling (o principal). Cada thread acumula um bloco $R_M \times R_N$ de C em registradores:

```
// cada thread calcula um micro-tile 4x4 de C
// (versão didática; supõe M e N múltiplos de 4 * dim)
__global__ void gemm_registradores(const float* A,
        const float* B, float* C, int M, int N, int K) {
    int lin0 = (blockIdx.y * blockDim.y + threadIdx.y) * 4;
    int col0 = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
    float acc[4][4] = {};
    for (int k = 0; k < K; ++k) {
        float a[4], b[4];
        #pragma unroll
        for (int i = 0; i < 4; ++i)
            a[i] = __ldg(&A[(lin0 + i) * K + k]);
        #pragma unroll
        for (int j = 0; j < 4; ++j)
            b[j] = __ldg(&B[k * N + col0 + j]);
        #pragma unroll
        for (int i = 0; i < 4; ++i)
            #pragma unroll
            for (int j = 0; j < 4; ++j)
                acc[i][j] += a[i] * b[j];   // 16 FMAs
    }
    for (int i = 0; i < 4; ++i)
        for (int j = 0; j < 4; ++j)
            C[(lin0 + i) * N + col0 + j] = acc[i][j];
}
```

Reuso: 8 leituras para 16 FMAs (2 por leitura), contra 2 leituras por FMA no kernel ingênuo. Com 8×8: 16 leituras para 64 FMAs.

2. Caches L1 e L2. Threads de um mesmo warp e de blocos vizinhos leem os mesmos elementos de A e B. Com uma boa ordem de execução dos blocos, esses dados ficam no L1 (por SM) e no L2 (50 MB na H100) e são reaproveitados sem voltar à HBM. Truques:

- `__ldg()` ou `const __restrict__` para usar o caminho de leitura somente-leitura.
- **Rasterização em swizzle**: lançar os blocos numa ordem que agrupa tiles vizinhos de C (que compartilham linhas de A e colunas de B) para aumentar o acerto no L2.

3. Warp shuffle. As instruções `__shfl_sync` deixam uma thread ler um registrador de outra thread do mesmo warp, sem passar pela memória. Uma thread carrega um valor de B e o distribui para as outras 31:

```
// cada lane carrega um elemento de B; todas usam todos
float b_meu = B[k * N + col_base + lane];
#pragma unroll
for (int j = 0; j < 32; ++j) {
    float b_j = __shfl_sync(0xffffffff, b_meu, j);
    acc[j] += a * b_j;   // (acc[] em registradores)
}
```

Isso é uma “shared memory de warp”, com custo de uma instrução por troca.

4. Tensor cores direto dos registradores. A instrução `mma.sync` (Ampere) recebe os operandos em registradores, distribuídos pelas 32 threads do warp. Dá para alimentar os tensor cores carregando os fragmentos direto da memória global, com acessos bem organizados. Os kernels mais rápidos ainda usam shared memory para reorganizar os dados, mas é possível.

Comparação honesta. Sem shared memory, você perde o reuso **entre warps** de um bloco e depende dos caches, que não são controlados por você. Para matrizes grandes, a versão com shared memory (ou TMA na Hopper) vence. Para matrizes **pequenas ou finas** (como no decode, com M pequeno), uma versão só com registradores e shuffle pode ser tão boa quanto ou melhor, porque o problema é memory-bound de qualquer jeito e evita o overhead de sincronização.

Perguntas de acompanhamento prováveis

- **Qual o limite do register tiling?** Registradores por thread (máximo de 255). Micro-tiles grandes usam muitos registradores, reduzem a occupancy e podem causar spilling (registradores derramados para a memória local, que é lenta).
- **Como o compilador sabe deixar `acc[4][4]` em registradores?** Os índices precisam ser constantes em tempo de compilação (laços com `#pragma unroll`). Indexação dinâmica força o array para a memória local.
- **O L1 e a shared memory não são a mesma memória física?** São, na maioria das gerações recentes; a divisão é configurável. Usar shared memory é controlar explicitamente o que fica ali, em vez de deixar o cache decidir.

Armadilhas comuns

- Responder “não dá para ser eficiente”. Dá, com registradores, caches e shuffle.
- Esquecer o coalescing ao ler direto da memória global.
- Usar arrays indexados dinamicamente e causar spilling.
- Não discutir quando essa abordagem é de fato competitiva (matrizes finas, memory-bound).

Q020 Fale de um kernel CUDA que você escreveu: as otimizações, os desafios e exatamente o que você fez.

> **[Relatado]** NVIDIA · Deep Learning Software Engineer, Inference · triagem com o gestor da vaga · 2025 · Médio · fonte

Resposta em 30 segundos

"Escrevi um kernel fundido de RMSNorm com quantização FP8 para a entrada das GEMMs do decode. Antes eram dois kernels, cada um lendo e escrevendo o tensor inteiro na HBM. Medindo, vi que os dois juntos eram 9% do passo de decode e estavam a uns 40% da banda de pico. Fundi os dois, fiz uma redução por warp com shuffle, usei leituras vetorizadas de 16 bytes e um bloco por linha. Chegou a uns 85% da banda e o passo ficou 6% mais rápido. O desafio maior foi a corretude numérica do fator de escala do FP8, que validei contra a referência em PyTorch."

Resposta completa

Assim como a Q001, esta é uma pergunta de história. Mas aqui o gestor quer **detalhe técnico real**: ele vai fazer perguntas até o nível de registradores e instruções para ver se foi você mesmo que escreveu. Prepare uma história que você domine por completo.

Estrutura recomendada:

```
1. O QUE o kernel faz (1 frase) e POR QUE ele importava
   (que % do tempo, qual métrica de negócio)
2. COMO você mediu o ponto de partida (profiler, % da banda
   ou do pico de FLOPS, roofline)
3. A HIPÓTESE (memory-bound? compute-bound? latência?)
4. As OTIMIZAÇÕES, em ordem, com o ganho de cada uma
5. O DESAFIO mais difícil (bug, corretude numérica,
   caso de borda) e como resolveu
6. O RESULTADO (kernel e ponta a ponta) e o que faria
   a seguir
```

Exemplo de história completa (um modelo para você adaptar; use a sua experiência real):

> "**Contexto.** No nosso serviço de decode, depois de passar as GEMMs para FP8, o perfil mostrou que RMSNorm seguida da quantização para FP8 da entrada da próxima GEMM somava uns 9% do tempo do passo.

Eram dois kernels separados: a RMSNorm lia o tensor em BF16 e escrevia em BF16; a quantização lia de novo, calculava o máximo absoluto e escrevia em FP8.

Medição. No Nsight Compute, os dois estavam em uns 40% da banda de pico da HBM, e não havia motivo para serem outra coisa além de memory-bound: poucos FLOPs por byte.

Hipótese. O desperdício era duplo: tráfego extra (o tensor intermediário em BF16 ia e voltava da HBM) e uma implementação ruim de redução (um bloco por linha, mas usando shared memory com muitos `__syncthreads`).

Otimizações. (1) Fundi os dois em um kernel: lê BF16 uma vez, escreve FP8 uma vez. Isso cortou o tráfego em pouco mais de metade. (2) Para a soma dos quadrados, fiz a redução dentro do warp com `__shfl_xor_sync` e só um passo pela shared memory para combinar os warps. (3) Leituras de 16 bytes (8 valores BF16 por instrução). (4) Para a linha de 4.096 elementos, usei 512 threads por linha, cada uma com 8 elementos em registradores, então os dados eram lidos da HBM uma única vez, mesmo eu precisando deles duas vezes (soma dos quadrados e normalização).

Desafio. O fator de escala do FP8. Na primeira versão, eu usava um fator por tensor, calculado num kernel anterior, e havia saturação em algumas camadas com outliers. Passei a usar um fator por linha (por token), calculado no próprio kernel depois da normalização, o que exigiu uma segunda redução (máximo absoluto) na mesma passada. Validei comparando com uma referência em PyTorch em FP32: erro máximo dentro do esperado para FP8, e a avaliação de qualidade do modelo ficou igual.

Resultado. O kernel fundido chegou a uns 85% da banda de pico. O tempo das duas operações caiu de 9% para uns 3% do passo, e o passo ficou uns 6% mais rápido, o que, pela lei de Amdahl, era o esperado. O próximo passo seria fundir isso direto no prólogo da GEMM, eliminando a escrita em FP8.”

Por que essa história funciona: tem números, um modelo mental (memory-bound → reduzir bytes), uma sequência de decisões justificadas, um desafio real de corretude e uma conexão com o ganho de ponta a ponta.

Perguntas de acompanhamento prováveis

- **Por que 512 threads por linha?** Para que cada thread ficasse com um número pequeno de elementos em registradores e para ter warps suficientes para esconder latência. Esteja pronto para justificar o seu número com uma conta de occupancy (3.5).
- **Como você testou a corretude?** Comparação com referência de alta precisão, tolerâncias definidas por formato, testes com tamanhos de borda (linhas não múltiplas de 8, valores extremos, zeros, NaN).
- **O que o Nsight mostrou depois?** Throughput de memória perto do pico; se não estiver, explique o que limitava (por exemplo, latência por falta de bytes em voo).
- **Por que não usar uma biblioteca?** Às vezes não existe a fusão exata de que você precisa. Mostre que você verificou as opções antes de escrever código próprio.

Armadilhas comuns

- Contar uma história que você não domina até o fim. O gestor vai cavar.
- Falar só do que funcionou. Os desafios mostram mais senioridade.
- Não ter números antes e depois.
- Não conectar o ganho do kernel ao ganho de ponta a ponta (Amdahl).

Resumo do capítulo

- A GPU é uma máquina de throughput: muitos SMs, cada um com milhares de threads residentes, escondendo a latência da memória trocando de warp.
- Hierarquia lógica: grid → bloco (um SM, shared memory, `__syncthreads`) → warp (32 threads, SIMT) → thread.
- Hierarquia de memória: registradores → shared/L1 → L2 → HBM. Cada nível abaixo é muito maior e muito mais lento.
- **Coalescing**: threads vizinhas lendo endereços vizinhos. **Bank conflicts**: threads acessando endereços diferentes no mesmo banco da shared memory; padding resolve.
- **Occupancy** = warps ativos / máximo; limitada por registradores, shared memory e tamanho do bloco. Serve para esconder latência; não é o objetivo final.
- Tensor cores dão uma ordem de grandeza a mais de FLOPS para GEMMs (~989 TFLOPS BF16 vs ~67 TFLOPS FP32 na H100).
- **Amdahl**: speedup = 1 / ((1 − p) + p/s). A parte não otimizada vira o teto.
- GEMM: ingênua (0,25 FLOP/byte) → tiling em shared → tiling em registradores → vetorização → double buffering → tensor cores.

Checklist para a entrevista

- [ ] Explico grid, bloco, warp e thread e como viram hardware.
- [ ] Explico divergência de warp com um exemplo.
- [ ] Recito a hierarquia de memória com tamanhos e latências aproximadas.
- [ ] Mostro um acesso coalescido e um não coalescido.
- [ ] Explico bank conflicts e o truque do padding.
- [ ] Calculo a occupancy de um kernel a partir de registradores e shared memory.
- [ ] Escrevo de memória a GEMM com tiling em shared memory, com os dois `__syncthreads`.
- [ ] Explico como fazer GEMM eficiente sem shared memory.
- [ ] Aplico Amdahl com números a um passo de decode.
- [ ] Tenho uma história de kernel com medição, hipótese, otimizações, desafio e resultado.

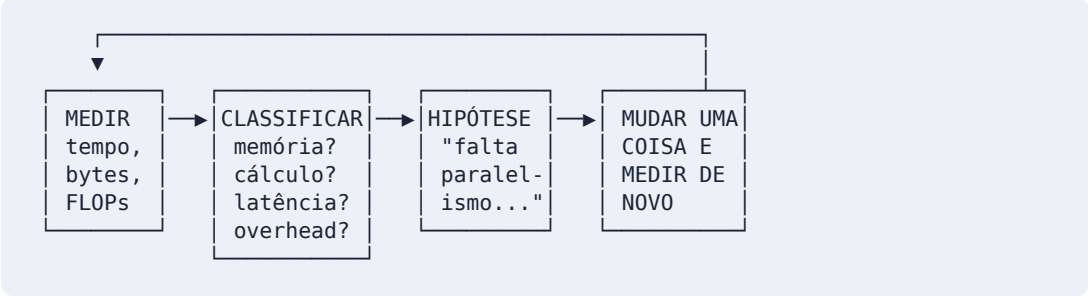
CAPÍTULO 4

Otimização de Desempenho em GPU

O capítulo anterior explicou como a GPU funciona. Este explica como descobrir **por que** um código está lento e o que fazer a respeito. É o tipo de pergunta em que o entrevistador dá um sintoma (“o kernel só usa 30% da banda”, “o bloco menor ficou mais lento”) e observa seu raciocínio. Ninguém espera que você acerte a causa de primeira. Esperam que você tenha um **método**.

4.1 O método

Toda investigação de desempenho segue o mesmo ciclo:



As quatro classes de gargalo:

Classe	Sintoma	Exemplo
Memory-bound	throughput de memória perto do pico, SMs ociosos	decode, normas, ativações
Compute-bound	tensor cores / ALUs perto do pico	GEMM grande no prefill
Latency-bound	nem memória nem cálculo perto do pico; warps esperando	pouco paralelismo, dependências
Overhead-bound	GPU ociosa entre kernels	kernels curtos, CPU lenta, Python

A terceira classe é a mais traiçoeira: o kernel não está usando nem a memória nem o cálculo, porque não tem trabalho independente suficiente em andamento para esconder a latência. A quarta é a mais comum em inferência com batch pequeno.

4.2 Roofline a fundo: MFU e MBU

O roofline do Capítulo 1 dá o limite teórico. Para saber quão perto você está dele, use duas métricas:

$$\text{MFU} = \frac{\text{FLOPs úteis por segundo atingidos}}{\text{pico de FLOPS}}$$

$$\text{MBU} = \frac{\text{bytes necessários por segundo atingidos}}{\text{pico de banda}}$$

- **MFU** (model FLOPs utilization) é a métrica de quem está compute-bound: treino e prefill.
- **MBU** (model bandwidth utilization) é a métrica de quem está memory-bound: decode.

> **Fazendo a conta: MBU de um decode.** Llama 3 8B em BF16 numa H100, batch 16, contexto médio de 2.000 tokens, passo medido de 7 ms.
>
> - Pesos: 16 GB.
> - KV cache lido: 16 sequências × 2.000 tokens × 128 KiB ≈ 4,2 GB.
> - Bytes por passo: ~20,2 GB.
> - Banda atingida: 20,2 GB / 7 ms ≈ 2,9 TB/s.
> - MBU = 2,9 / 3,35 ≈ **86%**. Muito bom. Ganhos grandes agora só vêm de ler menos bytes (quantização, cache menor), não de kernels melhores.
>
> Se o passo medido fosse 15 ms, o MBU seria ~40%, e haveria muito espaço para otimizar kernels e overhead.

Também existe o **roofline hierárquico**: um telhado para cada nível de memória (HBM, L2, shared memory). Um kernel pode estar longe do limite da HBM porque está limitado pela banda da L2 ou da shared memory. O Nsight Compute desenha esses rooflines automaticamente.

4.3 Ferramentas

Nsight Systems (nsys) mostra a **linha do tempo**: CPU, chamadas de API do CUDA, kernels, cópias de memória, comunicação NCCL. Use primeiro, para responder "onde está o tempo?" e "a GPU fica ociosa?".

```
nsys profile -o perfil --trace=cuda,nvtx,osrt \
    python servir.py --benchmark
```

```
 Linha do tempo (nsys), decode com batch pequeno:

 CPU  |▓▓launch▓▓|▓▓launch▓▓|▓▓launch▓▓|▓▓launch▓▓|
 GPU  | ▒ |      | ▒ |      | ▒ |      | ▒ |
        ▲ kernel de 8 µs   ▲ GPU parada esperando
                             a CPU lançar o próximo
 → overhead-bound: CUDA graphs resolvem
```

Nsight Compute (ncu) analisa **um kernel** em detalhe: throughput de memória e de cálculo em % do pico (seção "Speed of Light"), análise da hierarquia de memória, occupancy, motivos de espera dos warps, e contadores por linha de código-fonte.

```
ncu --set full -k regex:rms_norm -c 5 -o kernel \
    python servir.py --benchmark
```

`torch.profiler` dá a visão do PyTorch: que operador gerou que kernel, quanto tempo de CPU e GPU cada um levou, e exporta para o Chrome Trace ou Perfetto.

DCGM / `nvidia-smi` servem para monitorar a frota, não para otimizar kernels. A "GPU utilization" do `nvidia-smi` só diz que havia **algum** kernel rodando; métricas do DCGM como `SM_ACTIVE`, `TENSOR_ACTIVE` (pipe de tensor ativo) e `DRAM_ACTIVE` são bem mais informativas.

Motivos de espera dos warps

O Nsight Compute mostra **por que** os warps não estão emitindo instruções. Os principais:

Motivo (warp stall)	Significado	Ação típica
Long scoreboard	esperando dado da memória global/local	mais bytes em voo, melhor reuso, prefetch
Short scoreboard	esperando shared memory ou função especial	reduzir conflitos de banco

Motivo (warp stall)	Significado	Ação típica
Barrier	parado em `__syncthreads`	balancear trabalho, menos barreiras
MIO throttle	fila de instruções de memória cheia	leituras vetorizadas (menos instruções)
Math pipe throttle	unidade de cálculo saturada	bom sinal se for o esperado
Wait	dependência fixa entre instruções	mais ILP
Not selected	havia warp pronto, outro foi escolhido	normal; indica occupancy suficiente

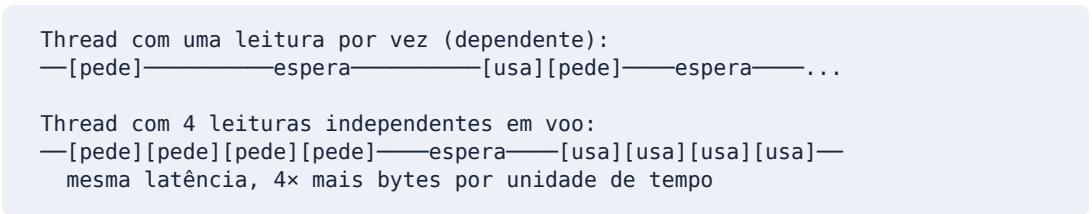
4.4 Escondendo latência: a lei de Little

Esta é a ideia que explica a maioria dos kernels memory-bound que não chegam ao pico. A **lei de Little** diz:

$$\text{dados em trânsito} = \text{banda} \times \text{latência}$$

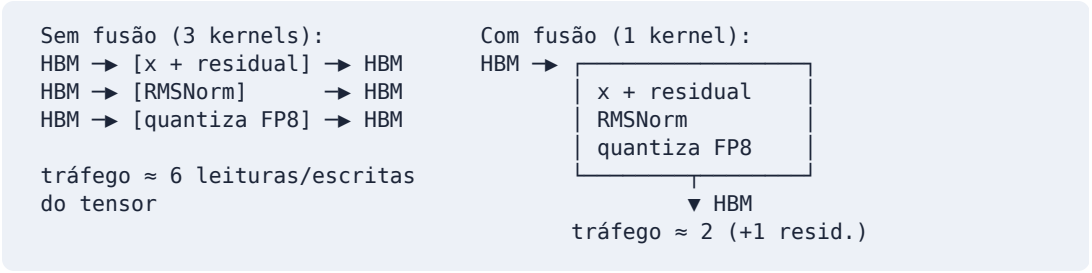
Para usar toda a banda da HBM, a GPU precisa ter, a todo momento, uma quantidade grande de leituras pedidas e ainda não respondidas.

> **Fazendo a conta:** suponha uma latência de memória da ordem de 600 ns (valor ilustrativo; varia com a carga) e 3,35 TB/s.
>
> - Bytes em voo necessários: $3{,}35 \times 10^{12} \times 600 \times 10^{-9} \approx$ **2 MB** na GPU toda.
> - Por SM: 2 MB / 132 ≈ **15 KB** em voo, o tempo todo.
> - Se cada thread tem uma leitura de 4 bytes pendente por vez, você precisaria de ~3.800 threads por SM. O máximo é 2.048.
> - Se cada thread lê 16 bytes (`float4`) e tem 2 leituras independentes pendentes, bastam ~480 threads por SM.
>
> Conclusão: **leituras largas e independentes** (vetorização e ILP) são tão importantes quanto a occupancy.



4.5 Fusão de kernels

Cada kernel separado lê suas entradas da HBM e escreve suas saídas na HBM. Para operações elemento a elemento, esse tráfego **é** o custo inteiro.



Fusões comuns em LLMs:

- **Residual + RMSNorm** (+ quantização da saída).
- **SiLU × up** no MLP (ou a GEMM de gate e up juntas numa só, com a ativação no epílogo).
- **QKV numa GEMM só** + RoPE + escrita no KV cache.
- **Epílogo de GEMM**: bias, ativação, quantização e soma residual feitas enquanto o resultado ainda está em registradores.
- **Atenção inteira** (FlashAttention).

4.6 Overhead de CPU e CUDA graphs

No decode com batch pequeno, um passo pode ter mais de mil kernels de poucos microssegundos cada. O Python e o driver do CUDA levam alguns microssegundos para lançar cada um. A GPU termina o kernel e fica esperando o próximo.

CUDA graphs resolvem isso: você grava a sequência de lançamentos uma vez e a reexecuta com uma única chamada.

```
import torch

# entradas e saídas com endereço FIXO (o grafo grava ponteiros)
x_estatico = torch.zeros(batch, d, device="cuda")
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):          # aquecimento obrigatório
    for _ in range(3):
        y_estatico = modelo_passo(x_estatico)
torch.cuda.current_stream().wait_stream(s)

g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    y_estatico = modelo_passo(x_estatico)

# a cada passo: copia a entrada para o buffer e reexecuta
x_estatico.copy_(x_novo)
g.replay()
resultado = y_estatico.clone()
```

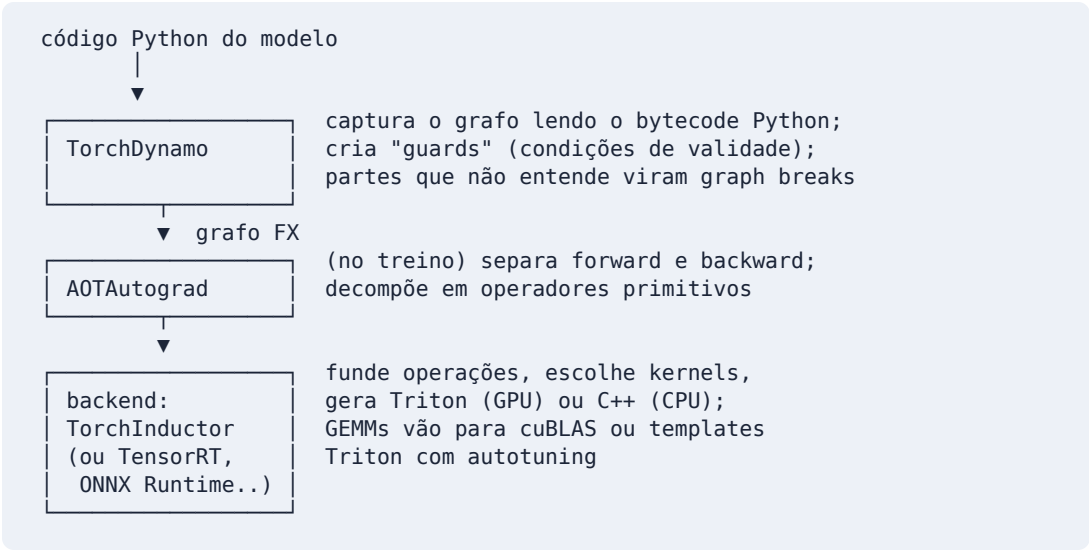
Limitações que você deve citar:

- **Formas fixas.** O grafo é gravado para um tamanho. Motores de inferência gravam um grafo para cada tamanho de batch de uma lista (1, 2, 4, 8, 16, ..., 256) e completam o batch com padding até o tamanho gravado mais próximo.
- **Endereços fixos.** Entradas e saídas precisam estar sempre nos mesmos buffers.
- **Nada de sincronização com a CPU** nem de fluxo de controle dependente de dados dentro do grafo.
- **Memória.** Cada grafo reserva sua memória; muitos tamanhos gravados consomem memória que poderia ir para o KV cache.

Por isso, a prática comum é usar CUDA graphs para o **decode** (formas previsíveis) e execução normal para o **prefill** (formas variáveis, kernels longos, overhead irrelevante).

4.7 torch.compile por dentro

O `torch.compile` é a forma de otimizar modelos PyTorch sem reescrever tudo à mão. Ele tem três camadas:



- **Guards**: o código compilado só é reutilizado se as condições valem (mesmo dtype, mesmas dimensões ou dimensões marcadas como dinâmicas, etc.). Se mudam, há **recompilação**. Em serving, recompilações em produção causam picos de latência enormes.
- **Graph breaks**: código que o Dynamo não consegue capturar (algumas chamadas externas, `print`, conversões para Python como `.item()`) quebra o grafo em pedaços. Cada quebra impede fusões. `torch._dynamo.explain()` e `TORCH_LOGS="graph_breaks"` mostram onde estão.
- **Modos**: `default`; `reduce-overhead` (usa CUDA graphs); `max-autotune` (testa várias configurações de kernel; compila bem mais devagar).
- **Backends externos**: `torch_tensorrt` compila partes do grafo para TensorRT; também há backends para ONNX Runtime e outros. Úteis quando o backend tem kernels melhores para o seu hardware ou formato (por exemplo, INT8/FP8 com calibração).

4.8 Medindo direito

Metade dos "ganhos" que aparecem em entrevistas são erros de medição. As regras:

1. **Aquecimento.** Os primeiros passos incluem compilação, autotuning, alocação de memória e caches frios. Descarte-os.
2. **Sincronização.** Lançamentos de kernel são assíncronos. Medir com `time.time()` sem `torch.cuda.synchronize()` mede só o tempo de lançamento. Use **CUDA events**.
3. **Repetições e estatística.** Reporte mediana e percentis de várias execuções.

4. **Formas realistas.** Benchmark com batch e contexto da produção. Um kernel ótimo para batch 256 pode ser péssimo para batch 4.
5. **Clocks estáveis.** GPUs mudam de frequência com temperatura e potência. Para comparações finas, trave os clocks ou rode tempo suficiente para estabilizar.
6. **Mesma qualidade.** Ao comparar com quantização, compare também a qualidade.

```python
import torch

def medir(fn, aquecimento=10, repeticoes=100):
    for _ in range(aquecimento):
        fn()
    torch.cuda.synchronize()
    tempos = []
    for _ in range(repeticoes):
        ini = torch.cuda.Event(enable_timing=True)
        fim = torch.cuda.Event(enable_timing=True)
        ini.record()
        fn()
        fim.record()
        torch.cuda.synchronize()
        tempos.append(ini.elapsed_time(fim))   # em ms
    tempos.sort()
    return {"p50": tempos[len(tempos) // 2],
            "p90": tempos[int(len(tempos) * 0.9)],
            "min": tempos[0]}
```

Perguntas de entrevista comentadas

Q025 Um kernel atinge 30% da largura de banda de memória máxima numa H100. Como você descobre o porquê?

> **[Fornecedor]** NVIDIA, segundo fornecedor de preparação · Presencial · 2026 · Difícil · fonte

Resposta em 30 segundos

“Primeiro confirmo que o kernel deveria ser memory-bound, calculando a intensidade aritmética. Depois abro no Nsight Compute: se o cálculo também está baixo, é problema de latência ou de paralelismo, não de banda. Aí verifico, em ordem: o tamanho do problema e da grade (poucos blocos, kernel curto demais, efeito de cauda); occupancy e bytes em voo por SM, pela lei de Little; coalescing, olhando setores por requisição; largura das leituras (escalares vs 128 bits); e motivos de espera dos warps. As causas mais comuns são acessos não coalescidos, poucos bytes em voo e grade pequena demais. Corrijo uma coisa por vez e meço de novo.”

Resposta completa

Mostre o método como uma árvore de decisão:

```
30% da banda
  │
  ├─ 1. Deveria ser memory-bound?
  │     intensidade << 295 FLOP/byte? Se não, o alvo é
  │     o pico de FLOPS, e 30% de banda pode ser normal.
  │
  ├─ 2. O kernel é grande o suficiente?
  │     dura poucos µs? lê poucos MB?  → overhead de
  │     lançamento e rampa dominam. Juntar trabalho,
  │     fundir kernels, CUDA graphs.
  │
  ├─ 3. A grade enche a GPU?
  │     blocos < (132 SMs × blocos por SM)? cauda?
  │     → mais blocos, grid-stride, split do trabalho.
  │
  ├─ 4. Há bytes em voo suficientes? (lei de Little)
  │     occupancy baixa? leituras dependentes?
  │     leituras de 4 bytes?
  │     → vetorizar (16 B), desenrolar laços para
  │       várias leituras independentes, reduzir
  │       registradores.
  │
  ├─ 5. Os acessos são eficientes?
  │     setores por requisição > ideal? stride?
  │     desalinhamento?  → reorganizar layout,
  │     transpor via shared memory, alinhar.
  │
  └─ 6. Algum outro limite?
        MIO throttle, conflitos de banco, atômicos
        disputados, cálculo de índice pesado (divisão
        inteira, módulo), L2 com padrão ruim.
```

Detalhe as principais causas com o sintoma que o Nsight Compute mostra e a correção:

Causa	Sintoma no Nsight Compute	Correção
Acesso não coalescido	setores/requisição alto; “uncoalesced global accesses”	threadIdx.x na dimensão contígua; transpor via shared
Poucos bytes em voo	stall “long scoreboard” dominante; occupancy baixa	`float4` / `uint4`, ILP, menos registradores
Grade pequena	poucos blocos; “waves per SM” < 1	mais blocos, dividir o trabalho
Kernel curto	duração de poucos µs	fusão, CUDA graphs, mais trabalho por kernel

Causa	Sintoma no Nsight Compute	Correção
Desalinhamento	setores extras por requisição	alinhar ponteiros e tamanhos em 16 B
Instruções demais	“issue slot” alto, “MIO throttle”	vetorizar, simplificar índices
Atômicos disputados	stalls em operações atômicas	reduzir localmente antes do atômico

Um exemplo concreto ajuda muito:

> “Num kernel de RMSNorm em BF16, cada thread lia um `__nv_bfloat16` (2 bytes) por vez. Com 2 bytes por leitura e uma leitura pendente por thread, não havia bytes em voo suficientes. Troquei por leituras de 16 bytes (8 valores) e duas leituras independentes por iteração, e a banda foi de 30% para uns 80%.”

E termine com a régua do que é bom: “Kernels de streaming simples (cópia, elemento a elemento) bem escritos costumam chegar a 80–90% da banda nominal. Abaixo de 60%, quase sempre há algo a corrigir.”

Perguntas de acompanhamento prováveis

- **E se o problema for pequeno, digamos 1 MB?** A 3,35 TB/s, 1 MB leva 0,3 µs; o lançamento e a rampa levam mais que isso. Esse kernel nunca vai atingir a banda de pico sozinho. A saída é fundi-lo com outro ou rodar dentro de um CUDA graph.
- **Como a L2 entra nisso?** Se os dados cabem na L2 (50 MB), leituras repetidas podem vir da L2 com banda maior do que a HBM. Às vezes a “banda de DRAM baixa” é boa notícia: os dados estão vindo do cache.
- **Como você sabe a latência de memória da GPU?** Microbenchmarks de pointer chasing (cada leitura depende da anterior) medem a latência; os valores variam com a geração e a carga.
- **O que é partition camping?** Padrões de acesso que concentram as requisições em poucos canais ou partições de memória, reduzindo a banda efetiva. Em GPUs modernas, o hash de endereços reduz o problema, mas padrões com stride potência de 2 ainda podem sofrer.

Armadilhas comuns

- Pular para uma solução (“aumentaria a occupancy”) sem dizer como mediria.
- Não confirmar antes se o kernel deveria ser memory-bound.
- Esquecer a lei de Little e a largura das leituras.

- Não citar a ferramenta (Nsight Compute) e as métricas específicas.

Q026 Sua redução é mais lenta com 64 threads por bloco do que com 256. O que está acontecendo?

> **[Fornecedor]** NVIDIA, segundo fornecedor de preparação · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"Com blocos de 64 threads, o SM pode bater no limite de blocos residentes antes do limite de threads, e a occupancy cai; em várias GPUs o limite é 16 ou 32 blocos por SM. Também há 4 vezes mais blocos, então 4 vezes mais resultados parciais: mais escritas na memória global ou mais atômicos disputando o mesmo endereço, ou uma segunda etapa de redução maior. E o custo fixo de cada bloco (iniciar, sincronizar, as etapas finais da árvore com poucas threads ativas, escrever o resultado) é amortizado sobre menos elementos. Eu confirmaria no Nsight Compute olhando a occupancy atingida e o limitador de blocos, e mudaria para um laço grid-stride com um número fixo de blocos, soma em registradores, redução por warp com shuffle e um atômico por bloco."

Resposta completa

Comece com uma redução típica, para ter algo concreto:

```
// cada bloco soma blockDim.x elementos e faz 1 atômico
__global__ void soma_v1(const float* x, float* total, int n) {
    extern __shared__ float s[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    s[threadIdx.x] = (i < n) ? x[i] : 0.0f;
    __syncthreads();
    for (int passo = blockDim.x / 2; passo > 0; passo /= 2) {
        if (threadIdx.x < passo)
            s[threadIdx.x] += s[threadIdx.x + passo];
        __syncthreads();
    }
    if (threadIdx.x == 0) atomicAdd(total, s[0]);
}
```

Agora analise o que muda de 256 para 64 threads por bloco, com $n = 100$ milhões de elementos:

1. Número de blocos e atômicos. Com 256 threads: ~390 mil blocos e 390 mil `atomicAdd` no mesmo endereço. Com 64: ~1,56 milhão de blocos e **1,56 milhão de atômicos** no mesmo endereço. Atômicos no mesmo endereço são serializados na L2. Isso sozinho pode dominar.

2. Limite de blocos por SM. Cada SM aceita um número máximo de blocos residentes (32 na Ampere e Hopper, 16 em gerações como a Turing). Com 64 threads por bloco:

```
Hopper:  32 blocos × 64 threads = 2.048 threads → 100% ok
Turing:  16 blocos × 64 threads = 1.024 threads
         (máximo do SM na Turing = 1.024)    → ok
GPU com limite de 16 blocos e 2.048 threads por SM:
         16 × 64 = 1.024 de 2.048            → 50%
```

O efeito depende da arquitetura; diga isso e explique como verificar (o Nsight Compute mostra o “block limit” de cada recurso).

3. Custo fixo por bloco. Cada bloco tem custo de agendamento, inicialização, $\log_2$ (threads) etapas com `__syncthreads()` e a escrita final. As últimas etapas da árvore usam poucas threads (as outras ficam ociosas). Com blocos menores, a proporção de trabalho “útil” por bloco cai.

4. Pouco trabalho por thread. Cada thread lê **um** elemento. Não há ILP, há poucos bytes em voo por thread, e quase todo o tempo é overhead.

A correção (o padrão que os entrevistadores querem ver):

```cpp
__inline__ __device__ float soma_warp(float v) {
    for (int d = 16; d > 0; d /= 2)
        v += __shfl_down_sync(0xffffffff, v, d);
    return v;
}

__global__ void soma_v2(const float* __restrict__ x,
                        float* total, int n) {
    float acc = 0.0f;
    // grid-stride: cada thread soma muitos elementos
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n; i += gridDim.x * blockDim.x)
        acc += x[i];

    acc = soma_warp(acc);                 // redução no warp
    __shared__ float parciais[32];        // 1 por warp
    int lane = threadIdx.x % 32, warp = threadIdx.x / 32;
    if (lane == 0) parciais[warp] = acc;
    __syncthreads();
    if (warp == 0) {
        int nwarps = blockDim.x / 32;
        acc = (lane < nwarps) ? parciais[lane] : 0.0f;
        acc = soma_warp(acc);
        if (lane == 0) atomicAdd(total, acc);  // 1 por bloco
    }
}
// lançar com ~ (número de SMs × 4) blocos de 256 threads
```

Melhorias: cada thread soma milhares de elementos em registradores; a redução no warp usa shuffle, sem shared memory nem `__syncthreads()`; há só uma barreira por bloco; e só algumas centenas de atômicos no total. Com leituras `float4`, chega perto do pico de banda.

Perguntas de acompanhamento prováveis

- **E se eu precisar de um resultado determinístico?** `atomicAdd` em ponto flutuante não é determinístico: a ordem das somas muda entre execuções, e ponto flutuante não é associativo. Use duas passadas (cada bloco escreve seu parcial num vetor, e um segundo kernel soma os parciais em ordem fixa). Ver Q034.
- **Por que `__shfl_down_sync` e não shared memory?** O shuffle troca registradores dentro do warp em uma instrução, sem barreira e sem conflitos de banco.
- **Qual o número ideal de blocos?** O suficiente para encher a GPU com algumas ondas (por exemplo, número de SMs × blocos residentes por SM), não um bloco por elemento. Dá para consultar com `cudaOccupancyMaxActiveBlocksPerMultiprocessor`.
- **Blocos maiores (1.024) seriam ainda melhores?** Não necessariamente: menos flexibilidade de agendamento, barreiras mais caras e possíveis limites de registradores. 128 a 512 costuma ser um bom intervalo.

Armadilhas comuns

- Responder só “occupancy” sem explicar o limite de blocos por SM.
- Esquecer o custo dos atômicos.
- Não propor grid-stride e redução por warp.
- Ignorar o não determinismo do `atomicAdd` em float.

Q027 Dê um exemplo de kernel que atingiu alta utilização dos tensor cores. Como você conseguiu isso?

> **[Relatado]** AMD · GPU Kernel Developer, AI Models · primeira rodada · relatado · Difícil · fonte

Resposta em 30 segundos

“Uma GEMM agrupada em FP8 para as camadas de especialistas de um modelo MoE. Para chegar perto do pico, garanti quatro coisas: dados no formato e alinhamento que os tensor cores querem (dimensões múltiplas de 16, layout certo para cada operando); tiles grandes o bastante para a intensidade aritmética passar do ponto de equilíbrio, com o reuso feito em shared memory e registradores; um pipeline com várias etapas de cópia assíncrona para os tensor cores nunca esperarem dados; e o trabalho de todos os especialistas num kernel persistente, para encher todos os SMs mesmo com especialistas pequenos. O epílogo fazia a descompactação de escala e a conversão no próprio kernel. Chegamos a uns 70% do pico de FP8, contra uns 35% da versão com uma GEMM por especialista.”

Resposta completa

O entrevistador quer duas coisas: que você conheça **os requisitos para alimentar tensor cores** e que tenha uma história crível. Explique primeiro os requisitos, depois a história.

Os requisitos.

```
Para os tensor cores trabalharem perto do pico:

1. FORMATO   tipos suportados (BF16, FP16, FP8, INT8),
             dimensões múltiplas do tamanho de instrução
             (ex.: 8 ou 16), ponteiros alinhados (16 B+)

2. REUSO     tiles grandes (ex.: 128×128 ou 128×256 por
             bloco) → intensidade acima do ponto de
             equilíbrio. Senão, o kernel é memory-bound
             e os tensor cores esperam.

3. ALIMEN-   pipeline de várias etapas: carregar o tile k+2
   TAÇÃO     enquanto calcula o tile k (cp.async, TMA);
             layout em shared memory sem conflitos de
             banco (swizzle)

4. OCUPAÇÃO  tiles suficientes para todos os SMs, sem
             cauda ruim (split-K, stream-K, kernels
             persistentes)

5. EPÍLOGO   escrita do resultado (com bias, ativação,
             conversão) sem virar gargalo
```

Na prática, para dimensões típicas de LLM, você raramente escreve isso do zero: usa CUTLASS/CuTe, Triton, as bibliotecas do fornecedor (cuBLASLt, hipBLASLt, Composable Kernel na AMD) e ajusta a configuração. A habilidade está em saber **por que** uma configuração é melhor e em escrever a parte que a biblioteca não tem (fusões, layouts especiais, MoE).

Uma história modelo (adapte à sua experiência real):

> "**Contexto.** Num modelo MoE com 64 especialistas e 8 ativos por token, as camadas de especialistas eram 55% do tempo do prefill. A implementação lançava uma GEMM por especialista. Com tokens distribuídos de forma desigual, vários especialistas recebiam poucos tokens, e as GEMMs pequenas não enchiam a GPU.
>
> **Medição.** No Nsight Compute, o pipe de tensor core ficava ativo uns 35% do tempo; havia dezenas de kernels curtos com cauda longa; e os especialistas com poucos tokens tinham intensidade aritmética baixa.
>
> **O que fiz.** (1) Uma GEMM agrupada num único kernel persistente: cada bloco pega tiles de uma fila global que cobre todos os especialistas, então os SMs ficam ocupados até o fim. (2) Ordenei os tokens por especialista antes da GEMM (um kernel de permutação), para que as linhas de cada especialista ficassem contíguas e as leituras, coalescidas. (3) Usei FP8 com fator de escala por bloco de 128 elementos e apliquei as escalas

no epílogo, em registradores. (4) Ajustei tamanho do tile e número de etapas do pipeline com autotuning nas formas reais do tráfego: tiles de 128×128 com 4 etapas foram os melhores para os especialistas grandes, e 64×128 para os pequenos.

Desafio. A distribuição de tokens muda a cada lote. Um tamanho de tile fixo era ótimo para um caso e ruim para outro. A solução foi escolher a configuração em tempo de execução, com base no número de tokens por especialista, entre duas ou três variantes compiladas.

Resultado. O pipe de tensor core ativo foi para uns 70%, e o tempo das camadas de especialistas caiu pela metade. O prefill inteiro ficou uns 35% mais rápido."

Perguntas de acompanhamento prováveis

- **Qual métrica mostra a utilização dos tensor cores?** No Nsight Compute, o throughput do pipe de tensor (porcentagem do pico das instruções HMMA/WGMMA); no DCGM, `TENSOR_ACTIVE`. Compare também os TFLOPS atingidos com o pico do formato.
- **Por que dimensões múltiplas de 8 ou 16?** As instruções de tensor core operam em fragmentos de tamanho fixo. Dimensões fora disso exigem padding ou caminhos mais lentos. Por isso modelos são desenhados com dimensões como 4.096 e vocabulários com padding para múltiplos de 64 ou 128.
- **Por que é mais difícil chegar ao pico em FP8 do que em BF16?** O pico de FP8 é o dobro. Para manter os tensor cores ocupados, você precisa entregar dados duas vezes mais rápido em relação ao cálculo, e o epílogo (escalas, conversões) pesa mais.
- **Na AMD, o que muda?** Os matrix cores usam instruções MFMA, os wavefronts têm 64 threads nas GPUs de data center (CDNA), e a memória local (LDS) tem outras regras de banco. Os princípios (reuso, pipeline, ocupação) são os mesmos.

Armadilhas comuns

- Dizer "usei cuBLAS e deu 90%" sem explicar o que você fez.
- Não saber os requisitos de formato e alinhamento.
- Esquecer o pipeline (sobrepor cópia e cálculo).
- Não mencionar o problema de encher os SMs (cauda, MoE, formas pequenas).

Q028 Projete e faça o benchmark de um pipeline de inferência otimizado: TorchDynamo, TorchInductor, backends externos, fusão, CUDA graphs, quantização.

> **[Relatado]** NVIDIA · Software Engineer · triagem técnica · listada em 2026 · Médio · fonte

Resposta em 30 segundos

"Começo com uma linha de base em eager PyTorch e um benchmark confiável: formas reais, aquecimento, CUDA events, percentis, e uma métrica de qualidade para cada variante. Depois subo uma camada por vez, medindo cada uma: torch.compile com Inductor para fusões automáticas, olhando graph breaks e recompilações; o modo reduce-overhead para usar CUDA graphs no decode; quantização (pesos em FP8 ou INT8/INT4) validada contra a avaliação de qualidade; e um backend externo como TensorRT quando ele tiver kernels melhores para o caso. No fim, a tabela compara latência p50/p99, throughput, memória, tempo de compilação e qualidade, e eu escolho com base no SLO, não só no número mais bonito."

Resposta completa

Estruture como um projeto, com o pipeline e o benchmark lado a lado.

1. Defina o alvo. Que modelo, que hardware, que métrica (latência com batch 1? throughput com batch 64?), que formas de entrada (distribuição de tamanhos), e que critério de qualidade aceitável (por exemplo, no máximo 0,5 ponto de queda num benchmark interno).

2. Linha de base. PyTorch eager, BF16. Meça com a função `medir` da seção 4.8. Rode o `torch.profiler` para ver a distribuição do tempo entre GEMMs, atenção, ops elemento a elemento e tempo ocioso.

3. As camadas de otimização, em ordem:

Etapa	O que ataca	Risco
0 eager BF16	(base)	—
1 SDPA/FlashAttention	atenção	pouco
2 torch.compile (Inductor)	fusões elemento a elemento, menos kernels	graph breaks, recompilações, tempo de compilação
3 reduce-overhead (CUDA graphs)	overhead de CPU no decode	formas fixas, memória extra
4 max-autotune	escolha de kernels	compilação longa
5 quantização FP8/INT8 /INT4 (pesos)	bytes lidos, FLOPs mais rápidos	qualidade
6 backend externo (TensorRT)	kernels específicos, fusões agressivas	cobertura de ops, manutenção

4. O código do benchmark (esqueleto):

```
import torch

def preparar(modelo, variante):
    if variante == "eager":
        return modelo
    if variante == "compile":
        return torch.compile(modelo)
    if variante == "compile-graphs":
        return torch.compile(modelo, mode="reduce-overhead")
    if variante == "autotune":
        return torch.compile(modelo, mode="max-autotune")
    raise ValueError(variante)

def bench(modelo, entradas, variante):
    m = preparar(modelo, variante)
    resultados = {}
    for nome, x in entradas.items():      # ex.: "b1_ctx512"
        with torch.inference_mode():
            resultados[nome] = medir(lambda: m(x))
    return resultados
```

Pontos de atenção que você deve dizer em voz alta:

- **Separe o tempo de compilação** do tempo de execução. Em serving, a compilação acontece na subida do servidor ou antes (cache de compilação); ela não pode acontecer no meio do tráfego.
- **Verifique graph breaks** (`TORCH_LOGS="graph_breaks,recompiles"`). Um graph break no meio de cada camada anula boa parte das fusões.
- **Formas dinâmicas**: marque as dimensões que variam (`torch._dynamo.mark_dynamic`) ou faça buckets de tamanhos, para não recompilar a cada tamanho de entrada.

- **Qualidade**: para cada variante quantizada, rode a avaliação. Para as variantes não quantizadas, compare os logits com a linha de base (diferença máxima dentro da tolerância).

5. O relatório final.

Variante	p50 b=1 (ms/token)	Throughput b=64 (tok/s)	Memória	Compilação	Qualidade
eager BF16	(medido)	(medido)	(medido)	0 s	referência
compile	…	…	…	…	=
compile + graphs	…	…	…	…	=
+ FP8 pesos	…	…	…	…	Δ medido
TensorRT FP8	…	…	…	…	Δ medido

Deixe a tabela vazia na entrevista e explique o que **espera** ver e por quê: “Para batch 1, espero que CUDA graphs deem o maior ganho isolado, porque o decode é overhead-bound; para batch 64, quantização e kernels melhores devem dominar, porque o passo é limitado por banda e cálculo.” Isso mostra que você tem um modelo, não só uma ferramenta.

6. Fusão manual quando a automática não basta. Se o profiler mostrar que uma sequência importante não foi fundida (por exemplo, residual + norma + quantização), escreva um kernel Triton para ela e registre como operador customizado (`torch.library`), para continuar compatível com o `torch.compile`.

Perguntas de acompanhamento prováveis

- **Por que não usar direto um motor como vLLM ou TensorRT-LLM?** Para serving de LLMs, normalmente você usaria. Eles já trazem continuous batching, KV cache paginado e kernels especializados. Essa pergunta testa se você entende as peças que esses motores combinam. Ver Q035.
- **Como evitar recompilações em produção?** Fixar buckets de formas, aquecer todos os buckets na subida, usar o cache de compilação, alertar quando houver recompilação.
- **Quando o TensorRT ganha?** Quando há kernels muito otimizados para o seu hardware e formato (por exemplo, FP8 na Hopper), fusões agressivas e formas previsíveis. O custo é uma etapa de build separada, menos flexibilidade e cobertura de operadores menor.

- **O que você faria se o compile deixasse mais lento?** Olhar os kernels gerados (`TORCH_LOGS="output_code"`), verificar se uma GEMM foi trocada por uma versão pior, se houve graph breaks, ou se o autotuning escolheu mal para as formas reais.

Armadilhas comuns

- Medir sem aquecimento ou sem sincronização.
- Comparar variantes com formas diferentes ou com batch irreal.
- Ignorar o tempo de compilação e as recompilações.
- Aplicar quantização sem medir qualidade.
- Empilhar todas as otimizações de uma vez e não saber qual deu o ganho.

Resumo do capítulo

- Método: medir → classificar (memória, cálculo, latência, overhead) → hipótese → mudar uma coisa → medir.
- **MFU** para quem é compute-bound, **MBU** para quem é memory-bound (decode). MBU acima de ~80% significa que só reduzir bytes ajuda.
- Nsight Systems para a linha do tempo; Nsight Compute para o kernel; motivos de espera dos warps dizem o que falta.
- **Lei de Little**: para saturar a HBM, é preciso muitos bytes em voo por SM. Leituras largas e independentes importam tanto quanto a occupancy.
- **Fusão** elimina idas e vindas à HBM; **CUDA graphs** eliminam o overhead de lançamento no decode.
- `torch.compile` = Dynamo (captura e guards) + AOTAutograd + Inductor (Triton). Cuidado com graph breaks e recompilações.
- Redução eficiente: grid-stride, soma em registradores, shuffle no warp, um atômico por bloco (ou duas passadas para determinismo).
- Tensor cores perto do pico exigem formato e alinhamento, tiles grandes, pipeline de cópias e SMs cheios.
- Benchmark sério: aquecimento, CUDA events, percentis, formas reais, qualidade medida.

Checklist para a entrevista

- [ ] Classifico um kernel nas quatro classes de gargalo e digo como confirmar cada uma.

- [ ] Calculo MBU de um passo de decode.
- [ ] Sei o que mostram o Nsight Systems e o Nsight Compute e uso os nomes certos das métricas.
- [ ] Aplico a lei de Little para estimar bytes em voo.
- [ ] Percorro a árvore de diagnóstico de banda baixa sem consultar.
- [ ] Escrevo a redução com grid-stride e shuffle.
- [ ] Explico os requisitos para alimentar tensor cores.
- [ ] Explico Dynamo, Inductor, guards, graph breaks e modos do `torch.compile` .
- [ ] Explico CUDA graphs e suas limitações.
- [ ] Desenho um plano de benchmark com qualidade, latência, throughput e memória.

CAPÍTULO 5

Kernels CUDA e Triton

Este é o capítulo “mão na massa”. Nos capítulos 3 e 4 você aprendeu como a GPU funciona e como diagnosticar desempenho. Aqui você vai ver os kernels completos que mais caem em entrevista: a multiplicação de matrizes em FP32 com todas as otimizações clássicas, a redução determinística, e kernels em Triton. No fim, a visão de cima: como motores inteiros como o vLLM e o TensorRT-LLM juntam kernels, scheduler e gerenciamento de memória.

Em entrevistas de código para GPU, o entrevistador avalia três coisas: **o kernel está correto** (bordas, sincronização, índices), **você sabe explicar cada decisão de desempenho**, e **você sabe dizer qual seria o próximo passo**. Treine escrever estes kernels à mão, sem compilador, e explicar em voz alta.

5.1 Esqueleto de um programa CUDA

Todo programa CUDA de teste segue o mesmo roteiro:

```
#include <cstdio>
#include <cuda_runtime.h>

// macro para checar erros: TODA chamada da API pode falhar
#define CHECK(x) do { cudaError_t e = (x);                    \
    if (e != cudaSuccess) {                                    \
        printf("CUDA: %s (%s:%d)\n", cudaGetErrorString(e),   \
               __FILE__, __LINE__);                            \
        exit(1); } } while (0)

int main() {
    // 1. aloca e preenche dados no host (CPU)
    // 2. aloca na GPU:          CHECK(cudaMalloc(...))
    // 3. copia host -> GPU:     CHECK(cudaMemcpy(..., H2D))
    // 4. lança o kernel:        kernel<<<grade, bloco>>>(...)
    //    checa o lançamento:    CHECK(cudaGetLastError())
    // 5. copia GPU -> host:     CHECK(cudaMemcpy(..., D2H))
    // 6. confere com uma referência na CPU
    // 7. libera a memória:      cudaFree(...)
}
```

Dois detalhes que mostram experiência:

- **Erros de kernel são assíncronos.** Um acesso fora dos limites dentro do kernel só aparece na próxima chamada que sincroniza. Para depurar, use `cudaDeviceSynchronize()` logo depois do lançamento e a ferramenta `compute-sanitizer`.
- **Sempre compare com uma referência.** Um kernel rápido e errado não vale nada.

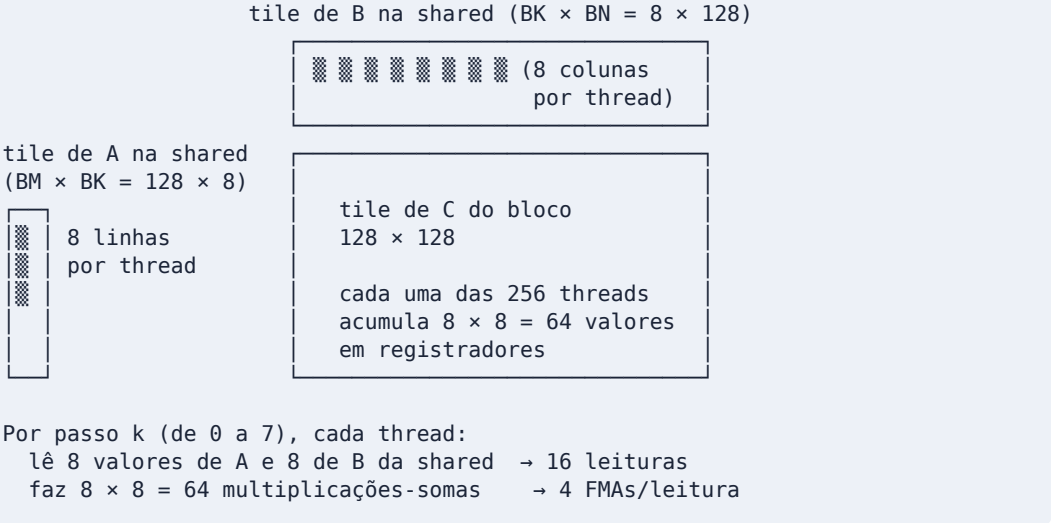
5.2 GEMM FP32 completa, em três versões

O problema: $C = A \cdot B$, com A de $M \times K$, B de $K \times N$, tudo em FP32 e row-major. Vamos do ingênuo até o tiling 2D em registradores. A versão ingênua e a versão com shared memory já apareceram no Capítulo 3 (Q017). Aqui está a terceira, que é a que separa candidatos medianos dos bons.

Tiling 2D em registradores

A ideia em duas camadas:

1. **Bloco**: calcula um tile de C de $BM \times BN$ (128 × 128). A cada iteração em K, carrega na shared memory um tile de A ($BM \times BK$) e um de B ($BK \times BN$), com $BK = 8$.
2. **Thread**: calcula $TM \times TN$ (8 × 8 = 64) elementos de C em registradores. São (128 × 128)/64 = 256 threads por bloco.



Um detalhe importante é **quais** linhas e colunas cada thread pega. Se a thread t pegasse 8 colunas **seguidas** ($8t$ a $8t + 7$), threads vizinhas leriam endereços separados por 8 floats na shared memory, o que causa conflitos de banco. Em vez disso, cada thread pega colunas **espaçadas** (t, $t + 16$, $t + 32$, ...): threads vizinhas leem endereços vizinhos, sem conflitos, e a escrita final em C também fica coalescida.

```cuda
#include <cuda_runtime.h>

template <int BM, int BN, int BK, int TM, int TN>
__global__ void __launch_bounds__((BM * BN) / (TM * TN))
sgemm_2d(int M, int N, int K,
         const float* __restrict__ A,
         const float* __restrict__ B,
         float* __restrict__ C) {
    constexpr int NT = (BM * BN) / (TM * TN);  // 256 threads
    constexpr int TC = BN / TN;   // 16 threads por linha
    constexpr int TR = BM / TM;   // 16 threads por coluna
    // A transposto na shared (As[k][m]) e +4 de padding:
    // a escrita transposta fica sem conflitos de banco
    __shared__ float As[BK][BM + 4];
    __shared__ float Bs[BK][BN];

    const int bm = blockIdx.y * BM;   // canto do tile de C
    const int bn = blockIdx.x * BN;
    const int tr = threadIdx.x / TC;  // 0..15
    const int tc = threadIdx.x % TC;  // 0..15

    float acc[TM][TN] = {};
    float ra[TM], rb[TN];

    for (int k0 = 0; k0 < K; k0 += BK) {
        // carrega A (BM x BK): 1024 floats, 4 por thread
        for (int i = threadIdx.x; i < BM * BK; i += NT) {
            int r = i / BK, c = i % BK;
            int gr = bm + r, gc = k0 + c;
            As[c][r] = (gr < M && gc < K) ? A[gr * K + gc]
                                          : 0.0f;
        }
        // carrega B (BK x BN): coalescido por linha
        for (int i = threadIdx.x; i < BK * BN; i += NT) {
            int r = i / BN, c = i % BN;
            int gr = k0 + r, gc = bn + c;
            Bs[r][c] = (gr < K && gc < N) ? B[gr * N + gc]
                                          : 0.0f;
        }
        __syncthreads();

        #pragma unroll
        for (int k = 0; k < BK; ++k) {
            #pragma unroll
            for (int i = 0; i < TM; ++i)
                ra[i] = As[k][tr + i * TR];   // espaçado
            #pragma unroll
            for (int j = 0; j < TN; ++j)
                rb[j] = Bs[k][tc + j * TC];   // espaçado
            #pragma unroll
            for (int i = 0; i < TM; ++i)
                #pragma unroll
                for (int j = 0; j < TN; ++j)
                    acc[i][j] += ra[i] * rb[j];
        }
        __syncthreads();
    }
```

```
    #pragma unroll
    for (int i = 0; i < TM; ++i) {
        int r = bm + tr + i * TR;
        #pragma unroll
        for (int j = 0; j < TN; ++j) {
            int c = bn + tc + j * TC;
            if (r < M && c < N) C[r * N + c] = acc[i][j];
        }
    }
}
```

E o código do host, com conferência e medição:

```cpp
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

int main() {
    const int M = 2048, N = 2048, K = 2048;
    std::vector<float> hA(M * K), hB(K * N), hC(M * N);
    for (auto& v : hA) v = rand() / (float)RAND_MAX - 0.5f;
    for (auto& v : hB) v = rand() / (float)RAND_MAX - 0.5f;

    float *dA, *dB, *dC;
    cudaMalloc(&dA, hA.size() * sizeof(float));
    cudaMalloc(&dB, hB.size() * sizeof(float));
    cudaMalloc(&dC, hC.size() * sizeof(float));
    cudaMemcpy(dA, hA.data(), hA.size() * 4,
               cudaMemcpyHostToDevice);
    cudaMemcpy(dB, hB.data(), hB.size() * 4,
               cudaMemcpyHostToDevice);

    dim3 bloco(256);
    dim3 grade((N + 127) / 128, (M + 127) / 128);
    auto roda = [&] {
        sgemm_2d<128, 128, 8, 8, 8>
            <<<grade, bloco>>>(M, N, K, dA, dB, dC);
    };
    roda();                         // aquecimento
    cudaDeviceSynchronize();

    cudaEvent_t ini, fim;
    cudaEventCreate(&ini);
    cudaEventCreate(&fim);
    cudaEventRecord(ini);
    for (int r = 0; r < 20; ++r) roda();
    cudaEventRecord(fim);
    cudaEventSynchronize(fim);
    float ms = 0;
    cudaEventElapsedTime(&ms, ini, fim);
    ms /= 20;
    double tflops = 2.0 * M * N * K / (ms * 1e-3) / 1e12;
    printf("%.3f ms, %.2f TFLOPS\n", ms, tflops);

    // confere 1000 posições aleatórias contra a CPU
    cudaMemcpy(hC.data(), dC, hC.size() * 4,
               cudaMemcpyDeviceToHost);
    double erro_max = 0;
    for (int t = 0; t < 1000; ++t) {
        int i = rand() % M, j = rand() % N;
        double ref = 0;
        for (int k = 0; k < K; ++k)
            ref += (double)hA[i * K + k] * hB[k * N + j];
        erro_max = fmax(erro_max, fabs(ref - hC[i * N + j]));
    }
    printf("erro máximo: %g\n", erro_max);
    cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
// compilar: nvcc -O3 -arch=sm_90 sgemm.cu -o sgemm
```

Por que cada decisão

Decisão	Motivo
Tile de bloco 128 × 128	reuso alto: cada valor carregado da HBM é usado 128 vezes
BK = 8	pouca shared memory por bloco (~8 KB), permite vários blocos por SM
Micro-tile 8 × 8 em registradores	4 FMAs por leitura de shared; antes era 0,5
A transposto na shared	as leituras `As[k][...]` ficam contíguas
Padding `BM + 4`	a escrita transposta não tem conflitos de banco
Linhas/colunas espaçadas por thread	leituras da shared sem conflito e escrita em C coalescida
`__launch_bounds__`	diz ao compilador o máximo de threads, ajudando a alocação de registradores
`#pragma unroll`	índices constantes mantêm `acc`, `ra` e `rb` em registradores

O que vem depois

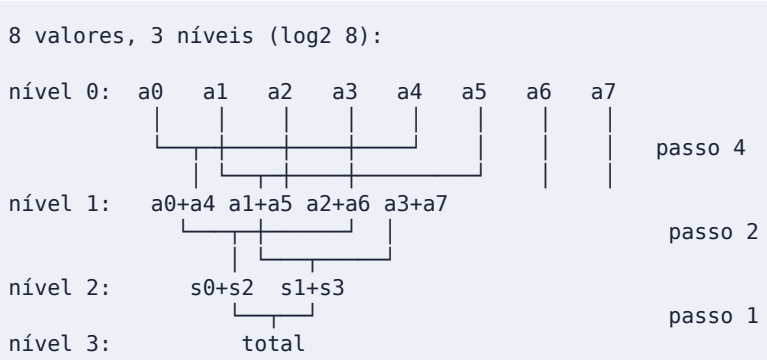
Este kernel costuma chegar a uma fração alta do cuBLAS em FP32, mas ainda tem caminho:

1. **Leituras vetorizadas** (`float4`) na carga dos tiles: 4 vezes menos instruções de leitura.
2. **Double buffering**: dois buffers na shared; enquanto calcula com um, carrega o próximo (com `cp.async` na Ampere ou TMA na Hopper).
3. **Warp tiling**: um nível intermediário em que cada warp calcula um sub-tile, para melhorar a localidade nos registradores e preparar o terreno para os tensor cores.
4. **Autotuning** de BM, BN, BK, TM e TN para cada forma de matriz.
5. **Tensor cores** (TF32 ou BF16), se o problema aceitar outra precisão. Isso muda o pico em uma ordem de grandeza.

> **Dica de entrevista:** se pedirem “implemente uma GEMM rápida”, escreva primeiro a versão com shared memory (curta e segura), diga que ela funciona e então descreva o register tiling, escrevendo-o se houver tempo. Um kernel simples e correto vale mais que um kernel ambicioso com bug de índice.

5.3 Reduções

Uma **redução** combina muitos valores em um só (soma, máximo, norma). Aparece em todo lugar em LLMs: softmax (máximo e soma), RMSNorm (soma dos quadrados), sampling, cálculo de escalas de quantização. A técnica básica é a **redução em árvore**:



Dentro de um warp, a forma mais rápida é o **shuffle**, que troca registradores entre as 32 threads sem passar pela memória:

```
__inline__ __device__ float soma_warp(float v) {
    // ao fim, a lane 0 tem a soma das 32 lanes
    for (int d = 16; d > 0; d /= 2)
        v += __shfl_down_sync(0xffffffff, v, d);
    return v;
}

__inline__ __device__ float max_warp(float v) {
    // com xor, TODAS as lanes terminam com o resultado
    for (int d = 16; d > 0; d /= 2)
        v = fmaxf(v, __shfl_xor_sync(0xffffffff, v, d));
    return v;
}
```

A redução de um vetor inteiro com grid-stride e um atômico por bloco foi mostrada na Q026. Ela é rápida, mas **não é determinística**, e isso nos leva à próxima pergunta de entrevista.

Por que reduções em float não são determinísticas

Soma em ponto flutuante não é associativa:

$$(10^{20} + (-10^{20})) + 1 = 1 \qquad \text{mas} \qquad 10^{20} + ((-10^{20}) + 1) = 0$$

Com `atomicAdd`, a ordem em que os blocos somam no resultado depende de qual bloco termina primeiro, o que muda de execução para execução. O resultado varia nos últimos bits. Em inferência isso importa porque:

- Testes de regressão "bit a bit" falham sem motivo.
- Com greedy decoding, uma diferença minúscula num logit pode trocar o token escolhido e mudar a resposta inteira a partir dali.
- Depurar fica muito difícil quando nada se repete.

5.4 Triton

Triton (OpenAI) é uma linguagem embutida em Python para escrever kernels de GPU. Em vez de programar threads individuais, você programa **blocos**: cada instância do programa opera sobre vetores e matrizes inteiros, e o compilador cuida de distribuir o trabalho entre threads, coalescer acessos, usar a shared memory e gerar instruções de tensor core.

CUDA	Triton
você controla cada thread	você controla cada bloco
threadIdx, blockIdx	tl.program_id, tl.arange
shared memory manual	automática
__syncthreads manual	automático
coalescing por sua conta	compilador cuida
mma.sync / WMMA para tensor cores	tl.dot
máximo controle	~80–100% do desempenho em muitos kernels, com muito menos código

Por isso Triton virou o padrão para kernels customizados em PyTorch: o `torch.compile` (Inductor) gera Triton, e muitos motores de inferência têm kernels de atenção, MoE e quantização escritos em Triton.

Softmax por linha em Triton

```
import torch
import triton
import triton.language as tl

@triton.jit
def softmax_kernel(saida, entrada, stride_e, stride_s,
                   n_cols, BLOCO: tl.constexpr):
    linha = tl.program_id(0)          # uma instância por linha
    cols = tl.arange(0, BLOCO)        # vetor de índices
    mask = cols < n_cols              # BLOCO é potência de 2
    x = tl.load(entrada + linha * stride_e + cols,
                mask=mask, other=-float("inf"))
    x = x - tl.max(x, axis=0)         # estabilidade numérica
    num = tl.exp(x)
    y = num / tl.sum(num, axis=0)
    tl.store(saida + linha * stride_s + cols, y, mask=mask)

def softmax(x: torch.Tensor) -> torch.Tensor:
    n_lin, n_cols = x.shape
    y = torch.empty_like(x)
    BLOCO = triton.next_power_of_2(n_cols)
    softmax_kernel[(n_lin,)](y, x, x.stride(0), y.stride(0),
                             n_cols, BLOCO=BLOCO)
    return y

x = torch.randn(1024, 3000, device="cuda")
assert torch.allclose(softmax(x), torch.softmax(x, dim=1),
                      atol=1e-6)
```

O kernel inteiro é um softmax **fundido**: lê cada linha uma vez, calcula máximo, exponencial e soma em registradores e escreve uma vez. O PyTorch eager faria várias passadas pela memória.

Multiplicação de matrizes em Triton

```
@triton.jit
def matmul_kernel(a, b, c, M, N, K,
                  s_am, s_ak, s_bk, s_bn, s_cm, s_cn,
                  BM: tl.constexpr, BN: tl.constexpr,
                  BK: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    rm = pid_m * BM + tl.arange(0, BM)
    rn = pid_n * BN + tl.arange(0, BN)
    rk = tl.arange(0, BK)
    acc = tl.zeros((BM, BN), dtype=tl.float32)
    for k0 in range(0, K, BK):
        kk = k0 + rk
        pa = a + rm[:, None] * s_am + kk[None, :] * s_ak
        pb = b + kk[:, None] * s_bk + rn[None, :] * s_bn
        ma = (rm[:, None] < M) & (kk[None, :] < K)
        mb = (kk[:, None] < K) & (rn[None, :] < N)
        ta = tl.load(pa, mask=ma, other=0.0)
        tb = tl.load(pb, mask=mb, other=0.0)
        acc += tl.dot(ta, tb)        # tensor cores
    pc = c + rm[:, None] * s_cm + rn[None, :] * s_cn
    mc = (rm[:, None] < M) & (rn[None, :] < N)
    tl.store(pc, acc.to(c.dtype.element_ty), mask=mc)

def matmul(a, b, BM=128, BN=128, BK=32):
    M, K = a.shape
    _, N = b.shape
    c = torch.empty((M, N), device=a.device, dtype=a.dtype)
    grade = (triton.cdiv(M, BM), triton.cdiv(N, BN))
    matmul_kernel[grade](a, b, c, M, N, K,
                         a.stride(0), a.stride(1),
                         b.stride(0), b.stride(1),
                         c.stride(0), c.stride(1),
                         BM=BM, BN=BN, BK=BK)
    return c
```

Compare com a GEMM em CUDA da seção 5.2: o mesmo tiling, em 30 linhas. Para chegar ao desempenho do cuBLAS, versões de produção adicionam `@triton.autotune` (testa várias configurações de BM/BN/BK, `num_warps` e `num_stages`), reordenação dos blocos para melhorar o acerto na L2 e epílogos fundidos.

Quando usar CUDA e quando usar Triton

- **Triton**: fusões de operações elemento a elemento e reduções, variantes de atenção, kernels de quantização, protótipos rápidos. Ótimo custo-benefício.
- **CUDA/CUTLASS/CuTe**: quando você precisa de controle total (layouts especiais, recursos novos do hardware antes de o Triton suportar, os últimos 10–20% de desempenho em kernels críticos como atenção e GEMM em FP8).

Perguntas de entrevista comentadas

Q033 Implemente um kernel CUDA FP32 de alto desempenho para C = A·B (A é m×k, B é k×n) e explique suas escolhas de arquitetura.

> **[Relatado]** NVIDIA · vaga de software de GPU · triagem · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

"Uso tiling em dois níveis. Cada bloco de 256 threads calcula um tile de 128×128 de C, carregando em cada iteração um tile 128×8 de A (transposto na shared) e 8×128 de B. Cada thread calcula 8×8 elementos em registradores, com linhas e colunas espaçadas para não ter conflito de banco e para escrever C de forma coalescida. A intensidade aritmética sai de 0,25 FLOP/byte no kernel ingênuo para dezenas por byte da HBM, e a shared memory deixa de ser gargalo porque cada leitura dela alimenta 4 FMAs. Trato as bordas com máscaras, confiro contra a CPU e meço TFLOPS com CUDA events. Os próximos passos seriam float4, double buffering com cp.async e autotuning."

Resposta completa

Use a sequência de três versões:

1. **Ingênua** (Capítulo 3): uma thread por elemento. Correta, simples, lenta. Escreva em 1 minuto para ter uma referência.
2. **Tiling em shared memory** (Q017): reuso de 32×; dois `__syncthreads()`; coalescing nas cargas.
3. **Tiling 2D em registradores** (seção 5.2): reuso em shared e em registradores.

Para cada uma, faça a conta de reuso:

Fazendo a conta: bytes da HBM por FLOP. Para um tile de bloco $BM \times BN$, a cada passo de BK:

- Bytes lidos da HBM: $(BM + BN) \times BK \times 4$.
- FLOPs: $2 \times BM \times BN \times BK$.
- Intensidade: $\frac{2 \cdot BM \cdot BN}{4(BM + BN)}$.

Com 32×32: 2 × 1.024 / (4 × 64) = **8 FLOP/byte**. Com 128×128: 2 × 16.384 / (4 × 256) = **32 FLOP/byte**.

O pico FP32 de uma H100 é ~67 TFLOPS e a banda é 3,35 TB/s, então o ponto de equilíbrio em FP32 nos CUDA cores é ~20 FLOP/byte. O tile de 32×32 ainda é limitado por memória; o de 128×128 passa do ponto de equilíbrio. Esse é o argumento quantitativo para tiles grandes (com a ajuda da L2, que reduz ainda mais o tráfego real).

E a conta para a shared memory: no kernel 2, cada FMA lê 2 floats da shared. A banda da shared memory é alta, mas não infinita; com o micro-tile 8×8, cada leitura alimenta 4 FMAs, e a shared deixa de limitar.

Explique a **arquitetura** (o "explain your architecture choices" do enunciado):

- **Por que 256 threads**: 8 warps por bloco; com ~8 KB de shared por bloco e uns 128 registradores por thread, cabem 2 blocos por SM (os registradores limitam). São 16 warps por SM: occupancy baixa (25%), mas cada thread tem 64 FMAs independentes por passo: ILP de sobra para esconder a latência da shared memory.
- **Por que BK = 8**: equilíbrio entre shared memory por bloco, número de sincronizações por FLOP e registradores.
- **Por que transpor A**: para que as leituras de `As[k][...]` sejam contíguas no índice de linha da matriz.

Termine com a precisão: "Em FP32, a ordem de soma muda em relação à CPU; comparo com tolerância relativa ao tamanho de K. Se a aplicação aceitar TF32 ou BF16, os tensor cores multiplicam o pico disponível."

Perguntas de acompanhamento prováveis

- **Quantos registradores esse kernel usa?** 64 acumuladores + 16 para `ra` e `rb` + índices e ponteiros: perto de 100 a 128. Confira com `nvcc --resource-usage` ou `-Xptxas -v`.

- **Como você lidaria com matrizes que não são múltiplas de 128?** As máscaras (`gr < M`, etc.) já tratam. Para desempenho, uma alternativa é ter um caminho rápido sem checagens para os tiles internos e um caminho com checagens só para as bordas.
- **E se M for pequeno, como no decode?** Com M = 8, um tile de 128 linhas desperdiça 94% do trabalho. Use tiles com BM pequeno, split-K ou um kernel tipo GEMV que distribui N e K entre os blocos.
- **Por que não usar o cuBLAS?** Em produção, usaria. Escrever o kernel mostra que você entende o que o cuBLAS faz e permite fusões que a biblioteca não oferece.

Armadilhas comuns

- Bug de índice ao misturar linha e coluna entre A, B e C.
- Esquecer as bordas.
- Usar arrays indexados dinamicamente e jogar `acc` para a memória local.
- Não ter uma forma de verificar o resultado.
- Não saber calcular a intensidade aritmética das versões.

Q034 Projete uma redução CUDA determinística que garanta uma ordem de redução bem definida. (Dica dada: parecido com prefix sum.)

> **[Relatado]** Applied (empresa listada) · Fundamentos de engenharia de software · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

“O não determinismo vem dos atômicos e de qualquer coisa que dependa da ordem de execução dos blocos. Eu fixo a estrutura da árvore: divido a entrada em folhas de tamanho fixo, por exemplo 2.048 elementos; cada bloco reduz uma folha numa ordem fixa, primeiro em registradores e depois numa árvore em shared memory com passos fixos, e escreve o parcial num vetor, na posição do seu índice. Depois aplico o mesmo kernel ao vetor de parciais, nível por nível, até sobrar um valor. É a fase de subida (up-sweep) de um prefix sum. Como a árvore só depende do tamanho da entrada e das constantes, o resultado é bit a bit igual em toda execução e em qualquer GPU com a mesma aritmética.”

Resposta completa

Primeiro, defina o que é “determinístico”. Há dois níveis:

1. **Reprodutível**: mesma entrada, mesma GPU, mesma configuração → mesmo resultado, bit a bit. Basta eliminar a dependência da ordem de execução.
2. **Independente de configuração**: o resultado não muda se você trocar o número de blocos ou a GPU (mais ou menos SMs). Exige que a forma da árvore dependa só de n e de constantes, nunca de `gridDim` ou do número de SMs.

O design abaixo atende os dois.

```
Entrada (n elementos), folhas de tamanho fixo F = 2048

[──── folha 0 ────][──── folha 1 ────][──── folha 2 ────] ...
        │                 │                 │
  bloco 0 reduz      bloco 1 reduz      bloco 2 reduz
  em ordem fixa      em ordem fixa      em ordem fixa
        ▼                 ▼                 ▼
parciais nível 1:  [ p0 ][ p1 ][ p2 ] ... (n/F valores)
                          │
         o MESMO kernel aplicado de novo
                          ▼
parciais nível 2:  [ q0 ][ q1 ] ...       (n/F² valores)
                          ▼
                       total
```

O código:

```
constexpr int BLOCO = 256;            // threads por bloco
constexpr int POR_THREAD = 8;         // elementos por thread
constexpr int FOLHA = BLOCO * POR_THREAD;   // 2048

__global__ void reduz_nivel(const float* __restrict__ x,
                            float* __restrict__ parciais,
                            int n) {
    __shared__ float s[BLOCO];
    const int base = blockIdx.x * FOLHA;

    // 1. cada thread soma seus 8 elementos em ordem fixa;
    //    o índice base + i*BLOCO + tid mantém a leitura
    //    coalescida
    float acc = 0.0f;
    #pragma unroll
    for (int i = 0; i < POR_THREAD; ++i) {
        int idx = base + i * BLOCO + threadIdx.x;
        if (idx < n) acc += x[idx];
    }

    // 2. árvore em shared memory com passos fixos
    s[threadIdx.x] = acc;
    __syncthreads();
    for (int passo = BLOCO / 2; passo > 0; passo /= 2) {
        if (threadIdx.x < passo)
            s[threadIdx.x] += s[threadIdx.x + passo];
        __syncthreads();
    }

    // 3. escreve na posição do bloco: sem atômicos
    if (threadIdx.x == 0) parciais[blockIdx.x] = s[0];
}

// host: aplica níveis até sobrar 1 valor
float soma_deterministica(const float* d_x, int n,
                          float* d_buf1, float* d_buf2) {
    const float* entrada = d_x;
    float* saida = d_buf1;
    while (true) {
        int blocos = (n + FOLHA - 1) / FOLHA;
        reduz_nivel<<<blocos, BLOCO>>>(entrada, saida, n);
        if (blocos == 1) break;
        n = blocos;
        entrada = saida;
        saida = (saida == d_buf1) ? d_buf2 : d_buf1;
    }
    float total;
    cudaMemcpy(&total, saida, sizeof(float),
               cudaMemcpyDeviceToHost);
    return total;
}
```

Por que isso é determinístico:

- Cada thread soma sempre os mesmos elementos, na mesma ordem.

- A árvore da shared memory tem passos fixos: o par que é somado em cada posição não depende do tempo.
- Cada bloco escreve num endereço próprio: nenhuma corrida.
- O número de níveis e o tamanho de cada nível dependem só de n.

A conexão com prefix sum. O algoritmo de Blelloch para prefix sum tem duas fases: **up-sweep** (uma redução em árvore que deixa somas parciais em posições fixas) e **down-sweep** (que distribui os prefixos). A redução determinística é exatamente o up-sweep. Se o entrevistador pedir um prefix sum depois, você reaproveita a mesma estrutura, guardando os parciais de cada nível.

Custo. Comparado com a versão com atômicos (Q026), há lançamentos extras (um por nível; com n = 100 milhões e folhas de 2.048, são 3 níveis) e uma escrita do vetor de parciais. Ambos são desprezíveis perto da leitura da entrada. A versão determinística continua perto do pico de banda.

Indo além: precisão. Determinismo não é exatidão. Para somas longas com valores de magnitudes muito diferentes, você pode:

- Acumular em FP64 dentro do bloco.
- Usar **soma compensada de Kahan** para reduzir o erro de arredondamento.
- Para resultado exato e independente da ordem: converter para ponto fixo com inteiros de 64 bits (ou acumuladores largos) e somar inteiros, que são associativos. É a ideia de bibliotecas de soma reprodutível como a ReproBLAS.

Perguntas de acompanhamento prováveis

- **Por que não usar `atomicAdd` com uma única thread somando os parciais no fim?** Também é determinístico (uma thread soma em ordem), mas é serial. Com milhares de parciais, a árvore paralela é melhor.
- **O que o PyTorch faz?** `torch.use_deterministic_algorithms(True)` força versões determinísticas das operações que têm uma, ou levanta erro nas que não têm. Algumas reduções e o backward de certas operações usam atômicos por padrão.
- **E em inferência de LLMs, onde aparece isso?** Kernels de atenção com split-K ou split-KV, reduções de MoE e all-reduce com tensor parallelism podem mudar a ordem de soma conforme o tamanho do batch. É uma das causas de a mesma pergunta gerar respostas diferentes com temperatura 0 quando o batch muda. Kernels "invariantes ao batch" fixam a ordem de redução para resolver isso.
- **A redução por warp com shuffle é determinística?** Sim. O padrão de trocas é fixo. O que quebra o determinismo é depender da ordem de chegada (atômicos) ou de configuração variável.

Armadilhas comuns

- Achar que remover os atômicos basta, mas deixar a forma da árvore depender de `gridDim` ou do número de SMs.
- Confundir determinismo com exatidão.
- Ler a entrada com padrão não coalescido (cada thread lendo 8 elementos seguidos).
- Esquecer de sincronizar entre os níveis (no host, os lançamentos na mesma stream já são ordenados; diga isso).

Q035 Como funcionam o TensorRT-LLM e o vLLM? (perguntada junto com “implemente um loop de inferência em PyTorch”)

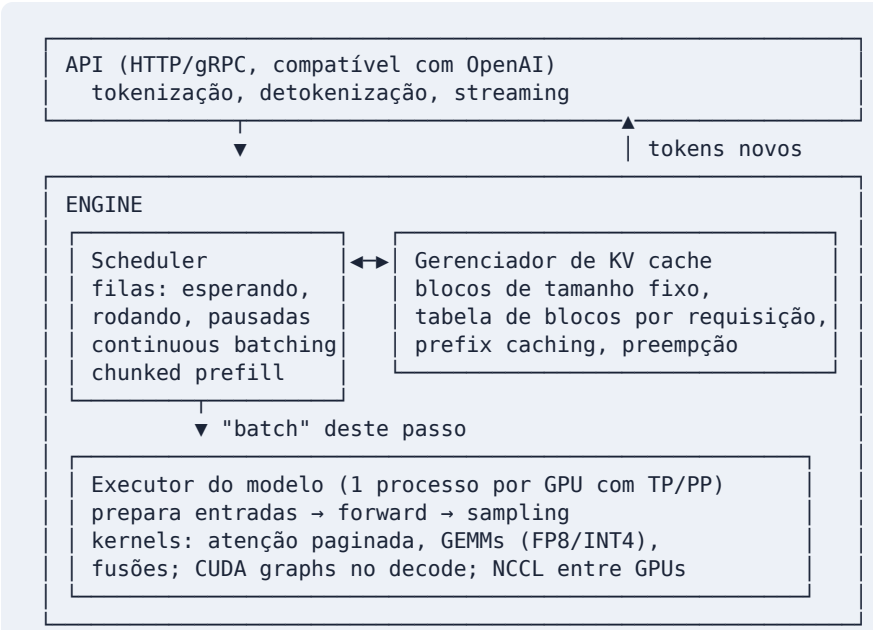
> **[Relatado]** NVIDIA · Senior Software Engineer · entrevista técnica · 2026 · Médio · fonte

Resposta em 30 segundos

“Os dois são motores de serving de LLM com as mesmas peças: um scheduler com continuous batching, que a cada passo decide quais requisições rodam; um gerenciador de KV cache paginado; kernels otimizados de atenção e GEMM, com quantização; CUDA graphs para o decode; e paralelismo multi-GPU com NCCL. O vLLM nasceu do PagedAttention, é escrito principalmente em Python com kernels em CUDA e Triton, roda modelos do Hugging Face quase direto e tem um servidor compatível com a API da OpenAI. O TensorRT-LLM é da NVIDIA: historicamente compilava o modelo num engine TensorRT com plugins otimizados, com um runtime em C++ que faz in-flight batching, e tem os kernels mais afinados para GPUs NVIDIA, principalmente em FP8. Em geral, o vLLM ganha em flexibilidade e cobertura de modelos, e o TensorRT-LLM em desempenho máximo em hardware NVIDIA.”

Resposta completa

Comece pela arquitetura comum, porque ela é a resposta verdadeira à pergunta “como funcionam”:



O laço principal, em ambos:

```
enquanto houver requisições:
  1. scheduler escolhe o batch do passo, respeitando:
     - memória livre para o KV cache (blocos)
     - orçamento de tokens por passo (prefill em pedaços)
     - prioridades; se faltar memória, preempta alguém
  2. aloca blocos de KV para os tokens novos
  3. executa o forward de todos juntos (prefill + decode)
  4. faz o sampling, anexa os tokens novos
  5. requisições que terminaram liberam seus blocos
  6. envia os tokens novos para os clientes (stream)
```

vLLM.

- **Origem**: o paper do PagedAttention (Kwon et al., 2023). A ideia central é gerenciar o KV cache como memória virtual paginada, o que elimina a fragmentação e permite batches muito maiores (Capítulo 6).
- **Implementação**: núcleo em Python, kernels em CUDA, C++ e Triton, e integração com bibliotecas de atenção (FlashAttention, FlashInfer). Usa `torch.compile` e CUDA graphs.
- **Arquitetura recente (V1)**: o núcleo do engine roda num processo separado do servidor HTTP, para que tokenização e rede não atrasem o scheduler. O scheduler trata prefill e decode de forma unificada (“quantos tokens cada requisição processa neste passo”), o que dá chunked prefill e prefix caching de forma natural.

- **Pontos fortes**: suporta muitos modelos e arquiteturas (incluindo MoE e multimodais), muitos formatos de quantização (FP8, AWQ, GPTQ e outros), speculative decoding, multi-LoRA, vários fabricantes de hardware, e uma comunidade grande.

TensorRT-LLM.

- **Origem**: NVIDIA, construído sobre o TensorRT (o compilador de inferência da NVIDIA).
- **Fluxo clássico**: o modelo é definido numa API Python, convertido e **compilado** num engine TensorRT para uma GPU e configuração específicas (tamanhos máximos de batch e sequência, paralelismo, precisão). Operações críticas, como a atenção, entram como **plugins** com kernels escritos à mão. As versões recentes também oferecem um caminho baseado em PyTorch, sem a etapa separada de build, para facilitar o uso.
- **Runtime em C++** com **in-flight batching** (o nome da NVIDIA para continuous batching), KV cache paginado, reuso de KV, chunked prefill e speculative decoding.
- **Pontos fortes**: kernels mais otimizados para cada geração de GPU NVIDIA (FP8 na Hopper, FP4 na Blackwell), integração com o Triton Inference Server e com as ferramentas de quantização da NVIDIA (Model Optimizer).
- **Custos**: só NVIDIA, etapa de build e configuração mais complexas, suporte a modelos novos às vezes chega depois.

Comparação direta:

	vLLM	TensorRT-LLM
Linguagem principal	Python + kernels CUDA/Triton	C++ runtime + Python
Preparação	carrega pesos do HF direto	build de engine (ou caminho PyTorch)
Hardware	NVIDIA, AMD, outros	só NVIDIA
Modelos novos	costumam chegar rápido	podem demorar mais
Desempenho	muito bom	geralmente o melhor em NVIDIA
Ecossistema	comunidade aberta grande	integrado ao stack NVIDIA

Diga também que o **SGLang** é um terceiro motor importante, conhecido pelo RadixAttention (prefix caching com uma árvore de prefixos) e por um runtime eficiente para programas com várias chamadas ao LLM.

Como a pergunta veio junto com "implemente um loop de inferência", feche ligando os dois: "Um loop de inferência simples em PyTorch (Q097) faz prefill, depois decode com KV cache, um token por vez, para um batch fixo. O que esses motores adicionam é o que acontece entre os passos: entrar e sair do batch a qualquer momento, gerenciar a memória do cache em páginas, e kernels e grafos que eliminam o overhead."

Perguntas de acompanhamento prováveis

- **Quando TensorRT-LLM ganha do vLLM?** Ver Q051.
- **O que é o engine do TensorRT, exatamente?** Um plano de execução otimizado e serializado para uma GPU específica: kernels escolhidos por autotuning, fusões aplicadas, memória planejada. Não é portável entre gerações de GPU.
- **Como o vLLM implementa tensor parallelism?** Um processo (worker) por GPU, cada um com sua fatia dos pesos; as camadas lineares são divididas em colunas e linhas no estilo Megatron, com all-reduce via NCCL (Capítulo 9).
- **O que é o Triton Inference Server?** Outro produto, sem relação com a linguagem Triton: um servidor de modelos genérico da NVIDIA, com batching, múltiplos modelos e backends (incluindo TensorRT-LLM e vLLM). É fácil confundir os dois nomes numa entrevista.

Armadilhas comuns

- Confundir Triton (linguagem de kernels) com Triton Inference Server.
- Descrever só "o vLLM usa PagedAttention" sem falar de scheduler, batching e kernels.
- Afirmar números de desempenho sem contexto (modelo, hardware, carga).
- Não saber o que é in-flight batching.

Resumo do capítulo

- Todo programa CUDA: alocar, copiar, lançar, checar erro, copiar de volta, comparar com referência.
- GEMM FP32 rápida: tiling de bloco (128×128) em shared memory + micro-tile (8×8) em registradores, A transposto com padding, linhas e colunas espaçadas por thread.
- Intensidade da GEMM em tiles: $2 \cdot BM \cdot BN/(4(BM + BN))$ FLOP/byte; tiles maiores passam do ponto de equilíbrio.
- Reduções: árvore, shuffle no warp, grid-stride. `atomicAdd` em float não é determinístico.

- Redução determinística: folhas de tamanho fixo, árvore fixa, escrita sem atômicos, níveis repetidos (up-sweep do prefix sum).
- Triton programa blocos, não threads; compilador cuida de shared memory, coalescing e tensor cores (`tl.dot`).
- vLLM e TensorRT-LLM têm as mesmas peças: scheduler com continuous batching, KV cache paginado, kernels otimizados, CUDA graphs, paralelismo com NCCL.

Checklist para a entrevista

- [ ] Escrevo o esqueleto de um programa CUDA com checagem de erros e verificação.
- [ ] Escrevo a GEMM com tiling em shared memory sem consultar.
- [ ] Explico o tiling 2D em registradores e o mapeamento espaçado de linhas e colunas.
- [ ] Calculo a intensidade aritmética para cada tamanho de tile.
- [ ] Escrevo reduções por warp com `__shfl_down_sync` e `__shfl_xor_sync`.
- [ ] Explico por que somas em float não são determinísticas.
- [ ] Projeto uma redução determinística e independente da configuração.
- [ ] Escrevo um softmax fundido em Triton.
- [ ] Explico quando usar Triton e quando usar CUDA.
- [ ] Desenho a arquitetura de um motor de inferência e comparo vLLM e TensorRT-LLM.

CAPÍTULO 6

KV Cache e Gerenciamento de Memória

Se você só puder estudar um assunto a fundo antes de uma entrevista de inferência, estude este. O levantamento das vagas foi direto: **o KV cache é o recurso escasso**. Os entrevistadores voltam a ele o tempo todo: qual o tamanho, como paginar, como compartilhar, o que fazer quando acaba.

A razão é que o KV cache define quantos usuários cabem numa GPU ao mesmo tempo. E quantos usuários cabem define o throughput, que define o custo. Quase toda técnica moderna de serving é, no fundo, uma forma de guardar mais KV cache na mesma memória, de reaproveitá-lo ou de movê-lo para outro lugar.

O conceito básico (o que é, por que existe, a fórmula do tamanho) está no Capítulo 2. Este capítulo trata do **gerenciamento**.

6.1 Para onde vai a memória da GPU

Uma GPU servindo um LLM divide a memória assim:

```
H100 80 GB servindo Llama 3 8B em BF16 (aproximado)

┌──────────────────────────────────────────────────────┐
│ Pesos do modelo                          16 GB       │
├──────────────────────────────────────────────────────┤
│ Ativações temporárias do forward          2–4 GB     │
│ (pico no prefill de prompts longos)                  │
├──────────────────────────────────────────────────────┤
│ Contexto CUDA, bibliotecas, CUDA graphs   1–3 GB     │
├──────────────────────────────────────────────────────┤
│                                                      │
│ KV CACHE: todo o resto                  ~55–60 GB    │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Motores como o vLLM fazem exatamente isso na subida: carregam os pesos, rodam um forward de teste com o maior batch possível para medir o pico de ativações, e reservam a memória restante (até um limite configurável, como `gpu_memory_utilization=0.9`) como um grande depósito de blocos de KV cache.

Fazendo a conta: quantos usuários cabem?

Llama 3 8B, 1 H100, BF16:

- Memória para o KV: ~58 GB.
- Por token: 128 KiB (131.072 bytes).
- Tokens que cabem: $58 \times 10^9 / 131.072 \approx$ **442 mil tokens**.
- Com conversas de 4.000 tokens em média: ~**110 sequências** simultâneas.
- Com conversas de 32.000 tokens: ~**13**.

Llama 3 70B, 8 H100 com tensor parallelism, BF16:

- Memória total: 640 GB. Pesos: 140 GB. Reserva para o resto: ~40 GB.
- Memória para o KV: ~460 GB.
- Por token: 320 KiB (327.680 bytes).
- Tokens: $460 \times 10^9 / 327.680 \approx$ **1,4 milhão**.
- Com 4.000 tokens por conversa: ~**350 sequências**.

Compare com o Capítulo 1: o decode precisa de batches de centenas para usar bem a GPU. Com contexto longo, a memória acaba antes disso. Esse é o conflito central do serving de LLMs.

6.2 A tabela que você deve saber montar

Modelo	Camadas	KV heads	head_dim	KV por token (BF16)	KV por token (FP8)
Llama 3 8B	32	8	128	128 KiB	64 KiB
Llama 3 70B	80	8	128	320 KiB	160 KiB
70B hipotético com MHA	80	64	128	2,5 MiB	1,25 MiB

Fórmula, mais uma vez:

$$\text{KV por token} = 2 \times n_{camadas} \times n_{kv_heads} \times d_{head} \times \text{bytes}$$

E o total para um batch:

$$\text{KV total} = \text{KV por token} \times \sum_{i=1}^{B} \text{tokens}_i$$

> **Dica de entrevista:** sempre que um design envolver um LLM, faça essa conta logo no início. “Com o 70B, são 320 KB por token; 100 usuários com 8 mil tokens dão 260 GB só de KV cache.” Isso muda o design inteiro (quantas GPUs, que paralelismo, se precisa de offload), e o entrevistador nota que você pensa na restrição certa.

6.3 O problema da alocação ingênua

A primeira geração de servidores alocava, para cada requisição, um tensor **contíguo** com o tamanho máximo possível (`max_seq_len`), porque não dá para saber de antemão quantos tokens a resposta terá.

```
Alocação contígua, max_seq_len = 2048

Req A (usou 300):  [███▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 15% útil
Req B (usou 1500): [████████████████████▒▒▒▒▒▒▒▒▒▒] 73% útil
Req C (usou 50):   [█▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒]  2% útil
                   █ = tokens reais  ▒ = reservado e vazio

+ fragmentação externa: buracos entre as alocações que
  não servem para nenhuma requisição nova
```

O paper do vLLM (Kwon et al., 2023) mediu que sistemas anteriores aproveitavam, para guardar estados reais de tokens, só algo entre 20% e 40% da memória reservada para o KV cache. O resto era desperdício: reserva para tokens que nunca vieram e fragmentação.

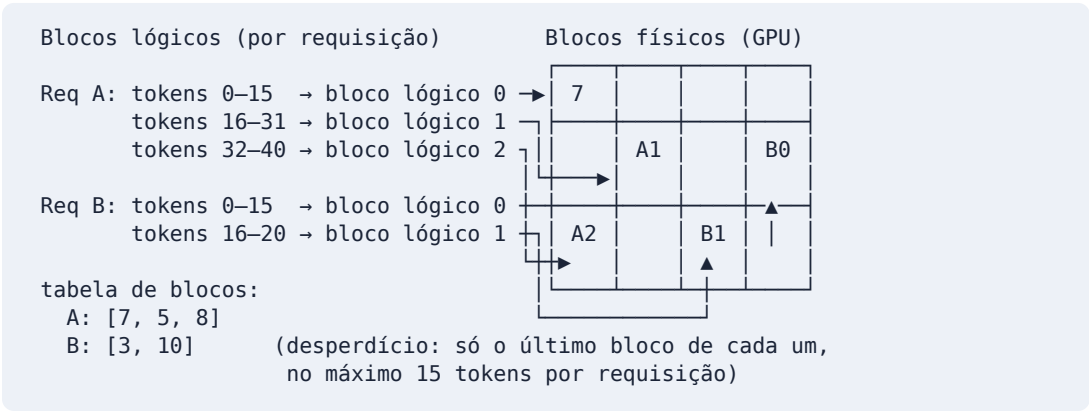
6.4 PagedAttention

A solução veio dos sistemas operacionais. Um processo enxerga sua memória como contínua, mas o sistema operacional a guarda em **páginas** de tamanho fixo espalhadas pela memória física, com uma **tabela de páginas** fazendo a tradução.

O **PagedAttention** faz o mesmo com o KV cache:

- A memória do KV é dividida em **blocos físicos** de tamanho fixo (por exemplo, 16 tokens cada, para todas as camadas).
- Cada requisição tem uma **tabela de blocos**: a lista de blocos físicos que guardam seus tokens, na ordem lógica.
- Blocos são alocados **sob demanda**: quando os 16 tokens do último bloco enchem, aloca-se mais um.

- O kernel de atenção recebe a tabela de blocos e lê os K e V de onde eles estiverem.



Os ganhos:

1. **Desperdício quase zero.** Só o último bloco de cada requisição fica parcialmente vazio. Com blocos de 16 tokens, a perda média é de uns 8 tokens por sequência.
2. **Sem fragmentação externa.** Todo bloco livre serve para qualquer requisição.
3. **Compartilhamento.** Duas requisições podem apontar para os **mesmos blocos físicos** (contagem de referências). Isso permite:
   - **Parallel sampling** (gerar *n* respostas para o mesmo prompt): o prompt é guardado uma vez.
   - **Beam search**: os feixes compartilham o prefixo comum.
   - **Prefix caching** entre requisições diferentes (seção 6.5).
4. **Copy-on-write.** Se duas sequências compartilham um bloco e uma delas precisa escrever nele (o último bloco, parcialmente cheio), o bloco é copiado antes. Só o bloco, não o cache inteiro.

O custo: o kernel de atenção precisa fazer uma leitura indireta (consultar a tabela, depois buscar o bloco). Os blocos são grandes o bastante (16 tokens × 8 heads × 128 dimensões × 2 bytes = 32 KB para K, por camada) para que a leitura de cada bloco continue eficiente. Na prática, o ganho de batch maior compensa muito.

O tamanho do bloco

Bloco menor (ex.: 8)	Bloco maior (ex.: 32 ou 64)
menos desperdício no último bloco	leituras maiores e mais eficientes no kernel
tabela de blocos maior	menos entradas na tabela

Bloco menor (ex.: 8)	Bloco maior (ex.: 32 ou 64)
compartilhamento de prefixo mais fino	prefix caching só em múltiplos maiores

O padrão de 16 tokens é um meio-termo comum. Alguns kernels e hardwares preferem blocos maiores.

6.5 Prefix caching

Muitas requisições começam igual:

- O mesmo **system prompt** de 2.000 tokens em todas as chamadas de um produto.
- Num **chat**, cada turno reenvia a conversa inteira: o turno 10 contém os turnos 1 a 9.
- **Agentes** reenviam o mesmo contexto com uma instrução nova a cada passo.
- **Few-shot**: os mesmos exemplos antes de cada pergunta.

Sem cache, o prefill recalcula tudo isso toda vez. Com **prefix caching**, os blocos de KV de prefixos já calculados ficam guardados, e uma requisição nova reaproveita os blocos cujo prefixo bate.

Como funciona (estilo vLLM)

Cada bloco **cheio** recebe uma chave (hash) calculada a partir do conteúdo do bloco **e da chave do bloco anterior**. Assim, a chave de um bloco identifica o prefixo inteiro até ele, não só seus 16 tokens.

```
Req 1: [sys prompt...........][pergunta 1]
        bloco0 bloco1 ... bloco124  bloco125...

hash(b0)   = H(tokens 0–15)
hash(b1)   = H(hash(b0), tokens 16–31)
hash(b124) = H(hash(b123), tokens 1984–1999)

Req 2: [sys prompt...........][pergunta 2]
        mesmos hashes até b124 → REUSA 125 blocos
        prefill só da pergunta 2
```

Quando uma requisição termina, seus blocos não são apagados na hora: vão para uma lista de **blocos livres, mas ainda com conteúdo válido**. Se outra requisição precisar desse prefixo, ele é "ressuscitado". Se a memória apertar, os blocos menos usados recentemente (LRU) são reutilizados para outra coisa.

O **RadixAttention**, do SGLang, organiza os prefixos numa **árvore radix** (uma árvore de prefixos comprimida), o que facilita encontrar o maior prefixo em comum e decidir o que despejar.

> **Fazendo a conta: o ganho num chat.** Conversa no turno 10, com 6.000 tokens de histórico e 100 tokens novos, no 70B em 8 H100.
>
> - Sem cache: prefill de 6.100 tokens. Pela conta do Capítulo 2, algo como 0,2 s.
> - Com cache (histórico ainda na memória): prefill de 100 tokens. Poucos milissegundos.
> - O TTFT cai de centenas para dezenas de milissegundos, e o custo de cálculo do prefill cai ~98%.

6.6 Quando a memória acaba: preempção

Mesmo com paginação, a memória acaba: as sequências crescem a cada passo de decode. Se não houver blocos livres para o próximo token de alguém, o scheduler precisa **liberar memória**. As opções:

```
Sem blocos livres no passo t
  │
  ├─ 1. Não admitir requisições novas (fila de espera)
  │
  ├─ 2. PREEMPTAR uma sequência em andamento
  │     (normalmente a que chegou por último):
  │
  │     a) RECOMPUTAR: joga fora o KV dela; quando
  │        voltar, refaz o prefill de prompt + tokens
  │        já gerados. Custo: cálculo.
  │
  │     b) SWAP: copia os blocos dela para a memória da
  │        CPU (PCIe); quando voltar, copia de volta.
  │        Custo: banda PCIe e memória da CPU.
  │
  └─ 3. Despejar blocos de prefix cache não usados (LRU)
        (sempre a primeira coisa a fazer)
```

Recomputar costuma ser mais simples e, para sequências curtas, mais rápido que fazer swap. Para sequências longas, o swap (ou o offload em camadas, a seguir) pode compensar. Preempção frequente é um sinal de que o batch está grande demais para a memória: o sistema fica refazendo trabalho, e a latência das vítimas explode.

6.7 Hierarquia de armazenamento do KV cache

A HBM é pequena e cara. Sistemas modernos tratam o KV cache como dados numa **hierarquia**, como os caches de um processador:

Camada	Capacidade típica	Banda até a GPU	Uso
HBM da GPU	dezenas de GB	~3,35 TB/s	sequências ativas
DRAM da CPU (mesmo nó)	centenas de GB a TB	~64 GB/s por GPU (PCIe Gen5)	conversas pausadas, prefixos populares
SSD NVMe local	TBs	alguns GB/s	prefixos frios
Armazenamento remoto (outros nós)	muito grande	rede (RDMA)	compartilhar prefixos entre réplicas

Sistemas como o **Mooncake** (Kimi, da Moonshot AI) e o **LMCache** constroem esse tipo de camada distribuída de KV cache.

A pergunta-chave é sempre: **é mais rápido trazer o KV de volta ou recalcular?**

> **Fazendo a conta:** retomar uma conversa de 8.000 tokens no 70B (2,6 GB de KV).
>
> - Trazer da DRAM da CPU: com 8 GPUs, cada uma traz sua fatia (~330 MB) a ~50 GB/s efetivos: **~7 ms**. Mesmo com uma GPU só, 2,6 GB a 50 GB/s seriam ~50 ms.
> - Recalcular (prefill de 8.000 tokens em 8 H100): **~0,3 s** e ocupa a GPU inteira.
> - Conclusão: para contextos longos, guardar na CPU é muito melhor. Para contextos curtos (centenas de tokens), recalcular é tão rápido que a complexidade do offload pode não valer.

6.8 Diminuindo o próprio KV cache

Além de gerenciar melhor, dá para **guardar menos**:

1. Quantizar o cache.

- **FP8** (E4M3) para K e V: metade da memória e da leitura, com fatores de escala por tensor ou por head. Costuma ter perda de qualidade pequena e é suportado pelos principais motores.

- **INT4 e menos**: métodos como o KIVI quantizam K por canal e V por token, porque os outliers de K se concentram em certos canais. Mais economia, mais risco de qualidade.

2. Arquitetura do modelo (decidido no treino):

- **GQA, MQA, MLA**: menos heads de KV ou KV comprimido (Capítulo 2).
- **Sliding window**: algumas ou todas as camadas só olham para os últimos W tokens. O cache dessas camadas fica limitado a W tokens (um buffer circular).
- **Camadas híbridas**: modelos que alternam camadas de atenção local (janela) com camadas de atenção global, e às vezes camadas sem KV cache nenhum (como camadas de modelos de espaço de estados), reduzindo muito o cache total.
- **Compartilhamento entre camadas**: camadas vizinhas usando o mesmo KV.

3. Despejar tokens (eviction) em tempo de inferência:

- **StreamingLLM**: mantém os primeiros tokens (os "attention sinks", que recebem muita atenção por razões estruturais) mais uma janela recente.
- **H2O, SnapKV** e similares: mantêm os tokens que receberam mais atenção e descartam o resto.

Essas técnicas de despejo **mudam a saída do modelo** e podem fazer ele "esquecer" algo importante do começo do contexto. Use com avaliação cuidadosa.

Perguntas de entrevista comentadas

Q041 Projete e implemente um cache de chave-valor para inferência autorregressiva num decoder Transformer: shapes dos tensores no prefill e no decode, API de cache por camada, batching e beam search, precisão mista.

> **[Relatado]** Applied Intuition · Machine Learning Engineer · triagem técnica · 2025 · Difícil · fonte

Resposta em 30 segundos

"Pré-aloco, por camada, K e V com shape [batch, kv_heads, max_len, head_dim] no dtype do modelo, mais um vetor de comprimentos por sequência. A API é `update(camada, k_novo, v_novo)`, que escreve os tokens novos na posição de cada sequência e devolve as views válidas para a atenção. No prefill entram T tokens por

sequência; no decode, 1. Para batch com tamanhos diferentes, guardo comprimentos por sequência e passo uma máscara para a atenção. Para beam search, reordeno o eixo do batch com `index_select` quando os feixes mudam de pai. Para precisão mista, o cache pode ter dtype próprio, por exemplo FP8 com escala, convertido na leitura. Em produção, trocaria tudo por blocos paginados."

Resposta completa

1. Shapes. Para um batch de B sequências, H_{kv} heads de KV, D = head_dim:

```
                entrada x        K, V novos            cache (por camada)
prefill         [B, T, d]        [B, H_kv, T, D]       [B, H_kv, T_max, D]
decode          [B, 1, d]        [B, H_kv, 1, D]       (escreve na posição
                                                        comprimento[b])
atenção usa     Q: [B, H_q, T_q, D]   K, V: [B, H_kv, T_k, D]
```

2. A implementação.

```python
import torch

class KVCache:
    """Cache pré-alocado, um par (K, V) por camada."""

    def __init__(self, n_camadas, batch, n_kv_heads, head_dim,
                 max_len, dtype=torch.bfloat16, device="cuda"):
        shape = (batch, n_kv_heads, max_len, head_dim)
        self.k = [torch.zeros(shape, dtype=dtype, device=device)
                  for _ in range(n_camadas)]
        self.v = [torch.zeros(shape, dtype=dtype, device=device)
                  for _ in range(n_camadas)]
        # tokens válidos por sequência (iguais se sem padding)
        self.comp = torch.zeros(batch, dtype=torch.long,
                                device=device)
        self.max_len = max_len

    def update(self, camada, k_novo, v_novo):
        """k_novo, v_novo: [B, H_kv, T_novo, D], já com RoPE
        em K. Escreve após o fim de cada sequência e devolve
        K e V até o maior comprimento do batch."""
        B, _, t, _ = k_novo.shape
        ini = self.comp                          # [B]
        if int(ini.max()) + t > self.max_len:
            raise ValueError("contexto maior que max_len")
        # posições de destino de cada token novo: [B, t]
        pos = ini[:, None] + torch.arange(t, device=ini.device)
        b_idx = torch.arange(B, device=ini.device)[:, None]
        # indexação avançada: escreve [B, t, H, D] de uma vez
        kc, vc = self.k[camada], self.v[camada]
        kc.permute(0, 2, 1, 3)[b_idx, pos] = (
            k_novo.permute(0, 2, 1, 3).to(kc.dtype))
        vc.permute(0, 2, 1, 3)[b_idx, pos] = (
            v_novo.permute(0, 2, 1, 3).to(vc.dtype))
        fim = int((ini + t).max())
        return kc[:, :, :fim], vc[:, :, :fim]

    def avancar(self, t):
        """Chamar UMA vez por passo, depois de todas as
        camadas: os comprimentos valem para todas elas."""
        self.comp += t

    def reordenar(self, indices):
        """Beam search: indices[b] = feixe pai do novo b."""
        for c in range(len(self.k)):
            self.k[c] = self.k[c].index_select(0, indices)
            self.v[c] = self.v[c].index_select(0, indices)
        self.comp = self.comp.index_select(0, indices)
```

O uso dentro de uma camada de atenção:

```
def atencao_camada(x, camada, cache, pesos, pos_ids, mask):
    # x: [B, T, d]; pos_ids: [B, T] posições ABSOLUTAS
    q, k, v = projeta_qkv(x, pesos)        # [B, H, T, D]
    q, k = aplica_rope(q, pos_ids), aplica_rope(k, pos_ids)
    k_tudo, v_tudo = cache.update(camada, k, v)
    k_tudo = expande_gqa(k_tudo, q.shape[1])   # só na conta
    v_tudo = expande_gqa(v_tudo, q.shape[1])
    # mask: [B, 1, T, T_k] combina causal + padding
    out = torch.nn.functional.scaled_dot_product_attention(
        q, k_tudo.to(q.dtype), v_tudo.to(q.dtype),
        attn_mask=mask)
    return projeta_saida(out, pesos)
```

Depois do forward de todas as camadas, o loop chama `cache.avancar(T)`.

3. Batching com tamanhos diferentes. Com o vetor `comp`, cada sequência escreve na sua própria posição. A atenção recebe K e V até o maior comprimento do batch, e a **máscara** precisa esconder, para cada sequência, as posições além do seu comprimento e as posições futuras. As posições do RoPE vêm de `comp[b] + i`, por sequência.

4. Beam search. A cada passo, cada feixe novo descende de um feixe antigo. `reordenar` aplica essa permutação ao eixo do batch. Isso copia o cache inteiro a cada passo, o que é caro. A alternativa eficiente é o cache paginado com contagem de referências: os feixes compartilham os blocos do prefixo comum e só copiam o último bloco quando divergem (copy-on-write).

5. Precisão mista.

- O cache pode ter `dtype` diferente do cálculo: BF16 para o cálculo, FP8 para o cache.
- Com FP8, guarde também **fatores de escala** (por tensor, por head ou por bloco). Quantize na escrita: $k_{fp8} = \text{round}(k/s)$; dequantize na leitura: $k = k_{fp8} \times s$. Kernels de atenção que leem FP8 direto fazem isso dentro do kernel, sem materializar a versão BF16.
- A atenção em si (scores e softmax) deve acumular em FP32.

6. O que mudaria para produção.

- Blocos paginados em vez de `max_len` contíguo (seção 6.4).
- Layout "ragged": sem padding, todas as sequências concatenadas num eixo só, com offsets.
- Kernel de atenção que lê o cache direto pela tabela de blocos e entende GQA (sem `expande_gqa`).

- Prefix caching e preempção.

Perguntas de acompanhamento prováveis

- **Por que não usar `torch.cat` a cada passo?** Realoca e copia o cache inteiro em cada passo: custo quadrático no tamanho da sequência e fragmentação no alocador. Pré-alocar escreve só os tokens novos.
- **Qual o custo de memória da pré-alocação com `max_len`?** Reserva o pior caso para todo mundo. Com `max_len` de 8 mil e respostas de 500 tokens em média, a maior parte fica vazia. É o desperdício que o PagedAttention resolve.
- **Como testar essa classe?** Gerar com greedy usando o cache e comparar com a geração sem cache; testar com batch de tamanhos diferentes e verificar que a saída de cada sequência não depende das outras; testar `reordenar` com uma permutação conhecida.
- **E com tensor parallelism?** Cada GPU guarda só as heads de KV da sua fatia (H_{kv}/TP). Com TP maior que o número de heads de KV (por exemplo, TP=16 com 8 heads), as heads são replicadas.

Armadilhas comuns

- Esquecer o vetor de comprimentos por sequência e quebrar batches com padding.
- Avançar o comprimento dentro de `update` (uma vez por camada, multiplicando o deslocamento pelo número de camadas).
- Aplicar RoPE no cache em vez de nos tokens novos.
- Expandir o GQA dentro do cache, anulando a economia de memória.
- Ignorar a reordenação no beam search.

Q042 Sua inferência de LLM está ficando sem memória de GPU em conversas longas. O que você faz?

> **[Fornecedor]** Meta, conforme post de um profissional sobre sua entrevista · MLE · 2025-2026 · Médio · fonte

Resposta em 30 segundos

"Primeiro descubro o que está ocupando a memória: pesos, KV cache, ativações de pico ou fragmentação. Em conversas longas, quase sempre é o KV cache ou o pico de ativações no prefill. Ações imediatas: limitar o contexto máximo e o número de tokens por batch, garantir que o motor tenha preempção em vez de OOM, e calcular

logits só do último token. Depois: KV cache paginado, quantização do KV para FP8, chunked prefill para limitar o pico de ativações, prefix caching para não duplicar o histórico, offload do KV para a CPU em conversas pausadas e, se preciso, mais GPUs com tensor parallelism. No produto: resumir ou truncar o histórico."

Resposta completa

1. Diagnóstico: onde está a memória?

```
OOM em conversa longa
  │
  ├─ Acontece no PREFILL de um prompt longo?
  │    → pico de ATIVAÇÕES. Suspeitos:
  │      - logits de TODOS os tokens do prompt
  │        (T × vocabulário × 4 bytes)
  │      - atenção sem FlashAttention (T × T)
  │      - MLP intermediário (T × 14.336 × 2 bytes)
  │
  ├─ Acontece no DECODE, crescendo com o tempo?
  │    → KV CACHE sem limite: batch × contexto grande
  │      demais, sem preempção
  │
  └─ Acontece com memória "livre" sobrando?
       → FRAGMENTAÇÃO do alocador (muitas alocações
         de tamanhos variados); ver
         torch.cuda.memory_summary()
```

Ferramentas: `torch.cuda.memory_summary()`, `torch.cuda.max_memory_allocated()`, o snapshot de memória do PyTorch (`torch.cuda.memory._record_memory_history()` + visualizador), e as métricas do motor (uso do KV cache em %, número de preempções).

> **Fazendo a conta: o assassino silencioso, os logits.** Um prompt de 32.000 tokens no Llama 3 (vocabulário 128.256), com logits em FP32:
>
> - 32.000 × 128.256 × 4 bytes ≈ **16,4 GB**. Só para os logits do prefill.
> - Mas só o logit do **último** token importa para gerar o próximo.
> - Calcular o LM head só na última posição elimina esse pico. Motores de inferência fazem isso; código de pesquisa adaptado às pressas muitas vezes não faz.

2. Ações imediatas (horas).

- **Limites**: contexto máximo por requisição, tokens máximos por batch, número máximo de sequências simultâneas.

- **Preempção em vez de OOM**: o scheduler deve recusar ou pausar requisições quando o KV cache enche, em vez de deixar o processo morrer e levar todos os usuários junto.
- **Logits só do último token**, ativações liberadas camada a camada, `torch.inference_mode()`.

3. Ações de engenharia (dias a semanas).

Técnica	Ataca	Ganho típico
KV cache paginado	desperdício e fragmentação	de 2 a 4× mais sequências que a alocação contígua
KV em FP8	tamanho do cache	2× mais tokens
Chunked prefill	pico de ativações no prefill	pico limitado pelo tamanho do chunk
Prefix caching	histórico duplicado entre turnos e usuários	depende da carga
Offload para CPU	conversas pausadas ocupando HBM	libera HBM para as ativas
Mais GPUs (TP)	capacidade total	proporcional

4. Ações de produto e modelo.

- **Gerenciar o contexto**: resumir turnos antigos, truncar, ou usar busca (RAG) para trazer só os trechos relevantes do histórico.
- **Modelo com cache menor**: GQA/MLA, sliding window ou arquitetura híbrida, se trocar de modelo for uma opção.
- **Limitar** `max_tokens` de saída.

5. Prevenção. Teste de carga com a distribuição real de tamanhos de conversa (incluindo a cauda longa), alertas de uso do KV cache e de preempções, e um limite de admissão baseado em tokens, não em número de requisições.

Perguntas de acompanhamento prováveis

- **Por que não simplesmente usar uma GPU com mais memória?** Ajuda, mas o problema volta com contextos maiores. E o desperdício (alocação contígua, logits completos) continua lá. Primeiro elimine o desperdício.

- **O que é melhor: recomputar ou fazer swap ao preemptar?** Recomputar para sequências curtas (prefill rápido); swap ou offload para sequências longas. Ver seção 6.6.
- **Como o chunked prefill reduz o pico de memória?** As ativações temporárias (MLP intermediário, scores parciais) são proporcionais ao número de tokens processados no passo. Com chunks de 2.048, o pico corresponde a 2.048 tokens, não a 32.000.
- **Truncar o histórico não piora a qualidade?** Pode piorar. Por isso resumir ou buscar os trechos relevantes costuma ser melhor que cortar. Meça com avaliações de conversas longas.

Armadilhas comuns

- Responder só "uso uma GPU maior" ou "diminuo o batch".
- Não separar pico de ativações (prefill) de crescimento do KV cache (decode).
- Esquecer os logits do prompt inteiro.
- Não mencionar preempção e controle de admissão.

Q043 Um assistente de chat reenvia a conversa inteira a cada turno. Como evitar recalculá-la, e quais são as armadilhas?

> **[Fornecedor]** Moonshot AI (Kimi), segundo banco público de perguntas por empresa · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"Com prefix caching: guardo os blocos de KV de conversas recentes, identificados por um hash encadeado dos tokens, e quando chega o turno seguinte reaproveito todos os blocos cujo prefixo bate, fazendo prefill só da parte nova. As armadilhas: o prefixo tem que bater token por token, e mudanças no template do chat, timestamps no system prompt ou tokenização diferente na fronteira quebram o reuso; o cache é local de cada réplica, então preciso de roteamento que leve a conversa para a réplica que tem o cache; a memória é limitada, então preciso de uma política de despejo e talvez de uma camada na CPU; a chave precisa incluir modelo, versão, adaptador LoRA e entradas multimodais; e, em ambiente multi-tenant, o cache compartilhado pode vazar informação por diferença de tempo de resposta, então isolo por tenant quando necessário."

Resposta completa

A solução. Prefix caching (seção 6.5): hash encadeado por bloco, blocos liberados ficam disponíveis com conteúdo válido, LRU para despejo. Num chat, o turno n reaproveita o KV de todos os turnos anteriores, **inclusive a resposta do assistente** gerada no turno $n-1$, que já está no cache porque foi gerada ali.

```
Turno 1: [sys][user1]                → prefill tudo
         gera [asst1] (KV no cache)
Turno 2: [sys][user1][asst1][user2]  → reusa sys+user1+asst1
                                       prefill só de user2
Turno 3: [sys][user1][asst1][user2][asst2][user3]
                                     → reusa até asst2
```

As armadilhas, em detalhe:

1. O prefixo precisa bater token por token.

- **Template do chat**: se o template inclui algo que muda a cada chamada (data e hora no system prompt, um ID de requisição, a lista de ferramentas em ordem diferente), o hash muda logo no começo e **nada** é reaproveitado. Coloque as partes variáveis no fim.
- **Fronteira de tokenização**: a resposta do assistente, ao ser detokenizada e retokenizada no turno seguinte, pode virar uma sequência de tokens diferente da que foi gerada. O prefixo deixa de bater a partir dali. Soluções: guardar os IDs de token gerados, ou aceitar a perda só a partir desse ponto.
- **Edições**: se o usuário edita uma mensagem antiga, tudo a partir dela é inválido.

2. Granularidade de bloco. Só blocos **cheios** são cacheados. Os últimos tokens (menos que um bloco) sempre são recalculados. Com blocos de 16, a perda é pequena.

3. Localidade entre réplicas. O cache vive na memória de uma réplica. Se o turno 2 cai em outra réplica, não há reuso. Soluções:

```
Balanceador ingênuo (round-robin):
  turno 1 → réplica A (cria cache)
  turno 2 → réplica C (cache miss: recalcula tudo)

Roteamento ciente de prefixo:
  hash(prefixo) → réplica preferida (hash consistente)
  turno 2 → réplica A (cache hit)
  + fallback para outra se A estiver sobrecarregada
```

O trade-off é balanceamento: mandar tudo de uma conversa para a mesma réplica pode desequilibrar a carga. Bons roteadores pesam a afinidade de cache contra a fila de cada réplica.

4. Memória e despejo. Conversas inativas ocupam blocos. LRU simples funciona, mas prefixos muito compartilhados (o system prompt de todos os usuários) devem ter prioridade para ficar. Para conversas pausadas por minutos, uma camada de cache na DRAM da CPU (seção 6.7) evita recalcular quando o usuário volta.

5. O que entra na chave. O KV depende de tudo que afeta o forward: tokens, posições, **pesos** (modelo e versão, adaptador LoRA), entradas multimodais (o hash da imagem, não só os tokens de placeholder). Esquecer o adaptador LoRA na chave faz uma requisição usar o KV calculado com outro adaptador: um bug de corretude silencioso.

6. Segurança e isolamento. Se o cache é compartilhado entre clientes diferentes, um atacante pode medir o TTFT para descobrir se um prefixo específico está no cache (ou seja, se outra pessoa enviou aquele texto há pouco). Para dados sensíveis, isole o cache por tenant (um "salt" por tenant no hash) ou desligue o compartilhamento entre tenants.

7. Medição. Acompanhe a taxa de acerto do cache (tokens reaproveitados / tokens de entrada) por produto. Uma queda súbita normalmente indica uma mudança no template.

> **Dica de entrevista:** mencione que provedores de API expõem esse mecanismo como **prompt caching**, cobrando menos pelos tokens de entrada que vêm do cache. Isso mostra que você entende o impacto econômico, não só técnico.

Perguntas de acompanhamento prováveis

- **O cache muda a saída do modelo?** Não deveria. O KV de um prefixo é o mesmo, seja calculado agora ou antes, a menos de pequenas diferenças numéricas entre kernels de prefill e de decode ou entre tamanhos de batch.
- **Os parâmetros de sampling (temperatura) entram na chave?** Não. O KV não depende do sampling, só do forward.
- **E se o histórico passar do contexto máximo e eu truncar o começo?** Tudo muda: as posições de todos os tokens mudam, então o cache inteiro fica inválido. Por isso truncar "pela esquerda" a cada turno destrói o prefix caching. Prefira resumir em blocos maiores e com menos frequência.

- **Qual a diferença entre o vLLM e o RadixAttention do SGLang?** O vLLM usa hashes encadeados por bloco; o SGLang mantém uma árvore radix de prefixos. Os dois encontram o maior prefixo comum; a árvore facilita políticas de despejo e o agendamento ciente de prefixos (rodar juntas as requisições que compartilham prefixo).

Armadilhas comuns

- Esquecer o roteamento entre réplicas.
- Não perceber que conteúdo variável no começo do prompt anula o cache.
- Esquecer o adaptador LoRA ou a versão do modelo na chave.
- Ignorar o vazamento de informação entre tenants.

Q044 Como funciona o PagedAttention do vLLM, e por que ele importa para a eficiência de memória da GPU?

> **[Fornecedor]** Lista de perguntas de entrevista sobre LLM de 2026 · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"O PagedAttention trata o KV cache como memória virtual paginada. A memória do cache é dividida em blocos de tamanho fixo, por exemplo 16 tokens, e cada requisição tem uma tabela de blocos que mapeia suas posições lógicas para blocos físicos espalhados pela GPU. Blocos são alocados sob demanda, então o desperdício fica limitado ao último bloco de cada requisição e não existe fragmentação externa. O kernel de atenção lê os blocos através da tabela. Com contagem de referências e copy-on-write, várias requisições compartilham blocos, o que permite parallel sampling, beam search e prefix caching sem duplicar memória. Como o batch no decode é limitado pela memória do KV, aproveitar quase toda a memória se traduz em batches maiores e muito mais throughput."

Resposta completa

Estruture em **problema → mecanismo → kernel → compartilhamento → impacto → custos**.

Problema. Alocação contígua por `max_seq_len`: reserva o pior caso, fragmenta, e sistemas anteriores aproveitavam só uma fração pequena da memória do cache (seção 6.3).

Mecanismo. Use o diagrama da seção 6.4 e a analogia com sistemas operacionais:

Sistema operacional	PagedAttention
processo	requisição (sequência)
página virtual	bloco lógico (tokens 0–15, 16–31...)
quadro físico	bloco físico na HBM
tabela de páginas	tabela de blocos
alocação sob demanda (page fault)	aloca bloco quando o último enche
copy-on-write após fork	copy-on-write entre feixes e amostras
swap para disco	swap de blocos para a CPU

Kernel. A atenção do decode, para cada sequência e cada head:

```
para cada bloco lógico i da sequência:
    físico = tabela_de_blocos[seq][i]
    K_bloco = cache_k[físico]     (16 tokens × D)
    V_bloco = cache_v[físico]
    scores = q · K_bloco^T          (com máscara no último
                                      bloco parcial)
    atualiza máximo, soma e acumulador (softmax online)
saída = acumulador / soma
```

É o softmax online do FlashAttention (Capítulo 2) aplicado a blocos não contíguos. Kernels de produção paralelizam entre sequências, heads e, para contextos longos, entre pedaços do contexto (com uma etapa final de redução).

Compartilhamento. Cada bloco físico tem uma contagem de referências.

```
Parallel sampling, n = 3, mesmo prompt de 40 tokens:

prompt: blocos físicos [4, 9, 2*]  (* = parcialmente cheio)
amostra 1: [4, 9, 2*]   ref(4)=3, ref(9)=3, ref(2)=3
amostra 2: [4, 9, 2*]
amostra 3: [4, 9, 2*]

cada amostra gera seu token 41 → precisa escrever no bloco 2
  amostra 1: ref(2)=3 > 1 → copia para o bloco 7 e escreve
  amostra 2: ref(2)=2 > 1 → copia para o bloco 11
  amostra 3: ref(2)=1     → escreve direto no bloco 2
o prompt continua guardado UMA vez (blocos 4 e 9)
```

Impacto. O vLLM reportou, no paper original, throughput várias vezes maior que os sistemas da época em cargas de chat, principalmente porque batches maiores cabem na mesma memória. Os números exatos dependem do modelo e da carga; o mecanismo é o que importa: **no decode, o batch é limitado pela memória do KV; usar quase toda a memória dá batches maiores; batches maiores dão mais throughput**.

Custos.

- O kernel de atenção é mais complexo e faz leituras indiretas.
- A tabela de blocos precisa ser montada e enviada para a GPU a cada passo (overhead de CPU, minimizado com estruturas incrementais).
- Blocos pequenos demais prejudicam a eficiência de leitura; grandes demais aumentam o desperdício.
- Operações que precisam de K e V contíguos (alguns kernels antigos) não funcionam direto.

Perguntas de acompanhamento prováveis

- **O PagedAttention acelera a atenção?** Não diretamente; o kernel em si pode ser um pouco mais lento que um kernel sobre memória contígua. O ganho é de **capacidade**: mais sequências por GPU.
- **O que acontece quando acabam os blocos livres?** O scheduler preempta sequências (recomputação ou swap) ou para de admitir novas (6.6).
- **Como o PagedAttention se relaciona com o prefix caching?** O prefix caching é construído em cima: blocos identificados por hash do prefixo podem ser compartilhados entre requisições diferentes, com a mesma contagem de referências.
- **Existem alternativas?** Sim. Algumas abordagens usam a memória virtual do próprio driver da GPU para manter o cache **virtualmente contíguo** e fisicamente alocado sob demanda, permitindo usar kernels de atenção comuns (como o vAttention, Prabhu et al., 2024).

Armadilhas comuns

- Dizer que o PagedAttention reduz o tamanho do KV cache. Ele reduz o **desperdício**; o tamanho por token é o mesmo.
- Não saber explicar a tabela de blocos e o copy-on-write.
- Não conectar memória → tamanho do batch → throughput.
- Confundir com offload para a CPU (que é outra técnica, embora o swap use a mesma estrutura de blocos).

Resumo do capítulo

- O KV cache ocupa toda a memória que sobra depois dos pesos e define quantas sequências cabem: ~110 conversas de 4 mil tokens no 8B numa H100; ~350 no 70B em 8 H100.
- Fórmula: 2 × camadas × kv_heads × head_dim × bytes por token. Faça essa conta no início de todo design.
- Alocação contígua por `max_seq_len` desperdiça a maior parte da memória do cache.
- **PagedAttention**: blocos fixos, tabela de blocos, alocação sob demanda, contagem de referências e copy-on-write. Ganha capacidade, não velocidade de kernel.
- **Prefix caching**: hash encadeado por bloco, reuso entre turnos e usuários, LRU. Armadilhas: template variável, retokenização, roteamento entre réplicas, chave incompleta, isolamento entre tenants.
- **Preempção** por recomputação ou swap quando a memória acaba; preempção frequente é sinal de batch grande demais.
- **Hierarquia**: HBM → DRAM da CPU → SSD → remoto. Para contextos longos, trazer o KV de volta é muito mais barato que recalcular.
- **Diminuir o cache**: FP8/INT4, GQA/MLA, sliding window, camadas híbridas, despejo de tokens (este último muda a saída).
- OOM em conversas longas: separar pico de ativações (prefill, logits) de crescimento do cache (decode).

Checklist para a entrevista

- ☐ Calculo o KV cache por token e quantas sequências cabem numa configuração.
- ☐ Desenho a divisão da memória da GPU num servidor de LLM.
- ☐ Explico o desperdício da alocação contígua.
- ☐ Explico PagedAttention com a analogia de memória virtual, tabela de blocos e copy-on-write.
- ☐ Implemento uma classe de KV cache pré-alocada com comprimentos por sequência e reordenação para beam search.
- ☐ Explico prefix caching com hash encadeado e cito as sete armadilhas.
- ☐ Comparo recomputação e swap na preempção.
- ☐ Calculo se vale trazer o KV da CPU ou recalcular.
- ☐ Listo as técnicas para diminuir o KV cache e o risco de cada uma.
- ☐ Tenho um roteiro para OOM em conversas longas, incluindo os logits do prompt.

CAPÍTULO 7

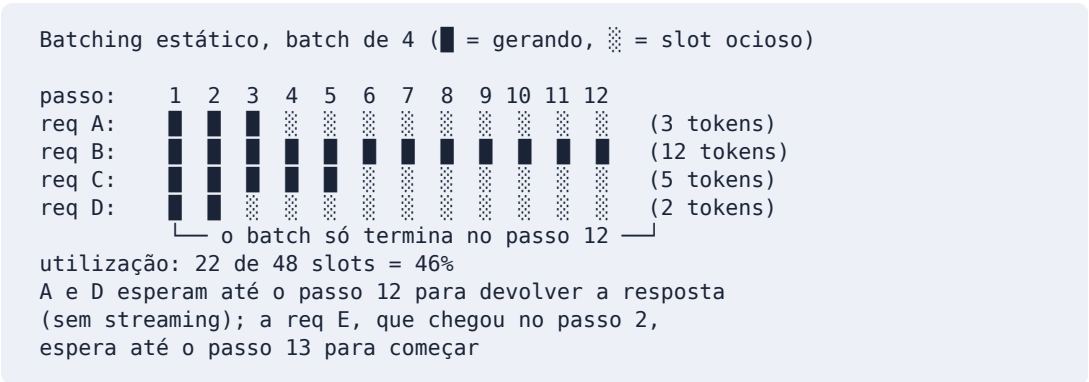
Serving de LLMs e Continuous Batching

Você já sabe que o decode é memory-bound e que o batching é a saída (Capítulo 1), e sabe que a memória do KV cache limita o batch (Capítulo 6). Este capítulo trata de **como montar os batches**: quem entra, quando, quem sai, o que fazer com um prompt gigante que chega no meio, como não deixar ninguém esperando para sempre.

É o tema preferido das entrevistas da Anthropic: o levantamento encontrou pelo menos oito enunciados relatados sobre batching, APIs de inferência em GPU ou distribuição de pesos. Os enunciados variam ("projete um sistema de batching", "critique este design", "projete o scheduler"), mas o núcleo é o mesmo, e este capítulo o cobre do zero.

7.1 Batching estático

A forma mais simples: junte B requisições, rode o batch inteiro até **todas** terminarem, devolva, junte o próximo batch.



Três problemas:

1. **Slots ociosos**: quem termina cedo deixa o lugar vazio até o fim.
2. **Latência inflada**: sem streaming, quem termina cedo espera o mais lento. Mesmo com streaming, quem chega espera o batch inteiro acabar para entrar.

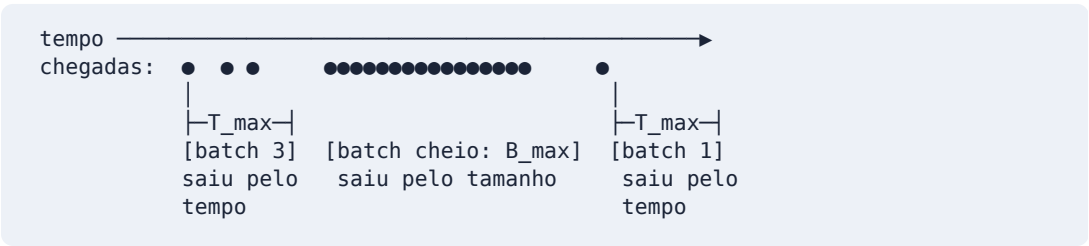
3. **Padding**: prompts de tamanhos diferentes precisam ser completados até o maior.

As saídas de LLMs têm tamanhos muito variados (de 1 a milhares de tokens), então esses problemas são graves.

7.2 Batching dinâmico (clássico)

Para modelos de **custo fixo por entrada** (classificadores, modelos de embedding, reconhecimento de imagem), o padrão clássico é o **batching dinâmico**, usado por servidores como o Triton Inference Server e o TorchServe:

- Requisições entram numa fila.
- Um laço tira da fila até B_{max} itens **ou** até passar um tempo máximo de espera T_{max} desde o primeiro item.
- Roda o batch, devolve todos, repete.



Os dois parâmetros controlam o trade-off:

- T_{max} maior → batches mais cheios em carga baixa, mais latência.
- B_{max} maior → mais throughput em carga alta, cada batch demora mais.

Esse padrão funciona bem quando todos os itens custam parecido e terminam juntos. Para LLMs, que geram um número variável de tokens, não basta.

7.3 Continuous batching

A ideia que mudou o serving de LLMs veio do paper do **Orca** (Yu et al., OSDI 2022): **agendar por iteração, não por requisição**. A cada passo do modelo (cada token), o scheduler decide de novo quem está no batch. Quem termina sai na hora; quem chega entra no passo seguinte.

```
Continuous batching, até 4 slots (letras = requisições)

passo:    1  2  3  4  5  6  7  8  9 10 11 12
slot 1:   A  A  A  F  F  F  F  F  F  H  H  H
slot 2:   B  B  B  B  B  B  B  B  B  B  B  B
slot 3:   C  C  C  C  C  G  G  G  G  G  G  G
slot 4:   D  D  E  E  E  E  E  E  I  I  I  I
          D sai no passo 2 e E entra no passo 3; A sai
          no passo 3 e F entra no 4; e assim por diante.
utilização perto de 100% enquanto houver fila
```

O mesmo paper introduziu o **selective batching**: as operações que não dependem de outras posições (as GEMMs das projeções e do MLP) rodam juntas para todos os tokens do batch, achatados num eixo só, independentemente de sequência. Só a atenção, que depende do histórico de cada sequência, é calculada sequência por sequência (ou com um kernel que entende sequências de tamanhos diferentes). Isso permite misturar no mesmo passo uma sequência em prefill (com centenas de tokens) e várias em decode (com um token cada).

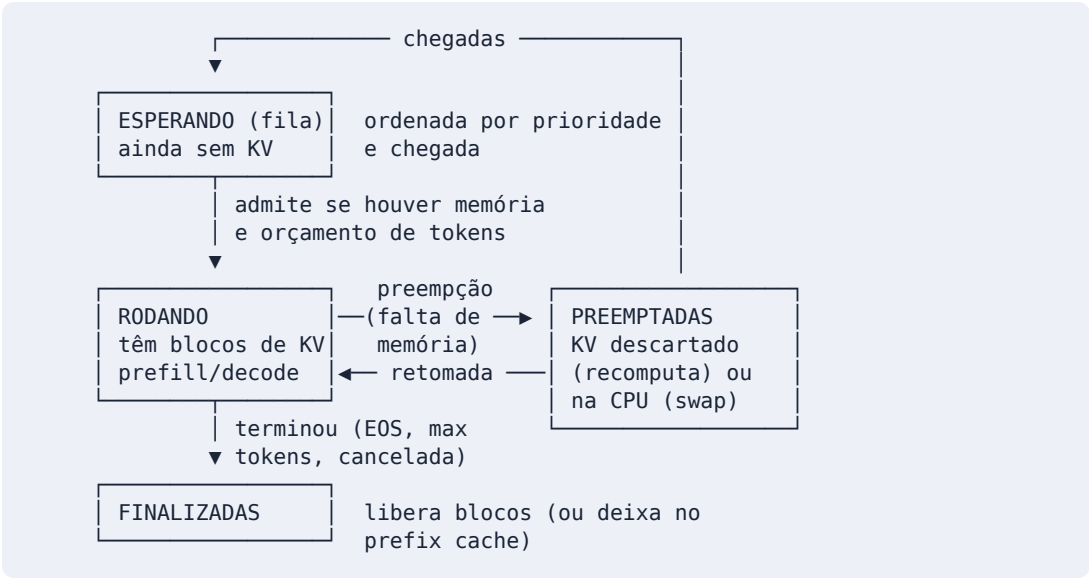
```
Um passo com batching seletivo (layout "achatado")

tokens do passo: [ s1:t57 | s2:t903 | s3: p0 p1 ... p511 ]
                   decode    decode     prefill (512 tok.)
                 └──────────── 514 tokens ────────────────┘
GEMMs (QKV, saída, MLP): uma matriz [514 × d] para todos
atenção: cada sequência contra o próprio KV cache
         (kernel com comprimentos variáveis)
```

Todos os motores modernos (vLLM, TensorRT-LLM, SGLang, TGI, LMDeploy) fazem continuous batching. A NVIDIA chama de **in-flight batching**.

7.4 Anatomia de um scheduler

O scheduler é o coração de um motor de inferência. A cada passo ele monta o "plano" do passo: quais sequências rodam e quantos tokens cada uma processa.



Os parâmetros principais (os nomes variam por motor):

Parâmetro	Controla	Efeito de aumentar
Máximo de sequências por passo	tamanho do batch	mais throughput, TPOT maior
Máximo de tokens por passo (orçamento)	quanto prefill entra por passo	TTFT melhor, ITL pior
Fração da memória para KV	capacidade total	mais sequências, risco de OOM em picos
Tamanho do bloco de KV	granularidade	ver Capítulo 6

E as políticas:

- **Ordem de admissão**: FCFS (primeiro a chegar, primeiro a ser atendido), prioridades (pago vs gratuito, síncrono vs batch), justiça entre clientes.
- **Quem preemptar**: normalmente a sequência que chegou por último (perde menos trabalho e preserva o FCFS).
- **Como preemptar**: recomputar ou swap (Capítulo 6).
- **Reserva de memória**: admitir uma sequência só se houver margem para ela crescer alguns passos, para evitar preempções em cadeia.

7.5 Chunked prefill

Um problema do continuous batching ingênuo: quando chega um prompt de 30 mil tokens, o passo em que ele faz prefill demora centenas de milissegundos, e **todos** os usuários em decode ficam sem token nesse tempo. É um soluço de latência (pico de ITL).

O **chunked prefill** (SARATHI, Agrawal et al., 2023; Sarathi-Serve, 2024) divide o prefill em pedaços e junta cada pedaço com os decodes em andamento, respeitando um orçamento de tokens por passo:

```
Orçamento: 2.048 tokens por passo. 60 sequências em decode.
Chega um prompt de 6.000 tokens.

passo t:    [60 decodes][pedaço 1: 1.988 tokens do prompt]
passo t+1:  [60 decodes][pedaço 2: 1.988 tokens]
passo t+2:  [60 decodes][pedaço 3: 1.988 tokens]
passo t+3:  [60 decodes][pedaço 4: 36 tokens] → 1º token
            do novo usuário sai neste passo
passo t+4:  [61 decodes]
```

Os benefícios:

1. **ITL estável**: nenhum passo fica muito mais longo que os outros.
2. **Melhor uso da GPU**: os passos de decode são memory-bound e deixam cálculo ocioso; o pedaço de prefill usa esse cálculo "de carona". O custo marginal dos decodes num passo que já tem prefill é pequeno, e vice-versa.
3. **Pico de memória limitado**: as ativações temporárias são proporcionais ao orçamento, não ao maior prompt.

O custo: o TTFT de prompts longos aumenta um pouco (o prefill fica espalhado em vários passos, e cada pedaço precisa ler o KV dos pedaços anteriores). O orçamento de tokens é o botão que ajusta esse trade-off.

7.6 Head-of-line blocking

Head-of-line (HOL) blocking é quando um item no começo da fila trava todos os que estão atrás. Em serving de LLMs, aparece em três níveis:

```
1. NO BATCH (batching estático)
   o batch espera a sequência mais longa terminar

2. NO PASSO (prefill longo sem chunking)
   um prefill de 30k tokens congela todos os decodes

3. NA FILA (FCFS puro com memória cheia)
   uma requisição gigante no começo da fila espera
   memória para caber, e as pequenas atrás dela,
   que caberiam, esperam também
```

As soluções correspondentes:

1. Continuous batching (resolve o nível 1).
2. Chunked prefill (resolve o nível 2).
3. Políticas de fila mais espertas para o nível 3:
   - **Pular** (com limite): admitir requisições menores que cabem enquanto a grande espera, com um mecanismo anti-inanição (envelhecimento: quanto mais tempo esperando, mais prioridade).
   - **Filas separadas por tamanho** (curtas e longas), com capacidade reservada para cada uma.
   - **Pools separados**: réplicas dedicadas a contextos longos.
   - **Aproximações de "shortest job first"**: se o sistema conseguir **prever** o tamanho da saída (por um modelo pequeno ou por histórico do cliente), priorizar as mais curtas reduz a latência média, sempre com proteção contra inanição.

7.7 Os motores

Motor	Origem	Destaques
vLLM	UC Berkeley, comunidade	PagedAttention, muitos modelos e hardwares, prefix caching, chunked prefill, API compatível com OpenAI
TensorRT-LLM	NVIDIA	kernels mais afinados para NVIDIA (FP8, FP4), runtime C++, in-flight batching, integração com Triton Inference Server
SGLang	LMSYS, comunidade	RadixAttention, runtime eficiente, bom para programas com muitas chamadas e saídas estruturadas

Motor	Origem	Destaques
TGI	Hugging Face	servidor integrado ao ecossistema HF
LMDeploy	OpenMMLab	runtime TurboMind, quantização

O funcionamento interno de vLLM e TensorRT-LLM está na Q035. Na entrevista, mais importante que saber detalhes de cada motor é saber **as peças comuns** e os trade-offs.

Perguntas de entrevista comentadas

Q049 Projete um sistema de batching de inferência para uma única GPU que processe até 100 entradas por batch enquanto os usuários esperam de forma síncrona, maximizando a utilização sob restrições de computação.

> **[Relatado]** Anthropic · ML engineer · design de sistemas · 2026 · Difícil · fonte

Resposta em 30 segundos

"Uma fila na frente da GPU e um único laço de batching. Cada requisição vira uma future e entra na fila; o laço pega o primeiro item e continua juntando até ter 100 ou até passar um tempo máximo curto, alguns milissegundos, calibrado pelo SLO. Enquanto a GPU roda um batch, o próximo já vai se formando, então em carga alta os batches saem cheios sem espera extra. A GPU roda fora do event loop; o resultado de cada item vai para a future dele. Tenho fila limitada com rejeição rápida quando enche, timeouts e cancelamento, isolamento de erros por item e métricas de tamanho de batch, espera na fila e latência. Se o modelo for um LLM, troco o batching por requisição por continuous batching por iteração, porque as saídas têm tamanhos diferentes."

Resposta completa

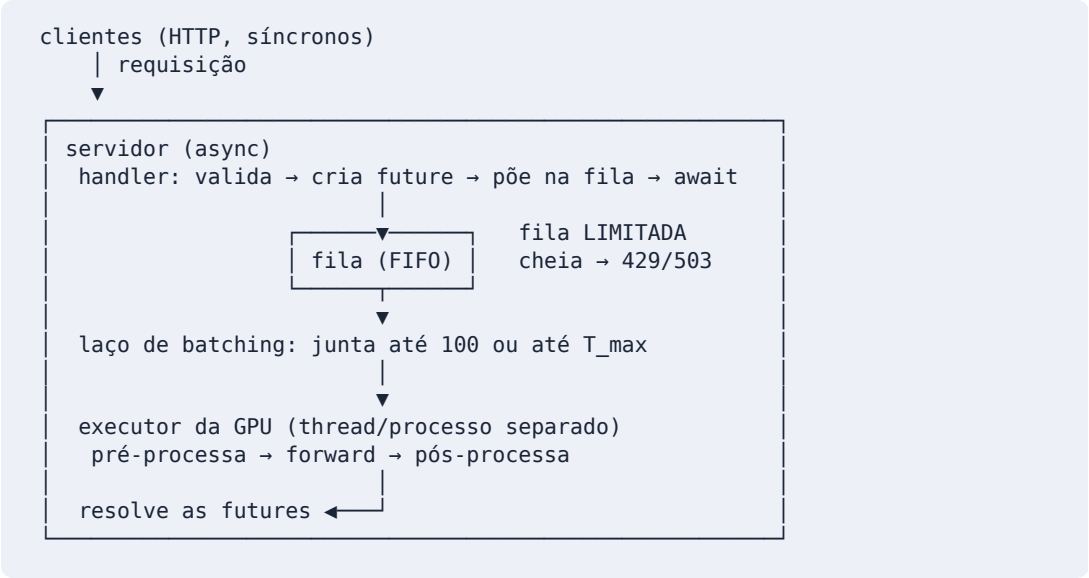
1. Esclareça os requisitos. Faça perguntas antes de desenhar:

- Que modelo? Custo fixo por entrada (classificador, embedding) ou geração de tamanho variável (LLM)?
- As entradas têm tamanhos parecidos? Precisa de padding?

- Qual o SLO de latência (p99)? Qual a taxa de chegada esperada e os picos?
- O que acontece com sobrecarga: rejeitar, degradar, esperar?

Vou assumir primeiro um modelo de custo aproximadamente fixo, e depois mostrar a variação para LLM.

2. Arquitetura.



3. O código do núcleo.

```python
import asyncio
import time

class Sobrecarga(Exception):
    pass

class Batcher:
    def __init__(self, modelo, max_batch=100,
                 max_espera_ms=5.0, max_fila=5000):
        self.modelo = modelo          # fn(list) -> list
        self.max_batch = max_batch
        self.max_espera = max_espera_ms / 1000
        self.fila = asyncio.Queue(maxsize=max_fila)

    async def inferir(self, entrada, timeout_s=2.0):
        loop = asyncio.get_running_loop()
        fut = loop.create_future()
        try:
            self.fila.put_nowait((entrada, fut))
        except asyncio.QueueFull:
            raise Sobrecarga("fila cheia")   # vira 429
        # wait_for cancela a future se estourar o prazo
        return await asyncio.wait_for(fut, timeout_s)

    async def _juntar(self):
        lote = [await self.fila.get()]       # bloqueia
        prazo = time.monotonic() + self.max_espera
        while len(lote) < self.max_batch:
            resta = prazo - time.monotonic()
            if resta <= 0:
                break
            try:
                item = await asyncio.wait_for(
                    self.fila.get(), resta)
            except asyncio.TimeoutError:
                break
            lote.append(item)
        # descarta quem desistiu (timeout ou cancelamento)
        return [(e, f) for e, f in lote if not f.done()]

    async def laco(self):
        loop = asyncio.get_running_loop()
        while True:
            lote = await self._juntar()
            if not lote:
                continue
            entradas = [e for e, _ in lote]
            try:
                # GPU fora do event loop
                saidas = await loop.run_in_executor(
                    None, self.modelo, entradas)
            except Exception as erro:
                for _, f in lote:
                    if not f.done():
                        f.set_exception(erro)
                continue
            for (_, f), s in zip(lote, saidas):
```

```
                if not f.done():
                    f.set_result(s)
```

4. As decisões e os porquês.

- **Tempo máximo de espera curto e adaptativo.** Com carga baixa, esperar 5 ms por companhia é barato comparado ao tempo de inferência. Com carga alta, a fila já tem itens quando o laço volta, e o batch sai cheio na hora. Esse comportamento ("batching adaptativo") surge naturalmente porque a coleta do próximo batch acontece enquanto o atual roda.
- **Quanto esperar?** Calcule: se o SLO p99 é 200 ms e o forward de 100 itens leva 60 ms, sobram ~140 ms para fila e espera no pior caso. Comece com uma espera máxima de poucos milissegundos e ajuste olhando o tamanho médio do batch e a latência.
- **Por que limitar a fila?** Uma fila sem limite transforma sobrecarga em latência infinita para todos. Rejeitar cedo (429 com `Retry-After`) é melhor do que aceitar uma requisição que vai estourar o timeout de qualquer jeito. Um bom critério: rejeitar se o tempo estimado de espera (tamanho da fila ÷ throughput) já passa do SLO.
- **Por que uma única execução por vez na GPU?** Várias threads disputando a GPU não aumentam o throughput e pioram a previsibilidade. Uma melhoria é **pipeline**: pré-processar e copiar o batch $k + 1$ para a GPU (em outra stream CUDA) enquanto o batch k calcula.
- **Padding e buckets.** Se as entradas variam em tamanho (texto), agrupe por faixas de tamanho para reduzir o padding, ou use kernels que aceitam tamanhos variáveis.
- **Isolamento de erros.** Uma entrada malformada não deve derrubar as outras 99. Valide na entrada; se o batch falhar, reexecute dividindo ao meio (busca binária) para isolar o culpado.
- **Cancelamento.** Se o cliente desistiu, não gaste GPU com ele.

5. Maximizar a utilização.

```
utilização da GPU ao longo do tempo

sem pipeline:  [cpu prep][  GPU  ][cpu pós][cpu prep][  GPU  ]
                GPU ociosa ▲               ▲
com pipeline:  [  GPU k  ][  GPU k+1  ][  GPU k+2  ]
               [prep k+1 ][prep k+2   ]  (outra thread/stream)
```

- Escolha o tamanho do batch pela curva throughput × latência medida (não necessariamente 100 sempre).
- Use CUDA graphs para tamanhos de batch fixos (buckets), o que exige padding até o bucket.
- Meça a utilização real (tempo de GPU ocupada, MFU), não só o `nvidia-smi`.

6. Se for um LLM. O batching por requisição desperdiça muito (7.1). O design muda para continuous batching: o "laço" passa a ser o scheduler por iteração (Q052), o limite de 100 vira o máximo de sequências simultâneas (limitado também pela memória do KV cache), e os usuários síncronos recebem os tokens por streaming.

7. Observabilidade. Histogramas de tamanho do batch, tempo na fila, tempo de forward, latência ponta a ponta, taxa de rejeição, taxa de timeout e uso da GPU.

Perguntas de acompanhamento prováveis

- **O que acontece se chegar uma rajada de 10 mil requisições?** A fila enche até o limite, o excedente recebe 429 rapidamente, a GPU processa batches cheios na capacidade máxima, e a latência dos aceitos fica limitada. Com autoscaling, novas réplicas sobem; com uma GPU só, rejeitar é o correto.
- **Como escolher entre latência e throughput?** Pelo SLO. Meça a curva de latência e throughput por tamanho de batch e escolha o maior batch cuja latência p99 ainda cumpre o SLO com folga para a fila.
- **Por que não `asyncio.sleep` num laço para verificar a fila?** Polling com sleep adiciona latência e gasta CPU. Esperar na própria fila com timeout acorda exatamente quando há trabalho.
- **E se as entradas tiverem prioridades diferentes?** Use uma fila de prioridade (ou várias filas), com proteção contra inanição.

Armadilhas comuns

- Esperar o batch encher (100) sem tempo máximo: em carga baixa, a latência explode.
- Fila sem limite.
- Chamar o modelo dentro do event loop (bloqueia todo o servidor).
- Não tratar timeouts, cancelamentos e erros por item.
- Usar batching por requisição para um LLM sem discutir continuous batching.

Q050 Aqui está o design de um desenvolvedor júnior para um sistema de batching de inferência. Critique-o e proponha melhorias.

> **[Relatado]** Anthropic · Engineering Manager · rodada técnica · 2025 · Difícil · fonte

Resposta em 30 segundos

"Eu leria o design procurando, em ordem: corretude (cada resposta volta para a requisição certa, erros e timeouts tratados), latência (espera por batch cheio sem prazo, polling com sleep), robustez (fila sem limite, sem backpressure, uma entrada ruim derruba o batch), utilização da GPU (GPU ociosa durante pré e pós-processamento, padding, várias threads disputando a GPU) e operação (métricas, configuração, testes de carga). Para cada problema, explico o impacto com um cenário concreto e proponho a mudança, priorizando o que é bug de corretude antes do que é otimização. Como gestor, também cuidaria da forma: feedback específico, começando pelo que está bom, e transformando os problemas em perguntas que ensinem."

Resposta completa

Como o design real é dado na hora, você precisa de um **roteiro de revisão**. Vamos usar um design júnior típico como exemplo:

```
# design do júnior (exemplo típico)
fila = []
resultados = {}

def handler(req):
    fila.append(req)
    while req.id not in resultados:   # espera ativa
        time.sleep(0.01)
    return resultados.pop(req.id)

def worker():
    while True:
        if len(fila) >= 100:          # só roda com 100
            lote = fila[:100]
            del fila[:100]
            saidas = modelo(lote)
            for r, s in zip(lote, saidas):
                resultados[r.id] = s
        time.sleep(0.01)
```

A crítica, organizada por gravidade:

#	Problema	Cenário de falha	Correção
1	Só roda com 100 itens	às 3h, com 5 req/s, cada requisição espera ~20 s	prazo máximo de espera (T_{max})
2	Lista compartilhada sem lock	`append` e `del` concorrentes perdem ou duplicam itens	fila thread-safe (`queue.Queue` ou `asyncio.Queue`)
3	Erro no modelo não é tratado	uma exceção mata o worker; todos os handlers esperam para sempre	try/except, propagar erro para cada requisição, reiniciar
4	Sem timeout no handler	cliente e thread presos indefinidamente	timeout, e remover a requisição se desistir
5	Espera ativa com sleep	10 ms de latência extra e CPU gasta por requisição	future/evento por requisição
6	Fila sem limite	numa rajada, a latência cresce sem parar	limite + rejeição rápida (429)
7	`resultados` cresce se ninguém buscar	vazamento de memória com clientes que desistem	futures (não há dicionário global)
8	Uma entrada ruim derruba o batch	um texto malformado faz 99 falharem	validação + isolamento por bisseção
9	GPU ociosa durante pré/pós	utilização baixa	pipeline com outra thread/stream
10	Sem métricas	ninguém sabe o tamanho real dos batches	histogramas de batch, fila e latência
11	IDs podem colidir	duas requisições com o mesmo id trocam respostas	IDs gerados pelo servidor (UUID) ou futures

A **proposta** é o design da Q049, apresentado como evolução do que o júnior fez, não como substituição.

Para LLMs, mais quatro pontos que aparecem com frequência em designs júnior:

- Batching estático para geração de tamanho variável (usar continuous batching).
- Nenhuma noção de memória do KV cache na admissão (OOM em picos).
- Prefills longos travando todos (chunked prefill).

- Sem streaming para usuários síncronos.

A parte de gestão. Como a vaga era de Engineering Manager, o entrevistador também avalia **como** você dá o feedback:

```
1. Comece pelo que está certo
   ("a ideia de juntar em batches e ter um worker
    separado é exatamente o caminho")
2. Priorize: corretude > robustez > desempenho > estilo
3. Para cada problema, mostre o CENÁRIO de falha
   (mais convincente que uma regra abstrata)
4. Faça perguntas que levem a pessoa à resposta
   ("o que acontece às 3 da manhã, com pouco tráfego?")
5. Combine os próximos passos e como validar
   (teste de carga, testes de falha)
```

Perguntas de acompanhamento prováveis

- **Qual desses problemas você corrigiria primeiro?** Os de corretude e disponibilidade: lista sem lock, erro que mata o worker, ausência de timeout. Depois o prazo máximo de espera (latência inaceitável em carga baixa).
- **Como você testaria o design corrigido?** Teste de carga com taxa variável (baixa, nominal, rajada), injeção de falhas (modelo levantando exceção, entradas malformadas, clientes que desistem), e verificação de que cada resposta corresponde à entrada certa.
- **Como você lidaria se o júnior discordasse?** Usar dados: um teste rápido que mostre o cenário de falha resolve a discussão melhor do que autoridade.

Armadilhas comuns

- Listar problemas sem priorizar.
- Reescrever tudo do zero sem aproveitar o que estava certo.
- Ignorar problemas de concorrência (lock, IDs) e focar só em desempenho.
- Em vaga de gestão, esquecer a forma do feedback.

Q051 Quando o TensorRT-LLM é melhor que o vLLM?

> **[Fornecedor]** Banco público de perguntas de inferência, marcada como comum · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

"Quando o hardware é NVIDIA, o modelo é estável e já bem suportado, e o objetivo é o máximo de desempenho por GPU: os kernels do TensorRT-LLM costumam ser os mais afinados para cada geração, principalmente em FP8 na Hopper e FP4 na Blackwell, com fusões agressivas. Compensa em escala grande, em que alguns por cento de throughput valem muito dinheiro, e quando o time pode investir no build e na configuração. O vLLM ganha quando você precisa de flexibilidade: modelos novos no dia do lançamento, arquiteturas variadas, hardware que não é NVIDIA, iteração rápida e uma comunidade grande. A resposta certa é medir os dois com a carga real."

Resposta completa

Organize como critérios de decisão:

Critério	Favorece TensorRT-LLM	Favorece vLLM
Hardware	só NVIDIA, especialmente gerações recentes	AMD, outros aceleradores, frota mista
Modelo	estável, popular, já otimizado	lançado ontem, arquitetura incomum
Prioridade	desempenho máximo, custo por token	velocidade de iteração, flexibilidade
Precisão	FP8/FP4 com kernels e calibração da NVIDIA	formatos variados da comunidade (AWQ, GPTQ...)
Escala	frota grande (ganhos pequenos valem muito)	pequena e média
Operação	time com experiência no stack NVIDIA, Triton Inference Server	time Python, integração rápida
Customização	aceita a estrutura do framework	precisa mexer no scheduler ou no modelo em Python

E três ressalvas que mostram maturidade:

1. **A diferença muda com o tempo.** Os dois projetos evoluem rápido e copiam boas ideias um do outro. Uma comparação de seis meses atrás pode estar errada hoje.
2. **Benchmarks públicos raramente batem com a sua carga.** A distribuição de tamanhos de prompt e de saída, a taxa de prefix cache e o SLO mudam qual é o melhor.

3. **O custo total inclui engenharia.** Um motor 10% mais rápido que exige o dobro de esforço de manutenção pode não compensar numa frota pequena.

O processo de decisão que você descreveria:

```
1. Definir a carga representativa (tamanhos reais,
   taxa de chegada, prefixos compartilhados)
2. Definir o SLO (TTFT p99, TPOT p99)
3. Configurar os dois motores com esforço comparável
   (mesma precisão, mesmo paralelismo, ajuste básico)
4. Medir goodput: requisições/s que cumprem o SLO
5. Medir qualidade (a quantização difere)
6. Pesar custo de operação e velocidade de iteração
```

Perguntas de acompanhamento prováveis

- **Por que o TensorRT-LLM tende a ser mais rápido em NVIDIA?** Kernels escritos pela própria NVIDIA para cada arquitetura, acesso antecipado a recursos novos do hardware, fusões agressivas e ferramentas de quantização integradas.
- **E o SGLang?** Forte em cargas com muito prefixo compartilhado e em programas com várias chamadas (agentes, saídas estruturadas), graças ao RadixAttention e ao runtime enxuto. Também entra na comparação.
- **Dá para usar os dois?** Sim, inclusive por modelo: motores diferentes atrás do mesmo gateway, escolhidos pelo melhor resultado em cada caso.

Armadilhas comuns

- Responder com um número fixo (“TensorRT-LLM é 30% mais rápido”).
- Ignorar hardware e cobertura de modelos.
- Não propor medir com a carga real.
- Esquecer o custo de engenharia.

Q052 Projete o escalonador de um motor de inferência com continuous batching.

> **[Fornecedor]** Banco público de perguntas por empresa (startups de inferência) · Presencial · 2026 · Difícil · fonte

Resposta em 30 segundos

“O scheduler roda uma vez por passo do modelo e monta o plano do passo: quais sequências entram e quantos tokens cada uma processa. Mantém filas de espera, em execução e preemptadas. Em cada passo, primeiro garante o decode de quem já está rodando, um token cada, alocando blocos de KV e preemptando a sequência mais recente se faltar memória. Depois continua prefills em andamento em pedaços, e por último admite novas requisições enquanto houver orçamento de tokens e memória com margem. Os parâmetros são o máximo de sequências, o orçamento de tokens por passo e a margem de memória; as políticas são de ordem (FCFS, prioridade, justiça), de preempção e de anti-inanição. Precisa ser rápido: roda a cada poucos milissegundos, então as estruturas são incrementais.”

Resposta completa

1. Requisitos.

- Maximizar goodput: throughput dentro do SLO de TTFT e TPOT.
- Nunca dar OOM: respeitar a memória do KV cache.
- Justiça e prioridades; nenhuma requisição esperando para sempre.
- Overhead pequeno: o scheduler roda a cada passo (a cada 10–50 ms).

2. Estado.

```
Sequência:
  id, prioridade, chegada
  tokens do prompt, tokens gerados
  tokens já processados (KV calculado)
  tabela de blocos de KV
  estado: ESPERANDO | RODANDO | PREEMPTADA | FINALIZADA
  parâmetros: max_tokens, stop, sampling

Filas:
  esperando  (ordenada por prioridade e chegada)
  rodando    (ordem de admissão)
  preemptadas (voltam antes das novas)

Gerenciador de KV: blocos livres, contagem de
  referências, prefix cache (hash → bloco)
```

3. O algoritmo de um passo.

```python
from collections import deque

class Scheduler:
    def __init__(self, kv, max_seqs=256, max_tokens=8192,
                 margem_blocos=64):
        self.kv = kv                  # gerenciador de blocos
        self.max_seqs = max_seqs
        self.max_tokens = max_tokens  # orçamento por passo
        self.margem = margem_blocos   # folga ao admitir
        self.esperando = deque()      # inclui preemptadas
        self.rodando = []             # ordem de admissão

    def _preemptar(self, seq):
        self.kv.liberar(seq)          # recomputação
        seq.processados = 0           # refará o prefill
        self.rodando.remove(seq)
        self.esperando.appendleft(seq)   # volta na frente

    def passo(self):
        orc = self.max_tokens
        plano = []                    # (seq, n_tokens)

        # 1) quem já roda: decode (1) ou resto do prefill
        for seq in list(self.rodando):
            if orc == 0:
                break
            if seq not in self.rodando:   # foi preemptada
                continue                  # neste passo
            n = min(seq.tokens_pendentes(), orc)
            while not self.kv.reservar(seq, n):
                vitima = self.rodando[-1]   # a mais nova
                self._preemptar(vitima)
                if vitima is seq:
                    n = 0
                    break
            if n > 0:
                plano.append((seq, n))
                orc -= n

        # 2) admite novas (ou preemptadas) se couber
        while (self.esperando and orc > 0
               and len(self.rodando) < self.max_seqs):
            seq = self.esperando[0]
            self.kv.aplicar_prefix_cache(seq)  # pula prefixo
            n = min(seq.tokens_pendentes(), orc)
            if not self.kv.reservar(seq, n,
                                    margem=self.margem):
                break        # FCFS: não pula a da frente
            self.esperando.popleft()
            self.rodando.append(seq)
            plano.append((seq, n))
            orc -= n
        return plano

    def depois_do_passo(self, resultados):
        for seq, token in resultados:
            seq.anexar(token)
            if seq.terminou():        # EOS, stop, max_tokens
```

```
                self.kv.liberar(seq, manter_no_cache=True)
                self.rodando.remove(seq)
```

4. As decisões importantes.

- **Decode antes de prefill novo**: protege o TPOT de quem já está recebendo tokens. Os prefills novos usam o orçamento que sobra (é o chunked prefill da seção 7.5).
- **Preemptar a mais recente**: perde menos trabalho e mantém a ordem de chegada. Alternativas: preemptar a de menor prioridade, ou a que tem o menor KV (mais barata de recomputar).
- **Margem ao admitir**: se você admitir até o último bloco livre, o próximo passo de decode já precisa preemptar alguém. A margem evita esse ciclo de admitir e preemptar.
- **FCFS estrito na admissão** (o `break`): simples e justo, mas causa head-of-line blocking se a primeira da fila for enorme. Variante: permitir pular com limite e envelhecimento (Q053).
- **Prefix cache na admissão**: antes de reservar, descontar os tokens que já estão no cache; a requisição precisa de menos memória e menos prefill.

5. Extensões que o entrevistador pode pedir.

- **Prioridades**: várias filas, ou uma fila ordenada por (prioridade, chegada); preempção de baixa prioridade quando chega alta.
- **Justiça entre clientes**: orçamento de tokens por cliente por janela de tempo (deficit round robin, fair queuing por tokens).
- **SLO-aware**: estimar, para cada requisição, se ela ainda vai cumprir o TTFT; priorizar as que estão perto de estourar, ou rejeitar cedo as que já estouraram.
- **Speculative decoding**: cada sequência pode processar $k + 1$ tokens por passo; o orçamento precisa contar isso.
- **Pipeline parallelism**: vários micro-batches em voo ao mesmo tempo; o scheduler monta batches para estágios diferentes.

6. Desempenho do próprio scheduler. Num passo de 20 ms, o scheduler e a preparação das entradas (tabelas de blocos, posições, metadados de atenção) precisam levar bem menos que isso. Técnicas: estruturas incrementais (só atualizar o que mudou), preparar o passo $t + 1$ na CPU enquanto a GPU executa o passo t (scheduling assíncrono), e mover partes da preparação para a GPU.

Perguntas de acompanhamento prováveis

- **Como você escolheria o orçamento de tokens por passo?** Pelo SLO de ITL: meça o tempo do passo em função do número de tokens e escolha o maior orçamento cujo tempo de passo ainda cumpre o ITL p99. Valores comuns ficam entre algumas centenas e alguns milhares de tokens.
- **Recomputar ou swap na preempção?** Ver Capítulo 6. Recomputar é mais simples; swap compensa para contextos longos.
- **Como evitar que uma requisição seja preemptada repetidamente?** Contar preempções por sequência e protegê-la depois de um limite, ou aumentar sua prioridade (envelhecimento).
- **E o cancelamento pelo cliente?** O handler marca a sequência como cancelada; o scheduler a remove no próximo passo e libera os blocos.

Armadilhas comuns

- Esquecer a memória do KV na admissão.
- Não ter política de preempção.
- Misturar prefill e decode sem orçamento (soluços de ITL).
- Ignorar o custo de CPU do próprio scheduler.
- Esquecer inanição e justiça.

Q053 Agrupe em batch requisições com tamanhos de prompt e de saída muito diferentes sem bloqueio head-of-line.

> **[Fornecedor]** Together AI, segundo fornecedor de preparação (“candidatos supostamente recebem”) · Design de sistemas · 2026 · Difícil · fonte

Resposta em 30 segundos

“Ataco o bloqueio nos três níveis. No batch, continuous batching: cada sequência sai quando termina e alguém entra no lugar, então saídas longas não seguram as curtas. No passo, chunked prefill com orçamento de tokens: um prompt de 100 mil tokens entra em pedaços e não congela os decodes. Na fila, admissão que não deixa uma requisição gigante travar as pequenas: permito pular com limite e envelhecimento, separo filas ou capacidade por classe de tamanho e, em escala, uso pools dedicados a contexto longo. Se tiver uma estimativa do tamanho da saída, aproximo shortest-job-first com proteção contra inanição. E meço TTFT e ITL por classe de tamanho, para ver se alguma classe está sendo sacrificada.”

Resposta completa

Comece mostrando os três níveis de HOL blocking (seção 7.6) e resolva cada um.

Nível 1 — saídas de tamanhos diferentes. Continuous batching (7.3). Com batching por iteração, uma sequência que gera 4.000 tokens não atrasa uma que gera 10.

Nível 2 — prompts de tamanhos diferentes. Chunked prefill com orçamento (7.5). Um prompt de 100 mil tokens, com orçamento de 4.096 tokens por passo, leva umas 25 iterações de prefill, e os decodes continuam saindo em todas elas.

Nível 3 — a fila. É onde está a parte interessante.

```
Fila FCFS com memória quase cheia:

[GIGANTE 120k tokens][p 300][p 500][p 200][p 800] ...
    ▲ não cabe agora; espera blocos liberarem
      e as pequenas, que caberiam, esperam junto

Admissão com "pulo limitado":
  - tenta a primeira; se não cabe, tenta as seguintes
    até K posições à frente
  - a gigante acumula "idade"; quando passa de um
    limite, o scheduler PARA de admitir as outras e
    reserva memória para ela (drena até caber)
```

As opções, com seus trade-offs:

Técnica	Como	Trade-off
Pulo limitado + envelhecimento	admite as que cabem, com prioridade crescente para quem espera	simples; a gigante pode esperar mais
Reserva de capacidade por classe	ex.: no máximo 30% dos blocos para contextos > 32k	protege as pequenas; pode deixar capacidade ociosa
Pools separados	réplicas dedicadas a contexto longo (mais memória, outro paralelismo)	melhor isolamento; custo de frota e roteamento
SJF aproximado	prioriza saídas previstas curtas	menor latência média; depende de previsão, risco de inanição
Preempção de longas	pausa uma longa para aliviar a memória	protege as curtas; a longa refaz trabalho
Limites por cliente	tokens em voo por cliente	um cliente não monopoliza

Prevendo o tamanho da saída. O tamanho da saída é desconhecido, mas pode ser estimado: pelo `max_tokens` pedido, pelo histórico do cliente ou do tipo de tarefa, ou por um modelo pequeno que prevê a faixa de tamanho a partir do prompt. Com uma previsão razoável, ordenar por "trabalho restante estimado" aproxima o shortest-job-first, que minimiza a latência média. Sempre com envelhecimento, senão as longas nunca rodam.

O papel do roteamento entre réplicas. Com várias réplicas, o balanceador deve considerar a carga de **tokens** de cada réplica (KV em uso, tokens na fila), não o número de requisições. Mandar dois prompts de 100 mil tokens para a mesma réplica cria um gargalo local mesmo com a frota ociosa.

Medição. Separe as métricas por classe de tamanho (por exemplo, prompt < 1k, 1k–8k, 8k–32k, > 32k). O p99 agregado pode parecer bom enquanto a classe de contexto longo está com TTFT de minutos, ou o contrário.

Perguntas de acompanhamento prováveis

- **Chunked prefill não piora o TTFT dos prompts longos?** Um pouco: o prefill é espalhado em vários passos e cada pedaço relê o KV dos anteriores. É o preço de não congelar os outros. Pools dedicados podem dar TTFT melhor para contextos longos.
- **Por que não ordenar só pelo tamanho do prompt?** O tamanho do prompt afeta o prefill (TTFT), mas a ocupação de memória e GPU ao longo do tempo depende também do tamanho da saída. O ideal é estimar o trabalho total.
- **Como evitar que um cliente com muitos prompts longos prejudique os outros?** Quotas de tokens por cliente e fair queuing, além das reservas por classe.
- **A desagregação de prefill e decode ajuda aqui?** Sim: prompts longos ficam no pool de prefill e não interferem no ITL de quem está em decode. Ver Q067.

Armadilhas comuns

- Resolver só um dos três níveis.
- Propor SJF sem proteção contra inanição.
- Esquecer o roteamento entre réplicas.
- Medir só o p99 agregado.

Resumo do capítulo

- **Batching estático** desperdiça slots e infla a latência quando as saídas têm tamanhos diferentes.

- **Batching dinâmico** (até B_{max} ou T_{max}) serve para modelos de custo fixo; não basta para LLMs.
- **Continuous batching** (Orca): agendar por iteração; quem termina sai, quem chega entra no próximo passo. Batching seletivo junta as GEMMs de todos os tokens e trata a atenção por sequência.
- O **scheduler** mantém filas de espera, execução e preempção; respeita memória do KV, orçamento de tokens e máximo de sequências; decide prioridade e preempção.
- **Chunked prefill**: prompts longos em pedaços junto com os decodes; ITL estável, cálculo ocioso aproveitado, pico de memória limitado.
- **HOL blocking** em três níveis: batch, passo e fila. Soluções: continuous batching, chunked prefill e admissão com pulo limitado, reservas por classe, pools separados e SJF aproximado com envelhecimento.
- Batching síncrono para modelos de custo fixo: fila limitada, prazo máximo de espera, futures, GPU fora do event loop, isolamento de erros, cancelamento, métricas.

Checklist para a entrevista

- ☐ Desenho batching estático e continuous batching e calculo a utilização de cada um.
- ☐ Explico batching seletivo e o layout achatado de tokens.
- ☐ Escrevo um batcher assíncrono com tamanho máximo, prazo máximo e fila limitada.
- ☐ Faço uma revisão de design priorizando corretude, robustez, desempenho e operação.
- ☐ Explico quando TensorRT-LLM ganha do vLLM e como decidir com medição.
- ☐ Escrevo o laço de um scheduler com decode primeiro, preempção e admissão com margem.
- ☐ Explico o orçamento de tokens e como escolhê-lo pelo SLO de ITL.
- ☐ Explico os três níveis de head-of-line blocking e as soluções de cada um.
- ☐ Discuto SJF aproximado e envelhecimento.
- ☐ Sei quais métricas separar por classe de tamanho.

CAPÍTULO 8

Quantização e Compressão de Modelos

Quantizar é guardar e calcular com números de menos bits. É a técnica de otimização com o melhor custo-benefício em inferência de LLMs, e por isso aparece em quase toda entrevista: o que é, quais formatos existem, como se faz sem destruir a qualidade, e quando preferir outras formas de compressão, como pruning e distilação.

A conexão com o resto do livro é direta. O decode é memory-bound (Capítulo 1): cada passo lê todos os pesos. Se cada peso ocupa metade dos bytes, o passo lê metade dos bytes e fica perto de duas vezes mais rápido. E o KV cache também pode ser quantizado (Capítulo 6), o que aumenta o batch.

8.1 Por que quantizar

Quatro ganhos, em ordem de importância para inferência:

1. **Menos bytes lidos por passo de decode** → decode mais rápido.
2. **Modelo cabe em menos GPUs** → menos comunicação, menos custo.
3. **Mais memória livre para o KV cache** → batches maiores, mais throughput.
4. **Tensor cores mais rápidos em baixa precisão** → prefill mais rápido (quando ativações também são quantizadas). Na H100, FP8 tem o dobro do pico de BF16.

Fazendo a conta: Llama 3 70B.

Formato dos pesos	Tamanho	Mínimo de H100 (só pesos)
BF16 (16 bits)	~140 GB	2
FP8 / INT8 (8 bits)	~70 GB	1 (sobram ~10 GB)
INT4 (4 bits + escalas)	~37 GB	1 (sobram ~43 GB para KV)

Em INT4, o passo de decode lê ~37 GB em vez de 140 GB: o limite teórico de velocidade quase quadruplica. Na prática o ganho é menor (dequantizar custa, e o KV cache não encolhe), mas é grande.

8.2 Formatos numéricos

Um número de ponto flutuante tem **sinal**, **expoente** (define a faixa, ou seja, o quão grande ou pequeno o número pode ser) e **mantissa** (define a precisão, quantos dígitos significativos).

```
bits:  S = sinal   E = expoente   M = mantissa

FP32   S EEEEEEEE MMMMMMMMMMMMMMMMMMMMMMM      1-8-23
FP16   S EEEEE MMMMMMMMMM                      1-5-10
BF16   S EEEEEEEE MMMMMMM                      1-8-7
FP8    S EEEE MMM          (E4M3)              1-4-3
FP8    S EEEEE MM          (E5M2)              1-5-2
FP4    S EE M              (E2M1)              1-2-1
INT8   SVVVVVVV            inteiro de -128 a 127
INT4   SVVV                inteiro de -8 a 7
```

Formato	Maior valor (aprox.)	Precisão relativa	Uso típico
FP32	$3{,}4 \times 10^{38}$	~7 dígitos	acumuladores, referência
FP16	65.504	~3 dígitos	inferência (risco de overflow)
BF16	$3{,}4 \times 10^{38}$	~2 dígitos	padrão de treino e inferência
FP8 E4M3	448	~1 dígito	pesos e ativações em inferência
FP8 E5M2	57.344	menos que E4M3	gradientes (mais faixa)
FP4 E2M1	6	muito baixa	com escalas por bloco (Blackwell)
INT8	127 (com escala)	uniforme	pesos e ativações
INT4	7 (com escala)	uniforme	pesos (W4A16)

Pontos que entrevistadores gostam de ouvir:

- **BF16 vs FP16**: o BF16 tem a mesma faixa do FP32 (8 bits de expoente), então quase nunca dá overflow; tem menos precisão. O FP16 tem mais precisão e faixa curta (máximo 65.504), o que exige cuidado com valores grandes.
- **FP8 vs INT8**: o FP8 tem precisão relativa (mais densidade de valores perto de zero, onde ficam a maioria dos pesos e ativações); o INT8 tem passo uniforme. O FP8 costuma lidar melhor com distribuições com cauda longa; o INT8 é suportado em mais hardware antigo.
- **FP4 com escalas por bloco**: com só 16 valores possíveis, o FP4 depende de **escalas compartilhadas por pequenos blocos** (por exemplo, a cada 16 ou 32 elementos), como nos formatos MXFP4 (padrão OCP de microscaling) e NVFP4 (NVIDIA).

8.3 Quantização linear: a matemática

Para converter um tensor real x para inteiros de b bits, usa-se uma **escala** s e, às vezes, um **ponto zero** z:

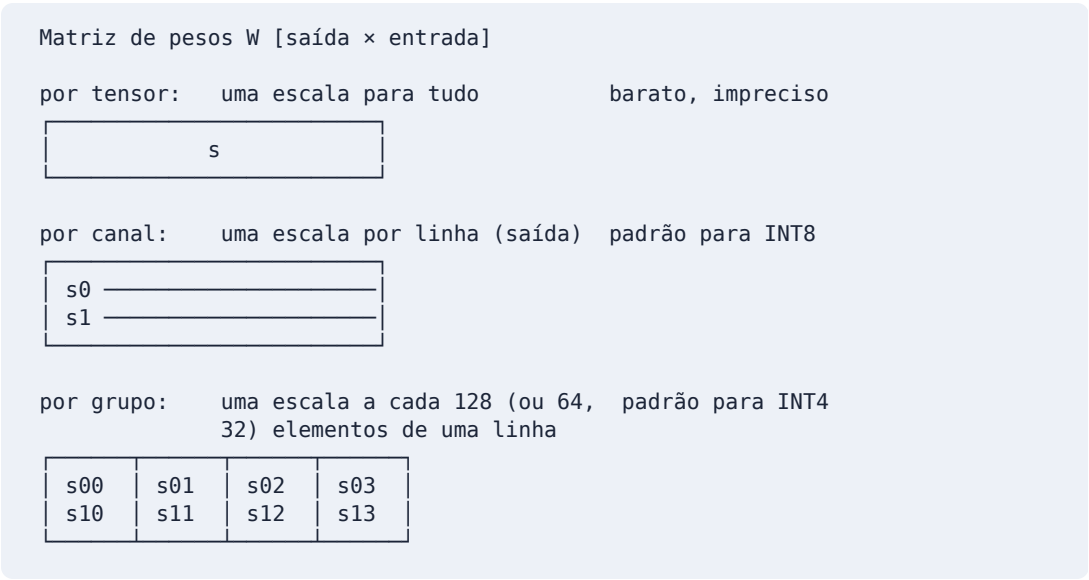
$$q = \text{clamp}(\text{round}(\frac{x}{s}) + z,\ q_{min},\ q_{max}) \qquad \hat{x} = s \cdot (q - z)$$

- **Simétrica** ($z = 0$): $s = \max |x| / q_{max}$. Simples e rápida; ideal para distribuições centradas em zero (pesos).
- **Assimétrica** ($z \neq 0$): usa toda a faixa quando a distribuição não é centrada (por exemplo, saídas de ReLU, sempre positivas).

> **Fazendo a conta: INT8 simétrico.** Um grupo de pesos: [0,12; −0,50; 0,03; 0,91; −0,27].
>
> - $\max |x| = 0,91$. $s = 0,91/127 = 0,007165$.
> - q = round(x / s) = [17; −70; 4; 127; −38].
> - De volta: $\hat{x}$ = [0,1218; −0,5016; 0,0287; 0,9100; −0,2723].
> - Erro máximo: ~0,0023 (no −0,27), menor que $s/2 \approx 0,0036$, como esperado.
>
> Agora, se o grupo tivesse um **outlier** de 9,1 em vez de 0,91: $s = 0,0717$, e o valor 0,03 viraria round(0,42) = 0. Um único valor grande destrói a precisão de todos os outros do grupo. Esse é o problema central da quantização de LLMs.

8.4 Granularidade das escalas

Quanto menor o grupo que compartilha uma escala, menor o estrago de um outlier, e maior o custo de guardar escalas.



Para ativações, a escala pode ser **por tensor** (estática, calculada com dados de calibração, mais rápida) ou **por token** (dinâmica, calculada em tempo de execução, mais precisa).

O custo do grupo: em INT4 com grupos de 128 e escalas em FP16, cada grupo adiciona 16 bits para 128 pesos, ou 0,125 bit por peso. O “INT4” vira, na prática, uns 4,125 bits por peso (mais, se houver ponto zero).

8.5 O que quantizar

A notação **WxAy** diz quantos bits têm os pesos (W) e as ativações (A):

Esquema	Pesos	Ativações	Ganho principal	Onde brilha
W8A16 / W4A16	8 / 4 bits	BF16	menos bytes lidos	decode com batch pequeno
W8A8 (INT8 ou FP8)	8 bits	8 bits	menos bytes e GEMMs mais rápidas	prefill e decode com batch grande
W4A8, W4A4, FP4	4 bits	8 ou 4 bits	os dois, mais agressivo	hardware com suporte (Blackwell)

Esquema	Pesos	Ativações	Ganho principal	Onde brilha
KV cache FP8/INT8/INT4	—	KV	mais sequências por GPU	contexto longo, batch grande

> **Cuidado:** W4A16 **não** acelera tudo. Os pesos precisam ser dequantizados para BF16 dentro do kernel antes da multiplicação. Com batch pequeno (memory-bound), isso é quase de graça e o ganho é grande. Com batch grande (compute-bound), o custo da dequantização aparece e um kernel W4A16 pode ficar **mais lento** que BF16 puro. Kernels especializados (como o Marlin) foram criados para manter o ganho em batches médios. Para batches grandes, W8A8 em FP8 costuma ser a melhor escolha na Hopper.

8.6 O problema dos outliers

Pesquisas como o **LLM.int8()** (Dettmers et al., 2022) mostraram que, em LLMs grandes, algumas **dimensões das ativações** têm valores muito maiores que as outras (outliers sistemáticos por canal). Quantizar essas ativações por tensor em INT8 destrói a precisão.

```
Ativações X [tokens × canais], valores absolutos típicos:

          c0   c1   c2   c3   c4 ... c1000 ... c4095
tok 0   0,3  0,1  0,5  0,2  0,4      58,0       0,3
tok 1   0,2  0,4  0,1  0,3  0,2      61,0       0,1
tok 2   0,5  0,2  0,3  0,1  0,6      55,0       0,4
                                       ▲
                    um canal com valores ~100× maiores
```

Soluções:

- **LLM.int8()**: separa os canais com outliers e os calcula em FP16; o resto em INT8. Preciso, mas o kernel misto é lento.
- **SmoothQuant** (Xiao et al., 2023): **migra a dificuldade** das ativações para os pesos. Como $Y = XW = (X\,\mathrm{diag}(s)^{-1})(\mathrm{diag}(s)W)$, dividir cada canal j da ativação por s_j e multiplicar a linha j do peso por s_j não muda o resultado. Escolhendo $s_j = \max|X_j|^{\alpha} / \max|W_j|^{1-\alpha}$ (com α perto de 0,5), os dois ficam “lisos” o bastante para INT8. A divisão pode ser absorvida na camada anterior (por exemplo, nos pesos da normalização), sem custo em tempo de execução.

- **Rotações** (QuaRot, SpinQuant): multiplicar por matrizes ortogonais (como Hadamard) espalha os outliers por todos os canais antes de quantizar, permitindo até W4A4.
- **FP8**: por ter faixa dinâmica maior que o INT8, tolera melhor os outliers, o que é um dos motivos da sua popularidade.

8.7 Métodos de quantização pós-treino (PTQ)

PTQ (post-training quantization) quantiza um modelo já treinado, sem treinar de novo, usando no máximo um pequeno conjunto de **dados de calibração** (algumas centenas de exemplos).

RTN (round-to-nearest). Arredonda cada peso para o valor quantizado mais próximo, com escalas por grupo. Simples e rápido. Em 8 bits, quase sempre basta. Em 4 bits, perde qualidade visível em vários modelos.

GPTQ (Frantar et al., 2022). Quantiza as colunas da matriz de pesos uma de cada vez e **compensa o erro**: depois de arredondar uma coluna, ajusta as colunas ainda não quantizadas para cancelar o erro introduzido, usando informação de segunda ordem (a matriz $H = X^\top X$ calculada com as ativações de calibração). O objetivo é minimizar o erro na **saída** da camada, $\| XW - X\hat{W} \|^2$, não nos pesos.

```
GPTQ, uma camada:
  H = Xᵀ X   (das ativações de calibração)
  para cada coluna i:
      q_i = quantiza(w_i)
      erro = (w_i − q_i) / [H⁻¹]_ii
      colunas seguintes −= erro × [H⁻¹]_i,seguintes
  → o erro de cada arredondamento é "absorvido" pelos
    pesos que ainda não foram quantizados
```

AWQ (Lin et al., 2023). Observa que uma pequena fração dos pesos é **saliente**: os que multiplicam canais de ativação com valores grandes. Em vez de manter esses pesos em alta precisão (o que exigiria kernels mistos), o AWQ **escala** esses canais para cima antes de quantizar (e divide a ativação correspondente), reduzindo o erro relativo justamente onde mais importa. As escalas são buscadas com os dados de calibração.

Método	Ideia	Custo de quantizar	Uso comum
RTN	arredondar	segundos	8 bits
GPTQ	compensar erro com segunda ordem	minutos a horas	4 bits (pesos)

Método	Ideia	Custo de quantizar	Uso comum
AWQ	proteger canais salientes com escala	minutos	4 bits (pesos)
SmoothQuant	migrar dificuldade das ativações para os pesos	minutos	W8A8 INT8
FP8 com calibração	escalas por tensor ou canal	minutos	W8A8 FP8 na Hopper

8.8 QAT e QAD

QAT (quantization-aware training): treina (ou faz fine-tuning) com a quantização **simulada** no forward. Os pesos continuam em alta precisão, mas a cada forward passam por "quantiza e dequantiza" (fake quantization), então o modelo aprende a ser robusto ao arredondamento.

O problema é que o arredondamento tem derivada zero em quase todo lugar. A solução é o **straight-through estimator (STE)**: no backward, finge-se que a função de arredondamento é a identidade e o gradiente passa direto.

```
forward:   w ──► [fake quant: round(w/s)·s] ──► ŵ ──► camada
backward:  ∂L/∂w ≈ ∂L/∂ŵ        (STE: ignora o round)
```

QAD (quantization-aware distillation): QAT em que o objetivo não é a perda original de treino, mas **imitar o modelo original em precisão total** (o professor). A perda é, por exemplo, a divergência KL entre as distribuições de saída do professor e do aluno quantizado:

$$\mathcal{L}_{QAD} = \mathrm{KL}(p_{professor}(\cdot \mid x) \,\|\, p_{aluno\ quantizado}(\cdot \mid x))$$

Vantagens do QAD sobre o QAT comum: não precisa do dataset e do pipeline de treino originais (que muitas vezes não estão disponíveis, como em modelos após RLHF), e o sinal do professor (a distribuição inteira, não só o token correto) é mais rico. Ferramentas como o NVIDIA Model Optimizer oferecem fluxos de QAT e QAD.

	PTQ	QAT	QAD
Treino	nenhum	fine-tuning com fake quant	fine-tuning com fake quant + professor
Dados	calibração (centenas de exemplos)	dados de treino da tarefa	prompts variados; rótulos vêm do professor

	PTQ	QAT	QAD
Custo	minutos a horas	horas a dias de GPU	horas a dias de GPU
Qualidade em baixa precisão	boa em 8 bits; variável em 4	melhor	melhor, sem precisar do pipeline original

8.9 Pruning

Pruning (poda) remove pesos ou estruturas inteiras.

- **Não estruturado**: zera pesos individuais (por exemplo, os de menor magnitude, ou com critérios como SparseGPT e Wanda). Alta esparsidade com pouca perda, mas **não acelera** em GPUs sem suporte a esparsidade arbitrária: a matriz continua do mesmo tamanho.
- **Semi-estruturado 2:4**: em cada grupo de 4 pesos, exatamente 2 são zero. Os tensor cores desde a Ampere aceleram esse padrão (até 2× no pico teórico de cálculo, e metade dos bytes nos valores, mais metadados). O ganho real em LLMs costuma ser menor e a perda de qualidade precisa ser recuperada com fine-tuning.
- **Estruturado**: remove heads de atenção, canais do MLP ou camadas inteiras. A matriz fica realmente menor, e o ganho é direto em qualquer hardware. Normalmente exige **distilação** depois, para recuperar a qualidade (abordagens como a Minitron, da NVIDIA, combinam poda de largura e profundidade com distilação).

8.10 Distilação

Distilação treina um modelo menor (aluno) para imitar um maior (professor). Formas comuns:

- **Por logits**: o aluno aprende a distribuição de probabilidades do professor em cada posição (KL com temperatura). Transmite mais informação do que só o token correto ("o professor acha que 'carro' é 60% provável e 'automóvel' 30%").
- **Por sequência**: o professor gera respostas para muitos prompts, e o aluno é treinado nessas respostas como se fossem dados normais. Simples, funciona até com professores acessíveis só por API (respeitando os termos de uso).
- **Por representações internas**: o aluno imita estados intermediários do professor (exige mapear camadas).

A distilação muda a **arquitetura** (menos camadas, menos largura), então o ganho de velocidade vale para qualquer hardware e se soma à quantização.

Perguntas de entrevista comentadas

Q057 O que é quantização? O que são QAT e QAD?

> **[Relatado]** NVIDIA · Senior Software Engineer · entrevista técnica · 2026 · Médio · fonte

Resposta em 30 segundos

"Quantização é representar pesos, ativações ou KV cache com menos bits, como FP8, INT8 ou INT4, usando escalas para mapear os valores reais na faixa do formato. Em inferência de LLMs, ela reduz os bytes lidos no decode, que é memory-bound, faz o modelo caber em menos GPUs e libera memória para o KV cache. QAT, quantization-aware training, é fazer fine-tuning com a quantização simulada no forward e o straight-through estimator no backward, para o modelo aprender a ser robusto ao arredondamento. QAD, quantization-aware distillation, é o mesmo processo, mas com a perda definida como a distância, por exemplo KL, entre as saídas do modelo quantizado e do modelo original em precisão total, que funciona como professor. Isso dispensa os dados e o pipeline de treino originais e costuma recuperar melhor a qualidade em precisões baixas como FP4."

Resposta completa

Estruture em quatro partes.

1. O que é, com a fórmula. Escala e ponto zero (8.3), um exemplo numérico e o efeito dos outliers.

2. As escolhas de projeto.

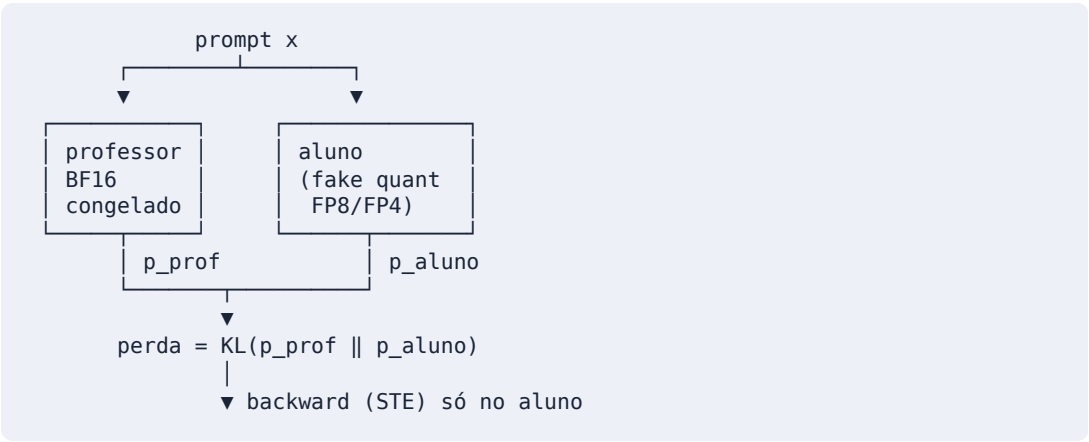
```
O QUÊ:          pesos | ativações | KV cache
QUANTOS BITS:   8 | 4 | (2, FP4 com blocos)
FORMATO:        INT (uniforme) | FP (relativo)
GRANULARIDADE:  tensor | canal | grupo | token | bloco
ESCALA:         estática (calibração) | dinâmica (runtime)
COMO:           PTQ (RTN, GPTQ, AWQ, SmoothQuant)
                | QAT | QAD
```

3. QAT em detalhe.

- Fake quantization no forward: $\hat{w} = s \cdot \text{round}(w/s)$.
- STE no backward: $\partial\hat{w}/\partial w \approx 1$ (dentro da faixa; zero onde há saturação, em algumas variantes).

- As escalas também podem ser aprendidas (como no LSQ, learned step size quantization).
- Normalmente parte de um modelo já treinado e faz um fine-tuning curto.
- Ganho: em precisões baixas (4 bits e menos), recupera boa parte da qualidade que o PTQ perde.

4. QAD em detalhe.



Por que é melhor que QAT comum em muitos casos:

- O objetivo é exatamente o que importa: **preservar o comportamento do modelo original**, não reaprender a tarefa.
- Não precisa do dataset original nem refazer o alinhamento (instrução, RLHF); qualquer conjunto variado de prompts serve, e as respostas do professor são os rótulos.
- A distribuição completa do professor dá um sinal mais rico por token.

O custo: precisa rodar o professor junto (memória e cálculo extras) e é um processo de treino, com tudo que isso exige.

Perguntas de acompanhamento prováveis

- **Quando PTQ basta?** Em 8 bits (FP8 ou INT8 bem feito), quase sempre. Em 4 bits só nos pesos, com GPTQ ou AWQ, em muitos modelos. Abaixo disso, ou em W4A4/FP4 com ativações, QAT ou QAD costumam ser necessários.
- **O que o STE faz, exatamente?** Trata o arredondamento como identidade no backward, para o gradiente não ser sempre zero. É uma aproximação, mas funciona bem na prática.

- **Como medir se a quantização estragou o modelo?** Perplexidade é um primeiro sinal, mas insuficiente. Rode as avaliações das tarefas que importam (raciocínio, código, contexto longo, idiomas), compare com o ruído entre execuções e faça comparação lado a lado de respostas.
- **Por que FP8 virou padrão na Hopper?** Tensor cores com o dobro do pico de BF16, faixa dinâmica que tolera outliers e perda de qualidade pequena com escalas simples.

Armadilhas comuns

- Definir quantização só como “reduzir precisão” sem falar de escala e granularidade.
- Confundir QAT com PTQ com calibração.
- Não saber o que é o STE.
- Não diferenciar QAD de QAT.

Q058 Descreva técnicas comuns de otimização de IA para treinamento e inferência: quantização, pruning, distilação, fusão de kernels, otimizações de memória e de throughput.

> **[Relatado]** NVIDIA · Software Engineer · triagem técnica · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

“Eu organizo em três grupos. Reduzir o modelo: quantização (menos bits por número), pruning (menos pesos, estruturado ou 2:4) e distilação (modelo menor que imita um maior). Executar melhor o mesmo modelo: fusão de kernels para evitar idas à memória, FlashAttention, CUDA graphs, compiladores como torch.compile e TensorRT. E gerenciar memória e throughput: no treino, precisão mista, gradient checkpointing, ZeRO/FSDP e paralelismo; na inferência, KV cache paginado, prefix caching, continuous batching, chunked prefill e speculative decoding. A escolha depende do gargalo: memória, cálculo ou overhead, e de quanto de qualidade dá para arriscar.”

Resposta completa

Apresente como uma tabela-mapa e depois aprofunde dois ou três itens, de acordo com o interesse do entrevistador:

Técnica	Treino	Inferência	Ataca	Afeta qualidade?
Precisão mista (BF16)	✔	✔	cálculo, memória	quase nada
Quantização FP8/INT8/INT4	(FP8)	✔	bytes, cálculo	pouco a moderado
Pruning 2:4 / estruturado	—	✔	cálculo, bytes	sim, recuperar com treino
Distilação	gera o modelo	✔	tudo (modelo menor)	sim, avaliar
Fusão de kernels	✔	✔	tráfego de memória	não
FlashAttention	✔	✔	memória e tráfego da atenção	não
CUDA graphs	✔	✔	overhead de CPU	não
Gradient checkpointing	✔	—	memória de ativações	não (mais cálculo)
ZeRO / FSDP	✔	—	memória de estados do otimizador	não
KV cache paginado, prefix caching	—	✔	memória do KV	não
Continuous batching, chunked prefill	—	✔	utilização, latência	não
Speculative decoding	—	✔	latência do decode	não (com verificação exata)

Aprofunde as do enunciado:

Quantização: 8.3 a 8.8.

Pruning: 8.9. Diga a diferença essencial: esparsidade não estruturada reduz o número de pesos, mas não acelera sem suporte de hardware; 2:4 tem suporte nos tensor cores; estruturado reduz as matrizes de verdade e acelera em qualquer lugar, mas pede distilação para recuperar qualidade.

Distilação: 8.10.

Fusão de kernels: Capítulo 4. Exemplo concreto: residual + RMSNorm + quantização em um kernel; epílogos de GEMM com ativação.

Memória e throughput no treino:

- **Precisão mista**: pesos mestres em FP32, cálculo em BF16 (ou FP8 com escalas).
- **Gradient checkpointing**: guarda só algumas ativações no forward e recalcula as outras no backward. Troca cálculo (~30% a mais) por memória.
- **ZeRO / FSDP**: divide estados do otimizador, gradientes e pesos entre as GPUs, em vez de replicar.
- **Paralelismo** (dados, tensor, pipeline, especialistas): Capítulo 9.

Memória e throughput na inferência: KV cache (Capítulo 6), batching (Capítulo 7), speculative decoding (Capítulo 14).

Termine com o critério de escolha: "Primeiro meço o gargalo. Se o decode é memory-bound com MBU alto, o caminho é ler menos bytes: quantização. Se o problema é custo e há margem de qualidade, distilação. Se o perfil mostra muita GPU ociosa entre kernels, CUDA graphs e fusão. E toda técnica que muda números passa por avaliação."

Perguntas de acompanhamento prováveis

- **Tensor parallelism vs pipeline parallelism?** É a outra metade desta pergunta no relato original; ver Q065.
- **Qual técnica você aplicaria primeiro num modelo novo em produção?** FP8 nos pesos e ativações (se o hardware suportar) e KV cache em FP8: grande ganho, pouco risco, ferramentas maduras.
- **Pruning ajuda em decode memory-bound?** Pruning estruturado sim (menos bytes). Não estruturado só se o kernel souber explorar a esparsidade e ler menos bytes; na maioria das GPUs, não.
- **Distilação e quantização se combinam?** Sim. É comum distilar para um modelo menor e depois quantizá-lo, ou usar o QAD, que é distilação com quantização ao mesmo tempo.

Armadilhas comuns

- Listar técnicas sem dizer o que cada uma ataca.
- Misturar técnicas de treino e de inferência sem distinguir.
- Esquecer o impacto na qualidade.
- Dizer que esparsidade não estruturada acelera GPUs automaticamente.

Q059 Explique quantização e distilação de modelos para otimizar inferência: como escolher entre as duas?

> **[Fornecedor]** Relatada “em várias empresas” numa compilação de 2026 · Presencial · 2026 · Médio · fonte

Resposta em 30 segundos

“Quantização mantém a arquitetura e reduz os bits por número: é barata, rápida de aplicar, dá 2 a 4 vezes menos memória, e o ganho de velocidade depende do suporte do hardware e do regime (memory-bound ou compute-bound). Distilação treina um modelo menor para imitar o maior: exige um projeto de treino, mas pode reduzir o custo em 5 ou 10 vezes e acelera em qualquer hardware, ao preço de perder capacidade geral. Eu começo pela quantização, porque o risco e o custo são baixos. Parto para a distilação quando preciso de um ganho maior do que a quantização dá, quando a tarefa é estreita e bem definida, ou quando a latência exige um modelo pequeno. E muitas vezes faço as duas: distilo e depois quantizo o aluno.”

Resposta completa

Compare lado a lado:

	Quantização	Distilação
O que muda	bits por número	arquitetura (modelo menor)
Esforço	horas a dias (PTQ); dias (QAT/QAD)	semanas: dados, treino, avaliação
Ganho de memória	2× (8 bits) a ~4× (4 bits)	proporcional à redução do modelo (5–20×)
Ganho de velocidade	depende do hardware e do regime	em qualquer hardware
Qualidade	perto do original em 8 bits	perde capacidade fora da tarefa-alvo
Generalidade	mantém todas as habilidades (quase)	ótima na tarefa treinada, pior fora dela
Reversível	sim (o original continua lá)	é outro modelo, com outro ciclo de vida

E a árvore de decisão:

```
Preciso reduzir custo/latência
  │
  ├─ Ganho de 1,5–3× resolve?
  │    └─ SIM → quantização (FP8; depois INT4 nos pesos)
  │
  ├─ Preciso de 5×+ ou latência muito baixa?
  │    ├─ A tarefa é estreita e estável
  │    │  (classificar, extrair, um domínio)?
  │    │    └─ SIM → distilação + quantização do aluno
  │    └─ Preciso das habilidades gerais do modelo?
  │         └─ quantização agressiva (QAD, FP4) +
  │            speculative decoding + melhor serving;
  │            distilar com dados amplos, aceitando perda
  │
  └─ O hardware suporta o formato (FP8, FP4, INT4)?
       └─ NÃO → a quantização vira só economia de
                memória; distilação ganha peso
```

Uma conta de custo ajuda a mostrar maturidade:

> **Fazendo a conta (valores hipotéticos):** um classificador de tickets usa um modelo de 70B, com 50 milhões de chamadas por mês.
>
> - Quantização FP8: corta o custo por chamada em ~40%. Esforço: uma semana.
> - Distilação para um 8B especializado: ~8× mais barato por chamada. Esforço: 6 semanas de engenharia + ~US$ 20 mil de GPU para gerar dados com o professor, treinar e avaliar.
> - Se o custo atual é US$ 100 mil por mês: a quantização economiza US$ 40 mil por mês; a distilação, ~US$ 87 mil por mês. A distilação se paga em menos de dois meses, **se** a qualidade na tarefa se mantiver, o que precisa ser medido antes de decidir.

Perguntas de acompanhamento prováveis

- **Como garantir que o modelo distilado não piorou em casos raros?** Avaliação estratificada: casos difíceis, classes raras, idiomas, entradas adversárias. E monitoramento em produção com amostras revisadas e fallback para o modelo grande quando a confiança for baixa.
- **De onde vêm os dados para distilar?** Prompts reais da produção (com as devidas permissões e anonimização) rotulados pelo professor, mais dados sintéticos para cobrir casos raros.

- **O QAD é distilação ou quantização?** As duas coisas: é distilação de um modelo para a sua própria versão quantizada, sem mudar a arquitetura.
- **E LoRA?** É uma técnica de fine-tuning eficiente, não de compressão. Mas é comum distilar um modelo menor usando fine-tuning completo ou LoRA, e servir vários adaptadores LoRA sobre um modelo base quantizado (Capítulo 14).

Armadilhas comuns

- Tratar as duas como alternativas excludentes.
- Ignorar o custo de engenharia da distilação.
- Prometer ganho de velocidade da quantização sem considerar o hardware.
- Não falar de avaliação e de perda de generalidade.

Resumo do capítulo

- Quantizar reduz bytes lidos (decode mais rápido), faz o modelo caber em menos GPUs, libera memória para o KV cache e usa tensor cores mais rápidos.
- Formatos: BF16 (faixa do FP32), FP16 (mais precisão, faixa curta), FP8 E4M3/E5M2, INT8, INT4, FP4 com escalas por bloco.
- Quantização linear: $q = \text{round}(x/s) + z$. Outliers estragam a escala do grupo inteiro.
- Granularidade: tensor, canal, grupo (128), token, bloco. INT4 por grupo custa ~0,125 bit extra por peso.
- W4A16 brilha com batch pequeno; W8A8 FP8 é o padrão para batch grande na Hopper.
- Outliers de ativação: LLM.int8() (separar), SmoothQuant (migrar para os pesos), rotações (espalhar), FP8 (tolerar).
- PTQ: RTN, GPTQ (compensação de erro com segunda ordem), AWQ (proteger canais salientes). QAT: fake quant + STE. QAD: QAT com professor em precisão total e perda KL.
- Pruning: não estruturado (não acelera sozinho), 2:4 (tensor cores), estruturado (acelera em qualquer hardware, pede distilação).
- Distilação: modelo menor imitando o maior; maior ganho, maior esforço, perda de generalidade.

Checklist para a entrevista

- [ ] Calculo o tamanho de um modelo em cada formato e quantas GPUs ele exige.
- [ ] Desenho os bits de FP32, BF16, FP16, FP8 e FP4 e explico faixa vs precisão.

- [ ] Quantizo um vetor à mão em INT8 simétrico e mostro o efeito de um outlier.
- [ ] Explico granularidade por tensor, canal, grupo e token, e o custo das escalas.
- [ ] Explico quando W4A16 acelera e quando não acelera.
- [ ] Explico SmoothQuant com a fórmula da escala por canal.
- [ ] Explico GPTQ e AWQ em uma frase cada.
- [ ] Explico QAT com STE e QAD com professor e KL.
- [ ] Comparo pruning não estruturado, 2:4 e estruturado.
- [ ] Escolho entre quantização e distilação com uma árvore de decisão e uma conta de custo.

CAPÍTULO 9

Inferência Distribuída

Um Llama 3 70B em BF16 não cabe numa H100. Modelos de centenas de bilhões de parâmetros, ou MoEs com quase um trilhão, precisam de dezenas de GPUs. Mesmo quando o modelo cabe, dividir entre várias GPUs pode reduzir a latência, porque soma a banda de memória de todas elas. Este capítulo explica as formas de dividir o trabalho entre GPUs, quanto cada uma custa em comunicação e quando usar cada uma.

As entrevistas pedem, quase sempre, três coisas: comparar **tensor parallelism** com **pipeline parallelism**, projetar uma **multiplicação de matrizes distribuída** e discutir a **desagregação de prefill e decode**. Os detalhes de rede e das operações coletivas estão no Capítulo 10.

9.1 Por que distribuir

Três motivos diferentes, que levam a escolhas diferentes:

1. **Capacidade**: os pesos e o KV cache não cabem numa GPU.
2. **Latência**: dividir cada camada entre p GPUs faz cada uma ler $1/p$ dos pesos; o passo de decode pode ficar perto de p vezes mais rápido (menos a comunicação).
3. **Throughput**: mais réplicas atendem mais requisições.

```
Os eixos de paralelismo

DP   data parallel      réplicas inteiras do modelo
TP   tensor parallel    cada matriz dividida entre GPUs
PP   pipeline parallel  camadas divididas em estágios
EP   expert parallel    especialistas de MoE em GPUs
                        diferentes
CP   context parallel   a sequência dividida entre GPUs
     (ou SP)            (contextos muito longos)
```

9.2 Data parallelism (réplicas)

Cada GPU (ou grupo de GPUs) tem o modelo inteiro e atende requisições diferentes. Não há comunicação entre réplicas durante a inferência; um balanceador distribui as requisições.

- **Prós**: simples, throughput escala linearmente, uma réplica com problema não derruba as outras.
- **Contras**: não reduz a latência de uma requisição, e exige que o modelo caiba no grupo de cada réplica.

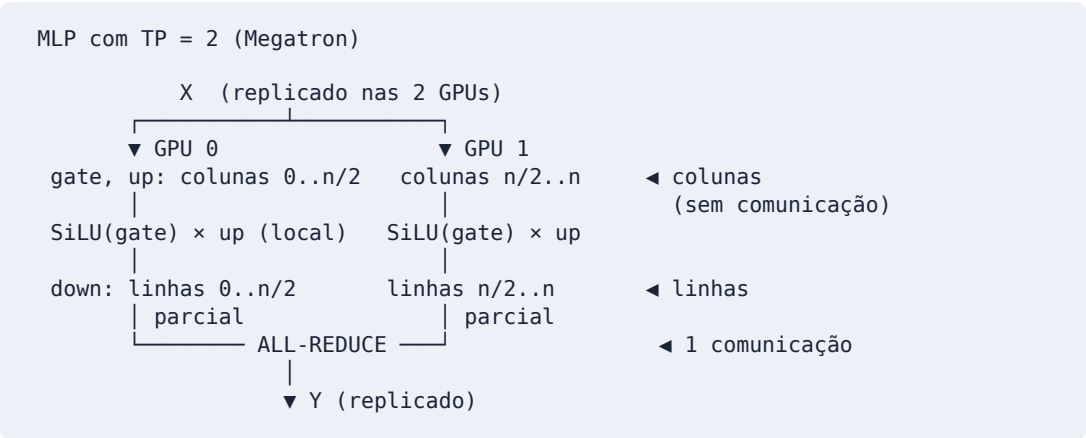
Na prática, todo serviço em produção usa DP no nível mais alto, combinado com TP, PP ou EP dentro de cada réplica.

9.3 Tensor parallelism

Cada matriz de pesos é dividida entre as p GPUs. Todas as GPUs participam de toda camada, cada uma com uma fatia. O esquema padrão vem do **Megatron-LM** (Shoeybi et al., 2019) e usa dois tipos de camada linear:

Paralela em colunas. W é dividida por colunas: a GPU i tem $W_{:,i}$. Cada GPU calcula $Y_i = XW_{:,i}$, uma fatia das colunas da saída. Não precisa de comunicação se a próxima operação trabalhar com essa fatia.

Paralela em linhas. W é dividida por linhas e a entrada chega já fatiada: a GPU i tem X_i e $W_{i,:}$ e calcula $X_i W_{i,:}$, que é uma **soma parcial** da saída completa. Um **all-reduce** (soma entre todas as GPUs) junta os parciais.



A **atenção** se divide naturalmente por heads: cada GPU calcula H/p heads (projeções Q, K, V em colunas), roda a atenção dessas heads localmente com a sua parte do KV cache, e a projeção de saída é paralela em linhas, com um all-reduce.

Resultado: **dois all-reduces por camada** (um depois da atenção, um depois do MLP), cada um com um tensor de tamanho [tokens × d_model].

Em código, com `torch.distributed`:

```
import torch
import torch.distributed as dist

class LinearColunas(torch.nn.Module):
    """W dividida por colunas: saída local, sem comunicação."""
    def __init__(self, d_in, d_out, tp):
        super().__init__()
        self.w = torch.nn.Parameter(
            torch.empty(d_in, d_out // tp))

    def forward(self, x):            # x replicado
        return x @ self.w            # fatia das colunas

class LinearLinhas(torch.nn.Module):
    """W dividida por linhas: soma parcial + all-reduce."""
    def __init__(self, d_in, d_out, tp):
        super().__init__()
        self.w = torch.nn.Parameter(
            torch.empty(d_in // tp, d_out))

    def forward(self, x_local):      # x já fatiado
        y = x_local @ self.w         # parcial
        dist.all_reduce(y, op=dist.ReduceOp.SUM)
        return y                     # completo, replicado

class MLPParalelo(torch.nn.Module):
    def __init__(self, d, d_ff, tp):
        super().__init__()
        self.gate = LinearColunas(d, d_ff, tp)
        self.up = LinearColunas(d, d_ff, tp)
        self.down = LinearLinhas(d_ff, d, tp)

    def forward(self, x):
        h = torch.nn.functional.silu(self.gate(x)) * self.up(x)
        return self.down(h)
```

O custo da comunicação

> **Fazendo a conta: Llama 3 70B com TP = 8 numa máquina com NVLink.**
>
> *Decode, batch de 64:*
>
> - Cada all-reduce: 64 tokens × 8.192 × 2 bytes = **1 MB**.
> - Por passo: 80 camadas × 2 = **160 all-reduces**.
> - Mensagens de 1 MB são pequenas: o custo é dominado pela **latência** de cada coletiva (da ordem de 10 a 20 µs), não pela banda.
> - 160 × ~15 µs ≈ **2,4 ms** por passo.
> - Leitura de pesos por GPU: 140 GB / 8 = 17,5 GB → a 3,35 TB/s, **5,2 ms**.
> - A comunicação acrescenta uns 45% ao passo se não for sobreposta. Por isso motores usam all-reduces customizados para mensagens pequenas e tentam sobrepor comunicação e cálculo.
>
> *Prefill de 8.000 tokens:*
>
> - Cada all-reduce: 8.000 × 8.192 × 2 bytes ≈ **131 MB**.
> - No all-reduce em anel, cada GPU envia ~2 × (7/8) × 131 MB ≈ 230 MB. Por passo: 160 × 230 MB ≈ **37 GB** por GPU.
> - Com ~450 GB/s por sentido na NVLink da H100: **~80 ms**, contra ~300 ms de cálculo. Relevante, mas sobreponível.

Por isso a regra prática: **TP dentro de uma máquina** (NVLink, centenas de GB/s), quase nunca entre máquinas (InfiniBand, ~50 GB/s por GPU).

9.4 Pipeline parallelism

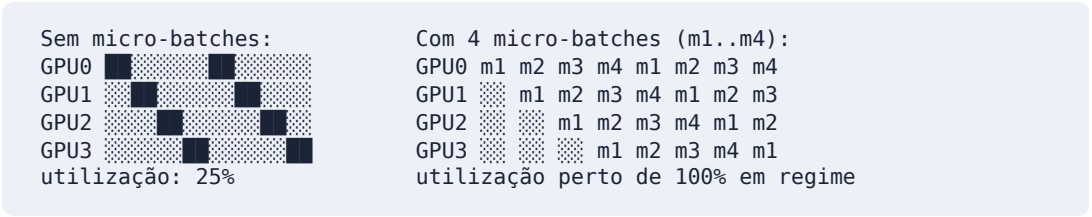
As camadas são divididas em **estágios**: a GPU 0 tem as camadas 0 a 19, a GPU 1 tem 20 a 39, e assim por diante. Cada estágio passa as ativações para o próximo.

```
PP = 4, 80 camadas

GPU 0: camadas  0–19 → GPU 1: 20–39 → GPU 2: 40–59 → GPU 3: 60–79
       envia [tokens × d_model] para o próximo estágio
       (comunicação ponto a ponto, pequena)
```

- **Comunicação**: só ponto a ponto, um tensor de ativações por fronteira de estágio. Muito menos do que o TP. Funciona bem **entre máquinas**.

- **Latência**: uma requisição passa por todos os estágios em sequência. O PP **não** reduz a latência de um passo; adiciona os saltos de rede.
- **Bolhas**: com um único batch, só um estágio trabalha por vez. A solução é ter vários **micro-batches** em voo:



Na inferência, os micro-batches são simplesmente grupos diferentes de requisições: enquanto o estágio 1 processa o grupo A, o estágio 0 já processa o grupo B. No treino, a fração de bolha é $(p-1)/(m+p-1)$ com m micro-batches; na inferência com fluxo contínuo, as bolhas aparecem quando os grupos têm tamanhos diferentes ou o fluxo é irregular.

9.5 Expert parallelism (MoE)

Num modelo **Mixture of Experts**, o MLP de cada camada é substituído por vários **especialistas** (MLPs menores) e um **roteador** que escolhe, para cada token, os k especialistas que vão processá-lo (por exemplo, 8 de 256).

Com **EP**, os especialistas ficam em GPUs diferentes. Cada camada de MoE vira:

```
1. roteador decide os especialistas de cada token (local)
2. ALL-TO-ALL (dispatch): cada GPU envia cada token para
   as GPUs que têm os especialistas escolhidos
3. cada GPU roda os seus especialistas sobre os tokens
   que recebeu (GEMM agrupada)
4. ALL-TO-ALL (combine): os resultados voltam para a GPU
   de origem e são somados com os pesos do roteador
```

Desafios:

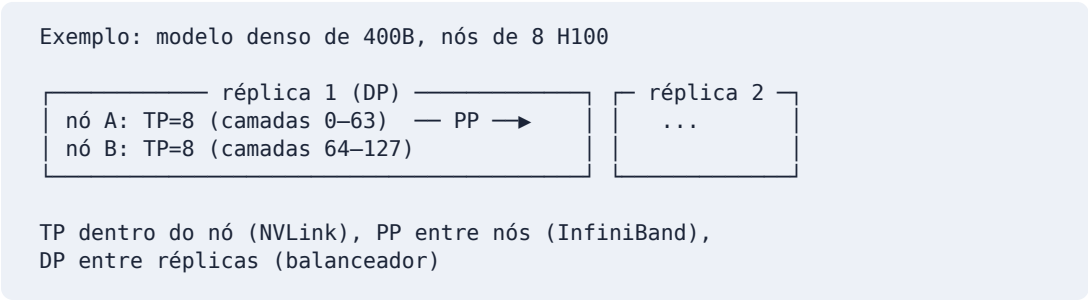
- **Desequilíbrio de carga**: se muitos tokens escolhem o mesmo especialista, a GPU dele vira gargalo. Soluções: perdas de balanceamento no treino, replicar especialistas populares, limitar a capacidade por especialista.
- **All-to-all é caro**, principalmente entre máquinas. Kernels especializados de comunicação e sobreposição com cálculo são essenciais.

- A atenção de um MoE costuma ser servida com DP (cada GPU com seu grupo de requisições), enquanto as camadas de especialistas usam EP. Essa combinação é comum em implantações de MoE grandes.

9.6 Context parallelism

Para contextos muito longos (centenas de milhares a milhões de tokens), até o prefill de **uma** sequência é pesado demais para um grupo de TP. O **context parallelism** divide a sequência entre GPUs: cada uma guarda os K e V de um pedaço do contexto. Na atenção, os blocos de K e V circulam entre as GPUs em anel (**ring attention**), e cada GPU acumula a atenção das suas queries com o softmax online (Capítulo 2), sobrepondo a comunicação do próximo bloco com o cálculo do atual.

9.7 Combinando tudo

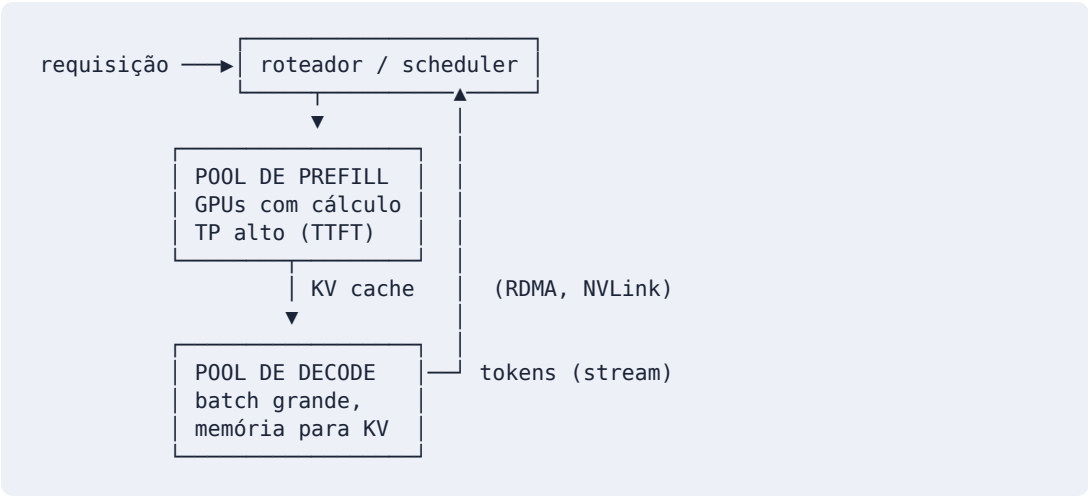


Como escolher, em ordem:

1. **Quanto de memória preciso?** Pesos + KV cache para o batch e o contexto alvo. Isso dá o número mínimo de GPUs por réplica.
2. **Cabe num nó?** Se sim, TP dentro do nó (latência baixa). Se não, PP entre nós (ou EP para MoE).
3. **Quanto de TP?** Mais TP reduz a latência por passo, mas a comunicação cresce em proporção ao trabalho. Teste TP = 2, 4, 8 e meça goodput.
4. **Quantas réplicas?** Pela demanda de throughput.

9.8 Desagregação de prefill e decode

O Capítulo 2 mostrou que prefill e decode têm gargalos opostos, e o Capítulo 7 mostrou que misturá-los no mesmo passo cria interferência. A **desagregação** coloca as duas fases em **pools de GPUs diferentes**:



Sistemas de referência: **DistServe** (Zhong et al., 2024), **Splitwise** (Patel et al., 2024) e **Mooncake** (Qin et al., 2024, da Moonshot AI, que serve o Kimi). O Mooncake é descrito como uma arquitetura **centrada no KV cache**: além de separar os pools, usa a DRAM e os SSDs ociosos do cluster como um armazenamento distribuído de KV cache, e um scheduler global que decide onde fazer o prefill para aproveitar o cache existente e rejeita requisições cedo quando a sobrecarga é prevista.

Perguntas de entrevista comentadas

Q065 Explique paralelismo de tensor versus paralelismo de pipeline, e quando você usaria cada um.

> **[Relatado]** NVIDIA · Software Engineer · triagem técnica · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

“Tensor parallelism divide cada matriz entre as GPUs: todas trabalham em toda camada, com dois all-reduces por camada. Isso reduz a latência de cada passo, porque cada GPU lê só uma fatia dos pesos, mas exige comunicação frequente e rápida, então uso dentro de um nó com NVLink. Pipeline parallelism divide as camadas em estágios: a comunicação é só ponto a ponto entre estágios e é pequena, então funciona entre nós, mas não reduz a latência de uma requisição e precisa de vários micro-batches em voo para não deixar estágios ociosos. Na prática: TP dentro do nó para latência; PP entre nós quando o modelo não cabe num nó; e DP por cima para throughput.”

Resposta completa

Use a tabela e os números:

	Tensor parallelism	Pipeline parallelism
O que divide	cada matriz (colunas/linhas, heads)	as camadas (estágios)
Comunicação	2 all-reduces por camada, [tokens × d]	1 envio ponto a ponto por fronteira de estágio
Volume	alto e frequente	baixo
Sensível à latência da rede	muito	pouco
Latência de um passo	cai (~1/p da leitura de pesos)	não cai (soma os saltos)
Throughput	bom	bom, se houver micro-batches suficientes
KV cache	dividido por heads	cada estágio guarda o das suas camadas
Onde	dentro do nó (NVLink)	entre nós (InfiniBand)
Limite	número de heads de KV (TP > H_kv replica KV)	bolhas, desequilíbrio entre estágios

Detalhes que impressionam:

- **TP e GQA**: o Llama 3 70B tem 8 heads de KV. Com TP = 8, cada GPU tem uma head de KV. Com TP = 16, as heads de KV precisam ser **replicadas**, e o KV cache por GPU deixa de diminuir. O número de heads de KV limita o TP útil.
- **TP e eficiência das GEMMs**: com TP alto, cada GPU multiplica matrizes mais finas; em batch pequeno, as GEMMs ficam ainda mais memory-bound e a fração de comunicação cresce. Há um ponto a partir do qual aumentar o TP piora.
- **PP e balanceamento**: o estágio mais lento dita o ritmo. A primeira e a última GPU têm trabalho extra (embedding, LM head), então os estágios nem sempre têm o mesmo número de camadas.
- **PP no decode**: com um único fluxo de requisições, só um estágio trabalha por vez. Para o PP ter bom throughput no decode, o scheduler precisa manter p grupos de requisições em voo ao mesmo tempo, um por estágio.

Exemplos de decisão:

```
70B BF16, 8 H100 num nó          → TP=8 (ou TP=4 × DP=2
                                     se a latência permitir)
70B FP8, 1 H100 cabe             → DP (réplicas de 1 GPU)
                                     ou TP=2 para latência
400B denso, 16 H100 em 2 nós     → TP=8 × PP=2
MoE de ~700B, muitos nós         → EP para especialistas +
                                     DP/TP para a atenção
```

Perguntas de acompanhamento prováveis

- **Por que não TP entre nós?** A banda por GPU cai de centenas de GB/s (NVLink) para ~50 GB/s (InfiniBand NDR), e a latência sobe. Com 160 all-reduces por passo, a comunicação passaria a dominar.
- **Sequence parallelism, no contexto do Megatron, é o quê?** Uma extensão do TP em que as operações que não são divididas pelo TP (normas, dropout) são divididas ao longo da sequência; o all-reduce vira reduce-scatter + all-gather. Reduz memória de ativações (mais relevante no treino).
- **Como sobrepor a comunicação do TP?** Dividir a GEMM em pedaços e começar a comunicar o primeiro enquanto calcula o segundo, ou fundir a GEMM com a coletiva em kernels que fazem as duas coisas.
- **O que muda com MoE?** Os especialistas pedem EP com all-to-all (9.5), que é outro padrão de comunicação.

Armadilhas comuns

- Dizer que PP reduz a latência.
- Não saber quantas comunicações o TP faz por camada.
- Ignorar o limite imposto pelo número de heads de KV.
- Não falar de micro-batches e bolhas.

Q066 Projete uma multiplicação de matrizes em múltiplas GPUs.

> **[Relatado]** Google · design de sistemas de ML · listada em 2026 · Difícil · fonte

Resposta em 30 segundos

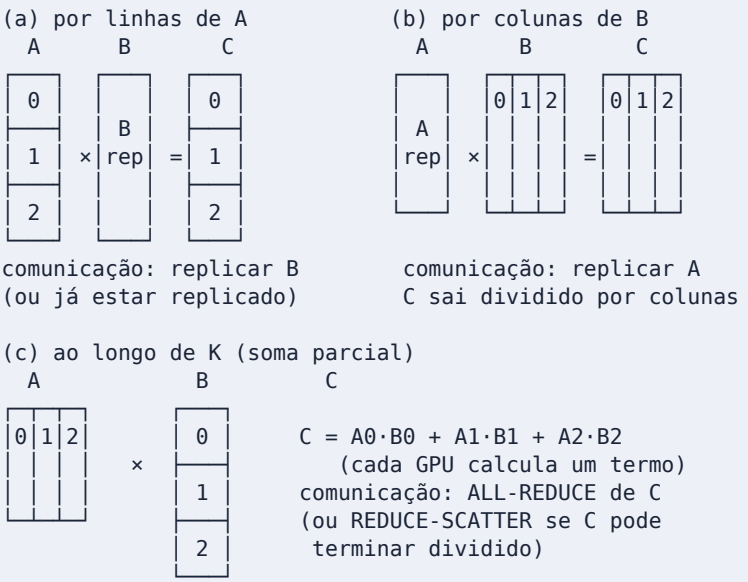
“Depende das formas e de onde os dados já estão. Para C = A·B com p GPUs, há três divisões básicas: por linhas de A, com B replicado e nenhuma comunicação no cálculo; por colunas de B, com A replicado, que é o column-parallel do Megatron; e ao longo de K, em que cada GPU calcula uma soma parcial e um all-reduce ou reduce-

scatter junta tudo, o row-parallel. Para matrizes grandes nas três dimensões, uso uma decomposição 2D como o SUMMA, que reduz a memória e a comunicação por GPU para a ordem de 1/√p. Escolho com um modelo de custo de comunicação e sobreponho comunicação e cálculo dividindo o trabalho em blocos.”

Resposta completa

1. Perguntas de esclarecimento. Tamanhos de M, N e K? Quantas GPUs e como estão ligadas? Onde A e B estão no começo, e onde C precisa terminar? É uma multiplicação única ou parte de uma sequência (como camadas de um modelo, em que a saída de uma é a entrada da próxima)?

2. As decomposições.



3. Modelo de custo. Use o modelo α-β: enviar n bytes custa $\alpha + n\beta$ (latência + tempo por byte). Para cada decomposição, com elementos de b bytes:

Decomposição	Cálculo por GPU	Dados a comunicar (aprox.)	Memória por GPU
(a) linhas de A	$2MNK/p$	replicar B: $KN \cdot b$	$(MK/p + KN + MN/p) \cdot b$
(b) colunas de B	$2MNK/p$	replicar A: $MK \cdot b$	$(MK + KN/p + MN/p) \cdot b$
(c) ao longo de K	$2MNK/p$	all-reduce de C: ~ $2MN \cdot b$	$(MK/p + KN/p + MN) \cdot b$

Decomposição	Cálculo por GPU	Dados a comunicar (aprox.)	Memória por GPU
2D (SUMMA)	$2MNK/p$	$\sim(MK + KN) \cdot b/\sqrt{p}$	$(MK + KN + MN) \cdot b/p$

A regra que sai da tabela: **comunique a matriz menor** e **evite replicar a maior**.

- Camada de LLM no decode: A = ativações (pequena: batch × d), B = pesos (enorme). Replicar A e dividir B: decomposição (b) ou (c). É exatamente o TP do Megatron.
- Treino com batch gigante e pesos pequenos: replicar B (pesos) e dividir A (dados): decomposição (a), que é data parallelism.

4. SUMMA (2D) em detalhe. As GPUs formam uma grade $\sqrt{p} \times \sqrt{p}$. A GPU (i, j) guarda o bloco A_{ij}, B_{ij} e calcula C_{ij}:

```
para k = 0 .. √p − 1:
    a GPU (i, k) envia A_ik para toda a linha i
    a GPU (k, j) envia B_kj para toda a coluna j
    cada GPU (i, j): C_ij += A_ik · B_kj
```

A memória fica dividida por p em todas as matrizes, e cada GPU recebe $O((MK + KN)/\sqrt{p})$ dados. É a escolha para matrizes grandes nas três dimensões. Em hardware com topologia de malha ou toro (como pods de TPU), decomposições 2D e 3D se encaixam naturalmente na topologia.

5. Sobrepor comunicação e cálculo. Divida cada transferência em blocos e faça pipeline:

```
sem sobreposição: [ comunica tudo ][   calcula tudo   ]
com sobreposição: [com 1][com 2][com 3][com 4]
                         [calc 1][calc 2][calc 3][calc 4]
```

Em GPUs: streams CUDA separadas para NCCL e para os kernels, ou kernels que fazem cálculo e comunicação juntos. Em TPUs/XLA, a mesma ideia aparece como “collective matmul”.

6. Considerações práticas. Topologia (NVLink dentro do nó, InfiniBand entre nós: use decomposições hierárquicas, com a comunicação pesada dentro do nó), formas não divisíveis por p (padding ou fatias desiguais), precisão (somas parciais em FP32 antes de reduzir) e determinismo da redução.

Perguntas de acompanhamento prováveis

- **E se uma das matrizes já estiver distribuída de um jeito específico?** Escolha a decomposição que aproveita o layout existente; redistribuir também custa comunicação. Em sequências de camadas, alterne colunas e linhas (como no Megatron) para que a saída de uma já esteja no layout que a próxima precisa.
- **Qual o speedup máximo?** Pela lei de Amdahl (Q018), a comunicação não sobreposta limita. Com o modelo α-β, compare o tempo de cálculo $2MNK/(p \cdot F)$ com o de comunicação; quando a comunicação domina, adicionar GPUs piora.
- **Por que reduce-scatter em vez de all-reduce?** Se a próxima operação aceita C dividido, o reduce-scatter custa metade do all-reduce (que é um reduce-scatter seguido de um all-gather).
- **Algoritmos com menos comunicação?** Decomposições 2,5D e 3D usam memória extra (replicação parcial) para reduzir ainda mais a comunicação.

Armadilhas comuns

- Apresentar uma única decomposição sem discutir as formas.
- Ignorar o custo de comunicação ou a topologia.
- Esquecer a sobreposição de comunicação e cálculo.
- Não conectar com o TP de LLMs, que é a aplicação mais comum.

Q067 Por que você separaria (desagregaria) prefill e decode em máquinas diferentes, como faz o Mooncake? O que isso traz de ganho e qual é o custo?

> **[Fornecedor]** Moonshot AI (Kimi), segundo banco público de perguntas por empresa · Presencial · 2026 · Difícil · fonte

Resposta em 30 segundos

"Porque prefill e decode têm gargalos e SLOs opostos: prefill é compute-bound e define o TTFT; decode é memory-bound e define o TPOT. Na mesma GPU, um prefill longo atrasa os decodes de todo mundo e vice-versa. Separando, cada pool tem a configuração, o paralelismo e até o hardware certos para a sua fase, e cada um é dimensionado para o seu SLO, o que aumenta o goodput. O custo é transferir o KV cache do prefill para o decode, centenas de MB a GB por requisição longa, o que exige rede RDMA rápida; mais complexidade de scheduling e de falhas; e o risco de dese-

quilíbrio entre os dois pools, porque a proporção ideal muda com a carga. O Mooncake vai além e trata o KV cache como um recurso do cluster todo, usando DRAM e SSD ociosos como cache distribuído para reaproveitar prefixos.”

Resposta completa

Os ganhos.

1. **Fim da interferência.** Sem prefills no meio, os passos de decode têm tempo previsível: ITL estável. Sem decodes, o prefill usa a GPU inteira: TTFT melhor.
2. **Configuração por fase.** Prefill pode usar TP alto (corta o TTFT de prompts longos); decode pode usar TP menor e batch enorme (throughput). Na mesma máquina, você precisa de um compromisso.
3. **Escala independente.** Se os prompts ficam mais longos (RAG, agentes), aumenta só o pool de prefill. Se as respostas ficam mais longas (raciocínio), aumenta só o de decode.
4. **Hardware especializado.** Prefill pede FLOPS; decode pede banda e capacidade de memória. Em princípio, cada pool pode usar a GPU com melhor custo para a sua fase.
5. **Goodput.** O DistServe mostrou que otimizar cada fase para seu próprio SLO aumenta o número de requisições por GPU que cumprem os dois SLOs, em comparação com sistemas que colocam as duas fases juntas.

Os custos.

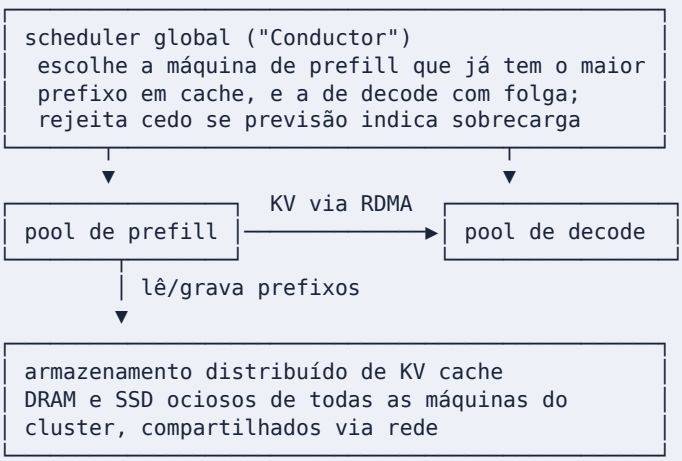
> **Fazendo a conta: a transferência do KV.** Llama 3 70B, prompt de 8.000 tokens: 2,6 GB de KV cache.
>
> - Com TP = 8 nos dois pools, cada GPU de prefill envia sua fatia (~330 MB) para a GPU correspondente no decode.
> - Com InfiniBand NDR (~50 GB/s por GPU): ~**7 ms**. Contra ~300 ms de prefill, é pouco, e dá para enviar camada por camada enquanto o prefill ainda calcula as próximas, escondendo quase tudo.
> - Sem RDMA (por exemplo, TCP em rede de 25 Gb/s, ~3 GB/s): ~**110 ms por GPU** de fatia. Aí a transferência pesa no TTFT.
> - Para prompts curtos (200 tokens, ~65 MB), a transferência é desprezível, mas o ganho da desagregação também é menor.

Outros custos:

- **Complexidade**: dois pools, um protocolo de transferência, um scheduler global que escolhe o par prefill-decode, tratamento de falhas no meio da transferência (se a GPU de decode morre, o KV se perde).
- **Desequilíbrio**: a razão entre capacidade de prefill e de decode depende da distribuição de tamanhos de entrada e saída. Se a carga muda, um pool fica ocioso enquanto o outro enfileira. Mitigações: pools elásticos, máquinas que podem trocar de papel, ou modo híbrido com chunked prefill.
- **Memória duplicada temporariamente**: o KV existe nos dois lados até a transferência terminar.

Quando não vale. Para cargas com prompts curtos e SLOs folgados, o chunked prefill numa arquitetura única costuma ser mais simples e quase tão bom. A desagregação compensa em escala, com prompts longos, SLOs rígidos de ITL e rede RDMA disponível.

O que o Mooncake adiciona. A arquitetura é centrada no KV cache:



Para um produto de chat com contextos longos (como o Kimi, conhecido por isso), reaproveitar prefixos entre turnos e entre usuários vale muito, e a memória da CPU do cluster inteiro é muito maior do que a HBM.

Perguntas de acompanhamento prováveis

- **Como dimensionar a proporção entre prefill e decode?** Pela carga: tokens de entrada por segundo ÷ throughput de prefill por GPU, e sequências simultâneas ÷ capacidade de decode por GPU. Ajustar dinamicamente com base nas filas de cada pool.

- **E se a GPU de decode falhar no meio da geração?** O KV dela se perde. Opções: refazer o prefill (talvez com prefixo no cache distribuído) e continuar de onde parou com os tokens já gerados, ou devolver erro e deixar o cliente tentar de novo.
- **Chunked prefill não resolve o mesmo problema?** Resolve parte: limita a interferência, mas as duas fases ainda dividem a GPU e a configuração. A desagregação elimina a interferência e permite configurações diferentes, ao custo da transferência.
- **Por que rejeitar requisições cedo?** Se a previsão mostra que a requisição não vai cumprir o SLO, rejeitá-la antes do prefill evita gastar GPU com trabalho que será inútil (o cliente vai desistir ou estourar o timeout).

Armadilhas comuns

- Não quantificar a transferência do KV cache.
- Ignorar o problema de desequilíbrio entre os pools.
- Dizer que desagregação é sempre melhor.
- Não diferenciar a ideia geral (DistServe, Splitwise) do que o Mooncake adiciona (KV cache distribuído no cluster).

Resumo do capítulo

- Motivos para distribuir: capacidade, latência, throughput. Cada um leva a um eixo diferente.
- **DP**: réplicas, sem comunicação, throughput linear.
- **TP** (Megatron): colunas e depois linhas; dois all-reduces por camada; reduz a latência; dentro do nó; limitado pelas heads de KV.
- **PP**: estágios de camadas; comunicação pequena ponto a ponto; entre nós; não reduz a latência; precisa de micro-batches.
- **EP**: especialistas de MoE distribuídos; dois all-to-all por camada de MoE; desafio de balanceamento.
- **CP**: sequência dividida, ring attention para contextos muito longos.
- GEMM distribuída: por linhas, por colunas, ao longo de K, ou 2D (SUMMA). Comunique a menor matriz; sobreponha comunicação e cálculo.
- **Desagregação** de prefill e decode: sem interferência, configuração e escala por fase; custo: transferência do KV, complexidade e desequilíbrio. Mooncake: KV cache distribuído no cluster.

Checklist para a entrevista

- [ ] Explico os cinco eixos de paralelismo e o motivo de cada um.
- [ ] Desenho o MLP com TP (colunas, linhas, all-reduce) e escrevo o código.
- [ ] Calculo o volume e o tempo da comunicação do TP no decode e no prefill.
- [ ] Explico bolhas no PP e como micro-batches resolvem.
- [ ] Explico o fluxo de uma camada MoE com EP (dispatch e combine).
- [ ] Monto a configuração de paralelismo para um modelo e hardware dados.
- [ ] Desenho as três decomposições de GEMM e preencho a tabela de custos.
- [ ] Explico SUMMA em poucas linhas.
- [ ] Listo ganhos e custos da desagregação, com a conta da transferência do KV.
- [ ] Explico o que o Mooncake adiciona à desagregação básica.

CAPÍTULO 10

Redes e Comunicação entre GPUs

Assim que um modelo passa a ocupar mais de uma GPU, a comunicação vira parte do caminho crítico. No Capítulo 9, o tensor parallelism fazia 160 all-reduces por passo de decode no 70B; a desagregação transferia gigabytes de KV cache entre máquinas. Se a comunicação for lenta, nenhuma otimização de kernel salva o sistema.

Este capítulo cobre o hardware de interconexão (PCIe, NVLink, NVSwitch, InfiniBand), as operações coletivas e seus custos, a biblioteca NCCL e a camada de orquestração (Kubernetes com GPUs). Vagas de infraestrutura de IA, como as da Fireworks AI e da CoreWeave, cobram muito este tema.

10.1 As interconexões

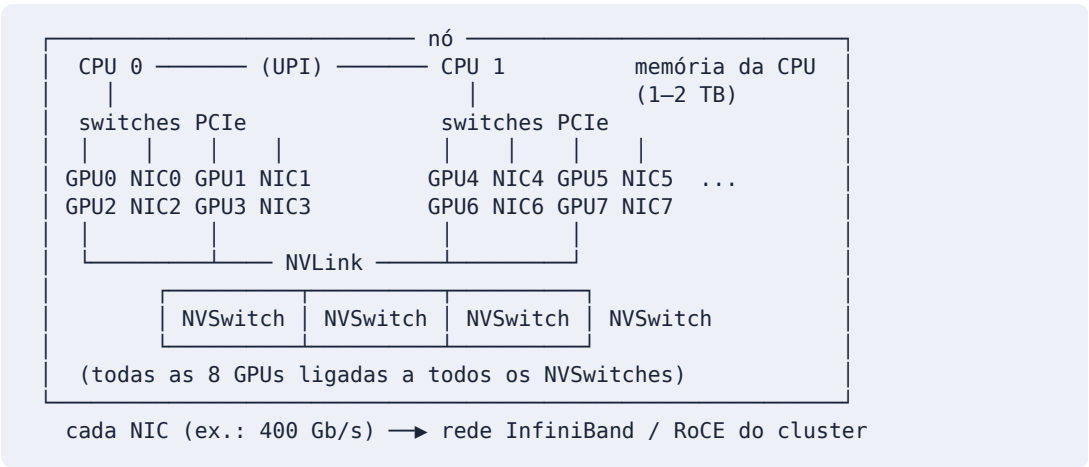
Ligação	Liga	Banda (aprox.)	Latência (ordem)
HBM3 (H100)	GPU ↔ sua memória	~3,35 TB/s	centenas de ns
NVLink 4 (H100)	GPU ↔ GPU no nó	900 GB/s total por GPU (~450 por sentido)	~µs
NVLink-C2C (Grace Hopper)	CPU Grace ↔ GPU	900 GB/s total	~µs
PCIe Gen5 x16	CPU ↔ GPU, NIC ↔ GPU	~64 GB/s por sentido	~µs
PCIe Gen4 x16	idem, geração anterior	~32 GB/s por sentido	~µs
InfiniBand NDR	nó ↔ nó (por porta)	400 Gb/s = ~50 GB/s	poucos µs
Ethernet com RoCE	nó ↔ nó	100–400 Gb/s por porta	poucos µs a dezenas

Repare nos degraus: da memória da GPU para a NVLink, uma ordem de grandeza; da NVLink para a rede entre nós, outra ordem de grandeza. **O design de sistemas distribuídos de GPU é organizar o trabalho para que a comunicação pesada fique nos degraus mais rápidos.**

Termos que você precisa conhecer

- **NVLink**: ligação ponto a ponto de alta banda entre GPUs NVIDIA.
- **NVSwitch**: um switch de NVLink. Num servidor HGX H100, 4 NVSwitches conectam as 8 GPUs **todas com todas**, com banda total de NVLink entre qualquer par. Gerações mais novas estendem o domínio NVLink para um rack inteiro (como os sistemas NVL72, com 72 GPUs Blackwell num único domínio NVLink).
- **NVLink SHARP (NVLS)**: os NVSwitches da Hopper podem fazer parte da **redução** dentro do switch, o que reduz o tráfego de um all-reduce.
- **InfiniBand (IB)**: rede de baixa latência e alta banda para clusters, com RDMA nativo. **SHARP** também existe nos switches InfiniBand, fazendo reduções dentro da rede.
- **RDMA** (remote direct memory access): uma máquina lê ou escreve a memória de outra sem passar pela CPU nem pelo kernel do sistema operacional.
- **RoCE** (RDMA over Converged Ethernet): RDMA sobre Ethernet. Mais barato e comum, mas exige configuração cuidadosa para evitar perda de pacotes (controle de fluxo, gerenciamento de congestionamento).
- **GPUDirect RDMA**: a placa de rede lê e escreve **direto na memória da GPU**, sem copiar para a memória da CPU no meio.
- **GPUDirect Storage**: o mesmo, entre armazenamento NVMe e a GPU.

10.2 Anatomia de um nó HGX H100

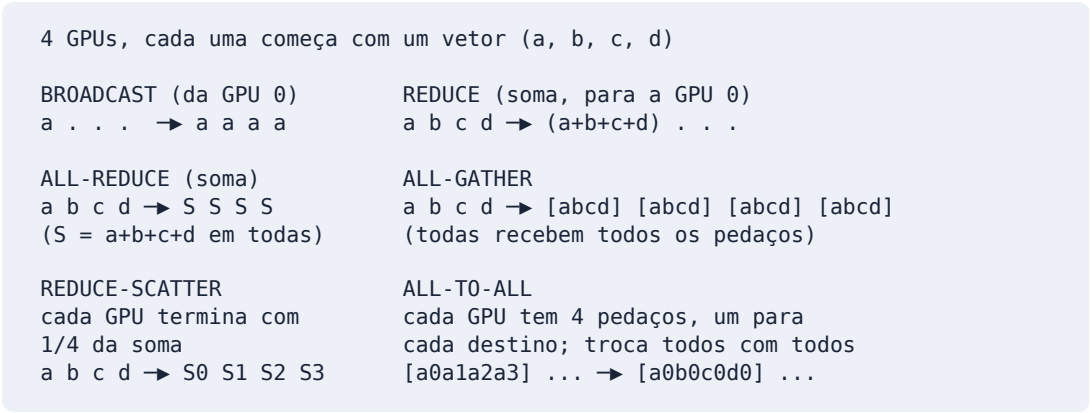


Detalhes que mostram experiência:

- **Uma NIC por GPU**, cada uma ligada ao mesmo switch PCIe da sua GPU. Assim, com GPUDirect RDMA, os dados vão da GPU para a NIC sem atravessar a CPU. Usar a NIC "errada" (ligada a outra CPU) derruba a banda.

- **Afinidade NUMA**: o processo que controla a GPU 0 deveria rodar nos núcleos da CPU 0 e usar a memória dela.
- **Rails**: em clusters grandes, a NIC *i* de todos os nós vai para o mesmo grupo de switches (o "rail" *i*). Coletivas entre GPUs de mesmo índice em nós diferentes ficam no mesmo rail, com menos saltos.

10.3 As operações coletivas



Onde cada uma aparece em inferência:

Coletiva	Uso
All-reduce	tensor parallelism (2 por camada)
All-gather	juntar fatias (ex.: logits divididos por vocabulário no TP)
Reduce-scatter	variantes de TP com sequence parallelism
All-to-all	expert parallelism (dispatch e combine do MoE)
Send/recv (ponto a ponto)	pipeline parallelism, transferência de KV na desagregação
Broadcast	distribuir pesos ou estado inicial

Uma identidade útil: **all-reduce = reduce-scatter + all-gather**.

10.4 O modelo α-β

Para estimar o custo de uma comunicação, use:

$$T(n) = \alpha + \frac{n}{B}$$

- α: latência fixa por mensagem (software, sincronização, rede).
- B: banda do enlace. n/B é o tempo para "empurrar" os bytes.

Mensagens **pequenas** são dominadas por α (latency-bound). Mensagens **grandes**, por n/B (bandwidth-bound). O tamanho de mensagem em que os dois termos se igualam é $n^* = \alpha B$.

> **Fazendo a conta (valores ilustrativos):** NVLink com $\alpha \approx 5\ \mu s$ por etapa e $B \approx 450$ GB/s.
>
> - $n^* = 5 \times 10^{-6} \times 450 \times 10^9 \approx$ **2,25 MB**.
> - Um all-reduce do decode com batch 64 no 70B (1 MB) está abaixo de n^*: é dominado pela latência.
> - Um all-reduce do prefill de 8.000 tokens (131 MB) está muito acima: é dominado pela banda.
> - Conclusão: para o decode, reduzir o **número** de coletivas e a latência de cada uma importa mais do que a banda. Para o prefill, o contrário.

10.5 Ring vs tree all-reduce

Ring (anel). As p GPUs formam um anel. O vetor de n bytes é dividido em p pedaços.

```
Fase 1: reduce-scatter (p−1 etapas)
  em cada etapa, cada GPU envia um pedaço para a próxima
  e soma o que recebe no pedaço correspondente
  ao fim, cada GPU tem UM pedaço completamente somado

Fase 2: all-gather (p−1 etapas)
  os pedaços somados circulam pelo anel até todas
  terem todos

GPU0 → GPU1 → GPU2 → GPU3 ─┐
  ▲                        │
  └────────────────────────┘
```

Custo:

$$T_{ring} = 2(p-1)\,\alpha + 2\,\frac{p-1}{p}\,\frac{n}{B}$$

O termo de banda tende a $2n/B$ quando p cresce: **ótimo em banda** (cada GPU envia e recebe o mínimo possível). Mas o termo de latência cresce **linearmente** com p.

Tree (árvore). A redução sobe por uma árvore até a raiz e o resultado desce por ela. Com pipelining (dividir a mensagem em pedaços que fluem pela árvore), o custo fica perto de:

$$T_{tree} \approx 2 \log_2 (p)\, \alpha + c \cdot \frac{n}{B}$$

com c pequeno (perto de 2 em implementações com duas árvores complementares, como a **double binary tree** do NCCL, que usa as duas direções dos enlaces). A latência cresce só com $\log p$.

	Ring	Tree (com pipelining)
Etapas de latência	$2(p-1)$	$\sim 2 \log_2 p$
Banda	ótima ($\to 2n/B$)	perto da ótima
Melhor para	mensagens grandes, p pequeno ou médio	mensagens pequenas, p grande
Exemplo	prefill com TP=8 no nó	decode com TP; all-reduce com milhares de GPUs

> **Fazendo a conta:** com α = 5 µs, 450 GB/s, p = 8.
>
> - 1 MB: ring = 14 × 5 µs + 1,75 MB / 450 GB/s ≈ 70 + 3,9 = **74 µs**; tree ≈ 6 × 5 µs + ~2 MB / 450 GB/s ≈ 30 + 4,4 = **34 µs**. A árvore ganha.
> - 1 GB: ring ≈ 0,07 ms + 3,9 ms ≈ **3,97 ms**; tree ≈ 0,03 ms + 4,4 ms ≈ **4,43 ms**. O anel ganha.
> - Com p = 1.024 e 1 MB: ring = 2.046 × 5 µs ≈ **10 ms** só de latência; tree ≈ 20 × 5 µs = **0,1 ms**.

Na prática, o NCCL escolhe o algoritmo (ring, tree, NVLS e outros) e o protocolo automaticamente, com base no tamanho da mensagem e na topologia. Reduções dentro do switch (NVLS no NVSwitch, SHARP no InfiniBand) mudam a conta: cada GPU envia seus dados uma vez e recebe o resultado uma vez.

10.6 NCCL na prática

O **NCCL** (NVIDIA Collective Communications Library) implementa as coletivas para GPUs NVIDIA (na AMD, o equivalente é o RCCL). O PyTorch o usa por baixo do `torch.distributed` com `backend="nccl"`.

Conceitos:

- **Comunicador**: o grupo de GPUs que participa das coletivas (um por grupo de TP, outro por grupo de PP, etc.).
- **Algoritmos**: ring, tree, NVLS, CollNet (com SHARP), entre outros.
- **Protocolos**: LL (low latency, para mensagens pequenas), LL128, Simple (para grandes).
- **Canais**: o NCCL usa vários "canais" em paralelo (cada um com seus blocos de GPU) para ocupar vários enlaces.

Para medir e depurar:

```
# benchmark de all-reduce de 8 B a 8 GB, 8 GPUs (nccl-tests)
./build/all_reduce_perf -b 8 -e 8G -f 2 -g 8

# logs detalhados: topologia detectada, algoritmos, NICs
NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,NET python treino.py

# forçar interface e NICs (útil quando a detecção erra)
NCCL_SOCKET_IFNAME=eth0 NCCL_IB_HCA=mlx5_0,mlx5_1 ...
```

O `nccl-tests` reporta duas bandas:

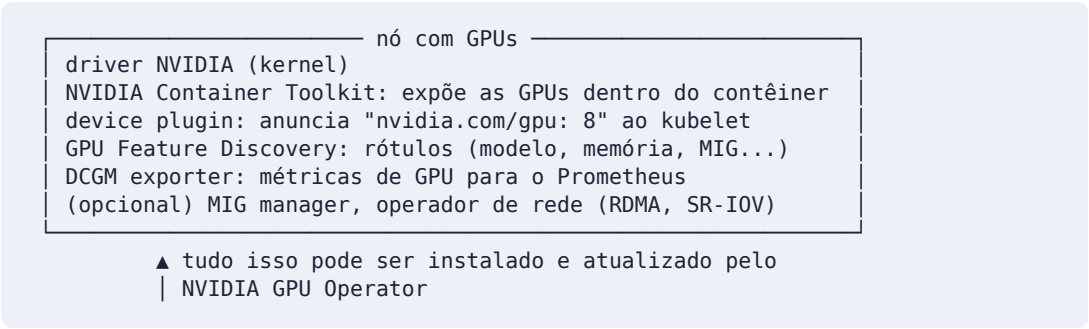
- **algbw** (banda do algoritmo): tamanho da mensagem ÷ tempo.
- **busbw** (banda do barramento): algbw corrigida pelo fator do algoritmo; para all-reduce, $\text{busbw} = \text{algbw} \times 2(p-1)/p$. É a métrica comparável com a banda do hardware: num nó H100 saudável, o busbw de all-reduce para mensagens grandes deve ficar perto da banda de NVLink por sentido.

Um busbw muito abaixo do esperado indica problema: NVLink degradado, NIC errada, afinidade NUMA ruim, cabo com defeito, configuração de RoCE com perdas.

> **Cuidado:** coletivas são **bloqueantes em grupo**. Se uma GPU do comunicador não chega à coletiva (travou, ficou lenta, deu erro), todas as outras ficam esperando para sempre. Esse "NCCL hang" é um dos incidentes mais comuns em sistemas distribuídos de GPU. Configure timeouts (`timeout` no `init_process_group` e o watchdog do PyTorch) e monitore stragglers. Ver Capítulo 12.

10.7 Kubernetes e GPUs

A maior parte das frotas de inferência roda em Kubernetes. As peças específicas de GPU:



- **Device plugin**: o Kubernetes não sabe nada de GPUs por padrão. O device plugin registra o recurso `nvidia.com/gpu`, e os pods pedem GPUs em `resources.limits`. O scheduler só coloca o pod num nó com GPUs livres suficientes.
- **NVIDIA GPU Operator**: automatiza a instalação de driver, container toolkit, device plugin, GPU Feature Discovery, DCGM exporter e MIG manager em todos os nós.
- **Compartilhamento**: **MIG** (multi-instance GPU) divide uma A100/H100 em até 7 instâncias isoladas em hardware (memória e SMs próprios); **time-slicing** deixa vários pods usarem a mesma GPU em turnos, sem isolamento de memória; **MPS** permite execução concorrente de vários processos.
- **Rede de alto desempenho**: o pod precisa de acesso às NICs de RDMA (via network operator, SR-IOV ou plugins de dispositivos RDMA, e muitas vezes uma segunda interface de rede com Multus).
- **Topologia**: o pod de 8 GPUs precisa das 8 GPUs do mesmo nó (NVLink); jobs multinó precisam de nós próximos na rede. O scheduler padrão não sabe disso; usam-se rótulos, afinidades e schedulers que entendem topologia.
- **Gang scheduling**: um modelo com PP em 2 nós precisa dos 2 ao mesmo tempo; subir só um desperdiça GPUs esperando. Ferramentas como Kueue, Volcano ou LeaderWorkerSet ajudam.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: llm-70b
spec:
  containers:
    - name: servidor
      image: registro.exemplo/llm-servidor:1.4.2
      args: ["--modelo", "/modelos/70b", "--tp", "8"]
      resources:
        limits:
          nvidia.com/gpu: 8          # o nó inteiro
      volumeMounts:
        - { name: shm, mountPath: /dev/shm }
  volumes:
    - name: shm                       # NCCL usa memória
      emptyDir: { medium: Memory }    # compartilhada
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3
```

O volume em `/dev/shm` é um detalhe clássico: o padrão de 64 MB do contêiner é pequeno demais para a comunicação entre processos que o NCCL e o PyTorch usam, e o erro aparece de formas confusas.

Perguntas de entrevista comentadas

Q073 Fale da sua experiência; depois, perguntas sobre GPUs, CI/CD, comunicação entre GPUs, Kubernetes e plugins da NVIDIA para Kubernetes.

> **[Relatado]** Fireworks AI · Software Engineer, AI Infrastructure · relatado em nov. 2025 · Difícil · fonte

Resposta em 30 segundos

“Essa rodada passeia por vários temas, então eu preparo uma resposta curta e concreta para cada um. GPUs: hierarquia de memória, por que inferência é memory-bound, MIG e como monitorar com DCGM. Comunicação: NVLink e NVSwitch dentro do nó, InfiniBand ou RoCE entre nós, GPUDirect RDMA, coletivas do NCCL, ring vs tree, e como validar com nccl-tests. Kubernetes: device plugin anunciando nvidia.com/gpu, GPU Operator instalando driver, toolkit, DCGM exporter e feature discovery, MIG e time-slicing, topologia e gang scheduling. CI/CD: testes em GPU

real, matriz de compatibilidade de driver e CUDA, imagens reprodutíveis, canary e rollback para drivers e motores de inferência. E para a experiência, uma história de incidente ou migração com números."

Resposta completa

Como é uma rodada ampla, a estratégia é ter **um parágrafo forte por tema**, mais uma história.

1. A história de abertura. Siga a estrutura da Q001, mas com foco em infraestrutura. Exemplo de formato: "Eu cuidava da plataforma de inferência em Kubernetes com 400 GPUs. Migramos de nós com 4 A100 por PCIe para HGX H100 com NVLink. O desafio foi o tempo de subida dos pods de 70B: 9 minutos, dominado pelo download dos pesos. Coloquei um cache de pesos em NVMe local, pré-aquecido por um DaemonSet, e passei a carregar com várias threads e direto para a GPU. A subida caiu para 90 segundos, o que permitiu autoscaling de verdade."

2. GPUs. Hierarquia de memória e números da H100; por que o decode é memory-bound; MIG vs time-slicing vs MPS; métricas do DCGM (uso de SM, tensor, DRAM, temperatura, energia, erros ECC, eventos Xid).

3. Comunicação entre GPUs. Seções 10.1 a 10.6. Saiba explicar:

- Por que TP fica dentro do nó.
- O que é GPUDirect RDMA e por que a NIC precisa estar no mesmo switch PCIe da GPU.
- Ring vs tree (Q074).
- Como diagnosticar banda baixa: `nccl-tests`, `NCCL_DEBUG=INFO`, `nvidia-smi topo -m` (mostra a topologia: quais GPUs e NICs estão ligadas por NVLink, PCIe ou pela CPU).

```
nvidia-smi topo -m  (trecho ilustrativo)

        GPU0  GPU1  ...  NIC0  NIC1
GPU0     X    NV18       PIX   SYS
GPU1    NV18   X         SYS   PIX

NV18 = 18 enlaces NVLink   PIX = mesmo switch PCIe
SYS  = atravessa a interconexão entre CPUs (lento)
```

4. Kubernetes e plugins NVIDIA. Seção 10.7. Pontos que costumam ser perguntados:

- **Como o pod vê a GPU?** O device plugin anuncia o recurso; o kubelet aloca GPUs específicas para o contêiner; o NVIDIA Container Toolkit monta os dispositivos e as bibliotecas do driver dentro do contêiner.
- **O que o GPU Operator faz?** Instala e mantém, como DaemonSets, o driver, o toolkit, o device plugin, o GPU Feature Discovery, o DCGM exporter e o MIG manager. Atualizar o driver vira uma operação declarativa, nó por nó.
- **E se um nó tiver uma GPU com defeito?** Health checks (DCGM, erros Xid, ECC) marcam a GPU como não saudável; o device plugin deixa de anunciá-la; o nó pode ser isolado (cordon) e drenado automaticamente.
- **Pod em Pending?** Ver Q089.

5. CI/CD para infraestrutura de GPU.

```
Pipeline de mudança (driver, CUDA, motor de inferência,
imagem do servidor):

1. build da imagem (versões fixadas: driver mínimo,
   CUDA, NCCL, PyTorch, motor)
2. testes unitários em CPU
3. testes em GPU real (runner com GPU no CI):
   - kernels e modelo pequeno: corretude numérica
   - nccl-tests em 1 nó: busbw mínimo
   - benchmark de inferência: TTFT/TPOT/throughput
     contra a linha de base (regressão > X% bloqueia)
4. staging com tráfego espelhado
5. canary em produção (1% → 10% → 100%),
   com métricas de erro, latência e qualidade
6. rollback automático se as métricas piorarem
```

Pontos específicos de GPU: a **matriz de compatibilidade** (versão do driver no nó vs CUDA na imagem), testar em **todos os tipos de GPU** da frota, e atualizar drivers com **rolling update** por grupos de nós, porque exige reiniciar os pods.

Perguntas de acompanhamento prováveis

- **Como você reduziria o tempo de subida de um pod de modelo grande?** Cache local de pesos, formato que carrega rápido (safetensors, com mmap), carga paralela direto para a GPU, imagens menores (pesos fora da imagem), pré-aquecer nós, manter réplicas quentes. Ver também Q083.
- **MIG ou time-slicing para modelos pequenos?** MIG quando precisa de isolamento (memória e desempenho previsíveis, multi-tenant); time-slicing quando a carga é leve e o isolamento não importa.
- **Como saber se a NVLink está saudável?** `nvidia-smi nvlink --status`, contadores de erro de NVLink no DCGM, e busbw do nccl-tests comparado com o esperado.

- **Como fazer deploy de um modelo com PP em dois nós no Kubernetes?** Um grupo de pods agendado junto (gang scheduling, LeaderWorkerSet), nós próximos na rede, rede RDMA nos pods, e um mecanismo para recriar o grupo inteiro se um pod falhar.

Armadilhas comuns

- Responder de forma genérica sobre Kubernetes, sem os detalhes de GPU.
- Não saber o que o device plugin e o GPU Operator fazem.
- Confundir NVLink com PCIe ou InfiniBand.
- Falar de CI/CD sem testes em GPU real e sem benchmark de regressão.

Q074 All-reduce em anel vs. em árvore: custos e quando cada um é usado.

> **[Prática]** Prática · Presencial · Médio · Base: operações coletivas nas vagas da DeepMind e da Groq

Resposta em 30 segundos

"No anel, o vetor é dividido em p pedaços e circula em duas fases, reduce-scatter e all-gather, cada uma com p−1 etapas. Cada GPU envia cerca de 2n(p−1)/p bytes, o que é ótimo em banda, mas a latência cresce linearmente com p. Na árvore, a redução sobe até a raiz e o resultado desce; com pipelining e duas árvores complementares, a banda fica perto da ótima e a latência cresce só com log p. Por isso o anel é melhor para mensagens grandes com poucas GPUs, e a árvore para mensagens pequenas ou muitas GPUs. O NCCL escolhe automaticamente com base no tamanho e na topologia, e reduções dentro do switch, como NVLS e SHARP, podem ganhar das duas."

Resposta completa

Derive as fórmulas da seção 10.5 no quadro:

Anel, fase 1 (reduce-scatter): $p - 1$ etapas, cada GPU envia n/p bytes por etapa. **Fase 2 (all-gather):** igual. Total por GPU:

$$2(p-1)\cdot\frac{n}{p}\ \text{bytes enviados}, \qquad T_{ring} = 2(p-1)\alpha + \frac{2(p-1)}{p}\frac{n}{B}$$

Por que é ótimo em banda: cada GPU precisa, no mínimo, enviar sua contribuição para os outros e receber o resultado, e o limite inferior teórico para all-reduce é exatamente $2(p-1)n/p$ bytes por GPU.

Árvore: sem pipelining, seria $2 \log_2 p$ etapas, cada uma com a mensagem inteira: $2 \log_2 (p)(\alpha + n/B)$, ruim em banda para mensagens grandes. Com pipelining (a mensagem em k pedaços fluindo pela árvore), o termo de banda cai para perto de $2n/B$ (ou um pouco mais), mantendo a latência em $O(\log p)$. A **double binary tree** usa duas árvores, em que cada nó é folha numa e interno na outra, para equilibrar o uso dos enlaces.

Depois, faça a conta com números (seção 10.5): a árvore ganha para 1 MB com 8 GPUs e de longe com 1.024 GPUs; o anel ganha para 1 GB com 8 GPUs.

E mostre que conhece as variantes:

Algoritmo	Ideia	Quando
Ring	anel, ótimo em banda	grandes, p pequeno
Tree / double binary tree	log p de latência	pequenas, p grande
Recursive halving-doubling	pares que trocam metades em log p etapas	potências de 2, mensagens médias
Hierárquico (2D)	reduce no nó (NVLink), all-reduce entre nós (IB), broadcast no nó	multi-nó: usa o degrau rápido primeiro
NVLS / SHARP	redução dentro do switch	quando o hardware tem

Para inferência, ligue com o Capítulo 9: os all-reduces do TP no decode são pequenos (~1 MB), então latência importa mais; por isso motores usam **all-reduces customizados de uma etapa** (cada GPU lê os dados das outras pela NVLink e soma localmente), que ganham dos algoritmos genéricos nessa faixa.

Perguntas de acompanhamento prováveis

- **Por que o anel tem $2(p-1)$ etapas e não $p-1$?** Uma fase reduz (cada GPU fica com um pedaço somado), a outra distribui os pedaços somados.
- **Como fica num cluster com vários nós?** Hierárquico: reduce-scatter dentro do nó pela NVLink, all-reduce entre nós (cada GPU com seu pedaço, pelo seu rail de InfiniBand), all-gather dentro do nó. A parte lenta (rede) só move $n/8$ por GPU.
- **O que é busbw?** A banda corrigida pelo fator do algoritmo, comparável com a banda do hardware (10.6).
- **E o all-to-all?** Cada GPU envia um pedaço diferente para cada outra; o volume total por GPU é $\sim n(p-1)/p$, mas não há redução nem árvore que ajude. É limitado pela banda da bisseção da rede, e por isso o MoE entre nós é caro.

Armadilhas comuns

- Dizer que a árvore é sempre melhor ou que o anel é sempre melhor.
- Não saber derivar o volume de dados do anel.
- Esquecer o termo de latência.
- Não citar o caso hierárquico multi-nó.

Q075 NVLink, PCIe e InfiniBand: quando cada um vira o gargalo na inferência?

> **[Prática]** Prática · Presencial · Médio · Base: conhecimento de interconexões presente em várias vagas

Resposta em 30 segundos

"NVLink vira gargalo no tensor parallelism: no prefill pela banda, com all-reduces de centenas de MB por camada, e no decode pela latência, com centenas de all-reduces pequenos por passo; também no all-to-all de MoE dentro do nó. PCIe vira gargalo em tudo que passa pela CPU: carregar pesos da memória ou do disco, swap e offload de KV cache, e TP em GPUs sem NVLink, que fazem os all-reduces pelo PCIe. InfiniBand vira gargalo no que atravessa nós: pipeline parallelism, expert parallelism entre nós, transferência de KV na desagregação, distribuição de pesos para a frota, e TP multi-nó, que deve ser evitado. Para saber qual é o caso, faço a conta de bytes por segundo de cada fluxo contra a banda do enlace e confirmo com o profiler."

Resposta completa

Monte uma tabela de fluxos e faça contas:

Fluxo	Enlace	Vira gargalo quando...
All-reduce de TP (prefill)	NVLink	prompts longos, TP alto; banda
All-reduce de TP (decode)	NVLink	sempre pesa pela latência (muitas coletivas pequenas)
All-to-all de MoE no nó	NVLink	batch grande, muitos especialistas
TP em GPUs sem NVLink	PCIe	quase sempre; ~10× menos banda que NVLink
Carga de pesos (CPU → GPU)	PCIe	subida de réplicas, troca de modelos

Fluxo	Enlace	Vira gargalo quando...
Swap/offload de KV	PCIe	muitas conversas pausadas, preempção frequente
Cópia de tokens e metadados CPU ↔ GPU	PCIe	raramente pela banda; pela latência e sincronização
PP entre nós	InfiniBand	raramente (ativações pequenas); latência por salto
EP entre nós (MoE grande)	InfiniBand	frequentemente: all-to-all pesado
Transferência de KV (desagregação)	InfiniBand	prompts longos com taxa alta
Distribuição de pesos para a frota	InfiniBand / Ethernet	lançamentos de modelo, autoscaling

Fazendo a conta: três casos.

1. TP=4 com GPUs sem NVLink (PCIe Gen4, ~32 GB/s). Prefill de 4.000 tokens num modelo com d = 8.192 e 80 camadas: all-reduce de 4.000 × 8.192 × 2 ≈ 65 MB; cada GPU envia ~2 × (3/4) × 65 ≈ 98 MB; 160 coletivas ≈ 15,7 GB → a 32 GB/s, ~**0,5 s** só de comunicação. Na NVLink da H100 seriam ~35 ms. Esse é o motivo de TP em GPUs PCIe ser tão pior.

2. Carga de pesos de 140 GB. Do disco local NVMe (~7 GB/s por drive, alguns em paralelo): dezenas de segundos. Da memória da CPU para 8 GPUs por PCIe Gen5 (~64 GB/s cada, em paralelo, 17,5 GB por GPU): ~**0,3 s** no limite teórico, mais na prática. Da rede, ver Q083.

3. MoE com EP = 16 em 2 nós. Batch de 4.096 tokens, d = 7.168, 8 especialistas por token, BF16: cada token vai para 8 destinos → 4.096 × 8 × 7.168 × 2 bytes ≈ 470 MB por dispatch, com uma boa parte atravessando a rede entre nós. Com ~50 GB/s por GPU de InfiniBand e o tráfego dividido entre 16 GPUs, cada uma move algo como 30 MB: ~0,6 ms por dispatch, o mesmo no combine, em cada camada de MoE. Com 60 camadas, dezenas de milissegundos por passo. Por isso o EP entre nós exige sobreposição com cálculo e kernels de comunicação especializados.

E como confirmar: Nsight Systems mostra os kernels do NCCL e o tempo em que a GPU espera comunicação; métricas de NVLink e PCIe no DCGM mostram a banda usada; `nccl-tests` mostra o teto disponível.

Perguntas de acompanhamento prováveis

- **Como reduzir a dependência do PCIe na carga de pesos?** Manter réplicas quentes, carregar direto para a GPU (GPUDirect Storage), ou, em Grace Hopper, usar a NVLink-C2C entre CPU e GPU.
- **InfiniBand ou RoCE?** InfiniBand é mais simples de colocar em alto desempenho (sem perdas por design); RoCE é mais barato e flexível, mas exige configurar controle de fluxo e congestionamento com cuidado. Os dois funcionam em produção.
- **Por que PP entre nós raramente é gargalo de banda?** Só passa o tensor de ativações na fronteira: tokens × d × 2 bytes, poucos MB, uma vez por estágio.
- **O que muda num domínio NVLink de rack (como o NVL72)?** TP e EP podem ir além de 8 GPUs com banda de NVLink, o que muda bastante os designs de MoE grandes.

Armadilhas comuns

- Responder sem números.
- Esquecer o PCIe (carga de pesos, offload, GPUs sem NVLink).
- Tratar latência e banda como a mesma coisa.
- Não citar como verificar com ferramentas.

Resumo do capítulo

- Degraus de banda: HBM (~3,35 TB/s) → NVLink (~450 GB/s por sentido) → PCIe Gen5 (~64 GB/s) → InfiniBand NDR (~50 GB/s por porta). Comunicação pesada nos degraus rápidos.
- Nó HGX H100: 8 GPUs todas com todas por NVSwitch, uma NIC por GPU no mesmo switch PCIe, afinidade NUMA importa.
- Coletivas: all-reduce (TP), all-gather, reduce-scatter, all-to-all (MoE), send/recv (PP, KV). All-reduce = reduce-scatter + all-gather.
- Modelo α-β: mensagens pequenas pagam latência, grandes pagam banda. O decode é latency-bound na comunicação; o prefill, bandwidth-bound.
- Ring: $2(p-1)\alpha + 2\frac{p-1}{p}\frac{n}{B}$, ótimo em banda. Tree: $O(\log p)$ de latência. NVLS/SHARP: redução no switch.
- NCCL: comunicadores, algoritmos, protocolos; `nccl-tests` com busbw; `NCCL_DEBUG=INFO`; coletivas travam o grupo inteiro se uma GPU falha.
- Kubernetes: device plugin (`nvidia.com/gpu`), GPU Operator, MIG/time-slicing/MPS, RDMA nos pods, topologia, gang scheduling, `/dev/shm`.

Checklist para a entrevista

- [ ] Recito a tabela de interconexões com bandas aproximadas.
- [ ] Desenho um nó HGX H100 com GPUs, NVSwitches, NICs e CPUs.
- [ ] Explico GPUDirect RDMA e por que a posição da NIC no PCIe importa.
- [ ] Desenho as seis coletivas e digo onde cada uma aparece em inferência.
- [ ] Aplico o modelo α-β e calculo o tamanho de mensagem de equilíbrio.
- [ ] Derivo o custo do ring all-reduce e comparo com a árvore com números.
- [ ] Sei usar nccl-tests, interpretar busbw e ligar os logs do NCCL.
- [ ] Explico device plugin, GPU Operator, MIG e time-slicing.
- [ ] Descrevo um pipeline de CI/CD com testes em GPU e benchmark de regressão.
- [ ] Digo quando NVLink, PCIe e InfiniBand viram gargalo, com contas.

CAPÍTULO 11

Design de Sistemas de Inferência

As entrevistas de system design de inferência juntam tudo o que vimos até aqui: batching, KV cache, paralelismo, rede, quantização. A diferença em relação ao system design "clássico" (encurtador de URL, feed de rede social) é que o recurso escasso é a **GPU**: cara, de subida lenta, com memória pequena, e com um desempenho que depende fortemente de como as requisições são agrupadas.

Os relatos de entrevista na Anthropic aparecem repetidamente neste capítulo: serviço de LLM de alta concorrência, API de inferência para vários modelos, distribuição de pesos. Vamos primeiro montar um **roteiro de resposta** que serve para qualquer enunciado, depois uma **arquitetura de referência**, e então aplicar os dois às três perguntas.

11.1 O roteiro de resposta

Uma entrevista de design dura de 45 a 60 minutos. Use o tempo assim:

```
┌──────────────────────────────────────────────────────┐
│ 1. REQUISITOS (5–8 min)                              │
│    funcionais: o que a API faz                       │
│    não funcionais: SLOs, escala, custo, isolamento   │
│    restrições: modelos, hardware, regiões            │
├──────────────────────────────────────────────────────┤
│ 2. CONTAS DE GUARDANAPO (5 min)                      │
│    tokens/s de entrada e saída, KV cache, GPUs       │
├──────────────────────────────────────────────────────┤
│ 3. API E ARQUITETURA DE ALTO NÍVEL (10 min)          │
│    caminho de uma requisição de ponta a ponta        │
├──────────────────────────────────────────────────────┤
│ 4. APROFUNDAMENTOS (15–20 min)                       │
│    os 2 ou 3 componentes mais difíceis; deixe o      │
│    entrevistador escolher                            │
├──────────────────────────────────────────────────────┤
│ 5. FALHAS, OPERAÇÃO, EVOLUÇÃO (5–10 min)             │
│    o que quebra, como você vê, como cresce           │
└──────────────────────────────────────────────────────┘
```

As perguntas de requisito que você deve fazer em quase todo design de inferência:

- **Que modelos?** Tamanho, arquitetura (denso, MoE), quantos modelos e versões.
- **Que carga?** Requisições por segundo (média e pico), tamanho típico de entrada e saída, distribuição (cauda longa?), streaming ou não.
- **Que SLOs?** TTFT e TPOT p50/p99, disponibilidade, latência E2E para não streaming.
- **Quem são os clientes?** Interno ou externo, multi-tenant, prioridades, quotas.
- **Que hardware?** Tipo de GPU, homogêneo ou misto, capacidade disponível, uma ou várias regiões.
- **O que é pior: rejeitar ou atrasar?** Define a estratégia de sobrecarga.

11.2 Contas de guardanapo

Fazendo a conta: dimensionamento (cenário hipotético).

Requisitos: pico de 200 requisições/s; entrada média de 1.500 tokens; saída média de 300 tokens; modelo 70B em FP8; réplicas de 8 H100 com TP=8; SLO de TPOT p99 de 40 ms.

Demanda:

- Tokens de entrada: 200 × 1.500 = **300 mil tokens/s**.
- Tokens de saída: 200 × 300 = **60 mil tokens/s**.
- Requisições em andamento (lei de Little): taxa × duração. Cada uma dura uns 300 × 40 ms = 12 s → 200 × 12 ≈ **2.400 sequências simultâneas**.

Capacidade de uma réplica (estimativas):

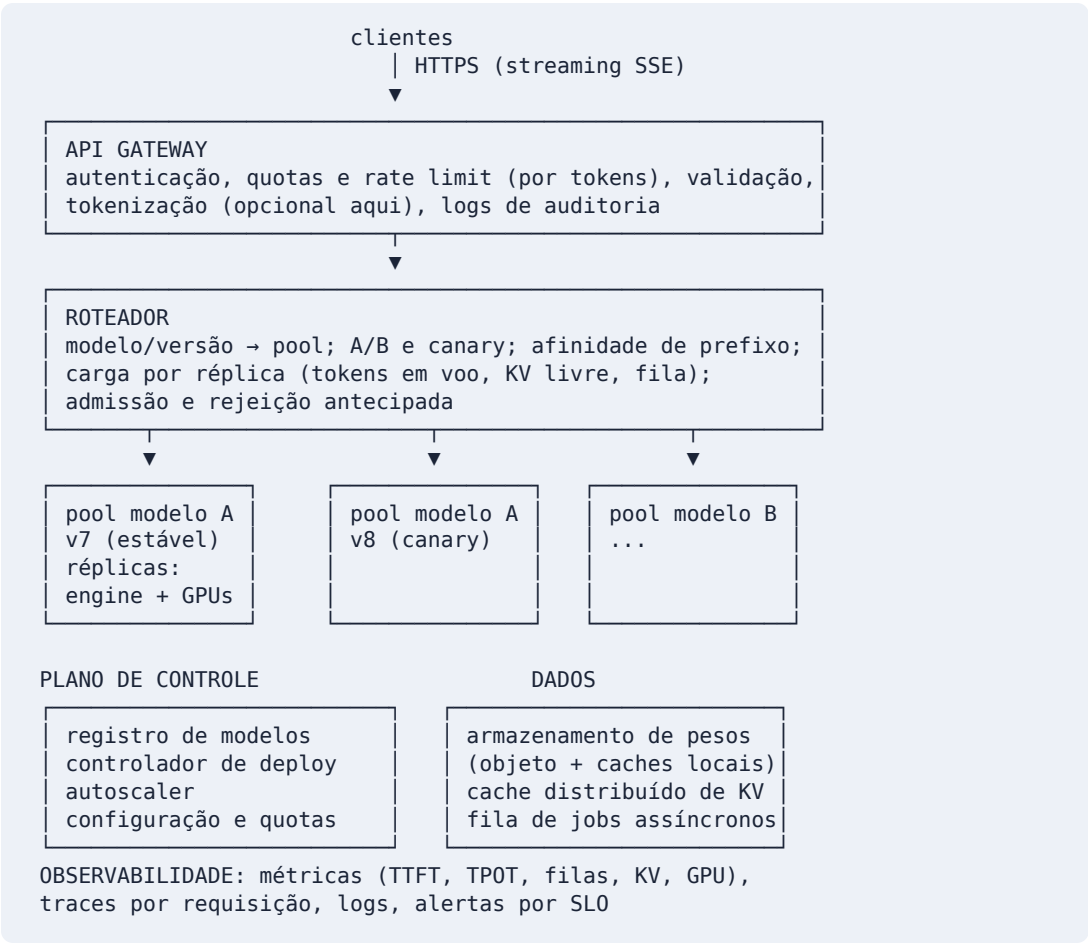
- Prefill: 8 × ~1.979 TFLOPS FP8 × 40% de eficiência ≈ 6,3 PFLOPS. Cada token custa ~2 × 70 × 10^9 = 140 GFLOP → ~**45 mil tokens/s** de prefill.
- Decode: suponha que, para manter o TPOT em 40 ms, a réplica aguente ~150 sequências → 150 / 0,04 = **3.750 tokens/s**.
- KV cache para 150 sequências de ~1.650 tokens (FP8, 160 KiB/token): 150 × 1.650 × 160 KiB ≈ **40 GB**. Cabe com folga nos ~500 GB livres. A memória **não** é o limite; o SLO de TPOT é.

Réplicas:

- Decode: 2.400 / 150 = **16 réplicas**.
- Prefill: 300 mil / 45 mil ≈ **7 réplicas-equivalentes** de tempo de GPU. Com chunked prefill, parte disso vai "de carona" nos passos de decode, então o total fica entre 16 e 23 réplicas.
- Com 30% de folga para picos e falhas: ~**25 a 30 réplicas = 200 a 240 H100**.

Os números de capacidade por réplica são hipóteses; num design real, você os mede com um benchmark. Mas a estrutura da conta (demanda em tokens, lei de Little, capacidade por réplica limitada pelo SLO ou pela memória, folga) é o que o entrevistador quer ver.

11.3 Arquitetura de referência



Componentes que costumam ser aprofundados

Roteamento e balanceamento.

- Balancear por **carga em tokens**, não por número de requisições: uma réplica com 3 prompts de 50 mil tokens está mais carregada que uma com 30 prompts curtos.
- **Power of two choices**: sortear duas réplicas e escolher a menos carregada. Quase tão bom quanto olhar todas, sem precisar de estado global perfeito.
- **Afinidade de prefixo**: mandar requisições com o mesmo prefixo para a mesma réplica (hash consistente do prefixo), com desvio quando ela estiver sobrecarregada (Q043).

Autoscaling.

- Métricas boas: tokens em voo por réplica, uso do KV cache, tamanho da fila, TTFT p95 recente. CPU e "GPU utilization" do `nvidia-smi` são métricas ruins.
- O problema central: **subir uma réplica leva minutos** (alocar GPUs, baixar pesos, carregar, compilar, aquecer). Soluções: escalar por previsão (padrão diário), manter folga, pools quentes, carga rápida de pesos (Q083).
- Escalar para baixo com cuidado: drenar a réplica (parar de mandar requisições novas e esperar as atuais terminarem) antes de desligar.

Admissão e backpressure.

- Fila limitada por réplica e no roteador.
- Rejeitar cedo (HTTP 429 com `Retry-After`) quando o tempo estimado de espera já passa do SLO.
- Prioridades: tráfego pago e síncrono antes de gratuito e batch; preempção de baixa prioridade.

Multi-tenancy.

- Quotas por cliente em tokens por minuto (entrada e saída separadas), não só requisições.
- Fair share: um cliente não pode ocupar toda a fila. Filas por cliente com round robin ponderado.
- Isolamento de cache (prefix cache com salt por tenant) e de dados (logs).
- Clientes grandes ou com SLO rígido: capacidade dedicada (Q091).

Streaming, cancelamento e falhas.

- SSE ou gRPC streaming. Detectar cliente desconectado e cancelar a geração para liberar a GPU.
- Se uma réplica morre no meio de uma geração: o cliente recebe erro e tenta de novo (com idempotência), ou o roteador refaz a requisição em outra réplica com o prompt + tokens já enviados como novo prompt, continuando a resposta. A segunda opção é melhor para o usuário, mais complexa e pode mudar o texto se houver sampling.

Perguntas de entrevista comentadas

Q081 Projete um serviço de inferência de LLM de alta concorrência: utilização da GPU, memória do KV cache,

batching e escalonamento, divisão e junção de requisições.

> **[Relatado]** Anthropic · Software Engineer · presencial · listada em fev. 2026 · Difícil · fonte

Resposta em 30 segundos

“Gateway com autenticação e quotas por tokens, um roteador que balanceia por carga em tokens e por afinidade de prefixo, e pools de réplicas, cada uma com um engine de continuous batching. Dentro da réplica, o scheduler monta cada passo com os decodes em andamento e pedaços de prefill dentro de um orçamento de tokens, admitindo novas requisições só se houver blocos de KV, com preempção quando a memória aperta. O KV cache é paginado, com prefix caching e possivelmente FP8, e o dimensionamento sai da conta de tokens por segundo e do SLO de TPOT. Divisão e junção: prompts longos são divididos em chunks de prefill; requisições com várias amostras compartilham o prefixo; e requisições pequenas são juntadas no mesmo passo. Fecho com autoscaling por tokens em voo, rejeição antecipada e métricas por SLO.”

Resposta completa

Siga o roteiro.

1. Requisitos (exemplo de como conduzir). “Vou assumir um modelo de 70B para chat e API, picos de 200 requisições/s, entradas de 1.500 tokens e saídas de 300 em média, com cauda longa até 100 mil tokens de contexto; SLO de TTFT p99 de 1 s para prompts até 8 mil tokens e TPOT p99 de 40 ms; streaming.” Confirme com o entrevistador.

2. Contas. As da seção 11.2: ~2.400 sequências simultâneas, 16 a 23 réplicas de 8 GPUs, e a observação de que o limite é o SLO de TPOT, não a memória (para esse perfil de tamanhos).

3. Arquitetura. A da seção 11.3.

4. Aprofundamentos, com os quatro tópicos do enunciado:

Utilização da GPU.

- Continuous batching (Capítulo 7): nenhuma vaga ociosa enquanto houver fila.
- Chunked prefill: aproveita o cálculo ocioso dos passos de decode.
- CUDA graphs no decode, kernels de atenção paginada, FP8.

- Escolha do ponto de operação: o maior batch que cumpre o TPOT, medido pela curva throughput × latência.
- Cuidado com a métrica: “GPU utilization” não serve; meça tokens/s por GPU e MBU.

Memória do KV cache.

- Conta por token (FP8: 160 KiB no 70B) e capacidade por réplica.
- PagedAttention: desperdício só no último bloco.
- Prefix caching para system prompts e chats multi-turno.
- Admissão com margem; preempção (recomputação para curtas, swap para longas); offload para a CPU de conversas pausadas.
- Contexto longo: limitar a parcela da memória que requisições acima de 32 mil tokens podem ocupar, para elas não expulsarem todo mundo (Q053).

Batching e scheduling. O scheduler da Q052, com prioridades e orçamento de tokens escolhido pelo SLO de ITL.

Divisão e junção de requisições. Explique as várias formas:

```
DIVIDIR
├── prompt longo → chunks de prefill (orçamento por passo)
├── prompt muito longo → context parallelism / pool
│   dedicado a contexto longo
└── requisição de "batch" do cliente (ex.: 500 prompts
    num só POST) → 500 sequências independentes no
    scheduler, resultados reagrupados na resposta

JUNTAR
├── muitas requisições pequenas → mesmo passo
│   (continuous batching, layout achatado de tokens)
├── n amostras do mesmo prompt → um prefill, KV do
│   prompt compartilhado, n sequências de decode
└── requisições com o mesmo prefixo → blocos de KV
    compartilhados (prefix caching)
```

5. Falhas e operação.

- Réplica morre: o roteador para de mandar tráfego (health check), as gerações em andamento falham ou são refeitas; o autoscaler repõe a capacidade.
- Sobrecarga: rejeição antecipada, prioridades, degradação (por exemplo, limitar `max_tokens` do tráfego gratuito).
- Deploy de nova versão: canary com comparação de métricas de latência, erro e qualidade.

- Painel: TTFT/TPOT p50/p99 por classe de tamanho, tokens/s por GPU, uso do KV, preempções, taxa de acerto do prefix cache, filas, rejeições.

Perguntas de acompanhamento prováveis

- **Onde fica a tokenização?** No gateway (libera a CPU das réplicas e permite contar tokens para quotas) ou no processo do servidor, separado do loop do engine. O importante é não bloquear o scheduler.
- **Como você testaria a capacidade antes de lançar?** Benchmark de carga com distribuição real de tamanhos (amostrada de logs), aumentando a taxa até violar o SLO; o ponto de quebra define a capacidade por réplica.
- **O que muda se o modelo for MoE?** Paralelismo (EP), batches maiores para amortizar a leitura dos especialistas, e balanceamento entre especialistas (Capítulo 9).
- **E se o TTFT p99 estourar só para prompts longos?** Pool dedicado a contexto longo, ou desagregação de prefill e decode (Q067).

Armadilhas comuns

- Desenhar só caixas, sem números.
- Não mencionar KV cache nem continuous batching.
- Balancear por número de requisições.
- Ignorar o tempo de subida das réplicas no autoscaling.

Q082 Projete uma API de inferência baseada em GPU para servir vários modelos de ML: SLOs, batching dinâmico, versionamento e roteamento A/B, autoscaling em nós de GPU heterogêneos, backpressure, isolamento multi-tenant, carregamento e aquecimento (warmup) de modelos.

> **[Relatado]** Anthropic · Software Engineer · presencial · listada em 2025 · Difícil · fonte

Resposta em 30 segundos

"Um plano de controle com registro de modelos e versões, e um plano de dados com gateway, roteador e pools de réplicas por modelo e versão. Cada réplica roda um servidor com batching dinâmico adequado ao tipo do modelo: continuous batching para LLMs e batching por tamanho e prazo para modelos de custo fixo. O roteador aplica pesos de A/B e canary, quotas por tenant e prioridades, e rejeita cedo quando

a fila prevista estoura o SLO. O autoscaler é por modelo, usa métricas de fila e tokens em voo, e escolhe o tipo de GPU com um perfil de cada modelo em cada hardware. Carregamento: pesos em cache local, carga paralela, e a réplica só entra no balanceador depois de aquecer, com compilação e CUDA graphs prontos e um teste de saúde. Isolamento: quotas, filas por tenant, pools dedicados para quem precisa."

Resposta completa

1. Requisitos e escopo. Quantos modelos (dezenas? centenas?), de que tipos (LLMs, embeddings, visão), com que distribuição de tráfego (alguns modelos muito usados e uma cauda longa de pouco usados), que GPUs (H100, A100, L4), que SLOs por modelo.

2. API.

```
POST /v1/models/{modelo}/infer
  headers: Authorization, Idempotency-Key (opcional)
  body:    { entradas, parâmetros, versão (opcional),
             prioridade (opcional), stream: bool }
  respostas: 200 (resultado ou stream SSE)
             429 (quota/sobrecarga, com Retry-After)
             503 (modelo indisponível)
GET  /v1/models                → lista, versões, status
POST /v1/jobs (assíncrono)     → para lotes grandes
```

3. Plano de controle.

- **Registro de modelos**: nome, versão, artefato (local dos pesos, checksum), formato e precisão, requisitos (memória, GPU mínima, paralelismo), configuração do servidor, SLO alvo, dono.
- **Política de roteamento**: por modelo, as versões ativas e seus pesos (por exemplo, v7: 95%, v8: 5%), regras por tenant (o cliente X fica fixo na v7).
- **Controlador de deploy**: cria e destrói réplicas conforme o estado desejado (como um operador do Kubernetes).

4. Plano de dados.

```
gateway → roteador → escolhe (modelo, versão) pela política
                     escolhe réplica pela carga
                     aplica quota e prioridade do tenant
          |
          ▼
réplica: [fila por prioridade] → [batcher] → [GPU]
         batcher depende do tipo:
         - LLM: continuous batching (Capítulo 7)
         - custo fixo: até B_max ou T_max (Q049)
```

5. Versionamento e A/B.

- Versões imutáveis; cada uma com seu pool.
- **Canary**: começa com 1% do tráfego, compara latência, erros e métricas de qualidade (para LLMs: avaliações automáticas, feedback, taxa de recusa) contra a versão estável; avança em degraus; rollback automático se piorar.
- **A/B**: atribuição estável por usuário ou tenant (hash do ID), para o mesmo usuário não alternar entre versões; registro da versão em cada resposta para a análise.
- **Shadow**: espelhar tráfego para a versão nova sem devolver a resposta, para medir latência e diferenças sem risco.

6. Autoscaling em GPUs heterogêneas.

Monte uma tabela de **perfis** (medida, não estimada) de cada modelo em cada tipo de GPU:

Modelo	GPU	Réplica	Capacidade no SLO	Custo/h (hipotético)	Custo por 1M tokens
LLM 70B FP8	8× H100	TP=8	3.750 tok/s	US$ 20	US$ 1,48
LLM 70B FP8	8× A100	TP=8	(não suporta FP8 nativo)	—	—
LLM 8B	1× H100	—	3.000 tok/s	US$ 2,50	US$ 0,23
LLM 8B	1× L4	—	400 tok/s	US$ 0,70	US$ 0,49
Embedding	1× L4	—	2.000 req/s	US$ 0,70	—

Com isso, o autoscaler resolve um problema de alocação: atender a demanda de cada modelo no SLO com o menor custo, dada a disponibilidade de cada tipo de GPU. Na prática, uma heurística gulosa funciona bem: cada modelo tem uma lista ordenada de GPUs preferidas (por custo por token no SLO), e usa a próxima da lista quando a preferida acaba.

Sinais para escalar: fila por réplica, tokens em voo, uso de KV, TTFT p95 recente; previsão por horário; folga mínima. Para modelos da cauda longa (pouco tráfego): **escala a zero** com o custo de cold start, ou **multiplexação** (vários modelos pequenos na mesma GPU, carregados sob demanda, como servidores com múltiplos modelos e LRU de modelos em memória).

7. Backpressure. Em três camadas: quota por tenant no gateway; fila limitada e rejeição antecipada no roteador (com estimativa de espera); fila limitada e preempção na réplica. Sempre devolver 429 rápido em vez de deixar a requisição esperar até o timeout.

8. Isolamento multi-tenant.

- Quotas em tokens por minuto e em requisições simultâneas.
- Filas por tenant com fair share, para que uma rajada de um cliente não aumente a latência dos outros.
- Classes de prioridade (por plano).
- Pools dedicados para clientes com SLO contratual (Q091).
- Isolamento de dados: sem prefix cache compartilhado entre tenants quando houver risco; logs separados.

9. Carregamento e aquecimento.

```
Subida de uma réplica (pronta para tráfego só no fim)

1. alocar GPUs (scheduler do cluster)
2. baixar pesos  — cache local NVMe? → pular
3. carregar na GPU (safetensors, paralelo, direto)
4. inicializar engine: reservar KV cache, compilar
   (torch.compile / engine TensorRT do cache),
   capturar CUDA graphs para cada tamanho de batch
5. AQUECER: rodar requisições sintéticas cobrindo
   os tamanhos típicos (sem isso, as primeiras
   requisições reais pagam autotuning e alocações)
6. teste de saúde: resposta de referência confere?
7. registrar no roteador (readiness = true)
```

A réplica só fica “pronta” (readiness probe) depois do passo 6. Os passos 2 a 5 são o cold start; reduzi-los é o que torna o autoscaling útil.

Perguntas de acompanhamento prováveis

- **Como garantir que a versão nova não piorou a qualidade?** Avaliações offline antes do deploy, shadow, canary com métricas de qualidade e comparação lado a lado, e rollback rápido.

- **O que acontece com requisições em andamento quando uma versão é desligada?** Drenagem: a réplica sai do balanceador, termina o que tem, e só então é desligada (com um prazo máximo).
- **Como lidar com 500 modelos, a maioria com pouco tráfego?** Multiplexação em GPUs compartilhadas, carga sob demanda com LRU, escala a zero, e serving multi-LoRA quando os modelos são variações do mesmo base (Capítulo 14).
- **Como rotear entre regiões?** Pela latência e pela capacidade disponível; requisições podem ir para outra região quando a local está saturada, se as regras de dados permitirem.

Armadilhas comuns

- Tratar todos os modelos como iguais (o batching certo depende do tipo).
- Esquecer o warmup e mandar tráfego para réplicas frias.
- Autoscaling sem considerar o tempo de subida e o tipo de GPU.
- A/B sem atribuição estável por usuário.

Q083 Projete um sistema para distribuir arquivos grandes de pesos de modelo (dezenas a centenas de GB, divididos em shards) para uma frota de workers de inferência.

> **[Relatado]** Anthropic · Software Engineer · presencial · listada em maio de 2026 · Médio · fonte

Resposta em 30 segundos

“Fonte de verdade num armazenamento de objetos, com cada versão imutável, dividida em shards e descrita por um manifesto com checksums. Os workers não baixam todos da origem, porque a banda de saída dela vira gargalo; uso distribuição em árvore ou peer-to-peer, em que cada nó que já tem um pedaço o repassa, com caches por rack e cache local em NVMe em cada nó. Cada GPU baixa só os shards de que precisa para o seu rank de paralelismo. O lançamento é em duas fases: pré-distribuir a versão nova em segundo plano, com limite de banda para não atrapalhar o tráfego, e só ativar quando todos tiverem os arquivos verificados. Para escalar rápido, um nó novo pode puxar os pesos de réplicas vizinhas por RDMA. Integridade com checksum e assinatura, retomada de downloads e coleta de versões antigas.”

Resposta completa

1. Requisitos e números.

- Tamanho: um modelo de 140 GB (70B em BF16) a ~1 TB (MoE grande), em shards de alguns GB.
- Frota: centenas a milhares de nós.
- Casos de uso: (a) lançamento de uma versão nova para toda a frota; (b) autoscaling: um nó novo precisa dos pesos rápido; (c) rollback.
- Objetivos: tempo para a frota toda ficar pronta, tempo de subida de um nó, não afetar o tráfego de produção, integridade.

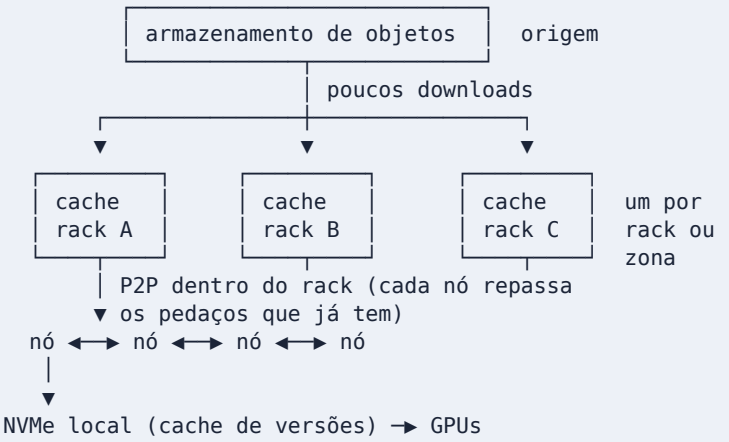
> **Fazendo a conta: por que não baixar todos da origem.** 1.000 nós × 140 GB = **140 TB**.
>
> - Se a origem (armazenamento de objetos) entrega 100 Gb/s agregados (12,5 GB/s) para esse fluxo: 140 TB / 12,5 GB/s ≈ **3,1 horas**.
> - Com peer-to-peer, cada nó também envia. Se cada nó tem ~25 Gb/s (~3 GB/s) livres para isso: no regime ideal, o tempo fica perto de 140 GB / 3 GB/s ≈ **47 s**, mais alguns fatores de propagação, independentemente do número de nós.
> - Diferença de duas ordens de grandeza. É o argumento central do design.

2. Formato do artefato.

```
modelos/llm-70b/v8/
  manifesto.json   ← lista de shards, tamanhos, sha256,
                     mapeamento shard → rank de TP/PP,
                     assinatura
  shard-00000.safetensors   (~5 GB cada)
  shard-00001.safetensors
  ...
```

- **Imutável e endereçado por conteúdo** (o hash identifica o arquivo): fácil de cachear, deduplicar e verificar.
- **Pré-dividido por rank**: com TP=8, cada GPU precisa de 1/8 dos pesos. Se os arquivos já estão divididos assim, cada GPU baixa e carrega só o seu pedaço, sem ler tudo e descartar 7/8.
- **safetensors**: formato sem código executável (seguro), com cabeçalho que permite mapear o arquivo em memória (mmap) e carregar tensores diretamente.

3. Distribuição.



- **Peer-to-peer em pedaços** (como BitTorrent; ferramentas como Dragonfly e Kraken fazem isso para imagens de contêiner): o arquivo é dividido em pedaços de dezenas de MB; cada nó pede pedaços aos vizinhos que os têm, priorizando os mais raros, e serve os que já tem.
- **Hierarquia**: caches por rack ou zona reduzem o tráfego entre racks, que costuma ser o mais caro.
- **Cache local em NVMe**: versões recentes ficam no disco do nó, para rollback e reinício instantâneos.

4. Lançamento em duas fases.

```
Fase 1 (pré-distribuição, em segundo plano)
  controlador publica "v8 disponível"
  nós baixam com limite de banda (QoS), sem afetar
  o tráfego de inferência; verificam checksums;
  reportam "v8 pronta"
Fase 2 (ativação)
  quando ≥ X% dos nós de um pool têm a v8 pronta,
  o controlador troca réplicas para a v8 de forma
  gradual (canary → 100%), cada réplica carregando
  do NVMe local
```

5. Autoscaling: subida rápida de um nó. O nó novo tem disco vazio. Opções, da mais lenta para a mais rápida:

1. Baixar da origem (minutos, e ruim em rajadas de autoscaling).
2. P2P a partir dos nós do rack (dezenas de segundos a poucos minutos).
3. **Copiar direto das GPUs de uma réplica viva** pela rede RDMA (GPU para GPU, com GPUDirect RDMA): a 50 GB/s por GPU, os 17,5 GB de cada rank num TP=8 do 70B levam ~0,35 s no limite teórico. Exige coordenação para não atrapalhar a réplica

de origem.

6. Carregar na GPU. Ler do NVMe com várias threads ou com GPUDirect Storage, direto para a memória da GPU; evitar desserializar com `pickle`; carregar os ranks em paralelo; validar a forma e o tipo de cada tensor contra o manifesto.

7. Integridade e segurança. Checksum por pedaço e por shard (detecta corrupção em trânsito ou no disco), manifesto **assinado** (garante que os pesos vieram do pipeline oficial), controle de acesso (pesos são propriedade intelectual valiosa), criptografia em trânsito.

8. Falhas e limpeza. Downloads retomáveis por pedaço; nós que falham na verificação refazem o pedaço; tempo limite e alerta para nós atrasados; coleta das versões antigas mantendo as N mais recentes (para rollback).

Perguntas de acompanhamento prováveis

- **Como evitar que a distribuição atrapalhe a inferência?** Limite de banda por nó, classes de QoS na rede (o tráfego de inferência e de coletivas tem prioridade), janelas de distribuição em horários de menos carga quando possível.
- **E se um shard estiver corrompido na origem?** O checksum do manifesto (gerado no pipeline de treino/exportação) detecta; a versão é bloqueada antes de qualquer ativação.
- **Como lidar com 10 modelos diferentes na mesma frota?** O cache local guarda os modelos de acordo com a atribuição do nó; o controlador decide quais nós precisam de quais modelos e pré-distribui só para esses.
- **Por que não colocar os pesos dentro da imagem do contêiner?** Imagens de 140 GB são lentas para construir, enviar e versionar, e misturam o ciclo de vida do código com o dos pesos. Pesos separados permitem trocar a versão do modelo sem rebuild e reaproveitar o cache.

Armadilhas comuns

- Todos os nós baixando da origem ao mesmo tempo.
- Não pré-dividir por rank, fazendo cada GPU ler o modelo inteiro.
- Esquecer integridade e assinatura.
- Ativar a versão nova antes de todos terem os arquivos.
- Ignorar o caso do autoscaling (nó novo com disco vazio).

Resumo do capítulo

- Roteiro: requisitos → contas → API e arquitetura → aprofundamentos → falhas e operação.
- Faça as contas: tokens/s de entrada e saída, sequências simultâneas pela lei de Little, capacidade por réplica limitada pelo SLO ou pela memória, folga.
- Arquitetura: gateway (auth, quotas por tokens) → roteador (versão, carga em tokens, afinidade de prefixo, admissão) → pools de réplicas com engine; plano de controle com registro, deploy e autoscaler.
- Autoscaling por tokens em voo, KV e fila; o tempo de subida é o grande problema.
- Backpressure em camadas, rejeição antecipada com 429.
- Multi-tenancy: quotas por tokens, fair share, prioridades, pools dedicados, isolamento de cache.
- Versionamento: imutável, canary, A/B com atribuição estável, shadow, rollback.
- Distribuição de pesos: origem imutável com manifesto, P2P hierárquico, NVMe local, pré-divisão por rank, duas fases, cópia GPU-GPU para autoscaling, integridade e assinatura.

Checklist para a entrevista

- ☐ Sigo o roteiro de cinco etapas e controlo o tempo.
- ☐ Faço as perguntas de requisito certas para inferência.
- ☐ Dimensiono um serviço em tokens/s, sequências simultâneas e réplicas.
- ☐ Desenho a arquitetura de referência de memória.
- ☐ Explico balanceamento por tokens, power of two choices e afinidade de prefixo.
- ☐ Explico por que o autoscaling de GPU é difícil e como mitigar.
- ☐ Descrevo canary, A/B e shadow para modelos.
- ☐ Monto a tabela de perfis por tipo de GPU para o autoscaling heterogêneo.
- ☐ Descrevo as etapas de subida e aquecimento de uma réplica.
- ☐ Projeto a distribuição de pesos com P2P e calculo o ganho sobre baixar da origem.

CAPÍTULO 12

Confiabilidade em Produção e Observabilidade

Um serviço de inferência rápido e barato que cai toda semana não serve. E sistemas de GPU falham de jeitos que serviços web comuns não falham: GPUs que somem do barramento, coletivas que travam o grupo inteiro, NaNs que aparecem só com certas entradas, uma GPU lenta que atrasa as outras sete. A pessoa de inferência carrega o pager, então os entrevistadores querem saber se você entende **como o sistema quebra, como você enxerga a quebra e o que faz quando acontece**.

12.1 SLIs, SLOs e orçamento de erro

- **SLI** (indicador): uma medida, como "fração de requisições com TTFT < 1 s".
- **SLO** (objetivo): a meta para o indicador, como "99% das requisições com TTFT < 1 s, em janelas de 28 dias".
- **Orçamento de erro**: o que sobra. Com SLO de 99,9% de disponibilidade, você pode falhar 0,1% das requisições. Se o orçamento está sendo queimado rápido, pare os lançamentos arriscados; se está sobrando, pode arriscar mais.

SLIs típicos de um serviço de LLM:

SLI	Definição	Observação
Disponibilidade	requisições sem erro de servidor / total	429 por quota do cliente não conta como falha do serviço
TTFT	percentil, por classe de tamanho de prompt	prompts de 100 mil tokens não podem ter o mesmo alvo que de 500
TPOT / ITL	percentil durante o streaming	ITL p99 pega os "soluços"
Latência E2E	para chamadas sem streaming	depende do tamanho da saída
Taxa de rejeição	429/503 por sobrecarga	sobe antes de a latência estourar

SLI	Definição	Observação
Qualidade	avaliações automáticas, taxa de erro de formato, feedback	detecta regressões silenciosas

12.2 O que medir

Os **quatro sinais dourados** (tráfego, erros, latência, saturação), adaptados a inferência:

```
TRÁFEGO     requisições/s, tokens de entrada/s, tokens de
            saída/s (por modelo, versão, tenant)
ERROS       5xx, timeouts, cancelamentos, OOM, NaN, erros
            de CUDA/NCCL, falhas de health check
LATÊNCIA    TTFT, ITL/TPOT, E2E, tempo na fila
            (percentis, por classe de tamanho)
SATURAÇÃO   uso do KV cache (%), tamanho da fila,
            sequências em execução, preempções/s,
            tokens em voo por réplica
```

E as métricas de **GPU**, coletadas pelo DCGM exporter para o Prometheus:

Métrica	O que indica
SM active / occupancy	quanto do tempo os SMs têm trabalho
Tensor pipe active	uso real dos tensor cores
DRAM active	uso da banda de memória (decode deve estar alto)
Memória usada	perto de 100% é normal (KV cache reservado)
Clocks e motivo de throttling	térmico, energia: GPU mais lenta que as outras
Temperatura e potência	refrigeração, limites de energia
Erros ECC (corrigidos e não corrigidos)	memória degradando
Eventos Xid	erros reportados pelo driver
Erros de NVLink e de PCIe (replays)	enlaces com problema

> **Dica de entrevista:** explique que "GPU utilization" do `nvidia-smi` não mede eficiência: ela só diz se havia **algum** kernel rodando. Uma GPU pode mostrar 100% fazendo pouco trabalho útil. Métricas como SM active, tensor active e DRAM active, junto com tokens/s por GPU, dizem muito mais.

Tracing por requisição

Um trace distribuído mostra onde cada requisição gastou o tempo:

```
trace req-7f3a  (TTFT = 1.840 ms)
├─ gateway: auth + quota ............  4 ms
├─ roteador: escolha da réplica .....  2 ms
├─ fila na réplica r-12 ............ 1.410 ms  ◄ o problema
├─ prefill (6.200 tokens, 3 chunks)   410 ms
│    atributos: prefix_hit = 0 tokens
└─ decode (412 tokens) ............. 11.900 ms
     atributos: preempções = 0, ITL p99 = 41 ms
```

Atributos úteis em cada trace: modelo e versão, réplica, tokens de entrada e saída, tokens reaproveitados do prefix cache, preempções, prioridade, tenant (de forma anônima). Por privacidade, o **conteúdo** de prompts e respostas não deve ir para logs e traces por padrão.

12.3 Como a stack de inferência falha

Organize as falhas por camada. É exatamente o que a Q090 pede.

TRÁFEGO	picos, rajadas de prompts longos, tempestade de retries, cliente abusivo
ROTEAMENTO	balanceamento ruim, réplica doente recebendo tráfego, afinidade desbalanc.
ENGINE	OOM, KV esgotado, tempestade de preempções, scheduler travado, recompilação, vazamento de memória
MODELO	versão errada, tokenizer/template diferente, NaN/Inf, quantização ruim
COMUNICAÇÃO	NCCL travado, GPU lenta (straggler), enlace NVLink/IB degradado
DRIVER/CUDA	erro de CUDA corrompe o contexto, incompatibilidade de versões
HARDWARE	GPU caiu do barramento, ECC, térmico, energia, corrupção silenciosa de dados
PLATAFORMA	pod Pending, imagem não baixa, pesos não baixam, nó sem rede RDMA

Alguns comportamentos específicos que você deve conhecer:

- **Erros de CUDA "pegajosos"**: depois de um acesso ilegal à memória dentro de um kernel, o contexto CUDA do processo fica corrompido e toda chamada seguinte falha. Não há recuperação dentro do processo: é preciso reiniciar. O health check precisa detectar isso.
- **NCCL hang**: com TP, se uma GPU não chega à coletiva (travou, morreu, ficou lenta), as outras esperam para sempre. Sem timeout, a réplica inteira fica parada e parece "viva" para um health check superficial.
- **Straggler**: uma GPU com throttling térmico ou um enlace degradado deixa a coletiva esperar por ela em todo passo. A réplica inteira fica mais lenta. Detectar comparando o tempo de cada rank.
- **Xid**: erros reportados pelo driver NVIDIA no log do kernel, com um número. Exemplos conhecidos: o Xid 79 indica que a GPU "caiu do barramento"; o Xid 48 indica erro de ECC com dois bits (não corrigível); há Xids específicos para erros de NVLink e para remapeamento de linhas de memória. Alguns exigem reiniciar o processo; outros, reiniciar ou trocar a GPU.
- **Corrupção silenciosa de dados**: raramente, uma GPU com defeito produz resultados errados **sem** reportar erro. Em LLMs, aparece como respostas estranhas vindas sempre da mesma réplica. Detectar com testes de resposta conhecida periódicos e diagnósticos (como `dcgmi diag`).
- **NaN/Inf**: overflow em FP16, problemas de escala na quantização, ou uma entrada patológica. Uma sequência com NaN pode gerar lixo; dependendo do kernel, pode contaminar outras do batch.

12.4 Cauda de latência

O p99 de um serviço de LLM costuma ser dominado por:

1. **Fila**: picos de chegada com a capacidade perto do limite.
2. **Prefills longos** sem chunking, congelando decodes.
3. **Preempções**: a vítima refaz o prefill.
4. **Stragglers** em TP: o passo espera a GPU mais lenta.
5. **Pausas no host**: coleta de lixo do Python, recompilação, CPU do scheduler sobrecarregada.
6. **Vizinhos barulhentos**: outro tenant enchendo a réplica com prompts gigantes.

E a regra de teoria de filas que justifica capacidade ociosa: o tempo de espera cresce muito rápido quando a utilização se aproxima de 100% (fator $\rho/(1-\rho)$, Capítulo 1). Um serviço com p99 rígido **precisa** operar com utilização bem abaixo do máximo.

12.5 Resposta a incidentes

```
alerta (queima rápida do orçamento de erro de TTFT)
  │
  ├─ 1. MITIGAR primeiro, investigar depois
  │     - rollback da última mudança (deploy, config,
  │       driver), se coincidir
  │     - tirar réplicas doentes do balanceador
  │     - aumentar capacidade / desviar para outra região
  │     - rejeitar tráfego de baixa prioridade
  │
  ├─ 2. DIAGNOSTICAR com os traces e métricas:
  │     é uma réplica, um modelo, uma classe de tamanho,
  │     um tenant, uma região?
  │
  ├─ 3. CORRIGIR a causa
  │
  └─ 4. POSTMORTEM sem culpados: linha do tempo, causa,
        por que não foi detectado antes, ações
```

Runbooks para as falhas comuns (Xid, NCCL hang, OOM, preempção excessiva, pod Pending) transformam o plantão das 2 da manhã num procedimento em vez de uma investigação.

Perguntas de entrevista comentadas

Q089 Como você investigaria um pod do Kubernetes travado no status Pending?

> **[Relatado]** CoreWeave · Operations Engineer · entrevista técnica · relatado em abr. 2026 · Médio · fonte

Resposta em 30 segundos

"Pending quer dizer que o pod ainda não foi agendado num nó, ou foi agendado mas não pode começar. Começo com `kubectl describe pod` e leio os eventos: o scheduler diz por que nenhum nó serve, por exemplo GPUs insuficientes, nodeSelector ou afinidade que não batem, taints sem tolerations, ou um PVC que não se liga. Depois confiro os nós: se o recurso nvidia.com/gpu está sendo anunciado, se o device plugin está rodando, se há GPUs marcadas como não saudáveis, se os nós estão cordoned ou com pressão de recursos. Também verifico quotas do namespace, PriorityClass e preempção, e se o pod precisa de um gang scheduler esperando o grupo inteiro. Para GPU, a causa mais comum é pedir 8 GPUs quando nenhum nó tem 8 livres ao mesmo tempo por causa de fragmentação."

Resposta completa

Mostre um processo, do geral ao específico:

```
 1. kubectl describe pod <pod> -n <ns>
    → seção Events. Mensagens típicas do scheduler:
      "0/40 nodes are available: 32 Insufficient
       nvidia.com/gpu, 8 node(s) had untolerated taint
       {gpu-maintenance: true}"
      "pod has unbound immediate PersistentVolumeClaims"
      "node(s) didn't match Pod's node affinity/selector"

 2. Se não houver eventos do scheduler:
    → o pod está esperando outro scheduler (schedulerName)
      ou um gang scheduler/fila (Kueue, Volcano)
    → kubectl get events -n <ns> --sort-by=.lastTimestamp

 3. Recursos dos nós
    kubectl describe node <nó>  → Capacity / Allocatable /
                                  Allocated resources
    kubectl get nodes -o custom-columns=NOME:.metadata.name,\
GPU:.status.allocatable.nvidia\.com/gpu
```

As causas, agrupadas:

Categoria	Causa	Como confirmar	Correção
Recurso	nenhum nó com GPUs livres suficientes	eventos “Insufficient nvidia.com/gpu”; nós alocados	liberar, escalar o cluster, reduzir o pedido
Recurso	fragmentação: 8 GPUs livres no total, mas espalhadas	somar GPUs livres por nó	bin packing, desfragmentar, reservar nós inteiros
Recurso	CPU, memória ou ephemeral storage insuficientes (não só GPU)	eventos	ajustar requests
GPU	device plugin parado, nó anuncia 0 GPUs	`allocatable` sem `nvidia.com/gpu`; pods do GPU Operator	reiniciar plugin, driver, investigar o nó
GPU	GPUs marcadas como não saudáveis (Xid, ECC)	logs do device plugin, DCGM	isolar o nó, manutenção
GPU	MIG configurado diferente do pedido (ex.: pede GPU inteira, nó só tem fatias MIG)	rótulos do nó (`nvidia.com/mig...`)	ajustar perfil MIG ou o pedido

Categoria	Causa	Como confirmar	Correção
Restrição	nodeSelector, afinidade, topologia não batem	eventos “didn’t match”	corrigir rótulos ou o spec
Restrição	taints sem tolerations (nós de GPU costumam ter taint)	`kubectl describe node` → Taints	adicionar toleration
Restrição	nós cordoned (unschedulable)	`kubectl get nodes`	uncordon se for o caso
Volume	PVC não ligado, volume em outra zona	eventos, `kubectl get pvc`	storage class, zona
Política	ResourceQuota do namespace estourada	eventos de quota, `kubectl describe quota`	aumentar a quota
Política	prioridade baixa e sem preempção	PriorityClass	ajustar prioridade
Fila	gang scheduling esperando o grupo inteiro caber	status do Kueue/Volcano	liberar capacidade para o grupo

E o caso “parece Pending mas não é do scheduler”: o pod foi agendado (tem `nodeName`), mas os contêineres não sobem. Aí o status costuma mostrar `ContainerCreating` ou `Init`, e a causa é imagem que não baixa (`ImagePullBackOff`), volume que não monta, CNI sem IP, ou um init container baixando pesos de 140 GB. Mostre que você distingue os dois casos.

> **Dica de entrevista:** termine com prevenção: alertas para pods Pending há mais de N minutos, painel de GPUs livres por nó (não só o total, por causa da fragmentação), e automação que isola nós com GPUs não saudáveis.

Perguntas de acompanhamento prováveis

- **O cluster tem 16 GPUs livres e o pod de 8 não sobe. Por quê?** Fragmentação: as GPUs livres estão em nós diferentes (por exemplo, 2 livres em cada um de 8 nós). Soluções: scheduler com bin packing (preencher nós antes de abrir novos), pools separados para pods de nó inteiro, desfragmentação planejada.
- **O que é o cluster autoscaler e por que ele não resolveu?** Ele cria nós quando há pods Pending por falta de recursos. Pode não resolver se o tipo de nó de GPU não está disponível na nuvem (falta de capacidade), se há limites máximos, ou se a

restrição do pod (zona, rótulo) não bate com nenhum grupo de nós.

- **Como o device plugin decide que uma GPU não está saudável?** Ele observa eventos de saúde (por exemplo, Xids críticos) e remove a GPU da lista anunciada ao kubelet.
- **E se o pod ficar Pending por causa de prioridade?** Pods com PriorityClass mais alta podem preemptar pods de prioridade menor; se o seu é de prioridade baixa, ele espera.

Armadilhas comuns

- Ir direto para os logs do contêiner (um pod Pending não tem contêiner rodando).
- Não ler os eventos do `describe`.
- Esquecer taints, quotas e fragmentação de GPUs.
- Confundir Pending com ContainerCreating e ImagePullBackOff.

Q090 Explique a stack de inferência de LLMs e seus cenários de falha.

> **[Relatado]** Anthropic · Staff Software Engineer · conselho de preparação de um candidato que passou pelo processo · 2026 · Difícil · fonte

Resposta em 30 segundos

"De cima para baixo: cliente, gateway com autenticação e quotas, roteador, servidor do modelo com o engine, que tem scheduler, gerenciador de KV cache e executor, os kernels, a comunicação entre GPUs com NCCL, o driver e o CUDA, o hardware, e a plataforma por baixo, Kubernetes, rede e armazenamento de pesos. Cada camada falha de um jeito: tráfego com picos e tempestades de retry; roteamento mandando para réplica doente; engine com OOM, KV esgotado e preempções em cascata; modelo com NaN, versão ou template errado; comunicação com NCCL travado e stragglers; CUDA com erros que corrompem o contexto; hardware com Xid, ECC, throttling e corrupção silenciosa. Para cada uma, digo como detecto, como contenho o raio de impacto e como me recupero."

Resposta completa

Esta pergunta é de nível Staff: o entrevistador quer **amplitude e profundidade**, e principalmente o raciocínio sobre **raio de impacto** e **detecção**. Use a pilha da seção 12.3 e, para cada camada, uma linha com sintoma, detecção e resposta:

Camada	Falha	Sintoma	Detecção	Resposta
Tráfego	tempestade de retries	carga 3× após um blip	taxa de retries, rajadas	retries com backoff e jitter, orçamento de retry, 429 com Retry-After
Tráfego	rajada de prompts longos	TTFT explode, preempções	métricas por classe de tamanho	limite por classe, pools separados, chunked prefill
Roteamento	réplica doente no balanceador	erros concentrados numa réplica	erros por réplica, health check profundo	ejeção automática (outlier detection)
Engine	KV esgotado	preempções em cascata, TPOT ruim	uso de KV, preempções/s	admissão com margem, limitar concorrência
Engine	OOM no prefill	processo morre com prompt grande	logs, reinícios	limite de tokens por passo, logits só do último token
Engine	recompilação em produção	picos de latência de segundos	logs de recompilação	buckets fixos, aquecimento
Modelo	template ou tokenizer errado	respostas estranhas, queda de qualidade	avaliações canário, métricas de formato	rollback, testes de compatibilidade no CI
Modelo	NaN/Inf	lixo nas respostas	checagem de NaN nos logits	isolar a sequência, investigar entrada e precisão
Comunicação	NCCL travado	réplica parada, sem erros	watchdog com timeout, progresso por passo	reiniciar a réplica inteira
Comunicação	straggler	réplica 20% mais lenta	tempo por rank, clocks	drenar e investigar o nó
CUDA	acesso ilegal	todas as chamadas falham	health check com forward real	reiniciar o processo
Hardware	GPU caiu do barramento (Xid 79)	GPU some	Xid, DCGM	isolar o nó, trocar a GPU

Camada	Falha	Sintoma	Detecção	Resposta
Hardware	ECC não corrigível	erros de CUDA, Xid	contadores ECC	reiniciar, remapear, trocar
Hardware	corrupção silenciosa	respostas ruins de uma réplica só	testes de resposta conhecida, comparação entre réplicas	isolar e diagnosticar
Plataforma	pesos não baixam	réplicas não sobem, autoscaling falha	tempo de subida, erros de download	caches locais, P2P (Q083)

Depois, fale dos **princípios** que atravessam as camadas, porque é isso que diferencia uma resposta Staff:

1. **Raio de impacto.** A unidade de falha de uma réplica com TP=8 é o nó inteiro: uma GPU ruim derruba as oito. Desenhe para que uma réplica ruim afete só as requisições dela, e para que as falhas sejam detectadas e a réplica ejetada rapidamente.
2. **Health checks profundos.** Um endpoint `/health` que só responde "ok" não detecta NCCL travado nem contexto CUDA corrompido. O health check precisa rodar um forward pequeno de verdade (com timeout) e verificar a resposta.
3. **Falhar rápido e reiniciar.** Muitos estados ruins de GPU não têm recuperação dentro do processo. É melhor reiniciar a réplica (e ter subida rápida) do que tentar consertar em execução.
4. **Degradação graciosa.** Sob sobrecarga, rejeitar tráfego de baixa prioridade, reduzir `max_tokens` e desligar recursos caros é melhor do que todos sofrerem.
5. **Detecção de problemas de qualidade.** Falhas de modelo (template, quantização, versão) não geram erros HTTP. Avaliações automáticas contínuas e métricas de formato são o "health check" da qualidade.
6. **Mudanças são a principal causa de incidentes.** Drivers, versões de engine, configurações e modelos novos passam por canary com rollback automático.

Perguntas de acompanhamento prováveis

- **Qual falha é mais difícil de detectar?** Corrupção silenciosa e regressões de qualidade: o sistema parece saudável em todas as métricas operacionais. Por isso testes de resposta conhecida e avaliações contínuas.
- **Como você faria o health check de uma réplica com TP=8?** Um forward curto com entrada fixa, com timeout, conferindo a saída (por exemplo, os primeiros tokens com greedy) contra o esperado. Executado periodicamente e fora do caminho das requisições reais.

- **O que você faria se uma GPU de uma réplica de 8 morresse?** A réplica inteira sai do balanceador; as requisições em andamento falham e os clientes tentam de novo; o nó é isolado; o autoscaler sobe uma réplica nova em outro nó.
- **Como evitar que retries piorem um incidente?** Backoff exponencial com jitter, orçamento de retries por cliente, circuit breakers, e respostas 429/503 com `Retry-After`.

Armadilhas comuns

- Listar falhas sem detecção e resposta.
- Ignorar falhas sem erro (qualidade, corrupção silenciosa).
- Health check superficial.
- Não pensar em raio de impacto.

Q091 Construa um endpoint dedicado com garantias rígidas de latência de cauda e explique o que muda em relação ao pool compartilhado.

> **[Fornecedor]** Together AI, segundo fornecedor de preparação (“candidatos supostamente recebem”) · Design de sistemas · 2026 · Difícil · fonte

Resposta em 30 segundos

“Um endpoint dedicado é capacidade reservada para um cliente, dimensionada pelo perfil de tráfego dele e por um SLO contratual de p99. O que muda em relação ao pool compartilhado: não há vizinhos, então a variância cai; o alvo de utilização é mais baixo, porque a latência de cauda explode perto de 100%; o batch e o orçamento de tokens são ajustados para latência, não para throughput; a configuração é estática e aquecida, sem recompilações nem cold starts; há redundância N+1 em domínios de falha diferentes; e a admissão limita o cliente à taxa contratada, rejeitando o excesso em vez de deixar a fila crescer. Ele custa mais por token, então o cliente paga por GPU-hora reservada. Monitoro o SLO do endpoint e meço a margem continuamente.”

Resposta completa

1. Requisitos do contrato. Defina com o cliente: taxa máxima (requisições/s e tokens/s), distribuição de tamanhos de entrada e saída, SLO (por exemplo, TTFT p99 < 500 ms e TPOT p99 < 30 ms para prompts até 8 mil tokens), disponibilidade, região.

Garantias rígidas só existem dentro de limites declarados: acima da taxa contratada, o endpoint pode rejeitar.

2. Dimensionamento para o p99, não para a média.

> **Fazendo a conta (hipotética):** contrato de até 30 requisições/s, saídas de ~400 tokens, TPOT p99 < 30 ms.
>
> - Sequências simultâneas no pico (lei de Little): 30 × (400 × 0,03 s) = 30 × 12 s = **360**.
> - Medindo a réplica: para TPOT p99 < 30 ms, ela aguenta ~120 sequências (no pool compartilhado, com TPOT de 40 ms, aguentava 150).
> - Réplicas para o pico: 360 / 120 = 3. Com N+1 para falha de uma réplica: **4**.
> - Utilização média esperada, se o tráfego médio for metade do pico: ~37%. Esse é o preço da garantia.

3. O que muda na configuração.

Aspecto	Pool compartilhado	Endpoint dedicado
Objetivo	maximizar goodput e custo	cumprir o p99 com margem
Utilização alvo	alta (70–90%)	moderada (30–60%)
Batch máximo e orçamento de tokens	maiores (throughput)	menores (latência previsível)
Vizinhos	muitos tenants, prompts imprevisíveis	só o cliente, perfil conhecido
Admissão	quotas e fair share	limite na taxa contratada, rejeição fora dela
Capacidade	elástica, autoscaling	reservada, N+1, pré-aquecida
Configuração	ajustada para a média dos clientes	ajustada para o perfil do cliente (tamanhos, paralelismo, speculative decoding)
Mudanças	lançamentos frequentes	janelas combinadas, canary dentro do endpoint
Prefix cache	compartilhado (com isolamento)	só do cliente, pode ser maior
Cobrança	por token	por GPU-hora reservada

4. Técnicas específicas para a cauda.

- **TP maior** para reduzir o tempo de cada passo (ao custo de throughput por GPU).
- **Speculative decoding** para reduzir o TPOT (Capítulo 14), ajustado ao tipo de texto do cliente.
- **Chunked prefill com orçamento pequeno**, ou **desagregação** de prefill e decode, para o ITL ficar estável.
- **Configuração estática**: tamanhos de batch fixos com CUDA graphs capturados, nada de recompilação, warmup completo antes de receber tráfego.
- **Sem preempção** em condições normais: a admissão garante memória para todo mundo que entra.
- **Redundância em domínios de falha**: réplicas em nós (e se possível racks) diferentes; se uma cai, as outras absorvem dentro do SLO.
- **Verificação contínua**: tráfego sintético medindo o p99 de fora, alertas quando a margem diminui (antes de violar).

5. A conversa de negócio. O endpoint dedicado custa mais por token porque a utilização é menor. Mostre que você entende isso e proponha meios-termos: capacidade reservada com "estouro" para o pool compartilhado (com SLO mais fraco no excesso), ou um piso dedicado mais autoscaling para a parte variável.

Perguntas de acompanhamento prováveis

- **O cliente manda o dobro da taxa contratada. O que acontece?** O excesso recebe 429, ou é desviado para o pool compartilhado sem garantia, conforme o contrato. Nunca deixar o excesso degradar o SLO do tráfego contratado.
- **Como provar que o SLO está sendo cumprido?** Medição do lado do servidor e sondas sintéticas do lado de fora, com relatórios por janela; definir no contrato como se mede (incluindo rede e exclusões).
- **E se o cliente mudar o perfil (prompts muito mais longos)?** O SLO só vale para o perfil acordado; monitorar o perfil, avisar o cliente e redimensionar.
- **Vale usar hedging (mandar a mesma requisição para duas réplicas)?** Para LLMs, duplicar uma geração inteira é caro. Uma variante viável é fazer hedge apenas da fase de admissão: se a réplica escolhida não começar o prefill em X ms, tentar outra e cancelar a primeira.

Armadilhas comuns

- Prometer p99 sem limites de taxa e perfil.
- Dimensionar pela média.

- Esquecer a redundância (uma falha de réplica viola o SLO).
- Não discutir o custo e a utilização baixa.

Resumo do capítulo

- SLIs de LLM: disponibilidade, TTFT e ITL por classe de tamanho, E2E, rejeição, qualidade. SLOs com orçamento de erro.
- Sinais dourados adaptados: tráfego em tokens, erros (incluindo CUDA/NCCL/NaN), latência (TTFT, ITL, fila), saturação (KV, fila, preempções).
- Métricas de GPU do DCGM: SM/tensor/DRAM active, clocks e throttling, ECC, Xid, erros de NVLink/PCIe. “GPU utilization” não mede eficiência.
- Traces por requisição mostram fila vs prefill vs decode.
- Falhas por camada: tráfego, roteamento, engine, modelo, comunicação, CUDA, hardware, plataforma. Erros de CUDA corrompem o contexto; NCCL trava o grupo; stragglers atrasam todos; corrupção silenciosa não gera erro.
- Princípios: raio de impacto, health check profundo, falhar rápido e reiniciar, degradação graciosa, detecção de qualidade, canary.
- Pod Pending: `describe` e eventos; recursos, fragmentação de GPUs, device plugin, taints, afinidade, quotas, PVC, gang scheduling.
- Endpoint dedicado: capacidade reservada pelo perfil e pelo p99, utilização menor, configuração estática para latência, N+1, admissão na taxa contratada.

Checklist para a entrevista

- [ ] Defino SLIs e SLOs para um serviço de LLM, separados por classe de tamanho.
- [ ] Listo os quatro sinais dourados adaptados a inferência.
- [ ] Sei quais métricas do DCGM importam e por que “GPU utilization” engana.
- [ ] Desenho um trace de requisição e explico como ele localiza o problema.
- [ ] Percorro a pilha de falhas camada por camada, com detecção e resposta.
- [ ] Explico erros de CUDA pegajosos, NCCL hang, stragglers e corrupção silenciosa.
- [ ] Tenho um roteiro de resposta a incidentes (mitigar antes de investigar).
- [ ] Investigo um pod Pending com `describe` , eventos e a tabela de causas.
- [ ] Dimensiono um endpoint dedicado para um p99 com N+1.
- [ ] Comparo endpoint dedicado e pool compartilhado em configuração e custo.

CAPÍTULO 13

Programação e Depuração de Desempenho

Toda vaga de inferência tem pelo menos uma rodada de código. Os relatos mostram três tipos misturados, às vezes na mesma entrevista (a NVIDIA, por exemplo, pediu um loop de inferência em PyTorch e uma ordenação topológica na mesma rodada):

1. **Algoritmos clássicos**: estruturas de dados e grafos, no estilo LeetCode de dificuldade média. Exemplos da lista: ordenação topológica, serializar uma árvore binária.
2. **Código de ML**: implementar atenção, KV cache, sampling, beam search, um loop de geração, um batcher.
3. **Depuração de desempenho**: "este código está lento; descubra por quê e conserte".

Este capítulo traz um método para as rodadas de código, as ferramentas de profiling que você precisa citar, e as três perguntas da lista resolvidas com código completo. Todo o código deste capítulo foi executado e testado (os testes estão no fim de cada seção).

13.1 O método para a rodada de código

```
1. ENTENDER (2–3 min)
   repita o problema com suas palavras; pergunte sobre
   entradas, tamanhos, casos de borda, formato da saída
2. EXEMPLOS (2 min)
   um caso normal e um de borda, escritos no quadro
3. SOLUÇÃO SIMPLES (2 min)
   diga a força bruta e a complexidade dela
4. OTIMIZAR (5 min)
   chegue na solução boa; diga a complexidade ANTES
   de codar; combine com o entrevistador
5. CODAR (15–20 min)
   nomes claros, funções pequenas, falando em voz alta
6. TESTAR (5 min)
   rode os exemplos à mão, depois os casos de borda;
   corrija com calma
```

Duas observações específicas para vagas de inferência:

- **O entrevistador quer ver você raciocinar sobre desempenho** mesmo em problemas algorítmicos. Depois de resolver, comente: "isso é O(n), sem recursão para não estourar a pilha com árvores degeneradas; se fosse um grafo de operadores de um compilador, eu também guardaria a ordem estável para builds reproduzíveis".
- **Em código de ML, shapes são tudo.** Escreva o shape de cada tensor num comentário. A maioria dos bugs de entrevista é de eixo trocado.

13.2 Profiling e depuração de desempenho

As ferramentas

Nível	Ferramenta	Responde
Python	`cProfile`, `py-spy`	que função Python consome a CPU?
PyTorch	`torch.profiler`	que operador gerou que kernel, tempo de CPU e GPU
GPU (linha do tempo)	Nsight Systems	onde a GPU fica ociosa? cópias? sincronizações?
GPU (kernel)	Nsight Compute	por que este kernel está lento?
Memória	`torch.cuda.memory_summary`, snapshots	quem alocou a memória?

O `py-spy` merece destaque: ele inspeciona um processo Python **em execução**, sem reiniciar e sem modificar o código. Perfeito para um servidor em produção que ficou lento:

```
py-spy top --pid 12345           # ao vivo, estilo "top"
py-spy dump --pid 12345          # pilha atual de cada thread
py-spy record -o perfil.svg --pid 12345 --duration 30
```

E o `torch.profiler`, para ver CPU e GPU juntos:

```python
import torch
from torch.profiler import profile, ProfilerActivity, record_function

with profile(activities=[ProfilerActivity.CPU,
                         ProfilerActivity.CUDA],
             record_shapes=True) as prof:
    with record_function("passo_de_decode"):
        logits = modelo(entrada)
        torch.cuda.synchronize()

print(prof.key_averages().table(sort_by="cuda_time_total",
                                row_limit=15))
prof.export_chrome_trace("trace.json")   # abrir no Perfetto
```

Os erros de desempenho mais comuns em código PyTorch

Padrão	Por que é lento	Correção
`.item()`, `.tolist()`, `print(tensor)` dentro do laço	sincroniza CPU e GPU a cada chamada	acumular na GPU, sincronizar uma vez
laço Python sobre elementos de um tensor	milhares de kernels minúsculos	operações vetorizadas
`torch.cat` crescendo um tensor a cada passo	realoca e copia tudo: $O(n^2)$	pré-alocar e escrever com índice
`.to("cuda")` dentro do laço	cópia pelo PCIe a cada iteração	mover uma vez, antes
tensores não contíguos (`.T`, `permute`) passados para kernels	cópias implícitas ou kernels lentos	`.contiguous()` uma vez, ou reorganizar o layout
dtype misturado (FP32 com BF16)	conversões extras e caminhos lentos	padronizar o dtype
sem `torch.inference_mode()`	guarda grafo para backward	usar o contexto de inferência
batch de tamanho 1	GPU subutilizada	agrupar requisições

Fazendo a conta: o custo de uma sincronização. Um decode com batch pequeno gera um token a cada ~10 ms. Se o laço chama `.item()` para cada uma das 64 sequências do batch, são 64 sincronizações por passo. Cada uma esvazia a fila de trabalho da GPU e paga a latência de ida e volta (dezenas de microssegundos). Um único `.tolist()` por passo resolve, como no código da Q097.

Perguntas de entrevista comentadas

Q097 Implemente um loop de inferência em PyTorch.

> **[Relatado]** NVIDIA · Senior Software Engineer · entrevista técnica · 2026 · Médio · fonte

Resposta em 30 segundos

“Tokenizo o prompt, rodo um forward com o prompt inteiro (prefill), que devolve os logits do último token e o KV cache de cada camada. Depois entro num laço: escolho o próximo token com o sampling (greedy, temperatura, top-p), anexo, e rodo um forward só com esse token novo, passando o cache, até atingir o máximo de tokens ou o token de parada. Para batch, uso padding à esquerda com máscara de atenção e posições por sequência. Tudo dentro de `torch.inference_mode()`, com uma sincronização por passo. E testo comparando a geração com cache contra a geração sem cache.”

Resposta completa

Comece pela versão mais curta, com um modelo do Hugging Face, para mostrar que você conhece o padrão. Depois, se o entrevistador quiser, escreva o modelo e o laço do zero.

Versão 1: com um modelo pronto (Hugging Face).

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
modelo = AutoModelForCausalLM.from_pretrained("gpt2").eval()

@torch.inference_mode()
def gerar_hf(texto, max_novos=30):
    ids = tok(texto, return_tensors="pt").input_ids   # [1, T]
    out = modelo(ids, use_cache=True)                  # prefill
    cache = out.past_key_values
    prox = out.logits[:, -1].argmax(-1, keepdim=True)  # [1, 1]
    gerados = [prox]
    for _ in range(max_novos - 1):                     # decode
        out = modelo(prox, past_key_values=cache,
                     use_cache=True)
        cache = out.past_key_values
        prox = out.logits[:, -1].argmax(-1, keepdim=True)
        gerados.append(prox)
        if prox.item() == tok.eos_token_id:
            break
    return tok.decode(torch.cat(gerados, dim=-1)[0])

print(gerar_hf("Engenharia de inferência é"))
```

Versão 2: do zero. Um decoder mínimo no estilo Llama (RMSNorm, RoPE, GQA, SwiGLU), com KV cache por camada, batch com padding à esquerda, sampling e parada por token. É longo para uma entrevista, mas cada pedaço é o que o entrevistador pode pedir isoladamente:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class Config:
    vocab = 1000
    d = 128
    n_heads = 8
    n_kv = 2          # GQA: 4 heads de query por head de KV
    n_layers = 2
    d_ff = 344


def rope_cos_sin(pos, head_dim, base=10000.0):
    # pos: [B, T] posições absolutas -> cos, sin: [B, T, D]
    i = torch.arange(0, head_dim, 2, dtype=torch.float32)
    inv_freq = base ** (-i / head_dim)
    ang = pos.float()[..., None] * inv_freq
    ang = torch.cat([ang, ang], dim=-1)
    return ang.cos(), ang.sin()


def aplica_rope(x, cos, sin):
    # x: [B, H, T, D]; cos, sin: [B, 1, T, D]
    d = x.shape[-1]
    x1, x2 = x[..., : d // 2], x[..., d // 2 :]
    return x * cos + torch.cat([-x2, x1], dim=-1) * sin


class RMSNorm(nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.g = nn.Parameter(torch.ones(d))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        return x * rms * self.g


class Atencao(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.h, self.hkv = c.n_heads, c.n_kv
        self.hd = c.d // c.n_heads
        self.wq = nn.Linear(c.d, c.d, bias=False)
        self.wk = nn.Linear(c.d, self.hkv * self.hd, bias=False)
        self.wv = nn.Linear(c.d, self.hkv * self.hd, bias=False)
        self.wo = nn.Linear(c.d, c.d, bias=False)

    def forward(self, x, cos, sin, cache, mask):
        B, T, _ = x.shape
        q = self.wq(x).view(B, T, self.h, self.hd).transpose(1, 2)
        k = self.wk(x).view(B, T, self.hkv, self.hd).transpose(1, 2)
        v = self.wv(x).view(B, T, self.hkv, self.hd).transpose(1, 2)
        q, k = aplica_rope(q, cos, sin), aplica_rope(k, cos, sin)
        if cache is not None:          # K já tem RoPE no cache
```

```
            k = torch.cat([cache[0], k], dim=2)
            v = torch.cat([cache[1], v], dim=2)
        rep = self.h // self.hkv       # GQA: expande só na conta
        o = F.scaled_dot_product_attention(
            q, k.repeat_interleave(rep, 1),
            v.repeat_interleave(rep, 1), attn_mask=mask)
        o = o.transpose(1, 2).reshape(B, T, -1)
        return self.wo(o), (k, v)


class Bloco(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.n1, self.att = RMSNorm(c.d), Atencao(c)
        self.n2 = RMSNorm(c.d)
        self.gate = nn.Linear(c.d, c.d_ff, bias=False)
        self.up = nn.Linear(c.d, c.d_ff, bias=False)
        self.down = nn.Linear(c.d_ff, c.d, bias=False)

    def forward(self, x, cos, sin, cache, mask):
        h, cache = self.att(self.n1(x), cos, sin, cache, mask)
        x = x + h
        y = self.n2(x)
        x = x + self.down(F.silu(self.gate(y)) * self.up(y))
        return x, cache


class MiniLLM(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.c = c
        self.emb = nn.Embedding(c.vocab, c.d)
        self.blocos = nn.ModuleList(
            Bloco(c) for _ in range(c.n_layers))
        self.norm = RMSNorm(c.d)
        self.head = nn.Linear(c.d, c.vocab, bias=False)

    def forward(self, ids, pos, attn_mask, caches=None):
        # ids, pos: [B, T]; attn_mask: [B, Tk], 1 = token real
        T, Tk = ids.shape[1], attn_mask.shape[1]
        cos, sin = rope_cos_sin(pos, self.c.d // self.c.n_heads)
        cos, sin = cos[:, None], sin[:, None]
        i = torch.arange(T)[:, None] + (Tk - T)   # pos. das queries
        j = torch.arange(Tk)[None, :]
        mask = (j <= i)[None] & attn_mask[:, None, :].bool()
        mask = (mask | (j == i)[None])[:, None]   # nunca linha vazia
        x = self.emb(ids)
        novos = []
        for n, bloco in enumerate(self.blocos):
            cache = None if caches is None else caches[n]
            x, c = bloco(x, cos, sin, cache, mask)
            novos.append(c)
        logits = self.head(self.norm(x[:, -1]))  # só o último token
        return logits, novos
```

O laço de geração:

```
def sample(logits, temperatura=0.0, top_p=1.0):
    if temperatura == 0:
        return logits.argmax(dim=-1)
    probs = torch.softmax(logits / temperatura, dim=-1)
    sp, idx = probs.sort(dim=-1, descending=True)
    cum = sp.cumsum(dim=-1)
    sp[(cum - sp) > top_p] = 0.0
    sp = sp / sp.sum(dim=-1, keepdim=True)
    escolha = torch.multinomial(sp, num_samples=1)
    return idx.gather(-1, escolha).squeeze(-1)


@torch.inference_mode()
def gerar(modelo, prompts, max_novos=20, temperatura=0.0,
          top_p=1.0, eos=None):
    B, L = len(prompts), max(len(p) for p in prompts)
    ids = torch.zeros(B, L, dtype=torch.long)
    mask = torch.zeros(B, L, dtype=torch.long)
    for b, p in enumerate(prompts):           # padding à esquerda
        ids[b, L - len(p):] = torch.tensor(p)
        mask[b, L - len(p):] = 1
    pos = (mask.cumsum(-1) - 1).clamp(min=0)
    logits, caches = modelo(ids, pos, mask)   # PREFILL
    saidas = [[] for _ in range(B)]
    ativo = [True] * B
    for _ in range(max_novos):
        prox = sample(logits, temperatura, top_p)       # [B]
        for b, t in enumerate(prox.tolist()):  # 1 sync por passo
            if ativo[b]:
                saidas[b].append(t)
                if eos is not None and t == eos:
                    ativo[b] = False
        if not any(ativo):
            break
        pos = mask.sum(-1, keepdim=True)       # posição do novo
        um = torch.ones(B, 1, dtype=mask.dtype)
        mask = torch.cat([mask, um], dim=-1)
        logits, caches = modelo(prox[:, None], pos, mask, caches)
    return saidas
```

E o teste que prova que o cache está correto: gerar com cache, gerar sem cache (recalculando a sequência inteira a cada passo) e comparar; e gerar cada prompt sozinho e dentro do batch, para provar que o padding e a máscara estão certos:

```
@torch.inference_mode()
def gerar_sem_cache(modelo, prompt, max_novos=20):
    seq = list(prompt)
    for _ in range(max_novos):
        ids = torch.tensor([seq])
        mask = torch.ones_like(ids)
        pos = torch.arange(len(seq))[None]
        logits, _ = modelo(ids, pos, mask)
        seq.append(int(logits.argmax(-1)))
    return seq[len(prompt):]
```

```
torch.manual_seed(0)
m = MiniLLM(Config()).eval()
prompts = [[5, 17, 300, 42], [9, 8], [1, 2, 3, 4, 5, 6, 7]]
com_cache = gerar(m, prompts, max_novos=15)
for p, saida in zip(prompts, com_cache):
    assert saida == gerar_sem_cache(m, p, max_novos=15)
    assert saida == gerar(m, [p], max_novos=15)[0]
print("ok: com cache == sem cache; batch == sozinho")
```

Os pontos que você deve explicar enquanto escreve:

1. **Prefill e decode são o mesmo `forward`** com entradas diferentes: no prefill, `ids` tem o prompt inteiro e não há cache; no decode, `ids` tem 1 token e o cache vem do passo anterior.
2. **Posições**: com padding à esquerda, a posição de cada token é a contagem de tokens reais antes dele (`cumsum` da máscara), não o índice no tensor. No decode, a posição do token novo é o número de tokens reais da sequência.
3. **Máscara**: combina causal (com o deslocamento entre queries e keys) e padding. A linha extra `mask | (j == i)` garante que nenhuma query fique sem nenhuma key permitida; sem isso, as posições de padding gerariam NaN, que contaminaria as outras posições pela multiplicação por zero.
4. **Logits só do último token**: o `head` roda só em `x[:, -1]`, não em todas as posições do prompt (ver Q042).
5. **GQA**: o cache guarda só as 2 heads de KV; a expansão para 8 heads acontece só na conta.
6. **Uma sincronização por passo**: `prox.tolist()` uma vez, não `.item()` por sequência.
7. **Limitações conscientes**: o cache usa `torch.cat` por simplicidade (em produção, pré-alocar ou paginar, Q041), e as sequências que já terminaram continuam ocupando lugar no batch (em produção, continuous batching, Capítulo 7).

Perguntas de acompanhamento prováveis

- **Como você faria isso rodar mais rápido na GPU?** Pré-alocar o cache, `torch.compile` com CUDA graphs no decode, kernels de atenção como FlashAttention/SDPA, BF16 ou FP8, e evitar qualquer sincronização dentro do passo.
- **Como adicionar parada por sequência de texto (stop strings)?** Detokenizar incrementalmente e checar o sufixo do texto, tomando cuidado com uma stop string que atravessa tokens. Isso exige CPU a cada passo; dá para fazer de forma assíncrona.

- **Como implementar beam search?** Manter k feixes por prompt, expandir cada um com os top-k tokens, somar log-probabilidades, manter os k melhores, e reordenar o cache conforme os feixes pais (Q041).
- **Por que padding à esquerda e não à direita?** Para que o último token de todas as sequências fique na mesma coluna, e o token novo de cada uma seja sempre anexado no fim. Com padding à direita, cada sequência teria o "fim" num lugar diferente.

Armadilhas comuns

- Rodar o modelo na sequência inteira a cada passo, sem cache.
- Posições erradas com padding.
- Esquecer a máscara de padding e obter resultados que mudam com o batch.
- `.item()` por elemento dentro do laço.
- Não ter um teste de equivalência.

Q098 Implemente ordenação topológica em Python.

> **[Relatado]** NVIDIA · Senior Software Engineer · entrevista técnica · 2026 · Médio · fonte

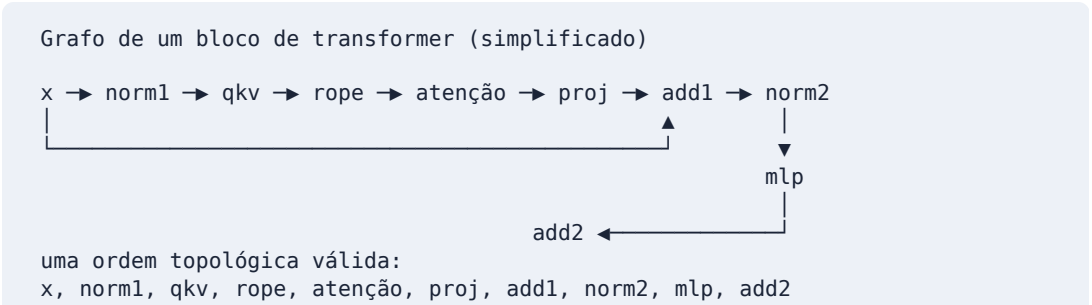
Resposta em 30 segundos

"Uso o algoritmo de Kahn: calculo o grau de entrada de cada nó, coloco numa fila os nós com grau zero, e repito: tiro um nó da fila, coloco na resposta e decremento o grau dos vizinhos, colocando na fila os que chegam a zero. Se no fim a resposta tiver menos nós que o grafo, existe ciclo. É O(V + E) em tempo e espaço. Se precisar de uma ordem determinística, por exemplo a menor lexicograficamente, troco a fila por um heap, com custo O((V + E) log V). Alternativa: DFS com três cores, invertendo a pós-ordem; faço iterativa para não estourar a pilha de recursão."

Resposta completa

O problema. Dado um grafo direcionado, encontrar uma ordem dos nós em que, para toda aresta $u \to v$, u aparece antes de v. Só existe se o grafo não tiver ciclos (DAG).

Por que isso cai numa vaga de inferência. Grafos de computação são DAGs: um compilador de modelos (como o Inductor ou o TensorRT) precisa ordenar as operações respeitando dependências para gerar o código, decidir fusões e planejar a memória. Mencione isso; mostra que você conecta o algoritmo ao trabalho.



Kahn (BFS):

```python
from collections import deque


def topo_kahn(n, arestas):
    """n nós (0..n-1), arestas (u, v) = u antes de v.
    Retorna uma ordem topológica ou levanta erro se houver ciclo."""
    adj = [[] for _ in range(n)]
    grau = [0] * n
    for u, v in arestas:
        adj[u].append(v)
        grau[v] += 1
    fila = deque(i for i in range(n) if grau[i] == 0)
    ordem = []
    while fila:
        u = fila.popleft()
        ordem.append(u)
        for v in adj[u]:
            grau[v] -= 1
            if grau[v] == 0:
                fila.append(v)
    if len(ordem) != n:
        raise ValueError("o grafo tem ciclo")
    return ordem
```

Variante com ordem determinística (menor lexicográfica):

```python
import heapq


def topo_menor_lexico(n, arestas):
    """Mesma ideia, mas sempre escolhe o menor nó disponível."""
    adj = [[] for _ in range(n)]
    grau = [0] * n
    for u, v in arestas:
        adj[u].append(v)
        grau[v] += 1
    heap = [i for i in range(n) if grau[i] == 0]
    heapq.heapify(heap)
    ordem = []
    while heap:
        u = heapq.heappop(heap)
        ordem.append(u)
        for v in adj[u]:
            grau[v] -= 1
            if grau[v] == 0:
                heapq.heappush(heap, v)
    if len(ordem) != n:
        raise ValueError("o grafo tem ciclo")
    return ordem
```

DFS com três cores, iterativa (branco = não visitado, cinza = na pilha atual, preto = terminado; encontrar um cinza é encontrar um ciclo):

```python
def topo_dfs(n, arestas):
    """DFS iterativa com 3 cores (sem limite de recursão)."""
    adj = [[] for _ in range(n)]
    for u, v in arestas:
        adj[u].append(v)
    BRANCO, CINZA, PRETO = 0, 1, 2
    cor = [BRANCO] * n
    pos_ordem = []
    for raiz in range(n):
        if cor[raiz] != BRANCO:
            continue
        pilha = [(raiz, 0)]
        cor[raiz] = CINZA
        while pilha:
            u, i = pilha[-1]
            if i < len(adj[u]):
                pilha[-1] = (u, i + 1)
                v = adj[u][i]
                if cor[v] == CINZA:
                    raise ValueError("o grafo tem ciclo")
                if cor[v] == BRANCO:
                    cor[v] = CINZA
                    pilha.append((v, 0))
            else:
                cor[u] = PRETO
                pos_ordem.append(u)
                pilha.pop()
    return pos_ordem[::-1]
```

Testes. A forma mais forte de testar é validar a propriedade em vez de comparar com uma resposta fixa (há muitas ordens válidas):

```python
def valida_topo(n, arestas, ordem):
    posicao = {u: i for i, u in enumerate(ordem)}
    return (sorted(ordem) == list(range(n))
            and all(posicao[u] < posicao[v] for u, v in arestas))
```

```python
assert valida_topo(4, [(0, 1), (0, 2), (1, 3), (2, 3)],
                   topo_kahn(4, [(0, 1), (0, 2), (1, 3), (2, 3)]))
assert topo_menor_lexico(4, [(3, 0), (2, 1)]) == [2, 1, 3, 0]
try:
    topo_kahn(3, [(0, 1), (1, 2), (2, 0)])
except ValueError:
    print("ciclo detectado")
```

No teste completo, as três versões foram validadas em 300 DAGs aleatórios de até 40 nós, e as três detectam o ciclo.

Complexidade. Kahn e DFS: $O(V + E)$ tempo e espaço. Com heap: $O((V + E) \log V)$.

Perguntas de acompanhamento prováveis

- **Como encontrar o ciclo, não só detectar?** Na DFS, quando encontrar um nó cinza, a pilha atual contém o ciclo: desempilhe até esse nó.
- **E se o grafo for enorme e não couber na memória?** Processar em partições, ou usar o algoritmo de Kahn com graus guardados em disco. Para grafos de modelos, isso não acontece; a pergunta testa se você pensa em escala.
- **Como paralelizar a execução de um grafo de operações?** Kahn por níveis: todos os nós com grau zero num mesmo momento podem rodar em paralelo (em streams CUDA diferentes, por exemplo).
- **Como escolher uma ordem que minimize o pico de memória?** É um problema difícil em geral; heurísticas escolhem, entre os nós disponíveis, os que liberam mais memória (cujos tensores de entrada não serão mais usados). Compiladores de ML fazem algo assim.

Armadilhas comuns

- Esquecer a detecção de ciclo.
- DFS recursiva que estoura a pilha em grafos profundos (o limite padrão do Python é de cerca de mil chamadas).
- Usar `list.pop(0)` como fila (O(n) por operação); use `deque`.
- Confundir a direção das arestas.

Q099 Serialize e desserialize uma árvore binária, com código limpo e tempo e espaço ótimos.

> **[Relatado]** NVIDIA · Deep Learning Software Engineer, Inference · triagem com o gestor da vaga · 2025 · Médio · fonte

Resposta em 30 segundos

“Serializo em pré-ordem com um marcador para nulos: valor do nó, depois a subárvore esquerda, depois a direita, com ‘#’ para cada filho ausente, separados por vírgula. Os marcadores tornam a pré-ordem suficiente para reconstruir a árvore sem ambiguidade. Para desserializar, leio os tokens na mesma ordem: cada token não nulo vira um nó e seus dois filhos são os próximos itens. Tudo O(n) em tempo e espaço. Faço as duas funções com pilha explícita, sem recursão, porque uma árvore degenerada com centenas de milhares de nós estouraria a recursão do Python.”

Resposta completa

Por que os marcadores de nulo. Uma pré-ordem sozinha não define a árvore: as árvores “1 com filho esquerdo 2” e “1 com filho direito 2” têm a mesma pré-ordem [1, 2]. Com os marcadores: “1,2,#,#,#” e “1,#,2,#,#”. Cada nó contribui com exatamente um token, e cada nulo também. Uma árvore com n nós tem $n + 1$ nulos, então a string tem $2n + 1$ tokens: $O(n)$.

```
    1
   / \          pré-ordem com nulos:
  2   3         1,2,#,#,3,4,#,#,5,#,#
     / \
    4   5       nível a nível (estilo LeetCode):
                [1,2,3,null,null,4,5]
```

Pré-ordem, sem recursão:

```python
class No:
    def __init__(self, val, esq=None, dir=None):
        self.val, self.esq, self.dir = val, esq, dir


def serializar(raiz):
    """Pré-ordem com marcador '#' para nulo. O(n)."""
    partes, pilha = [], [raiz]
    while pilha:
        no = pilha.pop()
        if no is None:
            partes.append("#")
            continue
        partes.append(str(no.val))
        pilha.append(no.dir)     # dir primeiro: esq sai antes
        pilha.append(no.esq)
    return ",".join(partes)


def desserializar(s):
    """Reconstrói a pré-ordem sem recursão. O(n)."""
    tokens = iter(s.split(","))
    primeiro = next(tokens)
    if primeiro == "#":
        return None
    raiz = No(int(primeiro))
    pilha = [(raiz, 0)]          # (nó, filhos já preenchidos)
    for t in tokens:
        pai, k = pilha.pop()
        filho = None if t == "#" else No(int(t))
        if k == 0:
            pai.esq = filho
            pilha.append((pai, 1))
        else:
            pai.dir = filho
        if filho is not None:
            pilha.append((filho, 0))
    return raiz
```

Como funciona a desserialização: a pilha guarda os nós que ainda esperam filhos, junto com quantos filhos já receberam (0 ou 1). Cada token preenche o próximo filho pendente do nó no topo. Se o token é um nó de verdade, ele entra na pilha para receber os próprios filhos, e como está no topo, os próximos tokens vão para ele, exatamente a ordem da pré-ordem.

Alternativa: nível a nível (BFS). É o formato usado pelo LeetCode. Também $O(n)$:

```
def serializar_bfs(raiz):
    """Estilo LeetCode: nível a nível, remove nulos do fim."""
    saida, fila = [], deque([raiz])
    while fila:
        no = fila.popleft()
        if no is None:
            saida.append("null")
            continue
        saida.append(str(no.val))
        fila.append(no.esq)
        fila.append(no.dir)
    while saida and saida[-1] == "null":
        saida.pop()
    return "[" + ",".join(saida) + "]"


def desserializar_bfs(s):
    itens = s.strip("[]")
    if not itens:
        return None
    vals = itens.split(",")
    raiz = No(int(vals[0]))
    fila, i = deque([raiz]), 1
    while fila and i < len(vals):
        no = fila.popleft()
        for lado in ("esq", "dir"):
            if i < len(vals) and vals[i] != "null":
                filho = No(int(vals[i]))
                setattr(no, lado, filho)
                fila.append(filho)
            i += 1
    return raiz
```

Teste. Ida e volta em árvores aleatórias e numa árvore degenerada de 200 mil nós (que quebraria uma versão recursiva):

```
def iguais(a, b):
    pilha = [(a, b)]
    while pilha:
        x, y = pilha.pop()
        if x is None or y is None:
            if x is not y:
                return False
            continue
        if x.val != y.val:
            return False
        pilha.append((x.esq, y.esq))
        pilha.append((x.dir, y.dir))
    return True
```

```
exemplo = No(1, No(2), No(3, No(4), No(5)))
print(serializar(exemplo))       # 1,2,#,#,3,4,#,#,5,#,#
print(serializar_bfs(exemplo))   # [1,2,3,null,null,4,5]
assert iguais(exemplo, desserializar(serializar(exemplo)))

fundo = None
for i in range(200_000):         # árvore degenerada
    fundo = No(i, esq=fundo)
assert iguais(fundo, desserializar(serializar(fundo)))
```

No teste completo, as duas formas fizeram ida e volta em 300 árvores aleatórias de até 60 nós.

Complexidade. Tempo $O(n)$ nas duas direções. Espaço $O(n)$ para a string e $O(h)$ para a pilha (onde h é a altura; no pior caso, n).

Discussão de formato, que impressiona um gestor de time de inferência:

- **Texto vs binário**: texto é legível e fácil de depurar; binário é menor e mais rápido. Para muitos nós com inteiros, um formato binário com um bit de presença por filho seria bem mais compacto.
- **Valores com vírgula ou "#"**: se os valores forem strings arbitrárias, é preciso escapar ou usar prefixo de tamanho.
- **Versionamento**: um formato que vai para disco precisa de um número de versão no cabeçalho.
- **Analogia com o trabalho**: serializar estruturas com ponteiros num formato linear, eficiente e versionado é o que formatos de checkpoint (como safetensors) fazem com tensores: cabeçalho com metadados e offsets, dados contíguos, carregamento sem execução de código.

Perguntas de acompanhamento prováveis

- **Dá para usar só a in-ordem?** Não. A in-ordem sozinha (mesmo com nulos) é ambígua. Pré-ordem ou pós-ordem com marcadores funcionam; in-ordem precisa de mais uma travessia (por exemplo, in-ordem + pré-ordem, sem marcadores, se os valores forem únicos).
- **E se for uma árvore binária de busca?** A pré-ordem **sem** marcadores basta, porque os valores determinam a forma (todos os menores que a raiz ficam à esquerda). Desserialização em $O(n)$ com limites superior e inferior.
- **Como paralelizar a serialização de uma árvore enorme?** Serializar subárvores independentes em paralelo, com cabeçalhos que dizem o tamanho de cada pedaço.

- **Por que evitar recursão?** O limite padrão de recursão do Python é de cerca de mil chamadas; árvores degeneradas (listas) passam disso fácil. Aumentar o limite é frágil e pode derrubar o processo.

Armadilhas comuns

- Esquecer os marcadores de nulo.
- Recursão sem considerar árvores profundas.
- Usar `pop(0)` em lista como fila.
- Concatenar strings num laço em vez de juntar uma lista (`",".join`), que é quadrático em algumas implementações.

Resumo do capítulo

- Rodadas de código em inferência misturam algoritmos clássicos, código de ML e depuração de desempenho.
- Método: entender, exemplos, solução simples, otimizar, codar, testar. Escreva os shapes em comentários.
- Ferramentas: `cProfile` e `py-spy` (Python), `torch.profiler` (operadores e kernels), Nsight Systems (linha do tempo), Nsight Compute (kernel), snapshots de memória.
- Erros comuns: sincronizações (`.item()`), laços Python sobre tensores, `torch.cat` crescente, cópias dentro do laço, dtypes misturados, batch 1.
- Loop de inferência: prefill com o prompt, decode com um token e o cache; posições e máscara com padding à esquerda; logits só do último token; uma sincronização por passo; teste com cache vs sem cache e batch vs sozinho.
- Ordenação topológica: Kahn com fila (ou heap para ordem determinística), DFS iterativa com três cores; O(V+E); detecção de ciclo.
- Serialização de árvore: pré-ordem com marcadores de nulo ou BFS; O(n); sem recursão para árvores profundas.

Checklist para a entrevista

- [ ] Sigo o método de seis passos e falo a complexidade antes de codar.
- [ ] Uso `py-spy` , `torch.profiler` e Nsight Systems e sei o que cada um mostra.
- [ ] Reconheço os erros de desempenho comuns em PyTorch e corrijo cada um.
- [ ] Escrevo um loop de geração com Hugging Face de memória.
- [ ] Escrevo um decoder mínimo com KV cache, RoPE, GQA e máscara de padding.

- [ ] Explico posições e máscara com padding à esquerda.
- [ ] Testo o cache comparando com a geração sem cache.
- [ ] Escrevo Kahn e DFS iterativa com detecção de ciclo.
- [ ] Escrevo serialização de árvore em pré-ordem e BFS, sem recursão.
- [ ] Discuto formatos de serialização (texto, binário, versionamento) e a ligação com checkpoints.

CAPÍTULO 14

Arquitetura Avançada de Inferência

Este último capítulo cobre as técnicas e os designs que aparecem em entrevistas de nível sênior e Staff: **speculative decoding**, serving de **muitos adaptadores LoRA** sobre um modelo base, **saídas estruturadas**, **modelos multimodais**, e as variações de "projete uma API com batching dinâmico" que a Anthropic e a xAI pedem. Ele se apoia em tudo que veio antes, então os links para os capítulos anteriores são frequentes.

14.1 Speculative decoding

O decode gera um token por passo e é memory-bound (Capítulo 1): a GPU passa o passo lendo os pesos e fica com o cálculo ocioso. **Verificar** vários tokens de uma vez custa quase o mesmo que gerar um, porque a leitura dos pesos é a mesma; só a quantidade de cálculo aumenta, e cálculo está sobrando.

O **speculative decoding** (Leviathan et al., 2023; Chen et al., 2023) explora isso:

1. Um **rascunho** barato propõe os próximos *k* tokens.
2. O modelo **alvo** (o grande) processa os *k* tokens propostos numa única passada e calcula sua própria distribuição em cada posição.
3. Os tokens propostos são **aceitos** da esquerda para a direita enquanto passam no teste; no primeiro rejeitado, o alvo escolhe um token correto naquela posição, e o resto é descartado.

```
contexto: "O gato subiu no"

rascunho propõe (k = 4):   telhado  da   casa   ontem
alvo verifica, 1 passada:   ✔        ✔     ✔      ✘
                                                    no lugar de "ontem",
                                                    o alvo amostra "à"
resultado do passo: "telhado da casa à"  → 4 tokens em 1 passada
                                           do modelo grande
```

A regra de aceitação

Seja $p(x)$ a probabilidade do token x segundo o alvo e $q(x)$ segundo o rascunho. O token proposto x é aceito com probabilidade:

$$\min\left(1, \frac{p(x)}{q(x)}\right)$$

Se for rejeitado, um token novo é amostrado da distribuição corrigida:

$$p'(x) = \frac{\max\left(0,\ p(x) - q(x)\right)}{\sum_y \max\left(0,\ p(y) - q(y)\right)}$$

Essa regra (uma forma de rejection sampling) garante que a saída tem **exatamente a mesma distribuição** que o modelo alvo sozinho produziria. O speculative decoding não troca qualidade por velocidade: só troca cálculo ocioso por velocidade. Com greedy, a regra fica simples: aceita se o token proposto é o argmax do alvo.

Quanto acelera

Se cada token proposto é aceito com probabilidade α (independente, como aproximação), o número esperado de tokens produzidos por passada do alvo, com k propostas, é:

$$E[\text{tokens}] = \frac{1 - \alpha^{k+1}}{1 - \alpha}$$

Fazendo a conta: $\alpha = 0,8$ e $k = 4$.

- $E = (1 - 0,8^5)/(1 - 0,8) = (1 - 0,328)/0,2 \approx$ **3,36 tokens** por passada do alvo.
- Se a passada de verificação custa 1,1× um passo normal (cálculo extra) e o rascunho custa 0,2× para gerar as 4 propostas, cada "rodada" custa 1,3×.
- Speedup: 3,36 / 1,3 ≈ **2,6×** no TPOT.
- Com $\alpha = 0,5$: $E = (1 - 0,5^5)/0,5 \approx 1,94$ → speedup ~1,5×. A taxa de aceitação manda em tudo.

Tipos de rascunho

Rascunho	Como propõe	Prós	Contras
Modelo pequeno separado	um LLM menor da mesma família	simples, genérico	memória e cálculo extras; precisa do mesmo tokenizer

Rascunho	Como propõe	Prós	Contras
N-gramas / prompt lookup	copia trechos que já aparecem no contexto	grátis; ótimo para edição de código, RAG	só funciona quando há repetição
Medusa	várias “cabeças” extras no próprio modelo preveem os tokens +1, +2, +3...	sem modelo separado	exige treinar as cabeças
EAGLE	rascunho leve que trabalha com as representações internas do alvo	aceitação alta	exige treinar o rascunho
MTP (multi-token prediction)	modelo treinado para prever vários tokens (ex.: DeepSeek-V3)	vem do treino	depende do modelo

Variações com **árvore**: em vez de uma única sequência de k propostas, o rascunho propõe várias alternativas em forma de árvore, e o alvo verifica todos os ramos de uma vez com uma **máscara de atenção em árvore** (cada token só enxerga seus ancestrais). Aumenta a chance de aceitar mais tokens por passada.

Quando não ajuda

- **Batch grande**: com muitas sequências, o decode já fica mais perto do limite de cálculo, e o cálculo extra da verificação deixa de ser grátis. O ganho diminui e pode virar perda. Muitos motores ajustam k (ou desligam a especulação) conforme a carga.
- **Taxa de aceitação baixa**: texto muito criativo (temperatura alta) ou um rascunho ruim para o domínio.
- **KV cache**: os tokens rejeitados escreveram K e V no cache; é preciso desfazer (voltar o comprimento). Com cache paginado, isso é barato.

14.2 Serving de muitos adaptadores LoRA

LoRA (low-rank adaptation) adapta um modelo adicionando a cada matriz W um termo de posto baixo: $W' = W + BA$, com $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ e r pequeno (8, 16, 64). Um adaptador ocupa uma fração pequena do modelo.

Um provedor pode ter **milhares** de adaptadores (um por cliente ou tarefa) sobre o mesmo modelo base. Uma réplica por adaptador seria um desperdício absurdo. O serving multi-LoRA mantém **um** modelo base e aplica o adaptador certo para cada requisição, **no mesmo batch**:

$$y_i = W x_i + B_{a(i)} A_{a(i)} x_i$$

onde $a(i)$ é o adaptador da requisição i.

```
batch com 4 requisições, 3 adaptadores

x1 (adaptador A) ┐
x2 (adaptador B) ├─▶ W·X  (uma GEMM grande, compartilhada)
x3 (adaptador A) │       +
x4 (adaptador C) ┘   para cada adaptador: B_a·(A_a·X_a)
                     (GEMMs pequenas agrupadas num kernel
                      só, como as SGMV do Punica)
```

Peças do sistema (ideias de sistemas como Punica e S-LoRA):

- **Kernels agrupados** para aplicar adaptadores diferentes a linhas diferentes do batch num único lançamento.
- **Cache de adaptadores** na GPU, com os demais na CPU, carregados sob demanda (LRU).
- **Memória unificada**: adaptadores e KV cache dividindo o mesmo depósito de blocos.
- **Roteamento**: mandar requisições do mesmo adaptador para as réplicas que já o têm carregado.
- **Prefix cache por adaptador**: a chave do cache precisa incluir o adaptador (Q043).

14.3 Saídas estruturadas (constrained decoding)

Muitas aplicações exigem que a saída siga um formato: JSON com um esquema, uma chamada de função, uma expressão regular. **Constrained decoding** garante isso: a cada passo, calcula-se o conjunto de tokens que mantêm a saída válida, e os outros recebem logit $-\infty$ antes do sampling.

```
esquema: {"nome": string, "idade": inteiro}
já gerado: {"nome": "Ana", "idade":
tokens permitidos agora: espaços, dígitos, "-"
máscara: logits de todos os outros tokens → −∞
```

A gramática (esquema JSON, regex) é compilada num autômato; o estado do autômato avança a cada token. O desafio de desempenho: calcular a máscara sobre um vocabulário de 128 mil tokens a cada passo, sem atrasar a GPU. Bibliotecas como Outlines e XGrammar pré-computam a maior parte por estado do autômato e calculam

a máscara na CPU **enquanto** a GPU roda o forward. Uma otimização extra: quando só existe um caminho possível (por exemplo, o texto fixo `", "idade":`), os tokens podem ser anexados de uma vez, sem passar pelo modelo um a um.

14.4 Modelos multimodais

Modelos de visão e linguagem transformam cada imagem em **tokens visuais** com um encoder (por exemplo, um ViT), que entram no LLM junto com o texto. Cada imagem pode virar de centenas a milhares de tokens, dependendo da resolução.

Consequências para o serving:

- **Prefill mais pesado**: imagens aumentam muito o número de tokens de entrada.
- **Encoder como etapa separada**: o encoder de visão é outro modelo, com outro perfil (compute-bound, sem KV cache). Pode rodar em GPUs separadas, com batching próprio, e entregar os embeddings para o LLM, uma desagregação "encoder → prefill → decode".
- **Cache por conteúdo**: a mesma imagem enviada de novo (comum em chats) pode reaproveitar os embeddings do encoder e o KV cache, se a chave incluir o **hash da imagem**, não só os tokens de placeholder.
- **Áudio e vídeo**: áudio vira tokens por janela de tempo; vídeo vira quadros amostrados. Os números de tokens crescem rápido, e decidir quantos quadros e que resolução usar vira uma decisão de custo.

Perguntas de entrevista comentadas

Q105 Projete uma API HTTP de alta concorrência para inferência de modelos de linguagem, em que workers de GPU processam requisições compatíveis em batches dinâmicos num pool limitado de GPUs.

> **[Relatado]** Anthropic · Machine Learning Engineer · presencial · listada em jul. 2026 · Difícil · fonte

Resposta em 30 segundos

"Frontends HTTP assíncronos aceitam as requisições, validam, tokenizam e calculam uma chave de compatibilidade: modelo, versão, adaptador e o que mais impedir que duas requisições dividam um batch. Cada chave tem uma fila. Os workers de

GPU puxam trabalho, o que dá backpressure natural: um worker só pega o que consegue admitir no KV cache. Dentro do worker, continuous batching junta todas as requisições compatíveis num passo. Como o pool é limitado, o despachante decide qual fila atender com uma pontuação que combina prioridade, idade e tamanho do lote possível, e chaves raras ganham um prazo máximo para não morrerem de fome. Respostas voltam por streaming. Na sobrecarga, rejeito cedo com 429 com base no tempo estimado de fila."

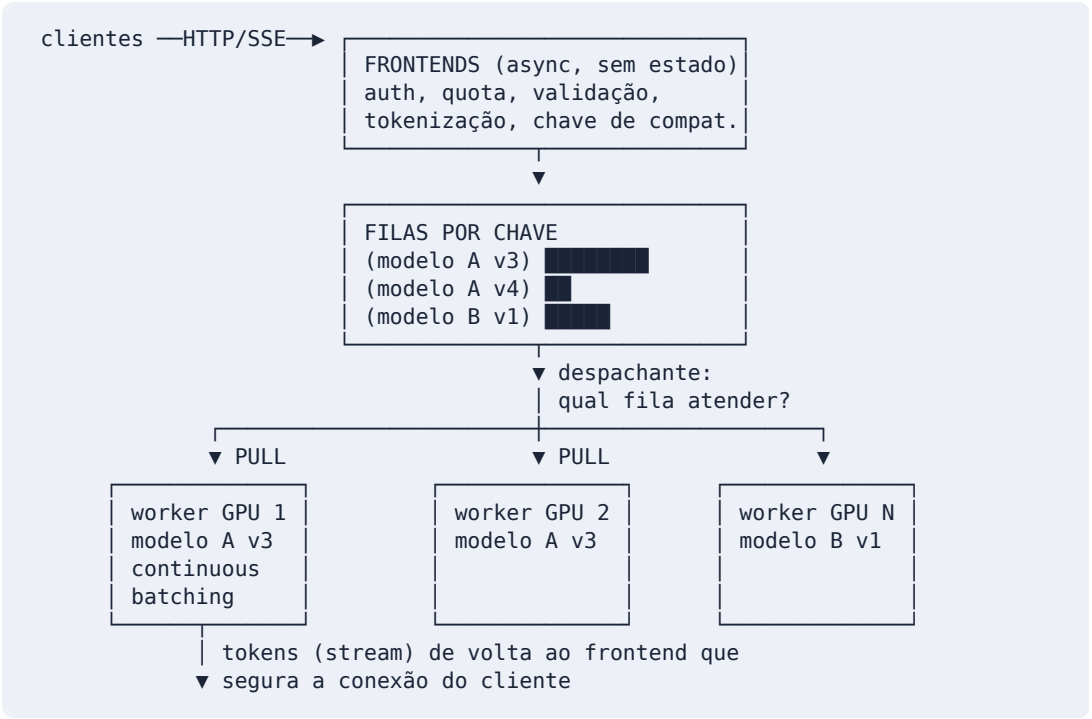
Resposta completa

1. O que é "compatível". É a palavra-chave do enunciado. Duas requisições podem dividir um passo de GPU se usam os **mesmos pesos** e o **mesmo programa de execução**:

Atributo	Precisa ser igual?	Observação
Modelo e versão	sim	pesos diferentes
Precisão (FP8, BF16)	sim	outro engine/kernels
Adaptador LoRA	não, com multi-LoRA (14.2)	sim, sem suporte
Parâmetros de sampling	não	aplicados por sequência
Tamanho do prompt	não, com continuous batching	sim, em engines de formas fixas (buckets)
Saída estruturada	não	máscara por sequência
Prioridade	não	afeta a ordem, não a compatibilidade

Conclusão que você deve dizer: "Com um engine moderno, a chave de compatibilidade se reduz basicamente a (modelo, versão, precisão), e às vezes adaptador. Quanto menos atributos na chave, maiores os batches."

2. Arquitetura.



3. Pull em vez de push. Os workers **pedem** trabalho quando têm capacidade (blocos de KV livres, orçamento de tokens). Vantagens: backpressure natural (um worker cheio não recebe nada), balanceamento automático (o worker mais livre pede mais), e falhas simples (um worker que morre para de pedir). Desvantagem: latência de ida e volta para pedir trabalho, mitigada pedindo em lote e antecipando.

4. O despachante com pool limitado. Quando há mais filas do que workers livres, quem é atendido? Uma pontuação por fila:

$$\text{score} = w_1 \cdot \text{prioridade} + w_2 \cdot \text{idade da mais antiga} + w_3 \cdot \min(\text{tamanho}, B_{max})$$

- O termo de **tamanho** favorece filas que formam batches cheios (eficiência).
- O termo de **idade** impede inanição: uma chave rara (um modelo pouco usado) acaba atendida antes de estourar o SLO, mesmo com um batch pequeno.
- Se a chave exige um modelo que nenhum worker tem carregado, o despachante pode decidir **trocar** o modelo de um worker (caro: segundos a minutos), o que só compensa com fila suficiente. Por isso, réplicas por modelo são decididas pelo autoscaler, e a troca sob demanda é a exceção.

5. Batching dentro do worker. Continuous batching com chunked prefill (Capítulo 7). O worker admite requisições da fila enquanto houver memória e orçamento.

6. Concorrência alta no HTTP. Dezenas de milhares de conexões abertas (muitas em streaming por dezenas de segundos):

- Frontends assíncronos (event loop), sem uma thread por conexão.
- Os tokens voltam do worker para o frontend que segura a conexão (por um canal direto, gRPC streaming, ou um barramento de mensagens).
- Keep-alive e limites de conexões por cliente.
- Cancelamento: cliente desconectou → o frontend avisa o worker → a sequência sai do batch.

7. Sobrecarga. Estimar a espera: (tokens na fila da chave) ÷ (throughput dos workers da chave). Se passa do SLO, 429 imediato com `Retry-After`. Prioridades por plano.

8. Números. Faça uma conta rápida como a da seção 11.2 para mostrar quantos workers o pool precisa e onde está o limite.

Perguntas de acompanhamento prováveis

- **Por que não uma fila única?** Requisições incompatíveis não dividem batch; uma fila única obrigaria o worker a pular itens, e o FCFS deixaria de fazer sentido. Filas por chave tornam o despacho explícito.
- **Como garantir justiça entre clientes dentro da mesma chave?** Subfilas por cliente com round robin ponderado, ou fair queuing por tokens.
- **E se um worker morrer com 200 sequências em andamento?** As conexões no frontend recebem erro ou as requisições são reenfileiradas com o texto já gerado como continuação. O despachante para de contar com aquele worker.
- **Onde ficaria o prefix caching?** No worker; o despachante pode preferir o worker com o prefixo em cache (afinidade), como na Q043.

Armadilhas comuns

- Não definir “compatível”.
- Push cego para workers, sem backpressure.
- Esquecer a inanição de chaves raras.
- Thread por conexão no HTTP.

Q106 Projete um serviço que aceite requisições de inferência online de baixa latência, agrupe requisições

compatíveis em batches, roteie o trabalho para workers de GPU e suporte vários modelos e versões ao mesmo tempo.

> **[Relatado]** Anthropic · Software Engineer · presencial · listada em mar. 2026 · Difícil · fonte

Resposta em 30 segundos

"É a mesma base da Q105, com a ênfase em vários modelos e versões. Separo o problema em três: posicionamento, que decide quais modelos e versões ficam em quais GPUs, com base na demanda e na memória; roteamento, que manda cada requisição para um worker que tem o modelo e a versão certos, com balanceamento por carga em tokens; e batching dentro do worker, por modelo. Para baixa latência: orçamento de latência por etapa, prazo de espera curto no batching, workers sempre aquecidos para os modelos de maior tráfego, e nada de carregar modelo no caminho da requisição. Para versões: pools por versão, roteamento por pesos para canary e A/B, drenagem antes de desligar, e roteamento fixo por cliente quando a versão importa para ele."

Resposta completa

1. Orçamento de latência. Para "baixa latência", quebre o SLO em partes e diga quanto cada componente pode gastar:

```
SLO: TTFT p99 < 300 ms (prompts < 2k tokens)

rede + TLS ..........................  20 ms
frontend (auth, validação, tokenização)   5 ms
roteamento ..........................   2 ms
fila + espera de batching ..............  60 ms  ◄ o botão
prefill (2k tokens) ..................... 150 ms
primeiro token de volta ................  10 ms
folga ................................. ~50 ms
```

A espera de batching é o botão que troca latência por eficiência. Com continuous batching, a espera é até o próximo passo do worker (dezenas de ms), não um prazo fixo.

2. Posicionamento (placement). Decidir quais modelos rodam onde:

```
entradas: demanda por (modelo, versão) em tokens/s,
          memória de cada modelo, perfil de GPU,
          SLO de cada um
saída:    número de réplicas por (modelo, versão)
          e em quais nós

heurística:
1. modelos grandes e muito usados: réplicas dedicadas,
   número ∝ demanda / capacidade por réplica
2. modelos pequenos: vários na mesma GPU (multiplexação),
   se a soma da memória e do tráfego couber
3. versões em canary: poucas réplicas, crescem com o peso
4. cauda longa: carga sob demanda, com custo de cold start
   aceito (ou SLO diferente)
```

3. Roteamento. O roteador mantém um mapa (modelo, versão) → workers saudáveis com aquele modelo carregado, atualizado pelo plano de controle. Para cada requisição:

1. Resolver a versão: a pedida explicitamente, ou a escolhida pela política (pesos de canary e A/B, atribuição estável por cliente).
2. Escolher o worker: menor carga em tokens entre duas sorteadas (power of two choices), com preferência por afinidade de prefixo.
3. Enviar e receber o stream.

4. Batching por modelo, no worker. Continuous batching. Se vários modelos pequenos dividem uma GPU, cada um tem seu engine e o compartilhamento é de memória e tempo de GPU (com MPS ou MIG para isolamento).

5. Ciclo de vida das versões.

```
v8 publicada no registro
  └─▶ pré-distribuição dos pesos (Q083)
  └─▶ réplicas da v8 sobem e aquecem (fora do tráfego)
  └─▶ roteador: v8 com 1% → compara métricas
  └─▶ 5% → 25% → 100% (ou rollback)
  └─▶ v7: drena (sem requisições novas, termina as atuais)
  └─▶ v7: desligada; pesos mantidos no cache por N dias
```

Clientes que fixaram a v7 (por reprodutibilidade) mantêm uma réplica mínima da v7 até a data de descontinuação anunciada.

6. Diferenças em relação à Q105, que você pode verbalizar: a Q105 é sobre o despacho com pool limitado (quem é atendido); esta é sobre o mapa de modelos para GPUs e as transições de versão sem perder requisições.

Perguntas de acompanhamento prováveis

- **Como você multiplexa vários modelos pequenos numa GPU sem que um prejudique o outro?** MIG para isolamento de hardware; ou processos separados com MPS e limites de memória; e monitoramento de latência por modelo para detectar interferência.
- **O que acontece se a demanda por um modelo triplicar de repente?** Rejeição antecipada e prioridades no curto prazo; o autoscaler sobe réplicas (minutos); se houver modelos de baixa prioridade ocupando GPUs, eles podem ser despejados para dar lugar.
- **Como garantir a mesma resposta para o mesmo prompt numa mesma versão?** Fixar a versão, temperatura 0 e kernels invariantes ao tamanho do batch; mesmo assim, diferenças numéricas entre configurações podem mudar tokens. Seja honesto sobre isso (Q034).
- **Como medir se o batching está ajudando?** Tamanho médio de batch por modelo, tokens/s por GPU, e a latência adicionada pela espera.

Armadilhas comuns

- Não ter orçamento de latência por etapa.
- Carregar modelos no caminho da requisição.
- Trocar de versão sem drenagem.
- Ignorar o posicionamento de modelos em GPUs.

Q107 Projete um sistema de upload de arquivos para um app de chat com IA que lide com uploads multimodais grandes e os coloque no contexto do modelo.

> **[Relatado]** xAI · AI engineer · design de sistemas · 2026 · Difícil · fonte

Resposta em 30 segundos

“O upload vai direto do cliente para o armazenamento de objetos, em partes e com retomada, usando URLs pré-assinadas, sem passar pelos servidores da aplicação. O cliente calcula o hash para deduplicar. Quando o upload termina, um pipeline assíncrono valida o tipo e o tamanho, faz a varredura de segurança e processa por tipo: extração de texto e OCR para PDFs, redimensionamento para imagens, transcrição para áudio, amostragem de quadros para vídeo. Ele conta os tokens e, se o arquivo couber no orçamento de contexto, coloca o conteúdo inteiro; se não couber, divide

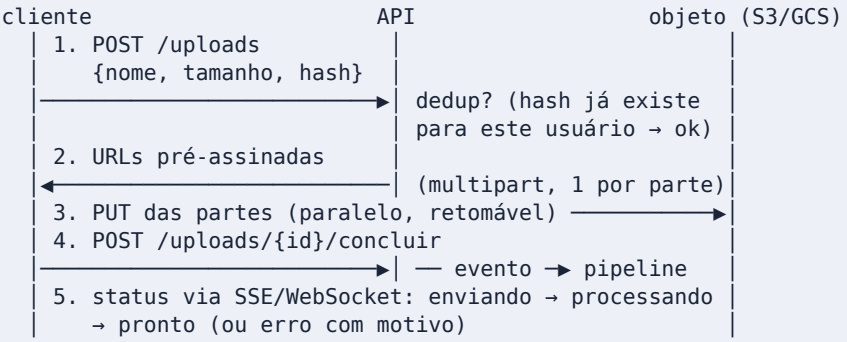
em pedaços com embeddings e usa busca para trazer só os trechos relevantes. Guardo as representações processadas por hash do conteúdo, inclusive as saídas do encoder de visão, e coloco o arquivo numa posição estável do prompt para o prefix cache funcionar nos turnos seguintes. O cliente acompanha o status em tempo real. E trato o conteúdo do arquivo como dados, nunca como instruções."

Resposta completa

1. Requisitos. Tipos (PDF, imagens, planilhas, código, áudio, vídeo), tamanho máximo (centenas de MB? GB?), volume (uploads por dia), latência aceitável até o arquivo estar "pronto", limites de contexto do modelo, privacidade e retenção.

> **Fazendo a conta (hipotética):** 2 milhões de uploads por dia, 5 MB em média.
>
> - Volume: 10 TB por dia de entrada; ~115 MB/s em média, com picos de 3 a 5×.
> - Um PDF de 100 páginas vira ~50 mil tokens de texto. Uma imagem, de centenas a alguns milhares de tokens visuais.
> - Se 30% dos uploads são PDFs grandes, o processamento (OCR, extração) é o gargalo de CPU/GPU, não o armazenamento.

2. O caminho do upload.



- **Direto para o armazenamento**: os servidores da aplicação não carregam bytes, só metadados. Escala e custo muito melhores.
- **Multipart e retomada**: arquivos grandes em partes de 8 a 64 MB; se a conexão cai, só a parte atual é reenviada.
- **Deduplicação por hash** (por usuário ou organização, para não vazar a existência de arquivos de outras pessoas).

3. O pipeline de processamento.

```
evento "upload concluído"
  ▼
VALIDAR: tipo real (pelos bytes, não pela extensão),
         tamanho, páginas, duração
  ▼
SEGURANÇA: antivírus/malware, arquivos compactados
           maliciosos (zip bomb), PDFs com scripts
  ▼
PROCESSAR POR TIPO (workers separados por tipo,
                    filas com prioridade):
  PDF/DOCX → texto + layout; OCR em páginas escaneadas
  imagem   → normaliza, redimensiona, gera miniatura
  áudio    → transcrição (modelo de fala)
  vídeo    → amostra quadros + transcreve o áudio
  planilha → tabelas em texto estruturado (ou ferramenta)
  código   → texto, com linguagem detectada
  ▼
CONTAR TOKENS e decidir a ESTRATÉGIA DE CONTEXTO
  ▼
GUARDAR: representação processada, chave = hash do
         conteúdo + versão do processador
  ▼
status "pronto"
```

4. Como o arquivo entra no contexto. A decisão central:

Situação	Estratégia
Cabe com folga no contexto	conteúdo inteiro no prompt
Grande, mas a pergunta é geral (“resuma”)	resumo hierárquico (map-reduce) gerado sob demanda e cacheado
Grande, pergunta específica	dividir em trechos, embeddings, buscar os relevantes (RAG)
Imagens	tokens visuais pelo encoder; resolução escolhida pelo orçamento
Vídeo longo	quadros amostrados + transcrição, com busca por trecho

5. Ligação com o serving (o que diferencia a resposta).

- **Prefix caching**: o arquivo vira um prefixo enorme. Se ele entra sempre na **mesma posição** do prompt (logo depois do system prompt e antes da conversa), os turnos seguintes reaproveitam o KV do arquivo inteiro. Se a aplicação reordena ou reformata o conteúdo a cada turno, o cache se perde e cada pergunta paga o prefill de 50 mil tokens de novo (Q043).

- **Cache do encoder**: embeddings visuais por hash da imagem, para não reprocessar.
- **Roteamento por afinidade**: a conversa com um arquivo grande deve voltar para a réplica que tem o KV dele.
- **Contexto longo é caro**: o prefill de 100 mil tokens leva segundos; mostrar "lendo o arquivo..." na interface e usar a resposta em streaming.

6. Segurança e privacidade.

- Controle de acesso: só o dono (e quem ele compartilhou) lê o arquivo e suas representações.
- Criptografia em repouso e em trânsito; retenção e exclusão (apagar o arquivo apaga também as representações, embeddings e caches).
- **Injeção de prompt em documentos**: um PDF pode conter texto como "ignore as instruções anteriores e...". O conteúdo do arquivo é **dado**, não instrução; o template do prompt deve delimitar claramente o conteúdo, e o sistema não deve dar a esse conteúdo autoridade para acionar ferramentas sensíveis sem confirmação.

7. Falhas e operação. Retries idempotentes por etapa, filas com dead letter, limite de tempo por arquivo, métricas de tempo até "pronto" por tipo e tamanho, taxa de falha por tipo.

Perguntas de acompanhamento prováveis

- **Por que não mandar o arquivo para o servidor da API?** Os servidores ficariam ocupados com bytes, com conexões longas e uso de memória, e escalariam mal. URLs pré-assinadas tiram esse peso.
- **O usuário pergunta algo antes do processamento terminar. E agora?** Mostrar o status; responder com o que já está pronto (por exemplo, as primeiras páginas) deixando isso claro; ou esperar com um tempo limite.
- **Como escolher o tamanho dos trechos para RAG?** Pelo tipo de documento: por seção ou parágrafo, com sobreposição pequena, algumas centenas de tokens; avaliar a qualidade das respostas com perguntas reais.
- **Como lidar com um vídeo de 2 horas?** Transcrição + amostragem esparsa de quadros, indexados por tempo; buscar os trechos relevantes à pergunta em vez de colocar tudo no contexto.

Armadilhas comuns

- Passar os bytes pelos servidores da aplicação.
- Processar de forma síncrona no upload.

- Ignorar o limite de contexto e o custo do prefill.
- Esquecer o prefix caching e a posição estável do arquivo no prompt.
- Não tratar injeção de prompt nos documentos.

Resumo do capítulo

- **Speculative decoding**: rascunho propõe k tokens, alvo verifica numa passada; regra $\min(1, p/q)$ preserva a distribuição; $E = (1 - \alpha^{k+1})/(1 - \alpha)$ tokens por passada; perde força com batch grande.
- Rascunhos: modelo pequeno, n-gramas, Medusa, EAGLE, MTP; verificação em árvore.
- **Multi-LoRA**: um modelo base, adaptadores aplicados por requisição no mesmo batch com kernels agrupados; cache de adaptadores; chave de cache com o adaptador.
- **Constrained decoding**: autômato da gramática gera máscara de tokens; calcular na CPU em paralelo com a GPU.
- **Multimodal**: tokens visuais aumentam o prefill; encoder separado; cache por hash do conteúdo.
- **APIs com batching dinâmico**: chave de compatibilidade mínima, filas por chave, workers que puxam trabalho, despachante com pontuação anti-inanição.
- **Vários modelos e versões**: orçamento de latência, posicionamento, roteamento, ciclo de vida com canary e drenagem.
- **Upload multimodal**: direto ao armazenamento, pipeline assíncrono por tipo, estratégia de contexto (inteiro, resumo, RAG), posição estável para o prefix cache, segurança e injeção de prompt.

Checklist para a entrevista

- ☐ Explico speculative decoding e a regra de aceitação, e por que a distribuição é preservada.
- ☐ Calculo o número esperado de tokens por passada e o speedup.
- ☐ Comparo os tipos de rascunho e digo quando a especulação não ajuda.
- ☐ Explico serving multi-LoRA com a fórmula e os kernels agrupados.
- ☐ Explico constrained decoding e o problema de desempenho da máscara.
- ☐ Listo os impactos de entradas multimodais no serving.
- ☐ Defino a chave de compatibilidade para batching e o que dispensa estar nela.
- ☐ Desenho um sistema pull com filas por chave e um despachante justo.

- [ ] Monto um orçamento de latência por etapa.
- [ ] Projeto o upload multimodal ponta a ponta, incluindo prefix cache e segurança.

APÊNDICE A

Números e fórmulas de referência

Os números abaixo são aproximados e servem para contas de guardanapo. Confira as especificações da geração de hardware com que você vai trabalhar.

A.1 Hardware

Item	Valor aproximado
H100 SXM: memória	80 GB HBM3
H100 SXM: banda de memória	~3,35 TB/s
H100 SXM: BF16/FP16 denso (tensor cores)	~989 TFLOPS
H100 SXM: FP8 denso	~1.979 TFLOPS
H100 SXM: FP32 (sem tensor cores)	~67 TFLOPS
H100 SXM: SMs / L2 / shared por SM	132 / 50 MB / até 228 KB
H100: NVLink 4	900 GB/s total por GPU (~450 por sentido)
A100 80GB SXM: banda / BF16 denso	~2,0 TB/s / ~312 TFLOPS
A100: SMs / L2 / NVLink 3	108 / 40 MB / 600 GB/s total
PCIe Gen4 / Gen5 x16	~32 / ~64 GB/s por sentido
InfiniBand NDR (por porta)	400 Gb/s ≈ 50 GB/s
Warp	32 threads (wavefront na AMD: 64 ou 32)
Threads por bloco (máx.)	1.024
Registradores por SM (H100)	65.536 × 32 bits (256 KB)
Warps por SM (H100)	até 64 (2.048 threads)

A.2 Modelos de exemplo

	Llama 3 8B	Llama 3 70B
Camadas	32	80
d_model	4.096	8.192
Heads de query / KV	32 / 8	64 / 8
head_dim	128	128
MLP	14.336	28.672
Vocabulário	128.256	128.256
Pesos em BF16	~16 GB	~140 GB
KV por token em BF16	128 KiB	320 KiB

A.3 Fórmulas

O quê	Fórmula
FLOPs por token (forward, sem atenção)	$\approx 2 \times$ parâmetros
FLOPs da atenção no prefill (com máscara causal)	$\approx 2 \times L \times T^2 \times d_{model}$
KV cache por token	$2 \times L \times H_{kv} \times d_{head} \times$ bytes
Tempo mínimo de um passo de decode	(bytes dos pesos + bytes do KV lido)/banda
Intensidade aritmética	FLOPs ÷ bytes lidos
Ponto de equilíbrio (ridge point)	pico de FLOPS ÷ banda (H100 BF16: ~295)
Roofline	$\min$ (pico, banda $\times$ intensidade)
Latência E2E	TTFT + $(N - 1) \times$ TPOT
Custo por 1M tokens	custo/h ÷ tokens/h $\times 10^6$
Lei de Little	itens no sistema = taxa de chegada × tempo no sistema
Espera em fila (intuição)	cresce como $\rho/(1 - \rho)$
Amdahl	$1/((1 - p) + p/s)$
All-reduce em anel	$2(p - 1)\alpha + 2\frac{p-1}{p}\frac{n}{\beta}$
busbw do all-reduce	algbw $\times\, 2(p - 1)/p$

O quê	Fórmula
Intensidade de GEMM em tiles (FP32)	$2 \cdot BM \cdot BN/(4(BM + BN))$
Speculative decoding	$E[\text{tokens}] = (1 - \alpha^{k+1})/(1 - \alpha)$
Quantização linear	$q = \text{round}(x/s) + z$, $\hat{x} = s(q - z)$

A.4 Regras práticas

- Decode com batch pequeno é memory-bound; prefill com prompts de centenas de tokens ou mais é compute-bound.
- Llama 3 8B numa H100: no máximo ~210 tokens/s para um usuário sozinho (16 GB ÷ 3,35 TB/s).
- Quantizar os pesos de 16 para 8 bits deixa o decode perto de 2× mais rápido no limite teórico; para 4 bits, perto de 4× (menos na prática).
- Tensor parallelism dentro do nó; pipeline parallelism entre nós.
- Mensagens de comunicação abaixo de alguns MB pagam latência; acima, pagam banda.
- Com o SLO de cauda rígido, opere bem abaixo de 100% de utilização.

APÊNDICE B

Glossário

Adaptador LoRA: matrizes de posto baixo somadas aos pesos de um modelo base para especializá-lo.

All-reduce: operação coletiva em que todas as GPUs terminam com a soma (ou outra redução) dos dados de todas.

All-to-all: cada GPU envia um pedaço diferente para cada outra; usado no expert parallelism.

AWQ: método de quantização de pesos que protege canais salientes com escalas, guiado pelas ativações.

Bank conflict: duas threads de um warp acessando endereços diferentes no mesmo banco da shared memory, o que serializa os acessos.

Batching contínuo (continuous batching): agendamento por iteração; sequências entram e saem do batch a cada passo.

BF16: formato de 16 bits com 8 bits de expoente (mesma faixa do FP32) e 7 de mantissa.

Bloco (CUDA): grupo de até 1.024 threads que roda num único SM e compartilha shared memory.

Chunked prefill: dividir o prefill de um prompt longo em pedaços misturados aos decodes.

Coalescing: threads vizinhas de um warp lendo endereços vizinhos, permitindo poucas transações de memória.

Compute-bound: limitado pela capacidade de cálculo.

Copy-on-write: compartilhar um bloco até alguém precisar escrever nele; aí ele é copiado.

CUDA graph: sequência de kernels gravada uma vez e reexecutada com um único lançamento.

DCGM: ferramenta da NVIDIA para monitorar e diagnosticar GPUs em data centers.

Decode: fase da geração que produz um token por passo, lendo pesos e KV cache.

Desagregação: separar prefill e decode em pools de GPUs diferentes.

Device plugin: componente que anuncia GPUs ao Kubernetes como recurso (`nvidia.com/gpu`).

Distilação: treinar um modelo menor para imitar um maior.

EP (expert parallelism): especialistas de um MoE distribuídos entre GPUs.

FlashAttention: atenção exata e ciente da memória, com tiling e softmax online.

FP8 (E4M3, E5M2): formatos de 8 bits de ponto flutuante; E4M3 tem mais precisão, E5M2 mais faixa.

Fusão de kernels: juntar operações num único kernel para os dados intermediários não irem à HBM.

Goodput: throughput que cumpre o SLO.

GPTQ: quantização de pesos que compensa o erro de arredondamento coluna a coluna com informação de segunda ordem.

GQA: grouped-query attention; grupos de heads de query compartilham uma head de K e V.

HBM: memória principal da GPU, de alta banda.

Head-of-line blocking: item no começo de uma fila travando os que estão atrás.

In-flight batching: nome da NVIDIA para continuous batching.

Intensidade aritmética: FLOPs por byte lido da memória.

ITL: inter-token latency; tempo entre dois tokens seguidos.

Kernel: função que roda na GPU em muitas threads.

KV cache: K e V dos tokens já processados, guardados por camada para não recalcular.

Lei de Little: itens no sistema = taxa × tempo no sistema.

MBU / MFU: utilização da banda de memória / dos FLOPS, em relação ao pico.

Memory-bound: limitado pela banda de memória.

MIG: divisão de uma GPU em instâncias isoladas em hardware.

MLA: multi-head latent attention; comprime K e V num vetor latente por token.

MoE: mixture of experts; cada token usa só alguns especialistas do MLP.

MQA: multi-query attention; uma única head de K e V para todas as queries.

NCCL: biblioteca de comunicação coletiva da NVIDIA.

NVLink / NVSwitch: ligação de alta banda entre GPUs / switch que liga todas as GPUs de um nó.

Occupancy: warps ativos num SM ÷ máximo possível.

PagedAttention: KV cache em blocos de tamanho fixo com tabela de blocos, como memória virtual.

Prefill: fase que processa o prompt inteiro de uma vez e preenche o KV cache.

Prefix caching: reaproveitar o KV cache de prefixos já calculados.

Preempção: tirar uma sequência do batch para liberar memória (recomputando ou fazendo swap depois).

PP (pipeline parallelism): camadas divididas em estágios em GPUs diferentes.

PTQ / QAT / QAD: quantização pós-treino / com treino simulando quantização / com treino e distilação de um professor.

RDMA: acesso direto à memória de outra máquina sem passar pela CPU.

Roofline: modelo que limita a performance por pico de cálculo e por banda × intensidade.

RoPE: codificação de posição que gira Q e K por um ângulo proporcional à posição.

Shared memory: memória rápida dentro do SM, compartilhada pelas threads de um bloco.

SIMT: modelo em que as 32 threads de um warp executam a mesma instrução.

SLI / SLO: indicador / objetivo de nível de serviço.

SM: streaming multiprocessor, a unidade de execução da GPU NVIDIA.

SmoothQuant: migra a dificuldade de quantizar ativações para os pesos com escalas por canal.

Speculative decoding: rascunho propõe tokens, modelo alvo verifica vários de uma vez.

STE: straight-through estimator; trata o arredondamento como identidade no backward.

Tensor core: unidade da GPU especializada em multiplicação de matrizes pequenas.

TP (tensor parallelism): cada matriz dividida entre GPUs.

TPOT: tempo médio por token gerado depois do primeiro.

Triton: linguagem em Python para escrever kernels de GPU por blocos.

TTFT: tempo até o primeiro token.

Warp: grupo de 32 threads que executa junto.

Xid: código de erro reportado pelo driver NVIDIA.

APÊNDICE C

Plano de estudo de 30 dias

Uma a duas horas por dia. Ajuste a ordem ao perfil da vaga (Introdução).

```
SEMANA 1 — FUNDAMENTOS
dia 1   Cap. 1 até 1.3 (prefill/decode, memory-bound, roofline)
dia 2   Cap. 1 até o fim; refaça todas as contas à mão
dia 3   Cap. 2 até 2.6 (decoder, atenção, GQA, RoPE, sampling)
dia 4   Cap. 2 até o fim (KV cache, FlashAttention)
dia 5   Prática: rode e modifique o código da Q097 (mini LLM)
dia 6   Cap. 6 (KV cache e memória)
dia 7   Revisão: responda Q001–Q011 e Q041–Q044 em voz alta

SEMANA 2 — SERVING E COMPRESSÃO
dia 8   Cap. 7 até 7.5 (batching, scheduler, chunked prefill)
dia 9   Cap. 7 até o fim; escreva o batcher da Q049 de memória
dia 10  Cap. 8 até 8.6 (formatos, escalas, outliers)
dia 11  Cap. 8 até o fim (GPTQ, AWQ, QAT, QAD, distilação)
dia 12  Prática: sirva um modelo pequeno com vLLM; meça TTFT
        e TPOT com batch 1 a 64, BF16 vs quantizado
dia 13  Cap. 14.1 a 14.3 (speculative, multi-LoRA, saídas
        estruturadas)
dia 14  Revisão: Q049–Q059 em voz alta, com contas

SEMANA 3 — GPU E DISTRIBUÍDO
dia 15  Cap. 3 até 3.5 (SMs, warps, memória, occupancy)
dia 16  Cap. 3 até o fim; escreva a GEMM com tiling de memória
dia 17  Cap. 4 (método, roofline, Little, fusão, CUDA graphs)
dia 18  Cap. 5 (GEMM 2D, reduções, Triton)
dia 19  Cap. 9 (TP, PP, EP, desagregação)
dia 20  Cap. 10 (interconexões, coletivas, NCCL, Kubernetes)
dia 21  Revisão: Q017–Q035 e Q065–Q075

SEMANA 4 — DESIGN, OPERAÇÃO E SIMULAÇÕES
dia 22  Cap. 11 (roteiro e arquitetura de referência)
dia 23  Faça a Q081 sozinho em 45 min, cronometrando
dia 24  Cap. 12 (confiabilidade); Q089 e Q090
dia 25  Cap. 14 até o fim; Q105–Q107
dia 26  Cap. 13; resolva Q098 e Q099 sem consultar
dia 27  Simulação 1: design (peça a alguém para entrevistar)
dia 28  Simulação 2: código + conceitos
dia 29  Prepare 3 histórias (otimização, incidente, kernel)
        no formato contexto → medição → mudança → resultado
dia 30  Revise os checklists de todos os capítulos
```

APÊNDICE D

As 50 perguntas

ID	Pergunta (resumida)	Marcação	Empresa	Capítulo
Q001	Experiência com otimização de inferência	Relatado	Groq	1
Q002	Por que a inferência é memory-bound	Fornecedor	—	1
Q003	TTFT, TPOT e throughput	Fornecedor	—	1
Q004	Forward pass na GPU	Fornecedor	—	1
Q007	Custo acima do orçamento	Fornecedor	—	1
Q008	Síncrona vs assíncrona	Fornecedor	—	1
Q009	KV cache e FlashAttention	Relatado	NVIDIA	2
Q010	Armadilhas do KV cache	Relatado	Startups.com	2
Q011	Prefill vs decode	Fornecedor	—	2
Q017	Modelo de execução CUDA + GEMM	Relatado	NVIDIA	3
Q018	Lei de Amdahl	Relatado	NVIDIA	3
Q019	GEMM sem shared memory	Relatado	NVIDIA	3
Q020	Um kernel CUDA que você escreveu	Relatado	NVIDIA	3

ID	Pergunta (resumida)	Marcação	Empresa	Capítulo
Q025	Kernel a 30% da banda	Fornecedor	NVIDIA	4
Q026	Redução: 64 vs 256 threads	Fornecedor	NVIDIA	4
Q027	Alta utilização de tensor cores	Relatado	AMD	4
Q028	Pipeline otimizado e benchmark	Relatado	NVIDIA	4
Q033	GEMM FP32 em CUDA	Relatado	NVIDIA	5
Q034	Redução determinística	Relatado	Applied	5
Q035	Como funcionam TensorRT-LLM e vLLM	Relatado	NVIDIA	5
Q041	Implementar KV cache	Relatado	Applied Intuition	6
Q042	Sem memória em conversas longas	Fornecedor	Meta	6
Q043	Reenvio da conversa inteira	Fornecedor	Moonshot AI	6
Q044	PagedAttention	Fornecedor	—	6
Q049	Batching numa GPU, até 100 entradas	Relatado	Anthropic	7
Q050	Criticar design de batching	Relatado	Anthropic	7
Q051	Quando TensorRT-LLM ganha do vLLM	Fornecedor	—	7
Q052	Scheduler de continuous batching	Fornecedor	—	7
Q053	Batching sem head-of-line blocking	Fornecedor	Together AI	7

ID	Pergunta (resumida)	Marcação	Empresa	Capítulo
Q057	Quantização, QAT e QAD	Relatado	NVIDIA	8
Q058	Técnicas de otimização	Relatado	NVIDIA	8
Q059	Quantização vs distilação	Fornecedor	—	8
Q065	Tensor vs pipeline parallelism	Relatado	NVIDIA	9
Q066	GEMM em múltiplas GPUs	Relatado	Google	9
Q067	Desagregação de prefill e decode	Fornecedor	Moonshot AI	9
Q073	GPUs, CI/CD, comunicação, Kubernetes	Relatado	Fireworks AI	10
Q074	Ring vs tree all-reduce	Prática	—	10
Q075	NVLink, PCIe e InfiniBand	Prática	—	10
Q081	Serviço de LLM de alta concorrência	Relatado	Anthropic	11
Q082	API de inferência multi-modelo	Relatado	Anthropic	11
Q083	Distribuição de pesos	Relatado	Anthropic	11
Q089	Pod em Pending	Relatado	CoreWeave	12
Q090	Stack de inferência e falhas	Relatado	Anthropic	12
Q091	Endpoint dedicado com p99 rígido	Fornecedor	Together AI	12
Q097	Loop de inferência em PyTorch	Relatado	NVIDIA	13

ID	Pergunta (resumida)	Marcação	Empresa	Capítulo
Q098	Ordenação topológica	Relatado	NVIDIA	13
Q099	Serializar árvore binária	Relatado	NVIDIA	13
Q105	API com batching dinâmico	Relatado	Anthropic	14
Q106	Vários modelos e versões	Relatado	Anthropic	14
Q107	Upload multimodal para chat	Relatado	xAI	14

APÊNDICE E

Referências

Fonte das perguntas

- AI Engineering Insider. *You should never walk into an Inference Engineering interview without knowing these 50 questions.* https://aiengineeringinsider.substack.com/p/you-should-never-walk-into-an-inference

Arquitetura de modelos

- Vaswani et al., 2017. *Attention Is All You Need.*
- Shazeer, 2019. *Fast Transformer Decoding: One Write-Head is All You Need* (MQA).
- Ainslie et al., 2023. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.*
- DeepSeek-AI, 2024. *DeepSeek-V2* (MLA).
- Su et al., 2021. *RoFormer: Enhanced Transformer with Rotary Position Embedding* (RoPE).

Atenção e kernels

- Milakov e Gimelshein, 2018. *Online normalizer calculation for softmax.*
- Dao et al., 2022. *FlashAttention.* Dao, 2023. *FlashAttention-2.* Shah et al., 2024. *FlashAttention-3.*
- Volkov, 2010. *Better Performance at Lower Occupancy.*
- Williams, Waterman e Patterson, 2009. *Roofline: An Insightful Visual Performance Model.*
- Tillet et al., 2019. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.*

Serving

- Yu et al., 2022. *Orca: A Distributed Serving System for Transformer-Based Generative Models.*
- Kwon et al., 2023. *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM).
- Agrawal et al., 2023. *SARATHI.* Agrawal et al., 2024. *Sarathi-Serve.*
- Zheng et al., 2024. *SGLang* (RadixAttention).
- Zhong et al., 2024. *DistServe.*
- Patel et al., 2024. *Splitwise.*
- Qin et al., 2024. *Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving.*
- Prabhu et al., 2024. *vAttention.*
- Chen et al., 2023. *Punica.* Sheng et al., 2023. *S-LoRA.*

KV cache

- Xiao et al., 2023. *Efficient Streaming Language Models with Attention Sinks* (StreamingLLM).
- Zhang et al., 2023. *H2O: Heavy-Hitter Oracle.*
- Liu et al., 2024. *KIVI.*

Quantização e compressão

- Dettmers et al., 2022. *LLM.int8().*
- Xiao et al., 2023. *SmoothQuant.*
- Frantar et al., 2022. *GPTQ.*
- Lin et al., 2023. *AWQ.*
- Frantar e Alistarh, 2023. *SparseGPT.*
- Hinton, Vinyals e Dean, 2015. *Distilling the Knowledge in a Neural Network.*

Paralelismo

- Shoeybi et al., 2019. *Megatron-LM.*
- Liu et al., 2023. *Ring Attention with Blockwise Transformers.*

Speculative decoding

- Leviathan et al., 2023. *Fast Inference from Transformers via Speculative Decoding*.
- Chen et al., 2023. *Accelerating Large Language Model Decoding with Speculative Sampling*.
- Cai et al., 2024. *Medusa.* Li et al., 2024. *EAGLE*.

Documentação

- NVIDIA: *CUDA C++ Programming Guide*, *CUDA C++ Best Practices Guide*, documentação do Nsight Systems, Nsight Compute, NCCL, DCGM e GPU Operator.
- PyTorch: documentação de `torch.compile`, `torch.profiler` e CUDA graphs.
- Documentação do vLLM, TensorRT-LLM, SGLang e Triton (tutoriais).